STUDY TEXTBOOK

Competitive Programming and Algorithms

From first submission to gold

First edition · 10 October 2026

Contents

1 The contest machine and the cost model **1**

1.1 How a judge sees your program . . . 1
1.2 The C++17 starter kit . . . 3
1.3 Time complexity . . . 7
1.4 From constraints to a plan . . . 9
1.5 The submit loop . . . 11
1.6 Common mistakes . . . 13
1.7 Chapter summary . . . 14
1.8 Additional exercises . . . 14
Notes . . . 15

2 Complete search **17**

2.1 The exhaustive idea . . . 17
2.2 Enumerating subsets . . . 19
2.3 Enumerating permutations . . . 23
2.4 Backtracking . . . 26
2.5 Search-space sketches . . . 28
2.6 Common mistakes . . . 29
2.7 Chapter summary . . . 29
2.8 Additional exercises . . . 29
Notes . . . 31

3 Sorting and greedy algorithms **33**

3.1 Sorting as a tool . . . 33
3.2 The greedy method . . . 35
3.3 How to argue correctness . . . 36
3.4 When greedy fails . . . 39
3.5 Sets and maps as greedy tools . . . 41

3.6 Chapter summary . . . 41
3.7 Common mistakes . . . 41
3.8 Additional exercises . . . 42
Notes . . . 43

4 Binary search **45**
4.1 The one correct template . . . 45
4.2 Monotone predicates: searching the answer . . . 46
4.3 Real-valued search . . . 52
4.4 Where the pattern ends . . . 53
4.5 Chapter summary . . . 53
4.6 Common mistakes . . . 54
4.7 Additional exercises . . . 54
Notes . . . 55

5 Prefix sums, difference arrays, and two pointers **56**
5.1 Prefix sums: build once, query forever . . . 56
5.2 Difference arrays: the inverse direction . . . 58
5.3 Two dimensions: rectangles by inclusion–exclusion . . . 59
5.4 Two pointers: the telescoping window . . . 61
5.5 Sliding windows with bookkeeping . . . 62
5.6 Chapter summary . . . 64
5.7 Common mistakes . . . 64
5.8 Additional exercises . . . 65
Notes . . . 65

6 Amortised structures: stacks, deques, sparse tables **67**
6.1 Amortised analysis: each element pays once . . . 67
6.2 The monotonic stack: nearest smaller everywhere . . . 67
6.3 The monotonic deque: sliding-window minimum . . . 70
6.4 Sparse tables: idempotent queries in $O(1)$ . . . 72
6.5 Square roots: the good-enough structure . . . 74
6.6 Chapter summary . . . 75
6.7 Common mistakes . . . 75
6.8 Additional exercises . . . 76
Notes . . . 77

7 Graphs: traversal and order **78**

7.1 Vocabulary . . . 78
7.2 Representations: adjacency lists only . . . 78
7.3 Depth-first search . . . 79
7.4 Breadth-first search . . . 79
7.5 Flood fill . . . 81
7.6 Bipartite checks . . . 83
7.7 Directed order: Kahn and the DAG . . . 84
7.8 Functional graphs: tail plus cycle . . . 85
7.9 Chapter summary . . . 87
7.10 Common mistakes . . . 87
7.11 Additional exercises . . . 88
Notes . . . 88

8 Shortest paths **90**

8.1 Relaxation . . . 90
8.2 BFS, revisited . . . 90
8.3 Bellman–Ford . . . 91
8.4 Dijkstra . . . 93
8.5 Floyd–Warshall . . . 95
8.6 State expansion . . . 97
8.7 Chapter summary . . . 99
8.8 Common mistakes . . . 99
8.9 Additional exercises . . . 100
Notes . . . 100

9 Disjoint sets and spanning trees **102**

9.1 The connectivity problem . . . 102
9.2 The forest representation . . . 102
9.3 The amortized bound . . . 103
9.4 The cut property . . . 104
9.5 Kruskal . . . 104
9.6 Prim . . . 105
9.7 Worked examples . . . 106
9.8 Chapter summary . . . 109

9.9 Common mistakes . . . 109
9.10 Additional exercises . . . 110
Notes . . . 110

10 Tree algorithms: subtrees, diameters, LCA 112
10.1 The shape of a tree . . . 112
10.2 Subtree aggregates . . . 112
10.3 Diameter: two DFS . . . 113
10.4 Rerooting . . . 114
10.5 The Euler tour . . . 115
10.6 Binary lifting and LCA . . . 115
10.7 Distance queries . . . 117
10.8 Chapter summary . . . 118
10.9 Common mistakes . . . 119
10.10Additional exercises . . . 119
Notes . . . 120

11 Dynamic programming I: the method and the classics 121
11.1 The same problem twice . . . 121
11.2 The DP recipe . . . 121
11.3 Coin problems . . . 121
11.4 Knapsack . . . 123
11.5 LIS in $O(n \log n)$ . . . 124
11.6 Grid DP . . . 125
11.7 Push vs pull . . . 126
11.8 Chapter summary . . . 126
11.9 Common mistakes . . . 126
11.10Additional exercises . . . 127
Notes . . . 128

12 Dynamic programming II: masks, digits, ranges, and trees 129
12.1 Subsets as states . . . 129
12.2 Digit DP . . . 130
12.3 Range DP . . . 130
12.4 DP on trees . . . 132

12.5 The optimisation horizon . . . 133
12.6 Chapter summary . . . 135
12.7 Common mistakes . . . 135
12.8 Additional exercises . . . 135
Notes . . . 136

13 Fenwick and segment trees **138**
13.1 The range-query family . . . 138
13.2 The Fenwick tree . . . 138
13.3 The segment tree . . . 139
13.4 Lazy propagation . . . 140
13.5 Walking the tree . . . 141
13.6 Order statistics . . . 143
13.7 Chapter summary . . . 144
13.8 Common mistakes . . . 144
13.9 Additional exercises . . . 144
Notes . . . 145

14 Number theory and combinatorics **147**
14.1 Divisibility and gcd . . . 147
14.2 Primes . . . 148
14.3 Modular arithmetic . . . 149
14.4 Binary exponentiation . . . 149
14.5 The `modint` struct . . . 149
14.6 Counting . . . 150
14.7 Linear recurrences . . . 152
14.8 Chapter summary . . . 153
14.9 Common mistakes . . . 153
14.10Additional exercises . . . 154
Notes . . . 155

15 Strings: hashing, KMP, Z **156**
15.1 Terminology . . . 156
15.2 Polynomial rolling hash . . . 156
15.3 The prefix function (KMP) . . . 159

15.4 The Z-function . . . 160
15.5 Tries . . . 160
15.6 Applications . . . 162
15.7 Common mistakes . . . 166
15.8 Additional exercises . . . 166
Notes . . . 167

16 Geometry basics **168**
16.1 The integer-only rule . . . 168
16.2 Points, vectors, orientation . . . 168
16.3 Segment intersection . . . 170
16.4 Convex hull . . . 171
16.5 Polygon measures: area, lattice points, containment . . . 173
16.6 Distances . . . 175
16.7 Common mistakes . . . 177
Notes . . . 178

17 Probability basics **180**
17.1 Finite probability: counting in disguise . . . 180
17.2 Expectation and linearity . . . 182
17.3 Conditioning, independence, Bayes . . . 184
17.4 Distributions as dynamic programs . . . 185
17.5 Exact answers vs. simulation . . . 187
17.6 Common mistakes . . . 188
Notes . . . 190

18 The contest playbook **192**
18.1 Before the contest . . . 192
18.2 Reading the scoreboard . . . 193
18.3 Time allocation and switching . . . 194
18.4 Stress testing . . . 195
18.5 Upsolving . . . 197
18.6 The practice ladder and contest styles . . . 198
18.7 Common mistakes . . . 199
Notes . . . 201

CHAPTER 1

The contest machine and the cost model

Competitive programming is a game played inside a strict machine: a judge compiles your program, runs it on hidden tests, and returns a one-word verdict in seconds. This chapter teaches you how that machine works and how to think inside it. By the end you will have a starter file you trust, a working sense of how fast a given algorithm is before you write it, and a routine for going from a problem statement to an accepted submission without flailing. Everything else in the book assumes this chapter's machinery.

1.1 How a judge sees your program

Forget the IDE, the debugger, the terminal prompt. To the judge your program is a *function*: it reads characters from standard input (`stdin`) and writes characters to standard output (`stdout`). No files, no network, no graphics, no prompts. A program that prints `please enter n:` gets marked wrong, because the judge compares your output byte-for-byte with the expected output.

A submission is run once per *test file*. Each test file is one complete input; your program must print the correct answer for it within the *time limit* (typically 1–2 seconds) and the *memory limit* (typically 256–512 MB). The tests are hidden from you. They are also written by people whose job is to break plausible-but-wrong programs, so "it worked on my three tests" means nothing.

After all runs, the judge reports one *verdict* per submission, sometimes per test:

Verdict	Meaning
Accepted (AC)	every test file passed
Wrong Answer (WA)	your output differed from the expected output
Time Limit Exceeded (TLE)	a run did not finish inside the time limit
Runtime Error (RE)	the program crashed (segfault, bad index, . . .)
Memory Limit Exceeded (MLE)	the program exceeded the memory budget
Compile Error (CE)	the judge could not build your code

Figure 1.1 shows the whole pipeline; Figure 1.2 annotates a typical statement so you can see where the machine's parameters are declared.

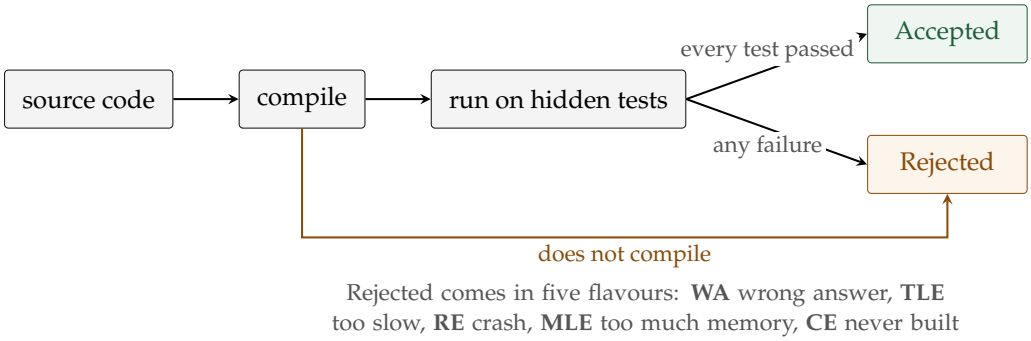


Figure 1.1: What the judge does with your code. Notice there is only one good outcome and five distinct ways to lose; each failure tells you where to look first. CE means the file did not build at all — fix that before anything else.

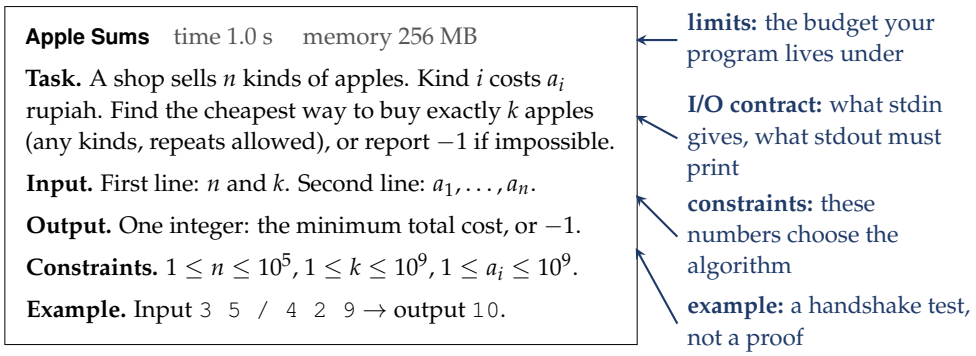


Figure 1.2: Anatomy of a problem statement (invented here, in the layout CSES and USACO use). Notice the constraints block: it is the most important paragraph on the page, and Section 1.4 teaches you to read it before reading anything else.

Scoring varies by contest. CSES and most archives are all-or-nothing: a submission is worth points only if every test passes. USACO gives partial credit per test file. IOI-style problems and some Codeforces subtasks give partial credit for easier constraint ranges — you can AC the small *subtasks* while the hard ones stay red. Always read which model a problem uses; it changes which submissions are worth your time.

Exercise 1.1 (easy)**.** A friend submits three programs to the same problem. Predict each verdict: (a) the program prints `answer: 42` instead of `42`; (b) the program is correct but takes 30 seconds on the largest test; (c) the program declares `int a[1000000000];` inside `main` (a four-gigabyte stack array).

Example 1.1 (WE 1.1 — CSES *Weird Algorithm*: the full loop)**.** A problem asks: given an integer n up to 10^6, print the sequence where each step halves an even number and sends an odd number to $3n + 1$, ending at 1. Input `3` should print `3 10 5 16 8 4 2 1`.

Strategy. There is nothing to optimise: the statement *is* the algorithm, so simulate it directly. The only question is the integer type — intermediate values can exceed 10^6 even when the input does not. (For the record: starting from $n = 704\,511$, legal under the limit, the sequence peaks at 56 991 483 520.) A 32-bit `int` would overflow on a fraction of tests, so use 64 bits.

Solution.

```
#include <iostream>
using namespace std;
using ll = long long;
int main() {
   ll n;
   cin >> n;
   while (n != 1) {
      cout << n << ' ';
      n = (n % 2 == 0) ? n / 2 : 3 * n + 1;
   }
   cout << 1 << '\n';
}
```

Run on the sample:

```
$ echo 3 | ./weird
3 10 5 16 8 4 2 1
```

It matches, so we also check the edge case $n = 1$ (prints `1` immediately) before submitting. The submission returns AC. The real workflow here is what matters: read, simulate, test on the sample, test one edge case, submit. You will repeat it hundreds of times.

1.2 The C++17 starter kit

Every solution in this book is C++17. Listing 1.1 is the starter file; it lives in `code/lib/` and every later chapter copies it.

```
// template.cpp -- the starter file for every solution in this book.
// Compile: clang++ -std=c++17 -O2 -Wall -Wextra -o sol template.cpp
#include <algorithm> // sort, min, max, ...
#include <iostream> // cin, cout
#include <vector> // vector
using namespace std;

using ll = long long; // 64-bit signed integers
#define rep(i, a, b) for (int i = (a); i < (b); ++i)
#define all(x) begin(x), end(x)

int main() {
   ios::sync_with_stdio(false); // untie cin/cout from stdio
   cin.tie(nullptr);

   // --- read input, compute, print answer ---
   return 0;
}
```

Listing 1.1: `code/lib/template.cpp`: the starter file. Minimal on purpose — every line earns its place.

Three choices in it deserve a defence. First, the includes are explicit. You will see `#include <bits/stdc++.h>` in other people's code; judges accept it, but it is a GCC-specific catch-all, it hides which facilities you actually use, and it fails on compilers that do not ship it. We name the four headers this chapter needs and add others as the book demands them.

Second, `using ll = long long`. The default integer type for counting should be 64-bit: `int` holds only about $\pm 2.1 \cdot 10^9$, while `ll` holds about $\pm 9.2 \cdot 10^{18}$. The price is a few characters and roughly twice the memory per number — cheap insurance against the single most common WA cause in the archive.

Third, the two macros. `rep(i,a,b)` is a counted loop; `all(x)` expands to `begin(x), end(x)` for calls like `sort(all(v))`. We keep the macro count at two because macros are invisible at read time — a template that does magic for you will do magic *to* you in a later chapter's debugging session.

Definition 1.2 (Integer overflow)**.** *Overflow* is arithmetic leaving the range of its type. In C++ signed overflow is *undefined behaviour*: the standard allows anything. In practice the value usually wraps to garbage ($2\,147\,483\,647 + 1$ becomes $-2\,147\,483\,648$), but "usually" is the point — the same code can misbehave differently across compilers and flags.

The rules of thumb: any product of two inputs near 10^5 or larger, any sum of up to 10^5 terms near 10^9, and any expression like `n*(n+1)/2` with n above 46 000 exceeds `int`. When in doubt, `ll`.

THE OVERFLOW TRIPWIRE

If a program passes the samples but fails the judge's big tests, check the types of every accumulator and every product *before* anything else. Worked Example 1.4 is this exact failure.

```
#include <bits/stdc++.h>                    ── headers
using namespace std;


void solve() {                              ── solve()
  // the actual logic
}

int main() {
  ios::sync_with_stdio(false);              ── fast I/O
  cin.tie(nullptr);
  solve();
}
```

one file, three parts: every solution in this book

Figure 1.3: The solution file, dissected: headers, `solve()`, fast I/O — every listing in this book follows it.

1.2.1 Fast input and output

By default, C++ streams are synchronised with the C stdio library — a convenience you pay for on every `cin`. Two lines in the template turn that off: `ios::sync_with_stdio(false)` detaches the streams, and `cin.tie(nullptr)` stops `cin` from flushing `cout` before each read. Together they make stream I/O fast enough for almost every problem in this book.

When input gets large — a million tokens or more — even that can be the bottleneck. Listing 1.2 compares the two levels on a file of one million integers: plain `cin` with sync disabled versus a small scanner that reads the raw `stdin` buffer itself (`code/lib/fastscan.h`, part of the capstone library).

```
// fastscan.cpp -- Listing 1.2: cin+sync off vs raw-buffer scanner, timed.
// Reads N integers from stdin and prints a checksum plus elapsed time.

// Build: clang++ -std=c++17 -O2 -o fastscan fastscan.cpp
#include <chrono>
#include <cstdio>
#include <iostream>
#include "lib/fastscan.h"
using namespace std;

int main(int argc, char** argv) {
   bool use_fast = argc > 1 && argv[1][0] == 'f';
   auto t0 = chrono::steady_clock::now();
   long long sum = 0;
   if (!use_fast) {
      ios::sync_with_stdio(false);
      cin.tie(nullptr);
      int n; cin >> n;
      for (int i = 0; i < n; ++i) { long long x; cin >> x; sum += x; }
   } else {
      FastScanner fs;
      int n = fs.nextInt();
      for (int i = 0; i < n; ++i) sum += fs.nextLongLong();
   }
   double ms = chrono::duration<double, milli>(
      chrono::steady_clock::now() - t0).count();
   printf("method=%s sum=%lld time_ms=%.1f\n",
        use_fast ? "fread" : "cin", sum, ms);
}
```

Listing 1.2: `code/fastscan.cpp`: two ways to read a million integers, timed.

Checkpoint 1.1 *Predict before you run.* Predict the ratio of the two timings: a factor of 2, of 10, or of 100? Then look at the measured run below.

```
$ ./fastscan < ints1e6.txt
method=cin sum=499884612927859 time_ms=919.0
$ ./fastscan f < ints1e6.txt
method=fread sum=499884612927859 time_ms=6.6
```

On this machine the buffer scanner is about 140× faster — nine hundred milliseconds spent purely on parsing. Take the lesson as a range, not a law: the ratio varies by machine and judge, but the direction never does. Use the template's two lines always; reach for `fastscan.h` when input is the bottleneck.

1.2.2 The STL you actually use

You can solve the first hundred problems of this book with five library tools. `vector` is the only array you need: contiguous, resizable, O(1) index access. `sort` orders a range in $O(n \log_2 n)$ and accepts a *comparator* — a function returning true when the first argument belongs before the second. `set` keeps distinct elements sorted (insert, erase, find all $O(\log_2 n)$); `map` is the same idea keyed by a value, which makes it a frequency table; `priority_queue` keeps the current maximum on top in $O(\log_2 n)$ per operation. Strings are `std::string`; small records are `pair` or `tuple`, unpacked by a *structured binding* (`auto& [k, v] : m`). Listing 1.3 exercises all of them.

```
// stl_tour.cpp -- Listing 1.3: the four STL tools used most, with output.
#include <algorithm>
#include <iostream>
#include <map>
#include <queue>
#include <set>
#include <vector>
using namespace std;

int main() {
   vector<int> v = {5, 2, 8, 1, 4};
   sort(v.begin(), v.end(), [](int a, int b) { // custom comparator:
      if ((a & 1) != (b & 1)) return (a & 1) < (b & 1); // even first,
      return a < b; // then ascending
   });
   cout << "sorted:";
   for (int x : v) cout << ' ' << x;
   cout << '\n';

   set<int> s = {4, 1, 4, 2, 1}; // dedupe + order
   cout << "set:";
   for (int x : s) cout << ' ' << x;
   cout << '\n';

   map<string, int> freq; // frequency table
   for (string w : {"b", "a", "b", "c", "a", "b"}) freq[w]++;
   for (auto& [word, count] : freq)
      cout << word << '=' << count << ' ';
   cout << '\n';

   priority_queue<int> pq; // largest on top
   for (int x : {3, 9, 4, 7}) pq.push(x);
   cout << "pop order:";
   while (!pq.empty()) { cout << ' ' << pq.top(); pq.pop(); }
```

```
35     cout << '\n';
36 }
```

Listing 1.3: `code/stl_tour.cpp`: the four containers and a custom comparator, with real output.

```
sorted: 2 4 8 1 5
set: 1 2 4
a=2 b=3 c=1
pop order: 9 7 4 3
```

Notice the comparator's effect on the first line: evens sorted before odds, each group ascending. Notice too that `freq[w]++` needs no initialisation — `map` value-initialises missing entries to zero, which is why frequency tables are one line of code.

Exercise 1.2 (easy)**.** Predict the output of this program, then check yourself by running it:

```
1 int a = 7;
2 cout << a/2 << ' ' << a/2.0 << ' ' << (a*a)/2 << '\n';
```

What type does the middle expression evaluate in, and why do the first and last answers differ?

1.3 Time complexity

"Is this algorithm fast enough?" has to be answerable *before* you write code, or you will spend contest minutes implementing something that was doomed from the statement. The tool is asymptotic analysis: count how the work grows with the input size, not how many milliseconds it takes on one machine.

Definition 1.3 (RAM model)**.** We count *operations*: arithmetic, comparisons, assignments, array reads and writes each cost one unit. A function call is the sum of the work it does. This is a model, not physics — real machines have caches, branches and vector units — but it predicts the shape of runtimes far better than it predicts their absolute values.

Count the operations in a nested loop exactly once and you will see why the abstraction helps:

```
1 long long s = 0; // 1 op
2 for (int i = 0; i < n; ++i) // n iterations
3    for (int j = i + 1; j < n; ++j) // n-i-1 iterations
4       s += a[i] * a[j]; // 3 ops each pass
```

The innermost line runs $\sum_{i=0}^{n-1}(n-1-i) = n(n-1)/2$ times, so the total is $1 + 3n(n-1)/2$. For large n the n^2 term dominates: at $n = 10^5$ the count is about $1.5 \cdot 10^{10}$ — seconds to minutes, not the microseconds a linear algorithm needs. The lower-order terms and the constants stop mattering quickly, which is exactly what the notation formalises.

Definition 1.4 (O, Θ, Ω)**.** For functions $f, g : \mathbb{N} \to \mathbb{R}_{>0}$:

- $f \in O(g)$: there exist $c > 0$ and n_0 with $f(n) \le c\,g(n)$ for all $n \ge n_0$ — g is an *asymptotic upper bound*.
- $f \in \Omega(g)$: there exist $c > 0$ and n_0 with $f(n) \ge c\,g(n)$ for all $n \ge n_0$ — an asymptotic lower bound.
- $f \in \Theta(g)$: both hold — f and g have the same growth rate.

Writing $f = O(g)$ is common shorthand for $f \in O(g)$.

Theorem 1.5. $3n^2 + 10n + 7 \in \Theta(n^2)$.

Proof. Upper bound: for $n \ge 1$, $10n \le 10n^2$ and $7 \le 7n^2$, so $3n^2 + 10n + 7 \le 20n^2$; take $c = 20$, $n_0 = 1$ and $f \in O(n^2)$. Lower bound: for $n \ge 1$, $3n^2 + 10n + 7 \ge 3n^2$; take $c = 3$, $n_0 = 1$ and $f \in \Omega(n^2)$. Both directions hold, so $f \in \Theta(n^2)$. ■

The same squeeze works for any polynomial: the leading term is the whole story. Inside $O(\cdot)$ the base of a logarithm never matters either, since $\log_a n = \log_b n / \log_b a$ differs only by a constant; in this book complexity uses $\log_2$ to keep the concrete meaning — the number of halvings — and every other log is natural.

The classes that matter, from cheapest to most expensive:

Class	Name	Produced by
$O(1)$	constant	a formula, one array lookup
$O(\log_2 n)$	logarithmic	halving the input each step (binary search)
$O(\sqrt{n})$	square root	per-block passes in sqrt decomposition
$O(n)$	linear	one pass over the input
$O(n \log_2 n)$	log-linear	sorting; divide and conquer without blowup
$O(n^2)$	quadratic	all pairs; doubly nested loops
$O(n^3)$	cubic	all triples; some DP on small n
$O(2^n)$	exponential	enumerating all subsets
$O(n!)$	factorial	enumerating all permutations

Definition 1.6 (The 10^8 heuristic)**.** As a budgeting rule of thumb, one second of judge time buys roughly 10^8 *simple* operations — integer adds, compares, array touches. It is a heuristic, not a law: memory-bound passes can be slower, vectorised inner loops can be several times faster, and the number says nothing about a two-second or ten-second limit. Use it to compare candidate algorithms, not to certify one.

Exercise 1.3 (medium)**.** Count the operations in this fragment exactly, then give its Θ-class:

```
for (int i = 1; i <= n; ++i)
   for (int j = 1; j <= i; j += 2)
      work();
```

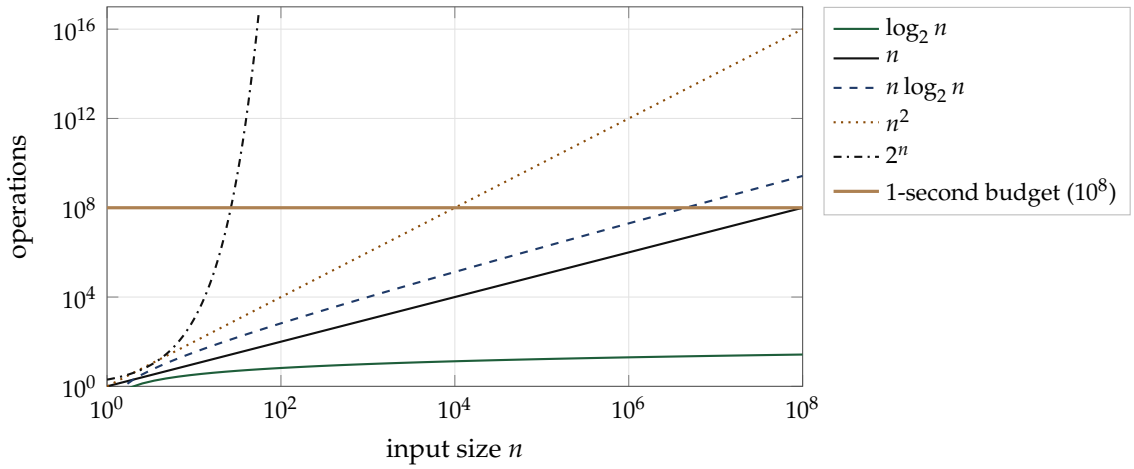


Figure 1.4: Operation counts on log–log axes against the one-second budget of 10^8 simple operations. Notice where each curve crosses the amber line: $\log_2 n$ never gets there, $n \log_2 n$ crosses near $n \approx 4 \cdot 10^6$, n^2 near 10^4, and 2^n already fails before $n = 30$. This single picture is the whole point of Section 1.4.

Hint: the inner trip count is $\lceil i/2 \rceil$.

1.4 From constraints to a plan

Statements hand you the input size n on a plate. The contest skill is converting n into the *largest* complexity class that survives the budget — Figure 1.5 is the table every experienced contestant has internalised — and then picking the simplest algorithm in that class.

largest n	**needs at most**	**typical shapes**
$n \le 12$	$O(n!)$	permutations, brute force
$n \le 25$	$O(2^n)$	subsets, meet-in-the-middle
$n \le 500$	$O(n^3)$	all triples, $n \times n$ DP
$n \le 5\,000$	$O(n^2)$	all pairs, quadratic DP
$n \le 2 \cdot 10^5$	$O(n \log_2 n)$	sorting, sets, Fenwick
$n \le 10^7$	$O(n)$	single linear pass
any n	$O(\log_2 n)$ or $O(1)$	binary search, closed formula

Assumes the 10^8-ops-per-second budget. Rows are rules of thumb, not laws: a $5 \cdot 10^7$-constant quadratic can still lose to a linear algorithm.

Figure 1.5: The constraint-to-complexity table. Notice it is a ceiling chart: $n \le 2 \cdot 10^5$ does not demand $O(n \log_2 n)$, it *forbids* anything slower. Rows are rules of thumb under the 10^8 heuristic; a $5 \cdot 10^7$-constant quadratic can still lose to a linear pass.

Two refinements before the table gets used. First, judge tests are built to stress the *worst case*: average-case reasoning is how quicksort dies on sorted input. Plan for the worst row in Figure's table, not the typical one. Second, *constant factors* are real — an $O(n \log_2 n)$ algorithm

whose inner step rebuilds a `set` can lose to a well-written $O(n)$ pass at the same n — but constants lose to growth: no constant rescues n^2 when $n = 10^6$.

CHOOSING THE ALGORITHM

Let C be the largest class that fits the constraint table for the given n. Among algorithms in C, implement the *simplest correct one you can explain* — the one with the fewest moving parts. If it TLEs, only then reach for the cleverer algorithm in the same class. Never climb the table first: a fancy $O(n \log_2 n)$ where $O(n^2)$ would have passed is wasted minutes.

Example 1.7 (WE 1.2 — CSES *Maximum Subarray Sum*, four ways)**.** Given an array of n integers (with negatives), find the largest sum of a contiguous subarray. On CSES, $n \le 2 \cdot 10^5$: the table says $O(n \log_2 n)$ or better.

Strategy. Same problem, four honest attempts, each removing one structural waste: (i) try every interval and resum — correct but cubic; (ii) keep the sum running while the right end moves — quadratic; (iii) split the array and combine — $O(n \log_2 n)$; (iv) notice that only the *best ending* matters at each step — linear.

Solution. The cubic version checks all $n(n+1)/2$ intervals and resums each:

```
for (int l = 0; l < n; ++l)
   for (int r = l; r < n; ++r) {
      ll s = 0;
      for (int k = l; k <= r; ++k) s += a[k];
      best = max(best, s);
   }
```

It is correct — it inspects every candidate — and it is the first submission most beginners write. The quadratic version keeps s running as r advances, deleting the inner loop. The divide-and-conquer version splits at the middle: the answer is the best of left half, right half, or an interval crossing the middle (extend left and right greedily from the split). The linear version is Kadane's:

```
ll best = a[0], cur = a[0];
for (int i = 1; i < n; ++i) {
   cur = max(a[i], cur + a[i]); // extend or restart
   best = max(best, cur);
}
```

Correctness, in one line: `cur` is the largest sum of any subarray *ending at* i — you either continue the previous best suffix or abandon it — and `best` remembers the largest of those. All four are in `code/mss_timing.cpp`; Listing 1.4 is the driver. Measured on this machine:

```
O(n^3) n=3000 ans=24382 time_ms=254.9
O(n^2) n=50000 ans=109297 time_ms=602.2
O(nlogn) n=200000 ans=218662 time_ms=1.8
O(n) n=200000 ans=218662 time_ms=0.1
O(n) n=5000000 ans=1565779 time_ms=3.4
```

Read the table backwards: the linear pass handles $n = 5 \cdot 10^6$ in less time than the cubic one needs for $n = 3000$ — and note the `ans` column is identical wherever both finished, which is the definition of a correct optimisation.

```
template <class F>
void run(const char* name, int n, F f) {
   vector<ll> a = make_input(n);
   auto t0 = chrono::steady_clock::now();
   ll ans = f(a);
   double ms = chrono::duration<double, milli>(
      chrono::steady_clock::now() - t0).count();
   printf("%-9s n=%-7d ans=%lld time_ms=%.1f\n", name, n, ans, ms);
}

int main() {
   run("O(n^3)", 3000, mss_cubic);
   run("O(n^2)", 50000, mss_quad);
   run("O(nlogn)", 200000,
      [](const vector<ll>& a) {
         return mss_dnc(a, 0, (int)a.size() - 1);
      });
   run("O(n)", 200000, mss_kadane);
   run("O(n)", 5000000, mss_kadane);
}
```

Listing 1.4: Listing 1.4: the benchmark driver (`code/mss_timing.cpp`, excerpt). Each algorithm is timed at the largest n it survives in about a second; the linear one is pushed to $n = 5 \cdot 10^6$ to show headroom.

We did not run the cubic version at $n = 2 \cdot 10^5$; extrapolating the measured scaling, it would take about a day. The quadratic at that n is roughly ten seconds. Both lose the contest to algorithm choice, not to coding skill — which is the lesson.

Exercise 1.4 (medium)**.** For each problem, name the largest complexity class the constraint allows under the 10^8 heuristic and the shape of an algorithm that fits: (a) $n \le 15$: count the subsets with a given property; (b) $n \le 3 \cdot 10^5$: find the maximum of a sliding window; (c) $n \le 2000$: longest common subsequence of two strings.

1.5 The submit loop

Acceptance is a loop, not an event. Figure 1.6 shows how a working five minutes splits on one problem; the dashed arrow is the part beginners get wrong — a WA sends you back to *thinking*, not to resubmitting.

The loop, expanded. *Compile like the judge*: the book's flags are `-std=c++17 -O2 -Wall -Wextra`; warnings are unheeded verdicts. *Test locally first*: run the samples, then one edge case you invented ($n = 1$, all-equal input, the largest legal n if generation is cheap). *Read a WA as a clue*, in this order:

- ☐ Did I handle the constraints' endpoints ($n = 1$, $n = 0$, maximum n)?

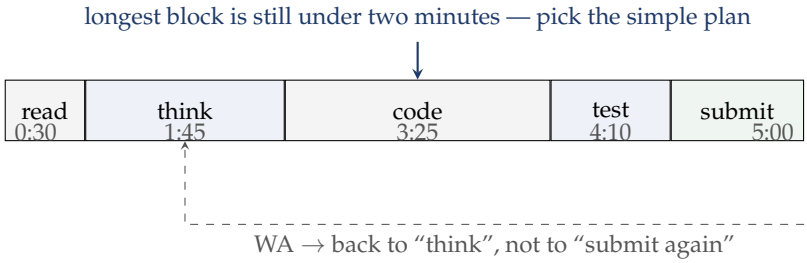


Figure 1.6: One problem, five minutes. Notice the blocks are small and the feedback edge goes to "think": a wrong answer is new information about your assumptions, not an invitation to submit again.

- ☐ Are all accumulators and products 64-bit? (The overflow tripwire.)
- ☐ Does my output match the format byte-for-byte — no extra words, spaces, or missing newline?
- ☐ Did I test anything the statement did not give me, or did I trust the sample?

And keep one thing straight: **Accepted means passed these tests, not correct**. Judges write strong test sets, but "AC" certifies nothing about inputs nobody tried. The deep fix — proving your candidate against a brute force on random small tests — is *stress testing*, and Chapter 18 makes it a habit; `code/test_ch01.sh` already runs 200 random rounds of the linear MSS code against the cubic one.

Example 1.8 (WE 1.3 — CSES *Missing Number*: two ways to be right)**.** Numbers 1 to n appear once each except one, which is missing. Given $n \le 2 \cdot 10^5$ and the $n-1$ present numbers, find the missing one.

Strategy. Two routes present themselves: a *formula* — the total should be $n(n+1)/2$, so subtract the given sum — and a *marking* pass — record which values appear and find the gap. Both are $O(n)$, both fit easily. Under time pressure choose the one with fewer ways to be wrong: the formula's product can overflow if types slip, while marking has no arithmetic at all beyond indexing. Marking is the safer submission; the formula is the faster insight. Knowing both lets you pick.

Solution. Formula (`code/missing_math.cpp`) and marking (`code/missing_set.cpp`); on the test of Section 1.5 both print `200000` for the crafted $n = 2 \cdot 10^5$ case. The point is not the answer — it is that two correct algorithms existed and the choice was a risk decision, not a knowledge gap.

Example 1.9 (WE 1.4 — the first bug hunt)**.** Here is a real Wrong Answer. The sum formula for *Missing Number* written with `int` throughout (`code/missing_int.cpp`) passes the sample happily — and prints this on the crafted large test:

```
$ ./missing_int < missing_big.txt
-2147283648
```

Negative two billion is not a number between 1 and n.

Strategy. Apply the checklist in order. Endpoints: n legal, fine. Types: `int total = n*(n+1)/2` overflows once the intermediate product $n(n+1) > 2^{31}$, i.e. already at $n \approx 46\,341$ — the judge's tests go to $2 \cdot 10^5$. That is the bug, found in one checklist step.

Solution. Change three `int`s to `ll` (`code/missing_math.cpp`); the same test now prints `200000`. The sample never had a chance to catch this: with $n = 4$, $n(n+1)/2 = 10$ fits in 16 bits, let alone 32. "Samples pass" and "types are right" are independent facts — only the checklist connects them.

Example 1.10 (WE 1.5 — CSES *Repetitions*: the scan is the algorithm)**.** Given a DNA string ($n \le 10^6$), find the longest run of equal characters. *Input:* `ATTCGGGA`. *Output:* `3`.

Strategy. The beginner's instinct reaches for a map of counts, but the question is about *runs*, not totals — T and G counts are irrelevant; only *consecutive* stretches matter. One left-to-right pass keeps two numbers: the current run length and the best seen. Compare each character to its predecessor; on a mismatch reset to 1, on a match increment. $O(n)$ time, $O(1)$ memory, no data structure at all. The lesson generalizes: when a problem says "longest/shortest *contiguous* something", a single scan with a running counter almost always beats a histogram.

Solution. `code/repetitions.cpp` does exactly that and prints `3` on the sample — the run is `GGG`. The whole algorithm is the loop you see; there is no cleverness to get wrong, which is why this is the right first archive submission after *Weird Algorithm*.

Exercise 1.5 (medium)**.** This program prints a million lines and gets TLE although it does almost no work per line. Find the one-token fix and say why it matters:

```
for (int i = 0; i < 1000000; ++i)
    cout << i << endl;
```

1.6 Common mistakes

- **`endl` in a loop.** `endl` writes a newline *and flushes* the stream; a flush per line turns microseconds into seconds. Use `'\n'`, and let `cin.tie(nullptr)` handle the rest.
- **`int` where the math needs 64 bits.** `n*(n+1)/2` overflows at $n \approx 46\,000$; products of two 10^5-scale inputs overflow always. When an accumulator can reach 10^9, it is `ll`. See WE 1.4.
- **Reading before understanding the format.** A common failure is slurping tokens before noticing the input is a grid, or that the first number is a *test count* and the loop is per-case. Sketch the parse on paper before coding.
- **Believing the sample proves the program.** A sample is a handshake test, not a proof: WE 1.4's bug was invisible at $n = 4$. The checklist exists precisely because samples cannot see overflows, edge cases, or the second half of a compound condition.

1.7 Chapter summary

Item	Rule of thumb	First check when wrong
Verdicts	AC = all tests; WA = wrong bytes; TLE = too slow; RE = crash	Read verdict before rereading code
Types	`int` $< 2.1 \cdot 10^9$; `ll` $< 9.2 \cdot 10^{18}$	Products and accumulators first
I/O	sync off + `tie(nullptr)` always; raw scanner past 10^6 tokens	`endl` → `'\n'`
Complexity	10^8 simple ops $\approx$ 1 s	count the innermost loop
Planning	$n \to$ largest class in Fig. 1.5 $\to$ simplest algorithm in it	worst case, never average
Submit loop	sample $\to$ edge case $\to$ submit $\to$ checklist	WA goes back to "think"

Table 1.1: Chapter 1 in one page. Keep this open during your first ten submissions; afterwards it is muscle memory.

1.8 Additional exercises

Named archive problems are cited by name only; look them up on `cses.fi` and `usaco.org`.

Exercise 1.6 (easy)**.** CSES *Increasing Array*: minimum increments to make the array non-decreasing. Trains a one-pass greedy.

Exercise 1.7 (easy)**.** CSES *Bit Strings*: count binary strings of length n, answer mod $10^9 + 7$. First taste of modular arithmetic.

Exercise 1.8 (easy)**.** CSES *Number Spiral*: value at (y, x) in the spiral. [derive a closed form — no loops needed]

Exercise 1.9 (easy)**.** CSES *Palindrome Reorder*: reorder into a palindrome or report impossible. Trains parity counting and a decision.

Exercise 1.10 (easy)**.** USACO-B *The Cow-Signal*: scale a grid by a factor. Trains I/O loops and indexing.

Exercise 1.11 (easy)**.** USACO-B *Speeding Ticket*: compare two piecewise-constant speeds over a road. Trains interval bookkeeping.

Exercise 1.12 (medium)**.** CSES *Trailing Zeros*: trailing zeros of $n!$. [derive the formula — then prove it]

Exercise 1.13 (medium)**.** CSES *Coin Piles*: can two piles be emptied by moves $(-2a, -a)$? [derive the test on paper first]

Exercise 1.14 (medium)**.** USACO-B *The Bucket List*: max cows milking at once from interval schedules. Trains sweep-line counting (preview of Chapter 5).

Exercise 1.15 (medium). USACO-B *The Bovine Shuffle*: undo a position permutation. Trains array-as-function thinking.

Exercise 1.16 (easy). CSES *Permutations* — construction, not search.

Exercise 1.17 (easy). CSES *Two Knights* — closed-form counting on the board.

Exercise 1.18 (medium). CSES *Two Sets* — split $\{1..n\}$ into equal-sum halves by residue classes.

Exercise 1.19 (medium). USACO-B *Square Pasture* — rectangle union via boundaries.

Exercise 1.20 (hard). CSES *Two Sets*: partition $1..n$ into two equal-sum sets or report impossible. [construct the split — constructive proof required]

Notes

The judge model and the submit-loop discipline follow the contest-craft chapters of Sannemo [1] and Laaksonen [2]; both books open with the same machine-first framing. The complexity treatment — RAM model, the squeeze proof for Θ, and the constraint table — parallels Laaksonen ch. 2 [2] and the USACO Guide's Bronze modules on time complexity and simulation [3]. The starter-file and fast-I/O conventions are the archive's de facto standard, documented at [6] and in the CSES problem set's own submission rules [4]; the `fastscan` routine is the standard contest idiom also catalogued at cp-algorithms [5]. Halim's text [7] covers the same ground with a different exercise mix — cited here, not consulted for content. All archive problems named in this chapter come from the CSES introductory set and the USACO-B archive; the rating-band mapping used throughout the book was computed from the public Codeforces problemset and is documented in `docs/sources.md`.

Further reading

- Laaksonen, *Competitive Programmer's Handbook*, ch. 1–2 (online edition, beginner) — the complexity convention and the contest-machine framing used in this chapter. https://cses.fi/book/book.pdf
- USACO Guide, "Introduction to Competitive Programming" (beginner) — the Bronze-on ramp this chapter assumes. https://usaco.guide/general/intro-cp
- USACO Guide, "Input and Output" (beginner) — alternative I/O recipes when `fastscan` is overkill. https://usaco.guide/general/input-output
- USACO Guide, "C++ With the Command Line" (beginner) — the compile/run loop every code listing in this book was produced with. https://usaco.guide/general/cpp-command
- CSES problem set (practice) — the archive whose samples anchor the worked examples here and in most later chapters. https://cses.fi/problemset/

- cp-algorithms (reference) — encyclopedia to consult as later chapters name their techniques.
  https://cp-algorithms.com
- cppreference, "Input/output library" (reference) — the language-level ground truth behind the I/O tricks of §1.2.1.
  https://en.cppreference.com/w/cpp/io

WATCH INSTEAD

Best first. "Lecture 1: Algorithmic Thinking, Peak Finding" — MIT OpenCourseWare, 53 min — https://youtu.be/HtSuA80QTyo. Covers this chapter's complexity framing (§1.3); OCW writes Θ/Ω formally where this book keeps to O — same meaning. "How to start Competitive Programming? For beginners!" — Errichto, 10 min — https://youtu.be/xAeiXy8-9Y8. Covers the submit-loop discipline of §1.5 and the contest minute; uses python in places, all advice transfers. Access date 2026-10-10.

CHAPTER 2

Complete search

The cheapest algorithm in the book: walk through every candidate answer and keep the ones that work. It sounds naive and it is — and for small enough search spaces it is also optimal, since the fastest correct program is the one you finish writing. This chapter teaches you the three big enumeration idioms — subsets, permutations, backtracking — and, more important, the habit of measuring the search space before committing to it. After this chapter you will be able to look at a constraint like $n \leq 20$, recognise it as an invitation rather than a warning, and enumerate correctly the first time.

2.1 The exhaustive idea

A *complete search* enumerates every element of the *search space* — the set of candidates the answer could be — and tests each one. Correctness is free: if the space really contains every candidate, the best survivor is the answer. The only question that matters is cost, and the cost has one formula:

$$\text{time} \;=\; (\text{size of search space}) \times (\text{work per candidate}).$$

Both factors are yours to shrink. The first is fixed by the problem, the second by your code; keep the per-candidate work small and the formula stays friendly surprisingly far up. How far? Figure 18.6 puts the three sizes you will meet most often next to the Chapter 1 budget.

The listing behind those measured points is `code/size_ladder.cpp`; its timing loop touches each subset once. Real output:

```
n=15 subsets=32768 time=0.007417 ms
n=20 subsets=1048576 time=0.235667 ms
n=24 subsets=16777216 time=3.76142 ms
n=27 subsets=134217728 time=35.3861 ms
n=29 subsets=536870912 time=143.94 ms
n=30 subsets=1073741824 time=285.851 ms
```

Read the doubling, not the absolute numbers: each extra element multiplies the work by exactly two, because the space itself doubles. That is the whole shape of exponential growth — Section 2.2 is about making that shape small enough to afford.

Remark (When $n \leq 20$ is a feature). Contest setters know Figure 18.6 too. A constraint like $n \leq 20$ is not a restriction on the input — it is a message: *the intended solution enumerates something*. When you see an unusually small bound, ask "what is the intended search space?"

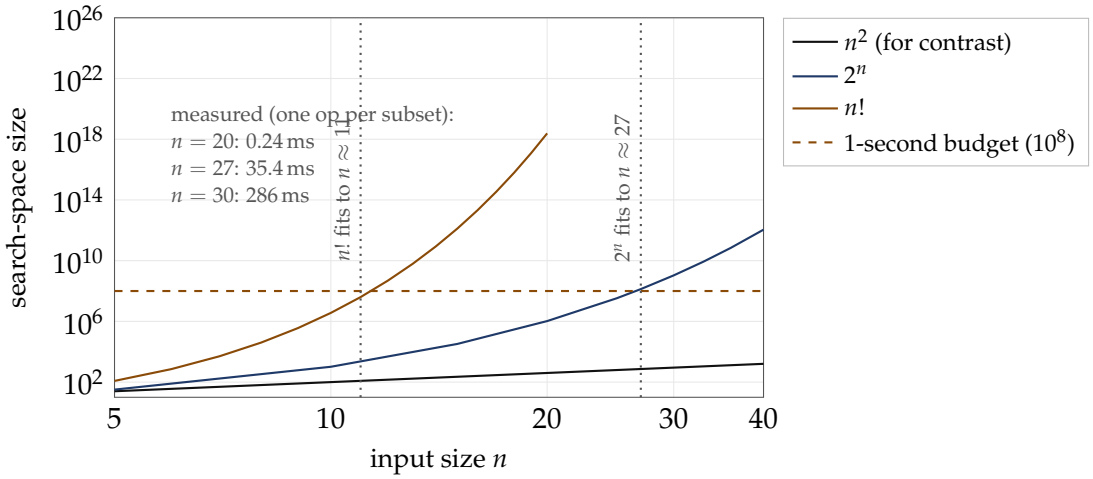


Figure 2.1: The size ladder. Notice where each curve crosses the amber budget line: n^2 is comfortable to $n \approx 10^4$, 2^n to $n \approx 27$, and $n!$ only to $n \approx 11$. The measured points are real: enumerating 2^{30} subsets with one cheap operation each took 286 ms on this machine — the shape of the curve, not your machine's speed, is the lesson.

before reaching for anything cleverer. The clever algorithm that takes an hour to find loses to the enumeration that takes ten minutes to write and a second to run.

Example 2.1 (WE 2.4 — USACO-B *Milk Pails*: bounds from the constraints)**.** Farmer John has two pails of sizes X and Y, which he may fill completely any number of times and pour into a third pail of capacity M ($M \le 1000$). What is the largest amount of milk he can measure into the third pail without overflowing it?

Strategy. What is the search space? Every outcome is "fill the X pail i times and the Y pail j times", giving $iX + jY$ units. The only question is how far i and j can run — and the constraint answers it: $iX + jY \le M \le 1000$, so $i \le M/X$ and $j \le M/Y$ — worst case a few hundred thousand pairs. That is a nested loop, not an algorithm: the constraint is small enough that the search space fits inside it twice over.

Solution. Two loops, both bounded by M itself so no legal combination is clipped:

```
// milk_pails.cpp -- USACO Bronze "Milk Pails" (February 2016).
// Pails of size X and Y can be filled any number of times and poured
// into a third pail of capacity M. Largest measurable amount <= M?
// Complete search over (i fills of X, j fills of Y): the bounds come
// straight from the constraint M <= 1000.
#include <iostream>
using namespace std;

int main() {
    int X, Y, M;
    cin >> X >> Y >> M;

    int best = 0;
    for (int i = 0; i * X <= M; i++) // i = fills of the X pail
        for (int j = 0; i * X + j * Y <= M; j++)
            best = max(best, i * X + j * Y);
```

```
    cout << best << "\n";
    return 0;
}
```

On the sample input `17 25 77`:

```
$ echo '17 25 77' | ./milk_pails
76
```

The inner loop's bound is the line that makes this correct rather than merely fast: `i * X + j * Y <= M` clips the search space to candidates that could still win, and `best` keeps the largest survivor. Notice what we did *not* do: no recursion, no pruning rule, no cleverness — the constraint gave the bound, the bound gave the loops.

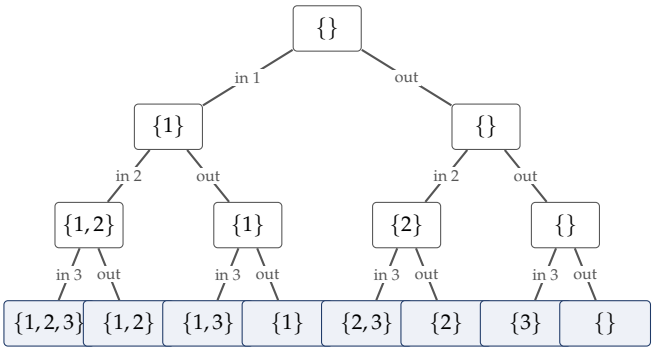


Figure 2.2: The subset tree: each level fixes one element, leaves are the 2^n subsets.

2.2 Enumerating subsets

The first enumeration shape you will use weekly: for a set of n elements, produce each of its 2^n *subsets*. Two idioms cover everything.

Recursion first. Walk the elements in order; at each one, branch into "take it" and "leave it". The walk is a binary tree of depth n — Figure 2.3 draws it for $n = 3$ — and every leaf is one subset.

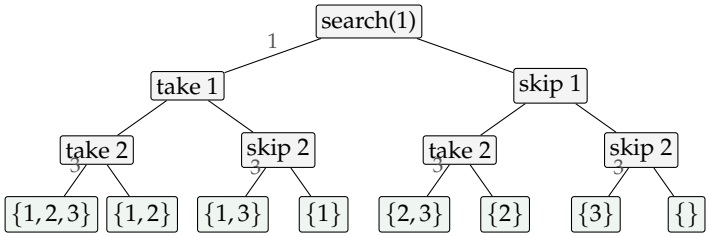


Figure 2.3: The recursion tree for subsets of $\{1, 2, 3\}$. Notice the shape: 2^n leaves at depth n, and every internal node is a decision. Drawing this tree before coding is Section 2.5's whole method.

```
// subsets_rec.cpp -- enumerate all subsets of {1,..,n} by recursion.
```

```
// Search tree: each level decides "take k" or "leave k".
#include <iostream>
#include <vector>
using namespace std;

int n;
vector<int> chosen;

void search(int k) {
   if (k == n + 1) {
      cout << "{";
      for (int i = 0; i < (int)chosen.size(); i++)
         cout << (i ? ", " : "") << chosen[i];
      cout << "}\n";
      return;
   }
   chosen.push_back(k); // take k
   search(k + 1);
   chosen.pop_back();
   search(k + 1); // leave k
}

int main() {
   n = 3;
   search(1);
   return 0;
}
```

Listing 2.1: `code/subsets_rec.cpp`: recursive take/leave. The skeleton of every backtracking program in this book.

Its real output — the eight subsets in take-first order:

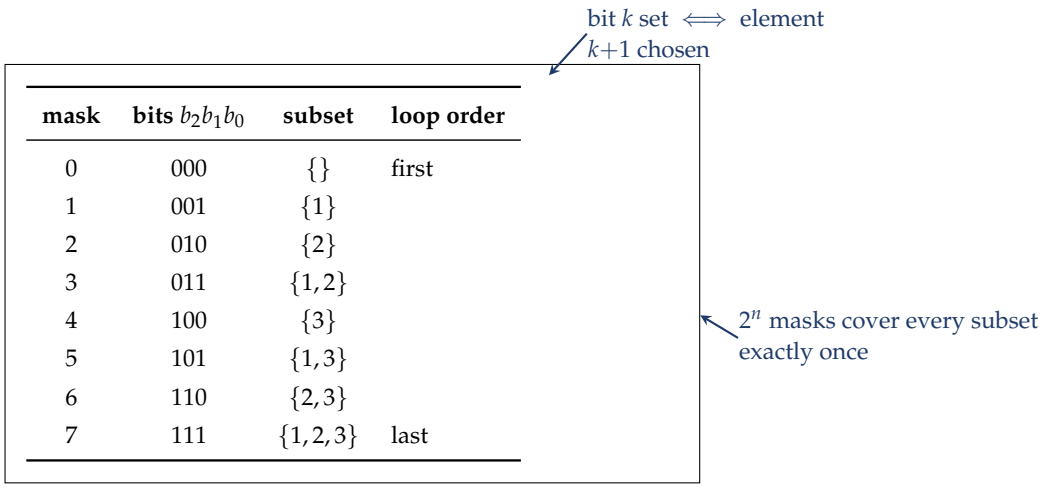
```
{1, 2, 3}
{1, 2}
{1, 3}
{1}
{2, 3}
{2}
{3}
{}
```

Then bitmasks. A subset of an n-element set is also a number: bit k of a mask says whether element k is in. Then "all subsets" is the loop `for mask in` $[0, 2^n)$ — the cleanest three lines in competitive programming, and Figure 2.4 shows the encoding.

```
// subsets_mask.cpp -- enumerate all subsets of {1,..,n} with bitmasks.

// mask bit k set <=> element k+1 is in the subset.
#include <iostream>
using namespace std;

int main() {
```



mask	bits $b_2b_1b_0$	subset	loop order
0	000	$\{\}$	first
1	001	$\{1\}$	
2	010	$\{2\}$	
3	011	$\{1,2\}$	
4	100	$\{3\}$	
5	101	$\{1,3\}$	
6	110	$\{2,3\}$	
7	111	$\{1,2,3\}$	last

Figure 2.4: Mask $\leftrightarrow$ subset for $n = 3$. Notice that the loop order is *numerical*, not the take-first order of recursion: mask 011 is $\{1,2\}$, not the next recursive leaf. If your output order matters, check which order your method produces.

```
7     int n = 3;
8     for (int mask = 0; mask < (1 << n); mask++) {
9        cout << "{";
10       bool first = true;
11       for (int k = 0; k < n; k++)
12          if (mask & (1 << k)) {
13             cout << (first ? "" : ", ") << k + 1;
14             first = false;
15          }
16       cout << "}\n";
17    }
18    return 0;
19 }
```

Listing 2.2: `code/subsets_mask.cpp`: the same eight subsets by bitmask. `mask & (1 << k)` is the membership test.

Real output — same subsets, different order:

```
{}
{1}
{2}
{1, 2}
{3}
{1, 3}
{2, 3}
{1, 2, 3}
```

Exercise 2.1 (easy). How many subsets does a 20-element set have? How many masks does the loop run for $n = 20$ — and for $n = 64$? What goes wrong in the code of Listing 2.2 for large n, and where exactly?

Example 2.2 (WE 2.1 — CSES *Apple Division*: the enumeration is the algorithm)**.** n apples weigh $w_1, \ldots, w_n$. Split them into two groups so the weight difference is as small as possible ($n \leq 20$).

Strategy. The answer is $\min |S_A - S_B|$ over partitions. Since $S_A + S_B$ is the fixed total T, this is $\min |T - 2S_A|$ — so we only need to look at one side: enumerate the subsets that could be group A and keep the best. The space is 2^n with $n \leq 20$: at most a million candidates, comfortably inside the budget by Figure 18.6. The constraint *is* the algorithm.

Solution.

```
// apple_division.cpp -- CSES Apple Division.
// Split n apples into two groups minimizing |sum A - sum B|.
// Bitmask enumeration: 2^(n-1) candidates (element 0 fixed in group A).
#include <algorithm>
#include <iostream>
#include <vector>
using namespace std;
using ll = long long;

int main() {
    int n;
    cin >> n;
    vector<ll> w(n);
    ll total = 0;
    for (ll& x : w) { cin >> x; total += x; }

    ll best = total; // |sumA - sumB| = |total - 2*sumA|
    for (int mask = 0; mask < (1 << (n - 1)); mask++) {
        ll sumA = w[0]; // element 0 fixed in A
        for (int k = 0; k < n - 1; k++)
            if (mask & (1 << k)) sumA += w[k + 1];
        best = min(best, llabs(total - 2 * sumA));
    }
    cout << best << "\n";
    return 0;
}
```

On the CSES sample (`5` then `3 2 7 4 1`, total 17 — best split puts $3 + 4 + 1 = 8$ in A, leaving $|17 - 16| = 1$):

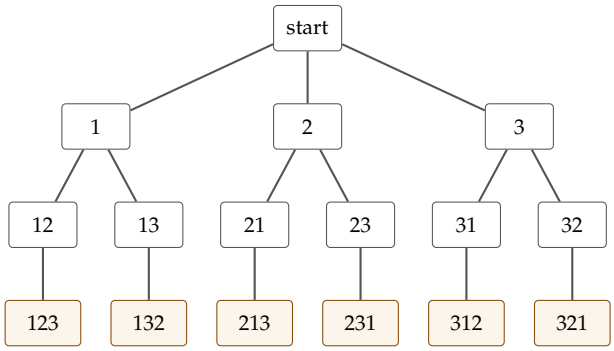
```
$ echo '5
3 2 7 4 1' | ./apple_division
1
```

Two details to copy. First, the loop runs to 2^{n-1} not 2^n: element 0 is pinned to group A, because swapping the two groups changes nothing — every answer appears twice in the free search, and the pin deletes the mirror half. Second, `best` starts at T (the trivial partition $A = \varnothing$), so even $n = 1$ works.

Remark (Is the symmetry real? Measure it.). Pinning element 0 halves the search space *if the objective is symmetric* — here $|S_A - S_B|$ does not change when groups swap, so the move is free. `code/apple_bench.cpp` times both versions on $n = 22$:

```
free: leaves=4194304 time=6.467 ms answer=1
fixed[0]: leaves=2097152 time=3.56679 ms answer=1
```

Same answer, half the leaves, roughly half the time. The speedup is modest because the search was already cheap — but the *technique* is the point: before exploiting a symmetry, verify the objective is actually invariant under it, or you have deleted half your answer space for nothing.



each level picks one of the unused elements: $3 \cdot 2 \cdot 1 = 3! = 6$ leaves (the last level is forced)

Figure 2.5: The permutation tree: each branch picks the next unused symbol; $n!$ leaves.

2.3 Enumerating permutations

Subsets cost 2^n; permutations cost $n!$ — a much harder wall (Figure 18.6 shows $n!$ crossing the budget near $n = 11$). When the constraint allows it, C++ does the walking for you: `next_permutation` advances a sequence to its next arrangement in *lexicographic order* — dictionary order — and returns `false` at the last one.

THE SORT CONTRACT

`next_permutation` only walks *upward*. Called on `"bca"`, it produces `cab`, `cba` — then stops, and you have silently missed four permutations. Sort first, always: from the smallest arrangement the walk covers all of them. This is the single most common submission bug in this chapter.

```
1 // perms_demo.cpp -- the next_permutation idiom, up close.
2 // Prints every distinct permutation of two words.
3 #include <algorithm>
4 #include <iostream>
5 #include <string>
6 using namespace std;
7
8 void all_perms(string s) {
```

```
    sort(s.begin(), s.end());
    int count = 0;
    do {
       cout << " " << s << "\n";
       count++;
    } while (next_permutation(s.begin(), s.end()));
    cout << "count: " << count << "\n";
}

int main() {
    cout << "abc:\n"; all_perms("cba"); // unsorted input is fine, we
        sort
    cout << "aab:\n"; all_perms("aab"); // duplicates: distinct only
    return 0;
}
```

Listing 2.3: `code/perms_demo.cpp`: the idiom up close. Notice `"aab"` yields three permutations, not six — duplicates make no duplicates.

```
abc:
  abc
  acb
  bac
  bca
  cab
  cba
count: 6
aab:
  aab
  aba
  baa
count: 3
```

The second half is worth staring at: repeated elements never produce repeated permutations, so the count is $n!/\prod(\text{multiplicity!})$ for free — `"aab"` gives $3!/2! = 3$ distinct strings with zero dedup code.

Example 2.3 (WE 2.2 — CSES *Creating Strings*: apply the idiom)**.** Given a string, print every distinct string that can be formed by rearranging its characters, alphabetically sorted ($n \leq 8$).

Strategy. $n \leq 8$ means at most $8! = 40\,320$ distinct strings — inside the budget with room to spare, and the problem's own output size is the real cost. Sort the input, then let `next_permutation` walk; the walk is already in alphabetical order, so printing as we go satisfies the output contract for free. Collect-or-print is a choice: we collect to learn the count first, since the answer's first line must be the total.

Solution.

```
// creating_strings.cpp -- CSES Creating Strings.
// All distinct permutations of a word, alphabetically sorted.
// next_permutation visits each distinct permutation of a multiset
    once,
// so duplicate letters need no special handling -- just sort first.
```

```cpp
#include <algorithm>
#include <iostream>
#include <string>
#include <vector>
using namespace std;

int main() {
   string s;
   cin >> s;
   sort(s.begin(), s.end()); // mandatory: start from smallest

   vector<string> out;
   do {
      out.push_back(s);
   } while (next_permutation(s.begin(), s.end()));

   cout << out.size() << "\n";
   for (auto& t : out) cout << t << "\n";
   return 0;
}
```

On `aabac` ($5!/(3! \cdot 1! \cdot 1!) = 20$ distinct strings), real output:

```
20
aaabc
aaacb
aabac
aabca
aacab
aacba
abaac
abaca
abcaa
acaab
acaba
acbaa
baaac
baaca
bacaa
bcaaa
caaab
caaba
cabaa
cbaaa
```

The lexicographic order you see is exactly the order the algorithm walks — *that's why* sorting first is the contract, not a nicety.

Exercise 2.2 (easy)**.** Write down the permutation `next_permutation` produces after `3142` — then after `4321`. What does each call return?

2.4 Backtracking

Recursive enumeration has a superpower the loop versions lack: it can see a dead end early and turn around. *Backtracking* is recursive search with *pruning* — before extending a partial solution, check whether it can still lead to a valid one, and skip the subtree entirely if not. The cost model is unchanged (space size $\times$ work per candidate); pruning attacks the first factor by shrinking the space itself.

A prune is only as good as its rule. Three requirements, in order of importance: the rule must be *sound* (never kills a branch that contains an answer — else the search is wrong, not fast); it should be *cheap* (a prune that costs more than the subtree it saves is a slowdown wearing a clever name); and ideally it is *measured* — which is why the code below counts.

Example 2.4 (WE 2.3 — CSES *Chessboard and Queens*: pruning with receipts)**.** Place eight queens on an 8×8 board with some squares reserved ($\star$), so that no two queens share a row, column, or diagonal. Count the valid placements.

Strategy. Place queens row by row — one per row is forced anyway, so the state at depth r is a partial placement in rows $0..r-1$. The blind version tries all 4^8-style assignments; the smart version checks three things before recursing: this column already used? this diagonal taken (two families, indexed $r+c$ and $r-c$)? this square reserved? Any "yes" and the branch dies instantly — a *prune*, and the program counts them so we can see the saving rather than guess at it.

Solution.

```
// queens.cpp -- CSES Chessboard and Queens, with an instrumented counter.
// 'leaves' = arrivals at depth 8. With all checks on, every leaf is a
// valid solution; with checks off it is just a surviving permutation.
// Place one queen per row; prune a branch as soon as a column, diagonal,
// or reserved square is violated. 'mode' toggles which checks are on so
// the counter can show what each pruning rule buys.
#include <iostream>
#include <string>
using namespace std;

const int N = 8;
string board[N];
bool col[N], d1[2 * N - 1], d2[2 * N - 1];
long long nodes = 0, pruned = 0, sols = 0;
bool useDiag = true, useCell = true;

void search(int r) {
   nodes++;
   if (r == N) { sols++; return; }
   for (int c = 0; c < N; c++) {
      bool bad = col[c]
         || (useDiag && (d1[r + c] || d2[r - c + N - 1]))
         || (useCell && board[r][c] == '*');
```

```
24        if (bad) { pruned++; continue; } // dead branch: counted, never
              explored
25        col[c] = d1[r + c] = d2[r - c + N - 1] = true;
26        search(r + 1);
27        col[c] = d1[r + c] = d2[r - c + N - 1] = false;
28     }
29  }
30
31  void run(const char* label) {
32     nodes = pruned = sols = 0;
33     for (int i = 0; i < N; i++) col[i] = false;
34     for (int i = 0; i < 2 * N - 1; i++) d1[i] = d2[i] = false;
35     search(0);
36     cout << label << " nodes=" << nodes
37          << " pruned=" << pruned << " leaves=" << sols << "\n";
38  }
39
40  int main(int argc, char** argv) {
41     for (int i = 0; i < N; i++) board[i] = ".......";
42     if (argc > 1 && string(argv[1]) == "stdin")
43        for (int i = 0; i < N; i++) cin >> board[i];
44
45     useDiag = true; useCell = true;
46     run("full checks (col+diag+cells): ");
47     useDiag = false; useCell = true;
48     run("columns only: ");
49     return 0;
50  }
```

Run on an empty board, then on the real CSES sample (four reserved squares), with the diagonal checks switched on and off for comparison:

```
full checks (col+diag+cells): nodes=2057 pruned=13664 leaves=92
columns only: nodes=109601 pruned=444648 leaves=40320

full checks (col+diag+cells): nodes=1607 pruned=10730 leaves=65
columns only: nodes=68069 pruned=288324 leaves=23520
```

Read the counter before the answer. With all checks on, 2057 nodes produce 92 leaves — and since only fully valid states reach depth 8, every leaf *is* a solution (CSES sample: 65). With columns alone, the search still walks all $8! = 40\,320$ leaf placements — $53\times$ more nodes visited for the same information. The diagonal rules did not make the algorithm cleverer; they deleted 98% of its work. That is what a pruning rule is for.

Example 2.5 (WE 2.5 — CSES *Gray Code*: construction as enumeration)**.** Print all 2^n bit strings of length n such that consecutive strings differ in exactly one bit — including the last versus the first. *Input:* `2`. *Output:* `00 01 11 10`.

Strategy. Backtracking works — walk the hypercube visiting every vertex — but the structure offers something cleaner: recursion *as* the construction. A length-n Gray code is the length-$(n-1)$ code written twice: first with a `0` prefix in the same order, then with a `1` prefix in

reverse order. The join is safe because the middle pair differ only in the new bit, and the wrap-around holds because the two ends were one bit apart already. This is the same "build the list, recurse on a smaller list" pattern as 2.3 — except here the recursion *is* the algorithm, not a search you prune.

Solution. `code/gray_code.cpp` runs the reflection and prints

```
$ echo 2 | ./gray_code
00
01
11
10
```

For n=16 it emits all 65 536 codes in well under a second — the output itself is the exponential, exactly the Section 2.1 trade: the list *is* the work.

Figure 2.6 draws the same story on the small board.

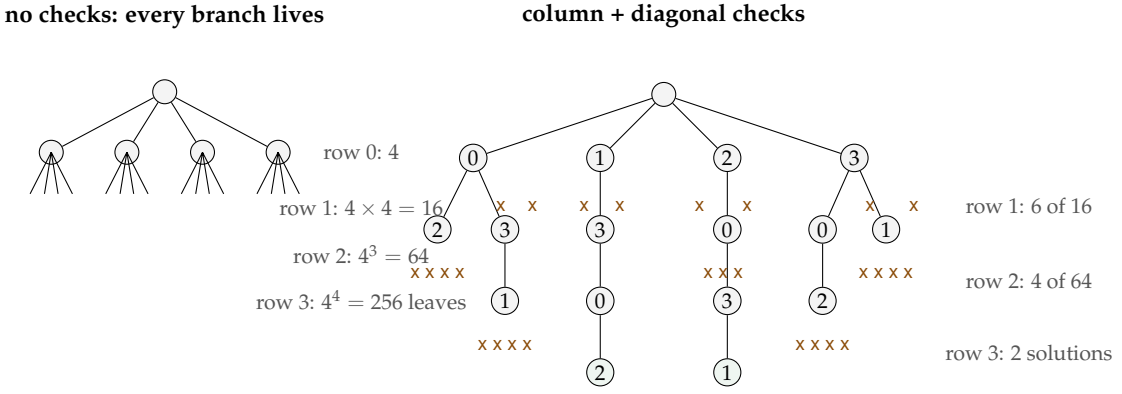


Figure 2.6: 4-queens, unpruned vs pruned (real shape, measured). Left: the blind tree fans out to $4^4 = 256$ leaves. Right: with column + diagonal checks, the row-1 level keeps 6 of 16 children, row 2 keeps 4 of 64, and exactly 2 solutions survive. Every x is a subtree that was never entered — the saving is the size of what is missing, not what is present.

Exercise 2.3 (medium)**.** In `code/queens.cpp`, the three prune rules live inside one boolean expression. Which exact line of `search()` fires when a candidate square shares a column with a placed queen? If you removed the `useDiag` term but kept `useCell`, would `leaves` still equal the answer? Why or why not?

2.5 Search-space sketches

Every problem in this chapter was planned the same way: draw the tree before coding it. The sketch costs a minute and answers three questions an IDE cannot: How big is the space? What does a node look like? Where can a check fire?

The method is mechanical. First, *name the state*: the minimal information a recursion call needs (for subsets, "depth k + what's been chosen"; for queens, "which columns are taken"). Second, *draw two levels of the tree* on a small instance ($n = 3$ or 4) — if the leaves are not

obviously the candidates, the state is wrong. Third, *count the leaves and the internal nodes*: leaves tell you the space size, internals tell you the real work, and their ratio tells you how much a per-node prune could save. Figure 2.3 and Figure 2.6 are exactly these sketches, left in the book on purpose — they are not illustrations of the code, they are the *design document* the code was written from.

Exercise 2.4 (medium)**.** Draw the full recursion tree for `subsets_rec.cpp` on $n = 4$, marking take/leave on every edge as in Figure 2.3. How many internal nodes and leaves does it have? Now mark the tree for the *fixed-element* version of WE 2.1's search: which half disappears, and which node does the pin live at?

2.6 Common mistakes

- **`1 << n` on the wrong side of 31.** `1` is an `int`; shifting it into bit 31 is undefined and bit 32+ is garbage. For $n \geq 31$ write `1LL << n` — and for $n \geq 64$ admit that a single mask cannot hold the subset at all (use `vector<bool>` or the recursive version, which never builds a mask).
- **`next_permutation` on unsorted input.** The walk goes upward only: from `"cba"` it ends immediately and you enumerate one permutation, silently. Sort first — then duplicates need no other handling.
- **Marking without unmarking.** Backtracking's bookkeeping is borrowed state: `col[c] = true` before the recursive call, `col[c] = false` after. Forget the second line and the next sibling inherits phantom conflicts — every subsequent branch dies for no reason and the count is quietly wrong. The mark/unmark pair is the price of the prune.
- **Halving on a non-symmetric objective.** The WE 2.1 pin worked because $|S_A - S_B|$ is unchanged by swapping the groups. If the objective distinguishes the sides — "first group's sum must not exceed the second's" — the same pin deletes exactly the answers you needed. Verify the invariance on paper before deleting half the tree.
- **Counting leaves as the cost.** The tree's work happens at *internal* nodes too: the subset tree of Figure 2.3 has 2^n leaves but $2^{n+1} - 1$ nodes total. When estimating a search, count nodes, not answers — and when claiming a prune helps, measure both counters as WE 2.3 does, because a rule that prunes the wrong branch is worse than none.

2.7 Chapter summary

2.8 Additional exercises

Named archive problems are cited by name only; look them up on `cses.fi` and `usaco.org`.

Exercise 2.5 (easy)**.** CSES *Tower of Hanoi*: print the optimal move sequence for n disks. [implement — this is a recursive walk that emits its own search tree]

Exercise 2.6 (easy)**.** USACO-B *Diamond Collector*: choose the two largest diamonds whose sizes differ by at most k. Trains bounded nested loops and the "sort then think" reflex.

Tool	Use when	Cost / first check when wrong
Nested loops	space is a small product of bounds ($\leq 10^6$ pairs)	bound clips a legal case: check $iX + jY \leq M$ style
Recursive subsets	you want take/leave order or per-node work	2^n leaves, $2^{n+1} - 1$ nodes; depth $= n$
Bitmask `for mask`	you want three lines and numerical order	`1 << n` overflows past $n = 30$
`next_permutation`	all $n!$ orders needed, $n \leq 11$	`sort` first or the walk is incomplete
Backtracking	dead branches are detectable early	prune must be sound — never kills an answer
Symmetry pin	objective invariant under some swap	verify invariance, then halve — measure it

Table 2.1: Chapter 2 in one page. The meta-rule sits above the table: estimate the space before you enumerate it — size times work per candidate decides everything.

Exercise 2.7 (easy)**.** USACO-B *Daisy Chains*: count plants whose height equals the average height of some contiguous interval. Nested loops over interval endpoints — the space is $O(n^2)$, the check is $O(n)$.

Exercise 2.8 (easy)**.** USACO-B *Counting Liars*: find the earliest true statement in a list of claims about a hidden position. Sort the claims and walk — complete search over n candidates.

Exercise 2.9 (medium)**.** CSES *Raab Game I*: construct a game history with given scores, or report impossible. Search meets construction.

Exercise 2.10 (medium)**.** CSES *Mex Grid Construction*: fill an $n \times n$ grid so each cell's mex condition holds. Enumeration under a local constraint — a backtracking-shaped construction.

Exercise 2.11 (medium)**.** USACO-B *Cow Gymnastics*: from K practice rankings, count pairs of cows whose order never changes. Triple nested loops bounded by the constraints.

Exercise 2.12 (medium)**.** USACO-B *Triangles*: sum the areas of all axis-aligned right triangles in a point set. Enumeration over $\binom{n}{2}$ or $\binom{n}{3}$ — choose which.

Exercise 2.13 (easy)**.** CSES *Two Sets* — partition by residue; when does a solution exist?

Exercise 2.14 (easy)**.** USACO-B *Livestock Lineup* — permutations filtered by constraints.

Exercise 2.15 (medium)**.** CSES *Grid Paths* — the 7×7 walk; your pruning from §2 pays here.

Exercise 2.16 (medium)**.** USACO-B *Cow Gymnastics* — all-pairs consistency check over rankings.

Exercise 2.17 (hard)**.** CSES *Digit Queries*: find the k-th digit of the concatenated sequence $1, 2, 3, \ldots$ — the search space is far past enumeration, so derive the block containing k instead. [derive — a lesson in when *not* to enumerate]

Exercise 2.18 (hard)**.** CSES *Grid Path Description*: count paths visiting every square of a 7×7 grid along a given pattern. Pure backtracking — and it only fits inside the budget once you add the pruning rules WE 2.3 taught. [implement]

Notes

The complete-search chapter of Laaksonen's handbook [2] covers the same three idioms in the same order — generating subsets both ways, permutations, then backtracking with pruning — and this chapter's worked examples are drawn from the CSES introductory set it accompanies [4]. Sannemo's treatment adds the "search space size times per-candidate work" cost model used in Section 2.1 and the "small constraint means intended enumeration" reading of Remark 2.1 [1]. The USACO Guide's Bronze modules on complete search and recursion supplied the nested-loop framing of WE 2.4 and several of the additional-exercise choices [3]; the backtracking instrumentation (count the nodes, count the prunes) follows the measure-first debugging style of cp-algorithms' backtracking articles [5]. All timings are real runs on this machine (Apple clang, `-O2`) and will differ modestly elsewhere — the ratios, not the numbers, carry the argument. Halim's text [7] organises the same material around UVa problem families — cited here, not consulted for content.

Exercise 2.19 (hard)**.** Extend the queens solver to $n = 14$: report your runtime and the first pruning that halves it.

Further reading

- Laaksonen, *Competitive Programmer's Handbook*, ch. 5 "Complete search" (beginner) — subset/permutation generation and pruning, the backbone of this chapter.
  https://cses.fi/book/book.pdf
- USACO Guide, "Complete Search" (beginner) — the enumerative half of the chapter in more exercises.
  https://usaco.guide/bronze/intro-complete
- USACO Guide, "Complete Search with Recursion" (beginner) — the recursive half, including the same subset/queen idioms.
  https://usaco.guide/bronze/complete-rec
- USACO Guide, "Introduction to Greedy Algorithms" (beginner) — when enumeration is too big: the Ch 3 bridge.
  https://usaco.guide/bronze/intro-greedy
- CSES 1623 *Apple Division* (practice) — the canonical subset-sum search used in 2.2 and a good first archive submission.
  https://cses.fi/problemset/task/1623
- cp-algorithms, "Generating all k-combinations" (reference). (article: "Generating all k-combinations) — bitmask iteration idioms beyond the two shown here.
  https://cp-algorithms.com/

- Wikipedia, “Backtracking” (reference) — the general vocabulary (state space tree, constraint propagation) this chapter specializes.
  https://en.wikipedia.org/wiki/Backtracking

WATCH INSTEAD

Best first. “6.2 Sum Of Subsets Problem - Backtracking” — Abdul Bari, 12 min — https://youtu.be/kyLxTdsT8ws. The state-space tree and prune-on-dead-end of §2.1, worked on the same problem shape as 2.2; hand-drawn trees, code-agnostic. “The N Queens Problem using Backtracking/Recursion - Explained” — Back To Back SWE, 14 min — https://youtu.be/wGbuCyNpxIg. Covers §2.4 end-to-end (place/backtrack on the diagonal constraints); Java-flavoured code, same recursion shape as the book’s C++. Access date 2026-10-10.

CHAPTER 3

Sorting and greedy algorithms

Half the problems at this level are won by two moves you already own: put the input in a useful order, then take whatever looks best next and never look back. The first move is `std::sort`; the second is a *greedy* algorithm. Sorting is safe — it always produces a sorted array. Greedy is *not* safe: it is a claim about the problem, and the claim needs a proof. This chapter teaches the tools (comparators, coordinate compression, `set` as a "best available" structure), the two standard proofs (the exchange argument and "greedy stays ahead"), and the discipline of hunting counter-examples before trusting a greedy rule. After it, you will sort fluently, compress fluently, and — more importantly — know which greedy ideas deserve your trust.

3.1 Sorting as a tool

`std::sort` runs in $O(n \log n)$ worst case (introsort: quicksort with a heapsort fallback) and is *unstable*: equal elements may reorder. When equal-order matters, `stable_sort` preserves it — a detail that costs a contest point exactly once, the time you forget it. A comparator must define a *strict weak ordering*: irreflexive, asymmetric, transitive. The classic violation is <= instead of <: equal elements then compare "less" than each other, the invariant breaks, and `sort` can read out of bounds — real undefined behaviour, not just a wrong answer.

Listing 3.1: `code/sort_cmp.cpp`: lambda comparator plus structured bindings. The comparator asks one question — "does *a* come before *b*?" — never "are they equal?"

```
// sort_cmp.cpp -- sorting with a lambda comparator + structured bindings.
#include <algorithm>
#include <iostream>
#include <string>
#include <vector>
using namespace std;

int main() {
   vector<pair<string,int>> team = {
      {"anni", 7}, {"bella", 9}, {"cato", 9}, {"dele", 4}
   };
   // by score descending, then name ascending on ties
   sort(team.begin(), team.end(),
       [](const auto &a, const auto &b) {
          return a.second != b.second ? a.second > b.second
```

```
                                  : a.first < b.first;
        });
    for (auto &[name, score] : team) // structured bindings
        cout << name << " " << score << "\n";
}
```

Its real output — note the tie broken by name:

```
bella 9
cato 9
anni 7
dele 4
```

The second tool is *coordinate compression*: when only the *order* of values matters, replace each by its rank among the distinct values. Values up to $4 \cdot 10^9$ become ranks $0 \dots k-1$ that index a Fenwick tree or a small array — essential from Chapter 12 on.

Listing 3.2: `code/compress_demo.cpp` using `code/lib/compress.h`. Ranks preserve every comparison ($<$, $=$, $>$); nothing else survives, by design.

```
// compress_demo.cpp -- coordinate compression with lib/compress.h.
#include "compress.h"
#include <iostream>
using namespace std;

int main() {
    vector<long long> v = {4000000000LL, 10, 10, 80, 4000000000LL};
    vector<long long> vals;
    vector<int> r = compress(v, vals);
    for (size_t i = 0; i < v.size(); i++)
        cout << v[i] << " -> " << r[i] << "\n";
    cout << "distinct values: " << vals.size() << "\n";
}
```

```
4000000000 -> 2
10 -> 0
10 -> 0
80 -> 1
4000000000 -> 2
distinct values: 3
```

Figure 9.1 draws the same run.

Exercise 3.1 (easy). Sort $\{(3,\mathrm{a}),(1,\mathrm{c}),(3,\mathrm{b})\}$ by first element only. Which of the two possible outputs can `sort` produce, and which does `stable_sort` guarantee?

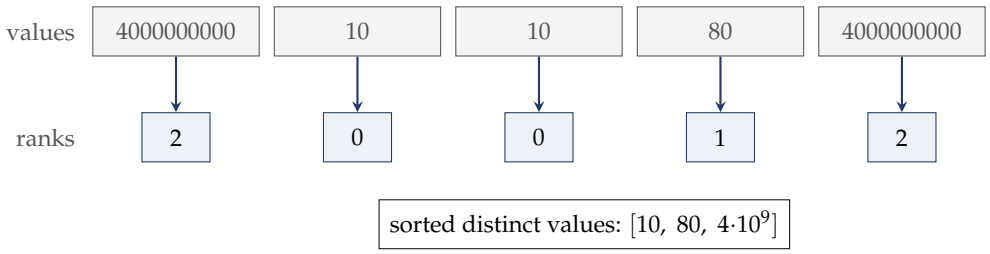


Figure 3.1: Values → ranks. Notice what is thrown away: magnitudes, gaps, units. If your algorithm ever reads a difference of two values, compression is the wrong tool.

3.2 The greedy method

Definition 3.1 (Greedy algorithm)**.** A *greedy algorithm* builds a solution in steps, at each step taking the locally best choice under some fixed rule — and never revisiting the choice. Correctness is not part of the definition.

Greedy is fast to write and fast to run — that is the appeal. The cost is outsourced to a proof: "locally best is globally right" is a theorem about the problem, not about your code. Chapter 2's complete search was the pessimist's answer (check everything); greedy is the optimist's answer (commit at every step). The skill is knowing which optimism is justified.

Example 3.2 (CSES *Ferris Wheel*)**.** Each gondola carries at most two children of total weight $\leq x$. Minimise gondolas. *Strategy.* The heaviest child must ride somewhere; the only question is whether anyone can join. If the lightest does not fit with them, nobody does — so pair heaviest with lightest whenever possible, else heaviest rides alone.

```
// ferris_wheel.cpp -- pair lightest + heaviest, else heaviest rides
   alone.
#include <algorithm>
#include <iostream>
#include <vector>
using namespace std;

int main() {
   int n;
   long long x;
   cin >> n >> x;
   vector<long long> p(n);
   for (auto &w : p) cin >> w;
   sort(p.begin(), p.end());
   int i = 0, j = n - 1, g = 0;
   while (i <= j) {
      if (p[i] + p[j] <= x) i++; // lightest fits with heaviest
      j--; // heaviest always boards now
      g++;
   }
   cout << g << "\n";
}
```

Real output on the CSES sample (`4 10 / 7 2 3 9`):

```
3
```

Why does pairing heaviest with lightest work? If the lightest fits, using any heavier partner instead cannot do better — the lightest is *at least* as hard to place as whoever replaces it. That is the "stays ahead" instinct of Section 3.3.

3.3 How to argue correctness

Two proofs cover most contest greedies:

- **Exchange argument.** Take an optimal solution O and the greedy solution G. Find the first place they differ, and show O can be *exchanged* into G's choice without making things worse. Repeating morphs O into something no worse that ends with G's choices — so greedy is optimal too.
- **Greedy stays ahead.** Maintain an invariant like "after k steps, greedy's partial solution is at least as good as any other partial solution", proved by induction on k.

Example 3.3 (CSES *Movie Festival* — the exchange argument)**.** Choose a maximum set of pairwise non-overlapping movies (a_i, b_i). *Strategy.* Sort by *ending* time and always take the movie that frees you earliest. Figure 3.3 is the proof in one picture.

```cpp
// movie_festival.cpp -- interval scheduling by earliest end time.
// argv[1] = "start" switches to the (wrong) earliest-start greedy.
#include <algorithm>
#include <cstring>
#include <iostream>
#include <vector>
using namespace std;

int main(int argc, char **argv) {
   bool by_start = argc > 1 && !strcmp(argv[1], "start");
   int n;
   cin >> n;
   vector<pair<int,int>> m(n);
   for (auto &[a, b] : m) cin >> a >> b;
   sort(m.begin(), m.end(), [by_start](const auto &x, const auto &y) {
      return by_start ? x.first < y.first : x.second < y.second;
   });
   int taken = 0, free_at = 0;
   for (auto &[a, b] : m)
      if (a >= free_at) { taken++; free_at = b; }
   cout << taken << "\n";
}
```

Real output on the CSES sample (`3 5 / 4 9 / 5 8`):

```
2
```

Theorem 3.4 (End-time greedy is optimal)**.** *Let $G = (g_1, \ldots, g_t)$ be the schedule produced by always taking the compatible movie with the smallest end time, and $O = (o_1, \ldots, o_m)$ any optimal schedule (both sorted by end time). Then $t = m$.*

Proof. Let j be the first index where G and O differ (if $t \geq m$ and they agree on m entries, we are done — greedy already has m). Since g_j is the compatible movie with the smallest end, $b(g_j) \leq b(o_j)$. Both fit after the common prefix: g_j and o_j each start no earlier than o_{j-1}'s end. Replace o_j by g_j in O: the new schedule is still feasible — o_{j+1} starts after $b(o_j) \geq b(g_j)$, so it also clears g_j — and has the same size m. Repeating the exchange turns O into a schedule that starts with G's first t movies and still has m entries. If $m > t$, the tail movie after g_t was still available when greedy stopped — contradiction. So $m = t$. ∎

The picture worth keeping: every exchange *frees time*, never consumes it — Figure 3.3. And the wrong greedy rule fails concretely: on $\{(0, 10), (1, 2), (3, 4)\}$, earliest-*start* takes the long movie and stops at 1, while the end rule takes both short ones — Listing 3.1's program prints 1 and 2 respectively, and Figure 4.5 draws both runs.

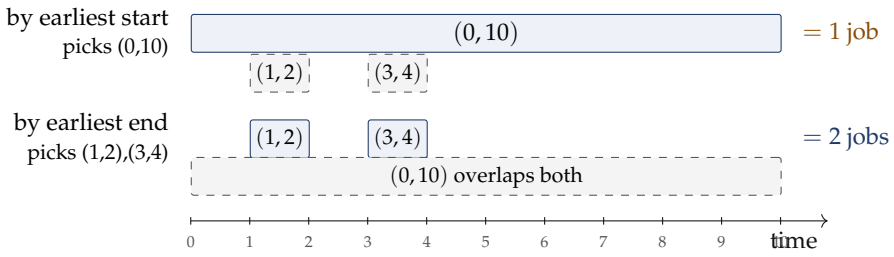


Figure 3.2: Same instance, two greedy orders. Notice the mechanism of the failure: the start rule burns the whole timeline on its first pick; the end rule's picks are exactly the ones that leave room.

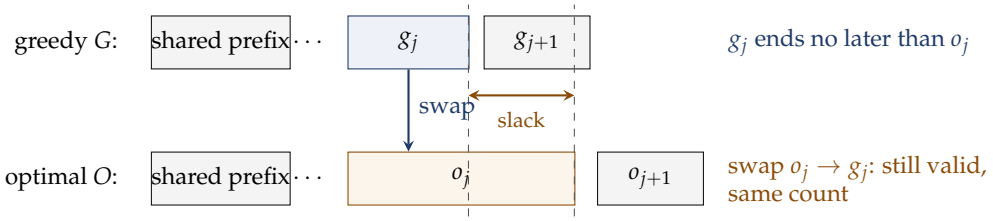


Figure 3.3: The exchange step. At the first difference, greedy's pick ends no later; swapping it in preserves feasibility *and* size. "No later" is the whole lemma — everything else is bookkeeping.

Exercise 3.2 (medium)**.** In the exchange above, where exactly is "$b(g_j) \leq b(o_j)$" used — and which step breaks if the greedy rule picked by earliest *start* instead?

Example 3.5 (CSES *Stick Lengths* — the pairing proof)**.** Make all p_i equal at cost $\sum |p_i - m|$. Which m? *Strategy.* Sort and pair the extremes. If m lies between a smallest and a largest element, that pair contributes exactly their own gap regardless of m's position inside it — so the middle element costs nothing to aim at, and the median is optimal.

Proof sketch. Sort p. For each pair (p_i, p_{n+1-i}) with $i \leq n/2$: if $p_i \leq m \leq p_{n+1-i}$, the pair contributes $(m - p_i) + (p_{n+1-i} - m) = p_{n+1-i} - p_i$, a constant. Moving m outside the pair

raises the sum by twice the overshoot. Every paired m between the two middle elements is optimal; the median $p_{\lceil n/2 \rceil}$ is one. ■

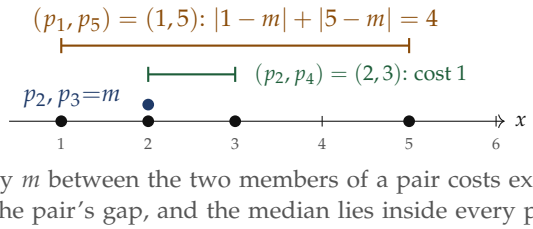
```cpp
// stick_lengths.cpp -- the median minimises sum |p_i - m|.
#include <algorithm>
#include <iostream>
#include <vector>
using namespace std;
using ll = long long;

int main() {
    int n;
    cin >> n;
    vector<ll> p(n);
    for (ll &x : p) cin >> x;
    sort(p.begin(), p.end());
    ll m = p[n / 2], cost = 0;
    for (ll x : p) cost += llabs(x - m);
    cout << cost << "\n";
}
```

Real output on the CSES sample (`2 3 1 5 2`):

```
5
```

Figure 3.4 shows the pairing on this data; the stress harness in `code/test_ch03.sh` checked the claim against an exhaustive minimiser on 300 random arrays — median won every round.



any m between the two members of a pair costs exactly the pair's gap, and the median lies inside every pair

Figure 3.4: The pairing picture. Notice the constant: within a pair, the two distances to m always sum to the pair's gap. The median sits inside *every* pair, so it inherits the minimum possible total.

Example 3.6 (CSES *Apartments* — greedy stays ahead)**.** Applicant i accepts $[a_i - k, a_i + k]$. Sort demands and sizes, walk two pointers: skip apartments too small for the current applicant; skip applicants the current apartment cannot satisfy; on a fit, match. *Stays-ahead invariant:* processing applicants in increasing demand, after greedy's i-th step it has matched at least as many applicants as any optimal solution restricted to the same first i — so when greedy stops, no unseen match can exist anywhere in the input.

```cpp
// apartments.cpp -- two pointers after sorting both lists.
#include <algorithm>
#include <iostream>
#include <vector>
using namespace std;
```

```
using ll = long long;

int main() {
   int n, m;
   ll k;
   cin >> n >> m >> k;
   vector<ll> a(n), b(m);
   for (ll &x : a) cin >> x;
   for (ll &x : b) cin >> x;
   sort(a.begin(), a.end());
   sort(b.begin(), b.end());
   int i = 0, j = 0, given = 0;
   while (i < n && j < m) {
      if (b[j] < a[i] - k) j++; // too small for everyone left
      else if (b[j] > a[i] + k) i++; // too big for this applicant
      else { given++; i++; j++; } // match: take it
   }
   cout << given << "\n";
}
```

Real output on the CSES sample (`4 3 5 / 60 45 80 60 / 30 60 75`):

```
2
```

Exercise 3.3 (easy)**.** On input `n=3, m=3, k=0`, demands $\{5,5,9\}$, sizes $\{5,9,9\}$ — does sorting the apartments change the greedy answer? Answer first, then check by hand.

3.4 When greedy fails

Example 3.7 (Coin systems — prove or distrust)**.** Coin change, minimum coins for amount A over a fixed system. For $\{1,5,10,25\}$, "always take the largest possible" is optimal (a canonical system — provable). For $\{1,3,4\}$ the same rule takes $4+1+1$ for 6: three coins where $3+3$ suffices. *Strategy.* Never trust the rule because it survives samples — test the *smallest* candidate failure, and if you cannot prove the rule, run DP (Chapter 11) instead.

```
// coins.cpp -- greedy coin change vs the true optimum on small
   amounts.
// usage: coins <amount> <denom>... e.g. coins 6 1 3 4
// Denominations are sorted descending internally: greedy is only
   defined
// for largest-first order, so caller order never changes the answer.
#include <algorithm>
#include <cstdlib>
#include <iostream>
#include <vector>
using namespace std;

int main(int argc, char **argv) {
   int amount = atoi(argv[1]);
```

```
    vector<int> d;
    for (int i = 2; i < argc; i++) d.push_back(atoi(argv[i]));
    sort(d.rbegin(), d.rend()); // largest first -- the definition
    // greedy: largest-first
    int left = amount, gc = 0;
    for (int c : d) { gc += left / c; left %= c; }
    // truth: DP over all amounts (correctness by construction)
    const int INF = 1e9;
    vector<int> best(amount + 1, INF);
    best[0] = 0;
    for (int v = 1; v <= amount; v++)
       for (int c : d) if (c <= v) best[v] = min(best[v], best[v - c] +
           1);
    cout << "amount=" << amount << " coins={";
    for (size_t i = 0; i < d.size(); i++)
       cout << d[i] << (i + 1 < d.size() ? "," : "");
    cout << "} greedy=" << gc << " optimal=" << best[amount] << "\n";
}
```

Real output — greedy vs the true optimum on both systems:

```
amount=6 coins={1,3,4} greedy=3 optimal=2
amount=99 coins={1,5,10,25} greedy=9 optimal=9
```

Figure 3.5 shows where the wrong branch starts.

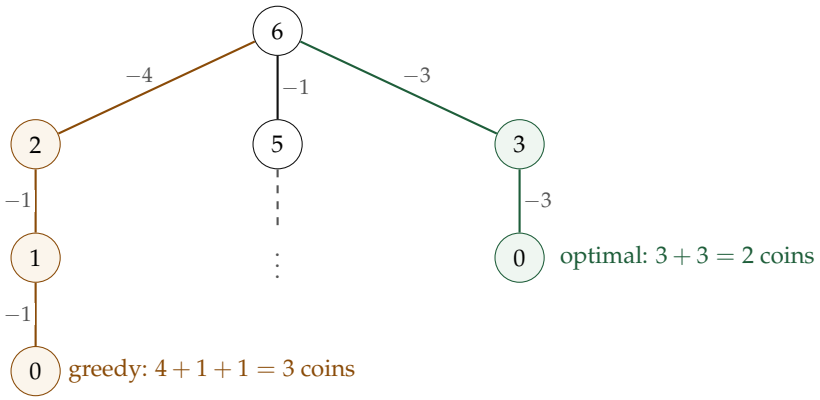


Figure 3.5: The decision tree for amount 6 on $\{1,3,4\}$. Notice the failure is *local*: the first greedy pick (4) already forfeits the optimum — there is no way back later. That is what "greedy fails" means mechanically.

PROVE OR DISTRUST

A greedy rule you cannot defend in two sentences gets a counter-example search before it gets submitted. Try the smallest meaningful instance; if it survives, try to *prove* the rule survives; only then code it.

Exercise 3.4 (medium). Find the smallest amount on which greedy fails for system $\{1,7,10\}$. (Write the three-candidate check before guessing.)

3.5 Sets and maps as greedy tools

A greedy step often asks "what is the best available option *right now*?" — which is exactly an ordered-set query: `lower_bound`/`upper_bound` in $O(\log n)$, then `erase` to consume the resource. CSES *Concert Tickets* is the archetype: sell each customer the most expensive seat $\leq$ their limit. Duplicates are real, so `multiset` — and `erase(it)` on an *iterator* removes one copy, while `erase(value)` removes them all.

Listing 3.3: `code/set_pattern.cpp`: the "best available" idiom. `upper_bound(t)` finds the first seat $> t$; stepping back one gives the largest $\leq t$.

```
// set_pattern.cpp -- "best available" with std::set (CSES Concert
    Tickets).
// For each customer price: sell the most expensive seat <= price.
#include <iostream>
#include <set>
#include <vector>
using namespace std;

int main() {
   int n, m;
   cin >> n >> m;
   multiset<int> seats; // duplicates exist -> multiset
   for (int i = 0, h; i < n; i++) { cin >> h; seats.insert(h); }
   for (int i = 0, t; i < m; i++) {
      cin >> t;
      auto it = seats.upper_bound(t); // first seat > t
      if (it == seats.begin()) { cout << "-1\n"; continue; }
      --it; // largest seat <= t
      cout << *it << "\n";
      seats.erase(it); // erase ONE copy via iterator
   }
}
```

Real output on `5 3 / 5 3 7 8 5 / 4 8 3` — seat 3 was already sold when the last customer asked:

```
3
8
-1
```

3.6 Chapter summary

3.7 Common mistakes

- **Comparator that is not a strict weak order.** <=, or a comparator that sometimes says both orders, corrupts `sort`'s internal invariant — silent wrong answers *and* possible crashes.

Situation	Greedy rule	Proof tool	When to distrust
intervals/schedules	earliest *end*	exchange argument	earliest start fails
equalise to a point	take the median	pairing (constant per pair)	weighted targets
two sorted lists	advance whichever pointer is behind	stays-ahead invariant	unsorted input
"best available"	`set::lower_bound`	step dominance	duplicates need `multiset`
coin-like systems	largest first	only if canonical	any non-canonical set

Table 3.1: The chapter in one table: the rule, the proof that licenses it, and the smell that says stop. The middle column is the one to memorise.

- **Sorting the wrong key.** Movie Festival by start time gives 1 on a 2-answer case. The sort key *is* the greedy rule — pick it by the proof, not by habit.
- **Trusting greedy because samples pass.** Two samples cannot witness a coin system's failure at amount 6. Counter-example hunt first, or proof, or DP.
- **Forgetting stability.** `sort` reorders equals; `stable_sort` does not. Only matters when ties carry meaning — exactly when you'll least expect it.
- **`erase(value)` on a multiset.** Removes *every* copy. Erase through the iterator the lookup returned.
- **Compressing before seeing all values.** Ranks are defined by the full set; an online stream needs a balanced structure, not compression.

3.8 Additional exercises

Exercise 3.5 (easy)**.** CSES *Distinct Numbers* — what does sorting buy you here that a `set` also gives, and which is faster to write?

Exercise 3.6 (easy)**.** CSES *Sum of Two Values* — after sorting, which two pointers, and when do they move? (This is the apartment pattern without the tolerance.)

Exercise 3.7 (easy)**.** CSES *Collecting Numbers* — sort values keeping original indices; count how many rounds a left-to-right pass needs. Which adjacency test detects a new round?

Exercise 3.8 (easy)**.** USACO-B *Cow College* — the optimal tuition is one of the offered prices; why does sorting make trying them all $O(n \log n)$ instead of $O(n^2)$?

Exercise 3.9 (easy)**.** USACO-B *Even More Odd Photos* — greedy rules and parity counting meet; identify the greedy rule and the one case it must handle carefully.

Exercise 3.10 (easy)**.** USACO-S *Lemonade Line* — cows tolerate a queue position only if enough room remains; sort and simulate. Prove the greedy order is optimal by exchange.

Exercise 3.11 (medium)**.** CSES *Restaurant Customers* — sweep line is sorting in disguise: sort all events together and walk a counter. Implement it; then say why "maximum prefix sum of the event order" is the answer.

Exercise 3.12 (medium)**.** CSES *Tasks and Deadlines* — which order minimises total finishing time weighted equally? (Shortest processing time first — prove it by exchanging one adjacent inversion.)

Exercise 3.13 (medium)**.** USACO-S *High Card Wins* — Bessie's strongest card should beat Elsie's weakest beatable card. Write the stays-ahead invariant for this greedy.

Exercise 3.14 (medium)**.** USACO-S *Rest Stops* — Farmer John stops only at tastiest-ahead stops. Sort or scan for "record" tastiness; which direction does the scan run?

Exercise 3.15 (medium)**.** USACO-B *Out of Place* — one cow is misplaced; sorting reveals her. Prove the answer is the count of cows off their sorted position, minus one boundary case — find that case.

Exercise 3.16 (hard)**.** CSES *Josephus Problem I* — simulate the elimination circle. Greedy helps nothing; the honest answer is a `set`/list walk or an order-statistics structure — implement one and compare with $O(n^2)$ simulation on $n = 2 \cdot 10^5$.

Notes

The greedy-proof taxonomy (exchange, stays-ahead, induction) and the canonical/non-canonical coin example follow Sannemo [1] and Laaksonen's chapter 6 [2]; the interval-scheduling exchange is the standard one, retold with the counter-example-first pedagogy of the USACO Guide [3]. `std::sort`'s introsort and strict-weak-ordering contract are from cppreference [6]; Halim's treatment of sort-and-greedy problem patterns informed the exercise set [7]. All problem names and samples are from the CSES Problem Set [4].

Further reading

- Laaksonen, *Competitive Programmer's Handbook*, ch. 6 "Greedy algorithms" (beginner). `https://cses.fi/book/book.pdf` — the scheduling/median/coin-canonical results quoted in this chapter.
- USACO Guide, "Introduction to Greedy Algorithms" (beginner). `https://usaco.guide/bronze/intro-greedy` — more greedy exercises at the Bronze level, same decision discipline.
- USACO Guide, "Introduction to Sorting" (beginner). `https://usaco.guide/bronze/intro-sorting` — comparators and sorting-based problem solving beneath §3.1.
- CSES 1629 *Movie Festival* (practice). `https://cses.fi/problemset/task/1629` — submit the earliest-finish schedule of 3.3 yourself.
- cppreference, "std::sort" (reference). `https://en.cppreference.com/w/cpp/algorithm/sort` — comparator contract (strict weak ordering) behind §3.1's rules.
- Wikipedia, "Greedy algorithm" (reference). `https://en.wikipedia.org/wiki/Greedy_algorithm` — general context for the exchange/stays-ahead proofs of §3.3.

WATCH INSTEAD

Best first. "3. Greedy Method - Introduction" — Abdul Bari, 12 min — https://www.youtube.com/watch?v=ARvQcqJ_-NY. The greedy-choice framing of §3.2 with small worked cases; he optimizes objectives on the same "choice + feasibility" axis this book uses. "Lec 16" — MIT OpenCourseWare 6.046J, 84 min — https://www.youtube.com/watch?v=FPEMBWg_WlY. Formal greedy-proof machinery (greedy-choice property + induction) matching §3.3; blackboard pace, uses more algorithm design notation than the book needs. Access date 2026-10-10.

CHAPTER 4

Binary search

Chapter 3 spent effort rearranging an input until it looked the way a greedy needed it. This chapter rearranges nothing at all: it takes a question whose answer is *one number*, notices that the number's candidates split cleanly into "too small" and "big enough", and finds the boundary in about sixty probes — whatever the range. Most of the problems a Silver contestant calls hard are this idea wearing a costume: the answer to *what is the smallest makespan* or *what is the largest safe gap* is easier to *verify* than to construct, and verification is enough. After this chapter you will be able to write the one template that never loops, spot a monotone predicate inside a word problem, and turn "minimise the maximum" into a routine search over the answer.

4.1 The one correct template

Binary search has a terrible reputation for off-by-one bugs, and every version of it deserves the reputation except this one. The trick is to search *predicates*, not elements: pick a question $P(x)$ that is false for a while and then true forever, and find the first true. On a sorted array the question "$a[i] \geq t$" is exactly that shape.

```
template <class F> ll first_true(ll lo, ll hi, F pred) {
   while (lo < hi) {
      ll mid = lo + (hi - lo) / 2;
      if (pred(mid)) hi = mid; else lo = mid + 1;
   }
   return lo;
}
```

Listing 4.1: `lib/bsearch.h`: first true in $[lo, hi]$, given a predicate monotone on the whole interval.

Three invariants carry the whole proof of correctness, and you should be able to recite them. **(a)** Every position in $[lo, hi)$ is still a candidate for the answer. **(b)** Everything left of lo is known false, everything at or right of hi is known true. **(c)** $hi - lo$ strictly shrinks, so the loop ends with $lo = hi$: one candidate, the first true. Two details do the real work: `mid` rounds *down* (`lo + (hi - lo)/2`, which also dodges overflow), and the update is `hi = mid` on true versus `lo = mid + 1` on false — the false side can skip `mid` entirely, the true side cannot.

The standard library sells the same idea as `lower_bound`, which returns an iterator to the first element $\geq t$; upper_bound returns the first $> t$. Both are the first-true shape.

Listing 4.2: `bsearch_template.cpp`: the template and `lower_bound` agree.

```cpp
// bsearch_template.cpp -- the template + lower_bound on a real array.
#include <algorithm>
#include <iostream>
#include <vector>
#include "lib/bsearch.h"
using namespace std;

int main() {
    vector<int> a = {2, 4, 7, 7, 11};
    int t = 7;
    ll i = first_true(0, (ll)a.size() - 1, [&](ll i){ return a[i] >= t;
        });
    ll j = first_true(0, (ll)a.size() - 1, [&](ll i){ return a[i] > t;
        });
    cout << "first >= " << t << " at index " << i
        << " (a[" << i << "]=" << a[i] << ")\n";
    cout << "first > " << t << " at index " << j
        << " (a[" << j << "]=" << a[j] << ")\n";
    cout << "lower_bound agrees: "
        << (lower_bound(a.begin(), a.end(), t) - a.begin()) << "\n";
}
```

Real output on $a = \{2, 4, 7, 7, 11\}$, $t = 7$:

```
first >= 7 at index 2 (a[2]=7)
first > 7 at index 4 (a[4]=11)
lower_bound agrees: 2
```

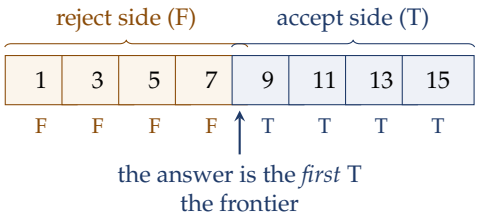


Figure 4.1: The predicate frontier for $P(T)$ = "can we finish by time T" on the Factory Machines instance. Everything the search does is walk the boundary between amber and blue — and it commits to the *first* T, never any T to its right.

4.2 Monotone predicates: searching the answer

Figure 4.1 is the whole idea; Figure 4.3 walks it once, probe by probe. When the predicate is monotone — false up to some boundary, true ever after (or the reverse) — the search runs on the *answer space*, the interval of every value the answer could take, not on the input array. Three questions design the search and nothing else matters:

1. **What is the answer's range?** $[lo, hi]$ must provably contain it — the fastest machine's time times t is a safe top.

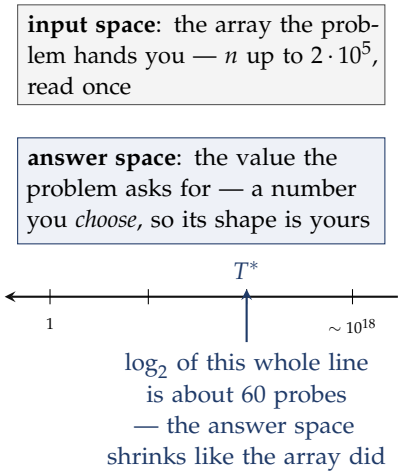


Figure 4.2: Two different spaces. The input array is data you must read; the answer space is a line you get to choose the probing on — and $\log_2$ of 10^{18} is only about 60.

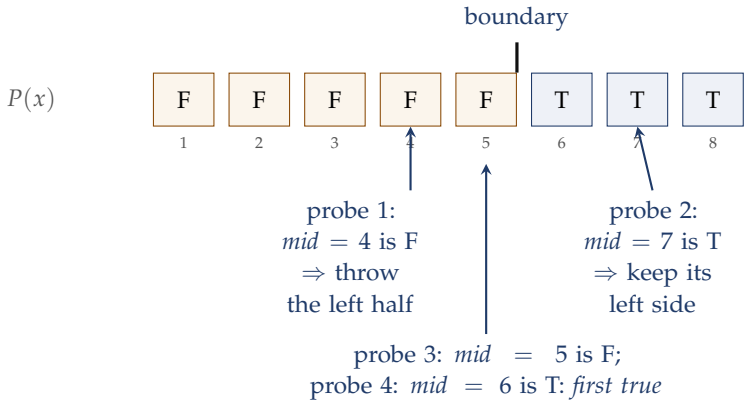


Figure 4.3: A first_true walk on the predicate row of Figure 4.1: each probe lands on a midpoint, the surviving half keeps the boundary inside it, and four probes suffice on eight cells — $\lceil \log_2 8 \rceil + 1$. Watch the invariant, not the values: the true/false boundary never leaves the live interval.

2. **What is the check?** $P(x)$ must be computable in far less time than constructing the answer.
3. **Is it monotone?** If P is a sawtooth, the search finds *a* boundary, not *the* answer (Section 4.4).

Example 4.1 (CSES *Factory Machines*, min makespan)**.** n machines make products, machine i taking k_i seconds each. Shortest time for t products total ($n \le 2 \cdot 10^5$, $t, k_i \le 10^9$)?

Strategy. The answer is a time T; the answer space is $[1, 10^{18}]$. The predicate "can we finish by T" is monotone — more time never hurts — and cheap: machine i produces $\lfloor T/k_i \rfloor$ units, so $\sum_i \lfloor T/k_i \rfloor \ge t$ decides it in $O(n)$. Search over 60 probes $\times$ one sum each.

Solution.

Listing 4.3: `factory_machines.cpp`: the check plus the driver.

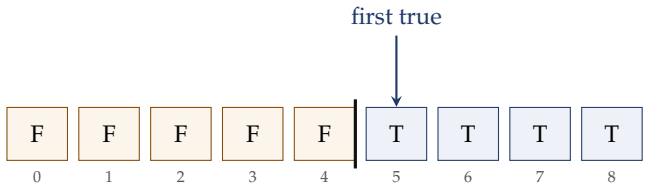


Figure 4.4: The predicate picture: F everywhere left, T right — binary search finds the boundary.

```
// factory_machines.cpp -- CSES Factory Machines via binary search on time.
#include <iostream>
#include <vector>
#include "lib/bsearch.h"
using namespace std;

int main() {
    ll n, t;
    cin >> n >> t;
    vector<ll> a(n);
    for (auto &x : a) cin >> x;
    auto check = [&](ll T) { // can we make t products by time T?
        ll made = 0;
        for (ll x : a) { made += T / x; if (made >= t) return true; }
        return made >= t;
    };
    cout << "check(7)=" << check(7) << " check(8)=" << check(8) << "\n";
    cout << first_true(1, (ll)4e18, check) << "\n";
}
```

Real output (CSES sample: 3 machines $\{3,2,5\}$, $t = 7$):

```
check(7)=0 check(8)=1
8
```

The check itself shows the frontier: 7 fails, 8 passes, and 8 is the answer. One overflow habit to steal: the accumulator stops at t instead of summing 10^9 terms it does not need.

Example 4.2 (CSES *Array Division*, minimise the maximum sum)**.** Split an array of n positive values into k contiguous groups minimising the largest group sum ($n \le 2 \cdot 10^5$).

Strategy. Answer space: $[\max a_i, \sum a_i]$. Predicate "does cap x per group suffice" is monotone — a bigger cap can only help — and checkable greedily: accumulate until the next element would overflow the cap, cut, count groups.

Solution.

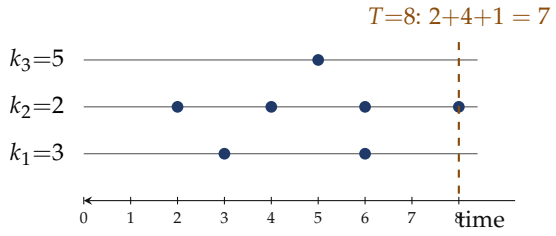


Figure 4.5: The check for Example 4.1 as a completion timeline at $T = 8$: machine k_1=3 finishes at 3, 6 (two products), k_2=2 at 2, 4, 6, 8 (four), k_3=5 at 5 (one). Counting ticks left of the dashed line *is* the predicate — nothing was scheduled.

```
// array_division.cpp -- CSES Array Division: minimise the largest group sum.
#include <algorithm>
#include <iostream>
#include <numeric>
#include <vector>
#include "lib/bsearch.h"
using namespace std;

int main() {
   int n, k;
   cin >> n >> k;
   vector<ll> a(n);
   for (auto &x : a) cin >> x;
   auto check = [&](ll cap) { // does cap per group suffice?
      int groups = 1; ll sum = 0;
      for (ll x : a) {
         if (x > cap) return false;
         if (sum + x > cap) { groups++; sum = 0; }
         sum += x;
      }
      return groups <= k;
   };
   ll lo = *max_element(a.begin(), a.end()),
     hi = accumulate(a.begin(), a.end(), 0LL);
   cout << "check(7)=" << check(7) << " check(8)=" << check(8) << "\n";
   cout << first_true(lo, hi, check) << "\n";
}
```

Real output (n=5, k=3, $\{2,4,7,3,5\}$):

```
check(7)=0 check(8)=1
8
```

The optimal split is $[2,4], [7], [3,5]$ — but notice the program never built it. The greedy's correctness rests on a minimality claim: each greedy group extends as far right as the cap allows, so no feasible division uses fewer groups. If the greedy count g is at most k, merging adjacent groups — always legal, each merge just concatenates two pieces — reduces the

count to exactly k; if g exceeds k, no division exists at all. Asking only "$g \le k$" is therefore sound.

Example 4.3 (USACO-S *Social Distancing*, maximise the minimum)**.** Free stalls at sorted positions $p_1 < \cdots < p_n$; place k cows to maximise the smallest gap between neighbours.

Strategy. The min-max pattern inverted: now the answer is the *largest* feasible gap d. Predicate "can k cows sit with gaps $\ge d$" is monotone the other way — feasible for small d, infeasible past the frontier. The check is the same greedy as ever: place the next cow at the first stall at least d past the last.

Solution.

```
// social_distancing.cpp -- USACO-S Social Distancing: maximise the
   min gap.
#include <algorithm>
#include <iostream>
#include <vector>
#include "lib/bsearch.h"
using namespace std;

int main() {
   int n, k;
   cin >> n >> k;
   vector<ll> p(n);
   for (auto &x : p) cin >> x;
   sort(p.begin(), p.end());
   auto fits = [&](ll d) { // can k cows sit with gaps >= d?
      int placed = 1; ll last = p[0];
      for (int i = 1; i < n; i++)
         if (p[i] - last >= d) { placed++; last = p[i]; }
      return placed >= k;
   };
   // largest d that still fits == last false of "!fits" == first_true
      -1
   ll d = first_true(0, p.back() - p[0] + 1,
               [&](ll d){ return !fits(d); }) - 1;
   cout << "fits(4)=" << fits(4) << " fits(5)=" << fits(5) << "\n";
   cout << d << "\n";
}
```

Real output (stalls $\{1, 2, 5, 8, 9\}$, $k = 3$):

```
fits(4)=1 fits(5)=0
4
```

Two habits worth copying: maximisation becomes first-true on the *negated* predicate (search for the first d that fails, answer is one less), and the candidate range runs to the span of the stalls — answers beyond it are impossible by construction.

Example 4.4 (CSES *Multiplication Table*, a counting predicate)**.** The $n \times n$ multiplication table (n odd, $n < 10^6$): find the median of its n^2 products without building them.

Strategy. Nothing is sorted and nothing can be — n^2 is too many elements to hold. The answer space is the product range $[1, n^2]$, and the predicate is a *count*: "at least $\lceil n^2/2 \rceil$ products are $\leq x$". Row i contributes $\min(n, \lfloor x/i \rfloor)$, so counting is $O(n)$ and monotone — larger x can only add products.

Solution.

```
// mult_table.cpp -- CSES Multiplication Table: median of n^2 products.

#include <iostream>
#include "lib/bsearch.h"
using namespace std;

int main() {
   ll n;
   cin >> n;
   auto le = [&](ll x) { // count products <= x, row by row
      ll c = 0;
      for (ll i = 1; i <= n; i++) c += min(n, x / i);
      return c;
   };
   ll k = (n * n + 1) / 2; // middle position (1-indexed)
   cout << first_true(1, n * n, [&](ll x){ return le(x) >= k; }) << "\
      n";
}
```

Real output:

```
3
```

n=3: the sorted products are $\{1, 2, 2, 3, 3, 4, 6, 6, 9\}$ and the middle is 3. The pattern to keep: *rank queries* ("how many are $\leq x$") are monotone almost by definition, so any "find the k-th" question is candidate answer-space search.

Example 4.5 (CSES *Concert Tickets*: the STL's own first_true)**.** Tickets cost $h_1, \ldots, h_n$; each of m customers in turn names a maximum price t_i and takes the most expensive ticket costing at most t_i, or none ($n, m \leq 2 \cdot 10^5$).

Strategy. Sort the tickets and binary-search each t_i — that is the template verbatim, but the STL already carries it: a `multiset` keeps prices sorted, and `upper_bound(t)` returns *the first element exceeding t* — the same first_true boundary as §4.1, with "too expensive" as the predicate. Step back one iterator for the answer. The custom walk still matters: here the library does it for you, and knowing that is the difference between a three-line solution and a forty-line one.

Solution. `code/concert_tickets.cpp`:

```
$ ./concert_tickets < tickets_sample.txt
3
8
-1
```

Customer three wants ≤ 3 but only $\{5,7,5\}$ remain — `upper_bound` lands on `begin()` and we print -1. Each query is $O(\log n)$; sorted input plus an ordered container is the whole trick.

Exercise 4.1 (easy)**.** The template keeps "left of `lo` known false, at-or-right-of `hi` known true". For each of these predicate answers at `mid`, state the correct update: (a) $P($`mid`$)$ false, (b) $P($`mid`$)$ true, (c) the invariant when the loop exits.

Exercise 4.2 (medium)**.** Which of these predicates is monotone, and on what range? (a) "x machines can finish by time T" as a function of x; (b) "x^2 is prime"; (c) "the greedy's group count is $\leq k$"; (d) "$a[i] \geq t$" on an *unsorted* array.

4.3 Real-valued search

Over a continuous interval the "first true" trick never terminates — there is no first true real number. The fix is to stop after a fixed number of iterations: each probe halves the bracket, so 60 rounds shrink $[0,2]$ by $2^{-60} \approx 10^{-18}$, well past double precision. Never write `while (hi - lo > eps)`: it compares the bracket to a tolerance that ignores the function's steepness, and `mid*mid` near 1e20 can overflow silently. Fixed iterations are cheaper to reason about and just as accurate.

Listing 4.4: `real_bsearch.cpp`: 60 iterations to $\sqrt{2}$ — the bracket, not the midpoint, is the deliverable.

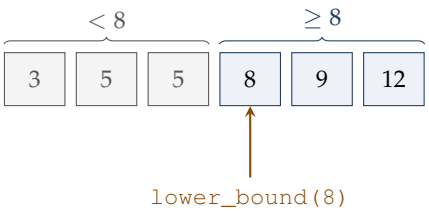
```
// real_bsearch.cpp -- 60 fixed iterations find sqrt(2); no eps needed.
#include <iomanip>
#include <iostream>
using namespace std;

int main() {
   double lo = 0, hi = 2;
   for (int it = 1; it <= 60; it++) {
      double mid = (lo + hi) / 2;
      if (mid * mid < 2) lo = mid; else hi = mid;
      if (it == 10 || it == 20 || it == 60)
         cout << "it=" << it << " bracket=[" << setprecision(17)
             << lo << ", " << hi << "]\n";
   }
}
```

Real output:

```
it=10 bracket=[1.4140625, 1.416015625]
it=20 bracket=[1.4142131805419922, 1.414215087890625]
it=60 bracket=[1.4142135623730949, 1.4142135623730951]
```

Note which side moved: the predicate is $x^2 < 2$, true below the root — so `lo` chases the boundary from below and `hi` from above. Reading off which endpoint is safe to report is the whole correctness story.



first element ≥ 8: skips both 5s and lands on 8; `upper_bound(8)` would land on 9

Figure 4.6: `lower_bound` on a sorted multiset: the first element ≥ the key.

4.4 Where the pattern ends

The template never fails; the *predicate* fails. A non-monotone P still returns a boundary, but it is the boundary of wherever `mid` happened to land — the answer space may hold several frontiers, and you find whichever the search path crossed first. Two honest warnings: (a) monotonicity is a *proof obligation*, not a property you get for free from writing `check()`; convince yourself the frontier is unique before you search it. (b) A monotone-sounding English description can still hide a sawtooth: "x workers finish in time" is monotone; "configuration x is valid" need not be. If the predicate wiggles, you need structure — convexity, a transform — not optimism.

4.5 Chapter summary

Problem	Answer space	Predicate	Check cost	WE
Factory Machines	$[1, 10^{18}]$ time	$\sum \lfloor T/k_i \rfloor \geq t$	$O(n)$	4.1
Array Division	$[\max, \sum]$ cap	greedy groups $\leq k$	$O(n)$	4.2
Social Distancing	$[0, \text{span}]$ gap	greedy placing $\geq k$	$O(n)$	4.3
Multiplication Table	$[1, n^2]$ value	count $\leq x \geq$ median	$O(n)$	4.4
$\sqrt{2}$ on $\mathbb{R}$	$[0, 2]$ real	$x^2 < 2$	$O(1)$	§4.3

Table 4.1: One template, five costumes. In every row the hard work was *writing the check* — the search was already solved.

4.6 Common mistakes

- **Mixing the updates.** `hi = mid - 1` on true throws away the only candidate you know is true; `lo = mid` on false leaves `mid` in play and loops forever. Memorise the pair: `true` keeps `mid`, `false` skips it.
- **Searching the wrong direction.** The template finds the first *true*. For a last-true or first-false problem, negate the predicate or subtract one — do not improvise a mirrored loop.
- **Predicate that is not monotone.** The search still returns something; it is just not the answer you wanted. Prove the frontier is unique first.
- **Overflow inside the check.** $\sum T/k_i$ with $T = 10^{18}$ and $n = 2 \cdot 10^5$ overflows a 64-bit sum several times over; cap the accumulator at the target, as Listing 4.3 does.
- **`while (hi-lo > eps)` on reals.** Epsilons couple the stop rule to the function's slope and invite infinite loops; fix the iteration count instead.

4.7 Additional exercises

Exercise 4.3 (easy)**.** CSES *Sum of Three Values* — reduce to two-sum via sorting.

Exercise 4.4 (easy)**.** USACO-S *Cow Dance Show* — feasibility as a simulation per candidate size.

Exercise 4.5 (easy)**.** USACO-S *Convention* — monotone coverage check.

Exercise 4.6 (medium)**.** CSES *Reading Books* — the answer space is a waiting time.

Exercise 4.7 (medium)**.** CSES *Movie Festival II* — multiset membership per step; where does the monotonicity live?

Exercise 4.8 (medium)**.** USACO-S *Loan Repayment* — search the repayment term.

Exercise 4.9 (medium)**.** USACO-S *Bakery* — feasibility linear program per cost bound.

Exercise 4.10 (medium)**.** USACO-S *Angry Cows* — the classic blast-radius search.

Exercise 4.11 (medium)**.** CF *Guess the K-th Zero* — an *interactive* first-true.

Exercise 4.12 (easy)**.** Hand-walk the first_true template on predicate `F F F T T` and list every probe.

Exercise 4.13 (medium)**.** USACO-S *Angry Cows* — search the blast radius directly.

Exercise 4.14 (hard)**.** CSES *Maximum Subarray Sum II* — combine with a sliding window.

Exercise 4.15 (hard)**.** CF *The Meeting Place Cannot Be Changed* — real-valued answer.

Exercise 4.16 (hard)**.** Baltic OI 2012 *Mobile* — heavy feasibility per candidate weight.

Notes

Binary search on the answer is the flagship technique of Sannemo's chapter on it [1] and Halim's greedy/search sections [7]; Laaksonen's guide covers the template's pitfalls [2]. The four archive problems are official CSES/USACO tasks [4, 3]; standard references on the idiom are the C++ reference's `lower_bound` page [6] and the cp-algorithms binary-search note.

Exercise 4.17 (hard)**.** CSES *Sliding Median* — order statistics inside the window.

Further reading

- USACO Guide, "Binary Search" (intermediate) — same first_true template, more answer-search problems at the Silver level.
  https://usaco.guide/silver/binary-search
- Laaksonen, *Competitive Programmer's Handbook*, ch. 4 "Binary search" (beginner) — the array half of the technique plus STL lower_bound idiom.
  https://cses.fi/book/book.pdf
- cp-algorithms, "Binary search" (reference) — implementation variants incl. the real-valued search of §4.3.
  https://cp-algorithms.com/num_methods/binary_search.html
- CSES 1085 *Array Division* (practice) — 4.2 yourself; CSES 1620 *Factory Machines* likewise.
  https://cses.fi/problemset/task/1085
- cppreference, "std::lower_bound" (reference) — the library realization of §4.1.
  https://en.cppreference.com/w/cpp/algorithm/lower_bound
- Bloch, "Nearly All Binary Searches and Mergesorts are Broken" (blog, intermediate) — the midpoint-overflow bug this book's template avoids.
  https://research.google/blog/ (post by Joshua Bloch, 2006)

WATCH INSTEAD

Best first. "Binary Search Algorithm" — Computerphile, 19 min — https://youtu.be/hDn8iOc30Tk. The halving invariant of §4.1 explained visually; Dr. Pound indexes from 1 where the book uses 0 — the boundary logic is identical. "Bug in Binary Search" — Computerphile, 12 min — https://youtu.be/_eS-nNnkKfI. The `(lo+hi)/2` overflow of the Common mistakes section — the real Java library bug, worth watching before you write `mid` once. Access date 2026-10-10.

CHAPTER 5

Prefix sums, difference arrays, and two pointers

Chapters 1–4 built answers by searching, sorting, and arguing. This chapter is quieter: three bookkeeping patterns that turn interval work from $O(n)$ per query into $O(1)$ per query — or a full scan into a single sweep. The prefix sum precomputes every cumulative total once so each later range-sum is one subtraction. The difference array is the same idea run backwards: one range update is two point writes, and a final prefix pass restores every cell. Two pointers replace a doubly nested scan with one march when the window's contents are monotone. After this chapter you will be able to answer a hundred thousand range queries in linear time total, batch range updates for free, and recognise exactly when the telescoping window is licensed.

5.1 Prefix sums: build once, query forever

Given an array $a_1, \ldots, a_n$, the *prefix table* $P_i = a_1 + \cdots + a_i$ (with $P_0 = 0$) stores every partial sum in one pass of adds. Any later range query is one lookup and one subtraction: $\sum_{i=l}^{r} a_i = P_r - P_{l-1}$. The whole bargain is $O(n)$ build, $O(1)$ query, forever — the oldest space-for-time trade in the book, and the first thing to try whenever a statement repeats "the sum over a range" more than once.

Listing 5.1: `prefix.h`: 1D and 2D prefix tables.

```
// prefix.h -- prefix tables: build once, query forever.
// 1D: P[i] = a[1]+...+a[i], sum(l..r) = P[r]-P[l-1].
// 2D: P[r][c] = sum of the (r x c) corner; rect (r1,c1)-(r2,c2) =
// P[r2][c2] - P[r1-1][c2] - P[r2][c1-1] + P[r1-1][c1-1].
#pragma once
#include <vector>
using ll = long long;

struct Prefix1D {
   int n; std::vector<ll> p;
   explicit Prefix1D(const std::vector<ll>& a) : n(a.size()), p(n + 1)
       {
      for (int i = 0; i < n; i++) p[i + 1] = p[i] + a[i];
   }
   ll sum(int l, int r) const { return p[r] - p[l - 1]; } // 1-indexed
};
```

```
16
17 struct Prefix2D {
18    int n, m; std::vector<std::vector<ll>> p;
19    Prefix2D(const std::vector<std::vector<ll>>& a)
20       : n(a.size()), m(a[0].size()), p(n + 1, std::vector<ll>(m + 1))
            {
21       for (int r = 1; r <= n; r++)
22          for (int c = 1; c <= m; c++)
23             p[r][c] = a[r - 1][c - 1] + p[r - 1][c] + p[r][c - 1]
24                   - p[r - 1][c - 1];
25    }
26    ll sum(int r1, int c1, int r2, int c2) const {
27       return p[r2][c2] - p[r1 - 1][c2] - p[r2][c1 - 1] + p[r1 - 1][c1
            - 1];
28    }
29 };
```

Example 5.1 (CSES *Static Range Sum Queries*)**.** n values, q queries "sum over $[a, b]$", $n, q \leq 2 \cdot 10^5$.

Strategy. q independent sums, $O(n)$ each, is $4 \cdot 10^{10}$ adds — dead. The constraint isn't the input size, it's the repetition: precompute P, then every query is one subtraction.

Solution.

```
1  // range_sum.cpp -- CSES Static Range Sum Queries (1D prefix).
2  #include <iostream>
3  #include <vector>
4  #include "prefix.h"
5  using namespace std;
6
7  int main() {
8     int n, q;
9     cin >> n >> q;
10    vector<ll> a(n);
11    for (auto &x : a) cin >> x;
12    Prefix1D pf(a);
13    while (q--) {
14       int l, r;
15       cin >> l >> r;
16       cout << pf.sum(l, r) << "\n";
17    }
18 }
```

Real output (CSES sample $\{3, 2, 4, 5, 1, 1, 5, 3\}$, queries $[2, 4]$, $[5, 6]$, $[1, 8]$, $[3, 3]$):

```
11
2
24
4
```

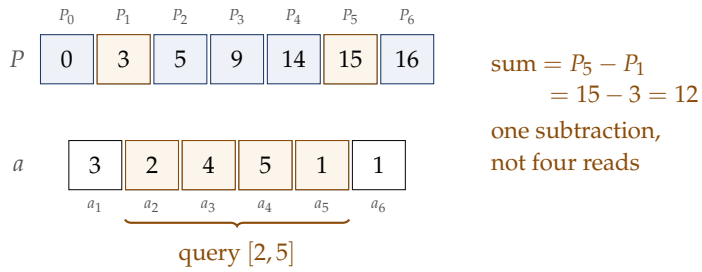


Figure 5.1: Prefix cells versus a range query. Notice the query reads *two* of the blue cells, none of the data cells: $P_r - P_{l-1}$ subtracts everything before the range at once.

Exercise 5.1 (easy). Given $a = \{4, 1, 7, 2, 6\}$, build P by hand and evaluate $\sum_3^5$ and $\sum_1^4$ with single subtractions.

5.2 Difference arrays: the inverse direction

The prefix sum reads a finished array per range. Its inverse writes: to add v to every a_i for $i \in [l, r]$, do *not* touch the range — write $d_l \mathrel{+}= v$ and $d_{r+1} \mathrel{-}= v$ into a difference array d. Two point writes per update, and a single prefix pass of d at the end restores every cell: position i's restored value is exactly the sum of all updates that started at or before i minus those that already closed. All updates batched, $O(1)$ each.

Listing 5.2: `diff.h`: range updates as two point writes, 1D and 2D.

```
// diff.h -- range updates as two point writes.
// To add v to a[l..r]: d[l] += v, d[r+1] -= v.
// The prefix sum of d restores every cell exactly once at the end.
#pragma once
#include <vector>
using ll = long long;

struct Diff1D {
    int n; std::vector<ll> d;
    explicit Diff1D(int n) : n(n), d(n + 2, 0) {}
    void add(int l, int r, ll v) { d[l] += v; d[r + 1] -= v; }
    std::vector<ll> restore() const {
        std::vector<ll> a(n);
        ll cur = 0;
        for (int i = 0; i < n; i++) { cur += d[i]; a[i] = cur; }
        return a;
    }
};

struct Diff2D {
    int n, m; std::vector<std::vector<ll>> d;
    Diff2D(int n, int m) : n(n), m(m), d(n + 2, std::vector<ll>(m + 2,
        0)) {}
    void add(int r1, int c1, int r2, int c2, ll v) { // 1-indexed
        corners
```

```
        d[r1][c1] += v; d[r2 + 1][c1] -= v;
        d[r1][c2 + 1] -= v; d[r2 + 1][c2 + 1] += v;
    }
    std::vector<std::vector<ll>> restore() const {
        std::vector<std::vector<ll>> a(n, std::vector<ll>(m));
        for (int r = 0; r < n; r++) {
            ll row = 0;
            for (int c = 0; c < m; c++) {
                row += d[r][c];
                a[r][c] = row + (r ? a[r - 1][c] : 0);
            }
        }
        return a;
    }
};
```

```
updates: [2,4]+3, [1,3]+5, [6,7]+2
restored: 0 5 8 8 3 0 2 2
```

Listing 5.3: `diff_demo.cpp` real output — three updates, one restore.

Check the restore by hand: cell 1 sees only $+5$; cell 3 sees $+3+5$; cell 5 sits outside every update. Figure 5.2 shows the pipeline.

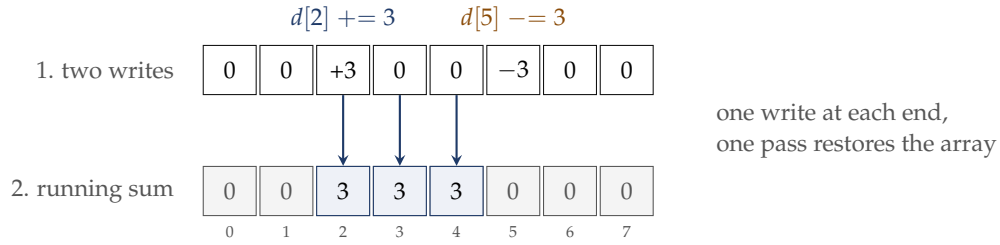


Figure 5.2: A difference-array update becomes two point writes; the prefix pass restores cells. Notice what the $-v$ write buys: the increase *ends*, which is what makes the restored plateau flat.

5.3 Two dimensions: rectangles by inclusion–exclusion

The same subtraction generalises to grids. $P_{r,c}$ = sum of the $r \times c$ top-left corner. A rectangle (r_1, c_1)–(r_2, c_2) then costs four lookups with the PIE sign pattern: $P_{r_2,c_2} - P_{r_1-1,c_2} - P_{r_2,c_1-1} + P_{r_1-1,c_1-1}$ — the two over-subtracted corner strips come back once, exactly the inclusion–exclusion of Figure 5.3. And the difference array generalises identically: a rectangle $+v$ update is four corner writes, restored by a 2D prefix pass.

Example 5.2 (CSES *Forest Queries*)**.** Count trees in axis-aligned rectangles of an $n \times n$ grid, $q \le 2 \cdot 10^5$ queries.

Strategy. "How many inside a rectangle" repeated q times is the 2D prefix pattern verbatim: trees are just values in $\{0, 1\}$, so build P once and answer each rectangle in four reads.

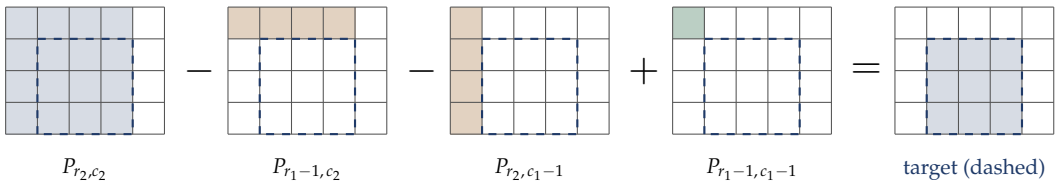
Solution.

```
// forest_queries.cpp -- CSES Forest Queries (2D prefix on a grid).
#include <iostream>
#include <string>
#include <vector>
#include "prefix.h"
using namespace std;

int main() {
   int n, q;
   cin >> n >> q;
   vector<vector<ll>> g(n, vector<ll>(n));
   for (int r = 0; r < n; r++) {
      string row; cin >> row;
      for (int c = 0; c < n; c++) g[r][c] = (row[c] == '*');
   }
   Prefix2D pf(g);
   while (q--) {
      int r1, c1, r2, c2;
      cin >> r1 >> c1 >> r2 >> c2;
      cout << pf.sum(r1, c1, r2, c2) << "\n";
   }
}
```

Real output (CSES sample):

```
3
1
2
```



the top-left strip is subtracted twice, so it is added back once — the signs alternate $+\ -\ -\ +$

Figure 5.3: Rectangle sums by corner algebra. Notice the signs $+, -, -, +$: the count over the target rectangle is the big corner minus the two flanking strips, plus the little corner both strips stole.

Example 5.3 (USACO-S *Painting the Barn*)**.** K coats visible: given a list of rectangles painted one coat at a time, count the unit squares covered by *exactly* K rectangles (grid $\leq 10^3$).

Strategy. The statement hands you an *update list*, not an array — that is the difference array's native habitat. Every rectangle is one 2D-range $+1$ update (4 corner writes); after all updates, restore the grid once and count cells $= K$.

Solution.

```
// painting_barn.cpp -- USACO-S Painting the Barn via 2D difference.
```

```
#include <iostream>
#include <vector>
#include "diff.h"
using namespace std;

int main() {
   int N, K;
   cin >> N >> K;
   vector<vector<ll>> diff(N + 1, vector<ll>(N + 1));
   int x1, y1, x2, y2;
   while (cin >> x1 >> y1 >> x2 >> y2) { // bottom-left / top-right
      diff[x1][y1]++; diff[x1][y2]--;
      diff[x2][y1]--; diff[x2][y2]++;
   }
   int cnt = 0;
   for (int x = 0; x < N; x++) {
      int run = 0;
      for (int y = 0; y < N; y++) {
         run += diff[x][y];
         diff[x][y] = run + (x ? diff[x - 1][y] : 0);
         if (diff[x][y] == K) cnt++;
      }
   }
   cout << cnt << "\n";
}
```

Real output (N=5, K=3, rectangles $(1,1)-(4,4)$, $(2,2)-(4,4)$, $(3,2)-(4,4)$ — cells $(3,2)$ and $(3,3)$ carry all three coats):

```
2
```

5.4 Two pointers: the telescoping window

Different machine, same moral: if a window only needs to slide, two indices can do what a double loop did. For "count subarrays summing to x" over *positive* values: extend r while the window is short of x, contract l while it overshoots. Each index moves only rightward, so the whole march is $O(n)$ — and each subarray counted is genuinely the only window at that l with the right sum, because growing or shrinking a positive window monotonically changes the sum.

Example 5.4 (CSES *Subarray Sums I*)**.** Count subarrays summing to x, all a_i positive.

Strategy. Positivity is the licence: window sums move *monotonically* as l or r advances, so each l has at most one valid r to find. March, don't scan.

Solution.

```
// twoptr_sums.cpp -- CSES Subarray Sums I: all values must be
   positive.
#include <iostream>
```

```cpp
#include <vector>
using namespace std;

int main() {
    int n; long long x;
    cin >> n >> x;
    vector<long long> a(n);
    for (auto &v : a) cin >> v;
    int cnt = 0; long long sum = 0;
    for (int l = 0, r = 0; r < n; r++) {
        sum += a[r];
        while (sum > x) sum -= a[l++];
        if (sum == x) cnt++;
    }
    cout << cnt << "\n";
}
```

Real output (CSES sample $\{2,4,1,2,7\}$, x=7 — windows $\{4,1,2\}$, $\{7\}$, $\{2,7\}$):

```
3
```

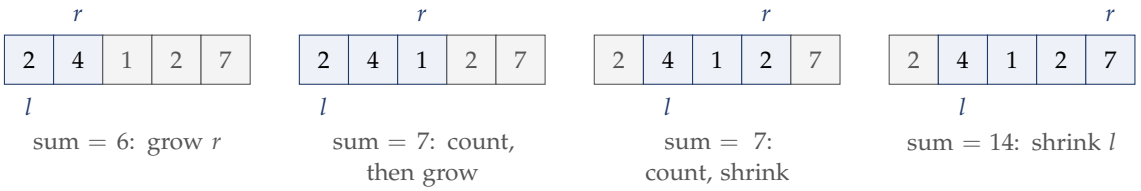


Figure 5.4: The two-pointer march on the CSES sample. Notice each pointer travels in one direction only — l never revisits an index, which is where the n^2 went.

Exercise 5.2 (easy)**.** The two-pointer code counts *wrongly* once negatives are allowed. Explain which step of the march breaks and why (find a counterexample with a single -1).

5.5 Sliding windows with bookkeeping

When values can be negative, the window stops telescoping — but the prefix table still answers, this time by *counting pairs*: subarray $[i{+}1, j]$ has sum x iff $P_j - P_i = x$. Walk j once, counting how many earlier prefixes equal $P_j - x$ in a `map`. The same trick with remainders counts subarrays divisible by n: equal prefix residues. Both are $O(n)$ or $O(n \log n)$ where the telescoping window was forbidden.

Example 5.5 (CSES *Subarray Sums II* + *Subarray Divisibility*)**.** Same count with negatives allowed; and subarrays whose sum is a multiple of n.

Strategy. Negatives kill monotonicity, so switch the predicate's frame: the window condition $P_j - P_i = x$ is a *pair* condition between prefix values — count prefix occurrences, not windows. For divisibility, the condition becomes equal residues mod n.

Solution.

```cpp
// prefix_map.cpp -- Subarray Sums II (negatives allowed) and
// Subarray Divisibility: prefix + map counting.
#include <iostream>
#include <map>
#include <vector>
using namespace std;
using ll = long long;

int main() {
   int n; ll x;
   cin >> n >> x;
   vector<ll> a(n);
   for (auto &v : a) cin >> v;
   // exact sum x: count pairs (P[i], P[j]) with P[j]-P[i] = x
   map<ll, int> seen;
   ll P = 0, cnt = 0;
   seen[0] = 1;
   for (ll v : a) {
      P += v;
      cnt += seen[P - x];
      seen[P]++;
   }
   cout << "sum=" << x << ": " << cnt << "\n";
   // divisibility by n itself (Subarray Divisibility): equal residues
   map<ll, int> rem;
   ll Q = 0; ll d = 0;
   rem[0] = 1;
   for (ll v : a) {
      Q = ((Q + v) % n + n) % n;
      d += rem[Q];
      rem[Q]++;
   }
   cout << "divisible by n=" << n << ": " << d << "\n";
}
```

Real output (CSES samples $\{2,-1,3,5,-2\}$, x=7; $\{3,1,2,7,4\}$ divisibility):

```
sum=7: 2
divisible by n=5: 1
```

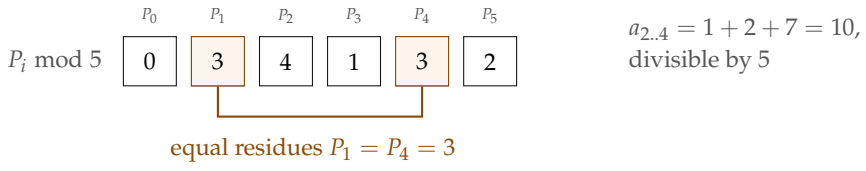


Figure 5.5: Prefix-remainder counting on $\{3,1,2,7,4\}$ mod 5. Notice the answer comes from a *repeated residue*, not from any single value: two equal prefix sums bracket a divisible subarray.

```
window[0..2] distinct=2 min=1
window[1..3] distinct=3 min=1
```

```
window[2..4] distinct=3 min=1
window[3..5] distinct=2 min=2
```

Listing 5.4: `window_demo.cpp`: sliding distinct-count and deque minimum preview.

The deque line is a preview: Chapter 6 formalises the monotonic queue that answers "window minimum" in amortised $O(1)$.

Exercise 5.3 (medium)**.** Show that if two prefix sums are congruent mod m, the subarray between them is divisible by m. Is the converse true?

5.6 Chapter summary

Asked	Tool	Cost	Needs
sum over $[l, r]$, q times	1D prefix P	$O(n) + O(1)$	static array
rectangle sums	2D prefix	4 lookups	static grid
range $+v$ updates	difference array	2 writes + 1 pass	batch offline
subarray sum $= x$, $a_i > 0$	two pointers	$O(n)$	positivity!
subarray sum $= x$, any a_i	prefix + `map`	$O(n \log n)$	pair count
divisible-sum subarrays	prefix residues	$O(n)$	mod n
window stats	deque/`map` slide	$O(n)$	Ch 6 preview

Table 5.1: Every row is the same idea at a different orientation: precompute cumulative structure once, answer forever.

5.7 Common mistakes

- **Two pointers on negative data.** The window's sum is not monotone in l once a negative value exists, so the march skips valid answers — Subarray Sums II exists precisely because the telescope fails. Reach for prefix-pair counting instead.
- P_0 **forgetting.** A prefix table without the sentinel $P_0 = 0$ cannot answer a range that starts at index 1 — every book keeps the zero cell.
- **Off-by-one on the difference close.** d_{r+1} ends the update, not d_r: the subtract write belongs one step *after* the range, so the restore is a prefix of d, never of a.
- **Rectangle corner confusion.** Half-open vs closed corners disagree by one in both axes; fix the convention (ours: $P_{r,c}$ is the $r \times c$ corner, query subtracts at $r_1 - 1$, $c_1 - 1$) and test on the 1×1 case.
- **Residues that go negative.** P mod m in C++ keeps the sign of P; always normalise $((P\%m) + m)\%m$ before bucketing.

5.8 Additional exercises

Exercise 5.4 (easy). USACO-S *Breed Counting* — three prefix tables, one per breed.

Exercise 5.5 (easy). USACO-S *Hoof, Paper, Scissors* — prefix counts per gesture.

Exercise 5.6 (easy). CSES *Maximum Subarray Sum* — prefix min so far, one pass.

Exercise 5.7 (easy). CF *Karen and Coffee* — difference array over the recipe range.

Exercise 5.8 (medium). CSES *Subarray Sums I* — implement the telescoping window.

Exercise 5.9 (medium). CSES *Subarray Sums II* — prefix + `map` counting.

Exercise 5.10 (medium). CSES *Subarray Divisibility* — residue buckets mod n.

Exercise 5.11 (medium). USACO-S *Subsequences Summing to Sevens* — first/last prefix per residue.

Exercise 5.12 (medium). SPOJ *Haybale Stacking* — difference array then read off.

Exercise 5.13 (medium). CSES *Distinct Values Subarrays* — sliding window + last-seen.

Exercise 5.14 (hard). USACO-S *Rectangular Pasture* — 2D prefix over a sparse grid.

Exercise 5.15 (hard). USACO-G *Haybale Feast* — window + monotone queue.

Exercise 5.16 (hard). CF *Queries for Number of Palindromes* — a prefix table with a cleverer cell content.

Notes

The prefix/difference pair and the telescoping window follow Sannemo's array-techniques sections [1] and Laaksonen's range-query chapter [2]; the USACO Guide's Silver prefix-sum and two-pointer modules [3] are the direct drill source, and Halim [7] covers the same machinery with more worked IDs. The five archive problems are official CSES/USACO tasks [4]; the remainder-counting trick appears throughout the cp-algorithms notes [5].

Further reading

- USACO Guide, "Prefix Sums" (intermediate). `https://usaco.guide/silver/prefix-sums` — 1-D and 2-D prefix sums with more graded problems at exactly this chapter's level.
- USACO Guide, "Two Pointers" (intermediate). `https://usaco.guide/silver/two-pointers` — the window idiom of §5.4 with a deeper problem bank.
- Laaksonen, *Competitive Programmer's Handbook*, ch. 8–9 (beginner). `https://cses.fi/book/book.pdf` — prefix sums and the sliding-window minimum-sum problems in one short read.
- CSES 1646 *Static Range Sum Queries* (practice). `https://cses.fi/problemset/task/1646` — the direct archive form of 5.1.
- CSES 1660 *Subarray Sums I* and 1644 *Maximum Subarray Sum II* (practice). `https://cses.fi/problemset/task/1660` — live exercises for §5.4 and §5.5.

WATCH INSTEAD

Best first. "Prefix Sum Array and Range Sum Queries" — Profound Academy, 8 min — https://www.youtube.com/watch?v=xbYr9JOC2Lk. The accumulation trick of §5.1 animated cell by cell; uses `sum[i]` where the book writes p_i — same array. "Two Pointers in 7 minutes" — AlgoMasterIO, 7 min — https://www.youtube.com/watch?v=QzZ7nmouLTI. The shrink-on-violation loop of §5.4 on interview-style problems; Python-flavoured, the pointer invariant is identical. Access date 2026-10-10.

CHAPTER 6

Amortised structures: stacks, deques, sparse tables

Chapter 5's prefix tables answer questions about an array that never changes. This chapter is about structures whose cost is invisible per-operation but bounded per-*element*: a monotonic stack that finds every position's nearest smaller neighbour in one pass, a deque that answers sliding-window minima in $O(n)$, a sparse table that answers static range minimums in $O(1)$ after an $O(n \log n)$ build, and a square-root split that trades theory for simplicity. The shared idea is amortisation — charge each index once for all the work it will ever cause, and the total is linear no matter how expensive a single step looks. After this chapter you will recognise the pop-everything-until pattern, argue its bound by counting pushes, and pick the right static-range structure for the operation's algebra.

6.1 Amortised analysis: each element pays once

A worst-case bound per operation can be badly pessimistic for a whole sequence: if each array index is pushed onto a stack at most once and popped at most once, then n operations cost at most $2n$ stack touches — a single push can trigger many pops, but those pops were *pre-paid* by earlier pushes. This accounting view (charge the element, not the call) is the licence for every structure in this chapter: where a per-call bound would say $O(n^2)$, the per-element bound says $O(n)$.

6.2 The monotonic stack: nearest smaller everywhere

Keep a stack of indices whose values are strictly increasing. When position i arrives, pop every index whose value is $\geq a_i$ — they can never again be the *nearest* smaller answer, because a_i is both smaller and closer. The surviving top is i's answer; push i. Every index enters once and leaves once, so the whole pass is $O(n)$ amortised (Figure 6.1).

Listing 6.1: `monostack.h`: nearest smaller to the left, and mirrored right.

```
// monostack.h -- monotonic stack for nearest-smaller queries.
// Invariant: the stack holds indices with strictly increasing values;
// each index is pushed once and popped once -> O(n) amortized.
#pragma once
#include <stack>
#include <vector>
```

```

// ans[i] = index of nearest position LEFT of i with a smaller value,
// or -1. Swap the comparison for "nearest greater" or right-side scans.
template <class T>
std::vector<int> nearest_smaller_left(const std::vector<T>& a) {
    int n = a.size();
    std::vector<int> ans(n, -1);
    std::stack<int> st;
    for (int i = 0; i < n; i++) {
        while (!st.empty() && a[st.top()] >= a[i]) st.pop();
        ans[i] = st.empty() ? -1 : st.top();
        st.push(i);
    }
    return ans;
}

template <class T>
std::vector<int> nearest_smaller_right(const std::vector<T>& a) {
    int n = a.size();
    std::vector<int> ans(n, n);
    std::stack<int> st;
    for (int i = n - 1; i >= 0; i--) {
        while (!st.empty() && a[st.top()] >= a[i]) st.pop();
        ans[i] = st.empty() ? n : st.top();
        st.push(i);
    }
    return ans;
}
```

Example 6.1 (CSES *Nearest Smaller Values*)**.** For each position, the nearest earlier position with a smaller value — $n \le 2 \cdot 10^5$.

Strategy. Per-position search is $O(n^2)$. The amortised insight: once a_i undercuts an earlier entry, that entry is dead for every later query — delete it on sight. What remains is exactly the monotone stack.

Solution.

```
// nsv.cpp -- CSES Nearest Smaller Values (1-indexed output, 0 if none).
#include <iostream>
#include <vector>
#include "monostack.h"
using namespace std;

int main() {
    int n;
    cin >> n;
    vector<long long> a(n);
    for (auto &x : a) cin >> x;
    auto L = nearest_smaller_left(a);
```

```
    for (int i = 0; i < n; i++)
       cout << L[i] + 1 << " \n"[i + 1 == n];
}
```

Real output (CSES sample $\{2, 5, 1, 4, 8, 3, 2, 5\}$):

```
0 1 0 3 4 3 3 7
```

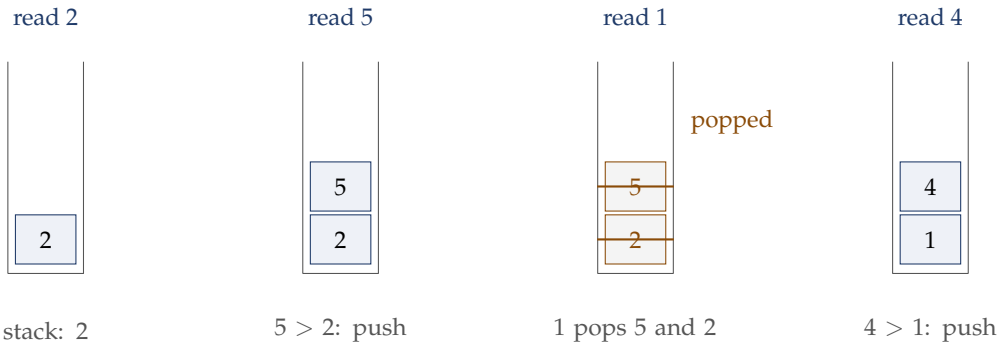


Figure 6.1: The stack pop trace on $\{2, 5, 1, 4\}$. Notice the popped entries are *gone*, not skipped: 1 is closer and smaller than either, so both can never answer again.

Example 6.2 (Largest rectangle in a histogram)**.** Heights $h_0, \ldots, h_{n-1}$; find the largest rectangle that fits under the skyline.

Strategy. Every candidate rectangle has a *limiting bar* — its shortest bar — so try each bar as the limiter: how far can bar i extend? Exactly until the first smaller bar on each side. Nearest-smaller to left and right is two monotonic-stack passes.

Solution.

```
// histogram.cpp -- largest rectangle: bar i extends to the first
// smaller bar on each side; area = h_i * (R_i - L_i - 1).
#include <iostream>
#include <vector>
#include "monostack.h"
using namespace std;
using ll = long long;

int main() {
   int n;
   cin >> n;
   vector<ll> h(n);
   for (auto &x : h) cin >> x;
   auto L = nearest_smaller_left(h);
   auto R = nearest_smaller_right(h);
   ll best = 0; int bi = -1;
   for (int i = 0; i < n; i++) {
      ll area = h[i] * (R[i] - L[i] - 1);
      if (area > best) { best = area; bi = i; }
   }
```

```
    cout << "best area " << best << " from bar " << bi
         << " (height " << h[bi] << ", spans [" << L[bi] + 1
         << "," << R[bi] - 1 << "])\n";
}
```

Real output (heights $\{2,1,5,6,2,3\}$ — bar 2's rays reach the first smaller bars, Figure 6.2):

```
best area 10 from bar 2 (height 5, spans [2,3])
```

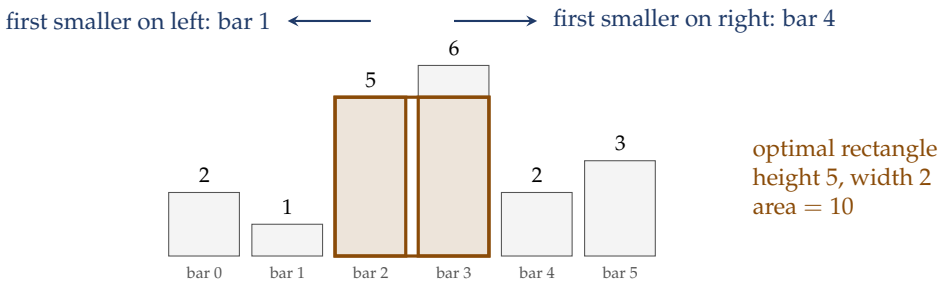


Figure 6.2: Bar 2's horizontal reach ends at the first smaller bar on each side. Notice the optimal rectangle is pinned by *one* limiting bar — which is why trying every bar suffices.

Exercise 6.1 (easy). The pop loop inside the push is not $O(1)$ — one push may pop k entries. Argue in one sentence why the total over n pushes is still $O(n)$.

6.3 The monotonic deque: sliding-window minimum

The same pop-until pattern slides a window: the deque keeps candidate indices in increasing value order, front = current minimum. New index i pops every back entry with $a_{\text{back}} \geq a_i$ (same dead-on-arrival argument), then the front drops when it leaves the window. Each index enters and exits once: $O(n)$ for all windows.

Listing 6.2: `monodeque.h`: window minimums in $O(n)$.

```
// monodeque.h -- sliding-window minimum in O(n).
// The deque holds candidate indices, front = current window min.
// On entry i: pop indices with a[idx] >= a[i] (they can never win
// again), push i, drop the front once it leaves the window.
#pragma once
#include <deque>
#include <vector>

template <class T>
std::vector<T> sliding_min(const std::vector<T>& a, int k) {
    std::deque<int> dq;
    std::vector<T> out;
    for (int i = 0; i < (int)a.size(); i++) {
        while (!dq.empty() && a[dq.back()] >= a[i]) dq.pop_back();
        dq.push_back(i);
```

```
        if (dq.front() <= i - k) dq.pop_front();
        if (i >= k - 1) out.push_back(a[dq.front()]);
    }
    return out;
}
```

Example 6.3 (CSES *Sliding Window Minimum*)**.** Window size k over a generated array up to $n = 10^7$; report the xor of all window minimums.

Strategy. $O(nk)$ is impossible at 10^7. The deque is the only structure that retires entries *both* by value (a newer, smaller element wins) and by age (the window forgets) — and it is linear.

Solution.

```
// sliding_min.cpp -- CSES Sliding Window Minimum (generated input, xor
// of window minimums).
#include <iostream>
#include <vector>
#include "monodeque.h"
using namespace std;
using ll = long long;

int main() {
    ll n, k, x, a, b, c;
    cin >> n >> k >> x >> a >> b >> c;
    vector<ll> arr(n);
    arr[0] = x;
    for (int i = 1; i < n; i++) arr[i] = (a * arr[i - 1] + b) % c;
    auto mins = sliding_min(arr, (int)k);
    ll acc = 0;
    for (ll v : mins) acc ^= v;
    cout << acc << "\n";
}
```

Real output (CSES sample, generator $x{=}3, a{=}7, b{=}1, c{=}11$, array $\{3,0,1,8,2,4,7,6\}$, $k{=}5$, minimums $0,0,1,2$):

```
3
```

Exercise 6.2 (medium)**.** Why does the sliding window need a *deque* rather than the monotonic stack of Section 6.2? Which operation does the window force that the stack never needed?

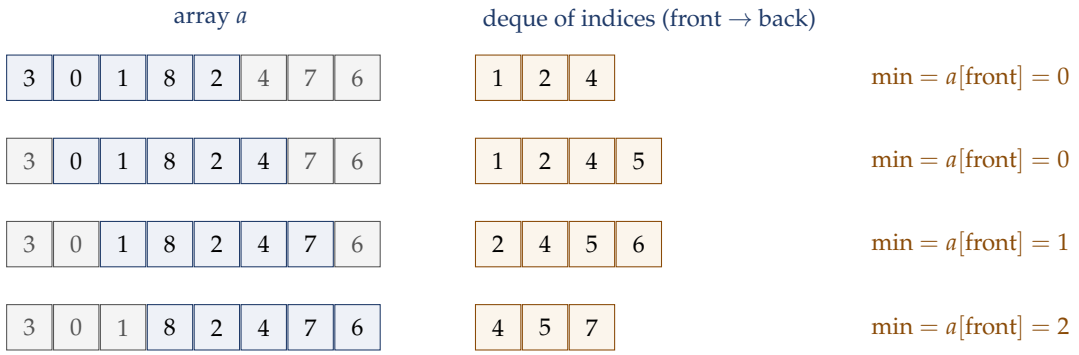


Figure 6.3: The deque's index march on the CSES sample. Notice expired indices leave through the front while dominated ones leave through the back — two retirements, one structure.

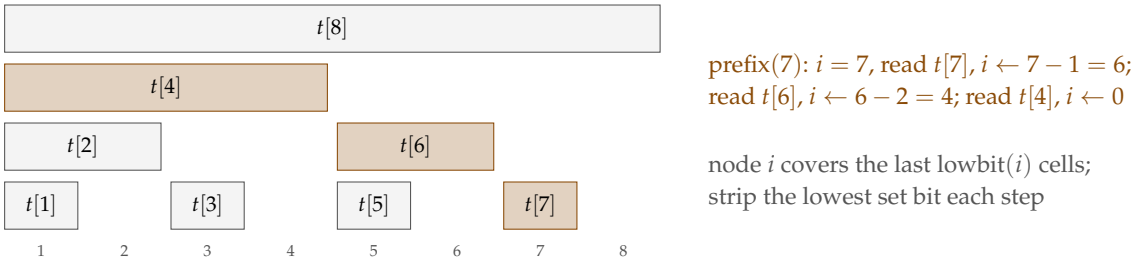


Figure 6.4: Fenwick decomposition: prefix(7) touches nodes $7, 6, 4$ — strip the lowbit each step.

6.4 Sparse tables: idempotent queries in $O(1)$

For a *static* array, a sparse table precomputes the answer on every interval of length 2^j: level j holds $n - 2^j + 1$ entries, each built from two level-$(j-1)$ halves, $O(n \log n)$ total. Any range $[l, r]$ is the union of two overlapping power-of-two pieces anchored at l and r — correct precisely when the operation is *idempotent* ($f(x, x) = x$: min, max, gcd, bitwise AND), because the overlap is then double-counted harmlessly. A sum double-counts: sparse tables are the wrong tool for range sums.

Listing 6.3: `sparse.h`: $O(n \log n)$ build, $O(1)$ query.

```
// sparse.h -- sparse table for idempotent range queries (min/max/gcd)
// O(n log n) build, O(1) query via two overlapping levels.
// Requires the operation satisfy f(f(x)) = f(x) -- overlapping pieces
// may double-count nothing.
#pragma once
#include <algorithm>
#include <vector>

template <class T, class F>
struct SparseTable {
    std::vector<std::vector<T>> st;
    F f;
    SparseTable(const std::vector<T>& a, F f) : f(f) {
        int n = a.size(), L = 32 - __builtin_clz(n);
```

```
        st.assign(L, std::vector<T>(n));
        st[0] = a;
        for (int j = 1; j < L; j++)
            for (int i = 0; i + (1 << j) <= n; i++)
                st[j][i] = f(st[j - 1][i], st[j - 1][i + (1 << (j - 1))]);
    }
    T query(int l, int r) const { // inclusive [l, r]
        int j = 31 - __builtin_clz(r - l + 1);
        return f(st[j][l], st[j][r - (1 << j) + 1]);
    }
};
```

Example 6.4 (CSES *Static Range Minimum Queries*)**.** Same shape as Chapter 5's sum queries, but minimum: $n, q \leq 2 \cdot 10^5$.

Strategy. Prefix tricks fail — min has no inverse to subtract with. But min *is* idempotent, so two overlapping precomputed halves cover any range exactly: sparse table.

Solution.

```
// rmq.cpp -- CSES Static Range Minimum Queries via sparse table.
#include <algorithm>
#include <iostream>
#include <vector>
#include "sparse.h"
using namespace std;

int main() {
    int n, q;
    cin >> n >> q;
    vector<long long> a(n);
    for (auto &x : a) cin >> x;
    SparseTable st(a, [](long long u, long long v) { return min(u, v);
        });
    while (q--) {
        int l, r;
        cin >> l >> r;
        cout << st.query(l - 1, r - 1) << "\n";
    }
}
```

Real output (CSES sample):

```
2
1
1
4
```

Example 6.5 (CSES *Josephus Problem I*: the queue does the rotating)**.** Children 1..n stand in a circle; walking clockwise, every second remaining child leaves. Print the removal order ($n \leq 2 \cdot 10^5$). *Input:* `7`. *Output:* `2 4 6 1 5 3 7`.

Strategy. Simulate the circle directly with the structure built for it: a `queue`. Pop the front child; if the step is "skip", push him to the back (he survives this round); if "eliminate", print and drop. Every queue operation is $O(1)$ and each of the n children is pushed at most n times across the whole run — amortized linear work, the theme of this chapter applied to a data structure you already own. A `vector` version needs an index walk and wrap-around; the queue version needs none of it because the circle *is* the enqueue/dequeue discipline.

Solution. `code/josephus1.cpp` prints `2 4 6 1 5 3 7` on the sample: odd positions leave first in order, then the rotation re-eliminates among survivors. No arithmetic beyond a toggle flag — the amortized cost of "rotate and inspect" is the point.

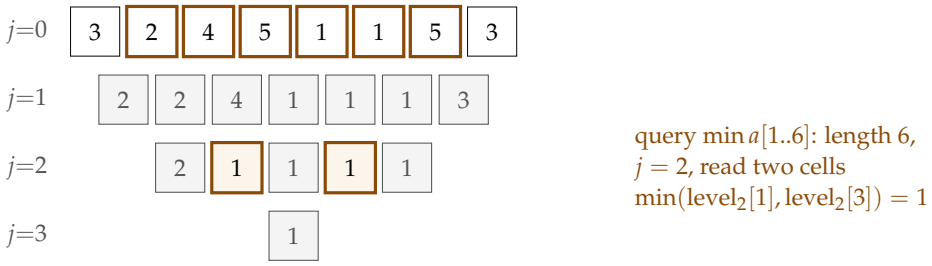


Figure 6.5: Sparse-table levels over $\{3,2,4,5,1,1,5,3\}$. Notice level j's i-th cell already contains the answer for the whole 2^j-block starting at i — a query is two reads, never more.

Exercise 6.3 (medium). Which of $\{\min, \max, \gcd, \text{sum}, \text{xor}, \text{lcm}\}$ are idempotent — i.e., safe for the sparse table's two-piece query? For each unsafe one, name the query where overlap double-counts.

6.5 Square roots: the good-enough structure

When the array *does* change, split it into $B \approx \sqrt{n}$ blocks with a stored sum per block. A range sum reads whole blocks in $O(1)$ each and scans at most two partial fringes: $O(\sqrt{n})$ per query, $O(1)$ per point update. That is worse than a segment tree's $O(\log n)$ — and often the right trade anyway: the code is a page of loops with no recursion, no trees, and no subtle bugs at 2 a.m.

Listing 6.4: `sqrtdec.h`: block sums plus point updates.

```
// sqrtdec.h -- square-root decomposition: block sums + point updates.
// Query: O(#blocks + #fringe) = O(sqrt n). Update a point: O(1).
#pragma once
#include <cmath>
#include <vector>
using ll = long long;

struct SqrtDec {
    int n, B;
    std::vector<ll> a, blk;
```

```
    explicit SqrtDec(std::vector<ll> v) : n(v.size()), a(std::move(v))
        {
       B = (int)std::sqrt(n) + 1;
       blk.assign(B + 1, 0);
       for (int i = 0; i < n; i++) blk[i / B] += a[i];
    }
    void set(int i, ll v) { blk[i / B] += v - a[i]; a[i] = v; }
    ll sum(int l, int r) const { // inclusive [l, r]
       ll s = 0;
       int bl = l / B, br = r / B;
       if (bl == br)
          for (int i = l; i <= r; i++) s += a[i];
       else {
          for (int i = l; i / B == bl; i++) s += a[i];
          for (int b = bl + 1; b < br; b++) s += blk[b];
          for (int i = br * B; i <= r; i++) s += a[i];
       }
       return s;
    }
};
```

```
sum[2..6] = 21
after set(4,0): sum[2..6] = 16
sum[0..9] = 34
```

Listing 6.5: `sqrt_sum.cpp` real output.

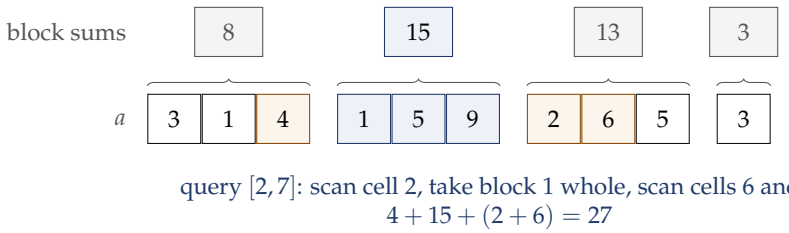


Figure 6.6: Block sums over 3-blocks. Notice a query touches at most two scanned fringes; everything in between is one table read per block.

6.6 Chapter summary

6.7 Common mistakes

- **Sparse table on a non-idempotent op.** Two overlapping pieces are only safe when $f(x, x) = x$; a sparse table for range sum double-counts the overlap and silently returns wrong answers.
- **Forgetting the expiry check.** The deque's front must be evicted when it leaves the window — separate from the domination pop; skipping it returns the minimum of a window that no longer exists.

Structure	Answers	Cost	Needs
monotonic stack	nearest smaller/greater	$O(n)$ total	one pass
monotonic deque	sliding-window min/max	$O(n)$	window slides
sparse table	static range min/max/gcd	$O(1)$ query	idempotent op
sqrt blocks	range sum + updates	$O(\sqrt{n})$	simplicity

Table 6.1: All four are the same accounting: charge the element, not the call — and read the operation's algebra before choosing.

- **Counting pops per call.** Amortised bounds are proven on the *sequence*, not the operation: "one push can pop k" is fine because each index is popped at most once ever.
- **Strictness flips.** $\geq$ vs $>$ in the pop condition decides whether equal values count as "smaller-or-equal neighbours"; pick per the statement and test on duplicates.
- **Block-size blind spots.** The fringe scan is $O(B)$ — queries shorter than B inside one block must take the scalar path (handled by the $bl = br$ branch); skipping it sums whole blocks for a two-cell query.

6.8 Additional exercises

Exercise 6.4 (easy)**.** CSES *Towers* — longest non-increasing run per cube (`upper_bound` or monotonic multiset).

Exercise 6.5 (easy)**.** USACO-S *Convention II* — sorted queue mechanics.

Exercise 6.6 (easy)**.** LC *IPO* — two heaps as amortised selection.

Exercise 6.7 (medium)**.** CSES *Traffic Lights* — ordered set of segments.

Exercise 6.8 (medium)**.** CSES *Josephus Problem II* — order-statistics elimination.

Exercise 6.9 (medium)**.** USACO-S *Milk Measurement* — ordered leaderboard updates.

Exercise 6.10 (medium)**.** CF *William and Robot* — greedy selection structure.

Exercise 6.11 (easy)**.** CSES *Playlist* — the sliding window with a set membership test.

Exercise 6.12 (easy)**.** CSES *Range Xor Queries* — xor prefix; why does the same trick work?

Exercise 6.13 (medium)**.** Derive the amortized bound for a queue built from two stacks: assign each element a potential.

Exercise 6.14 (medium)**.** Implement sliding-window minimum with a plain array + head index (no `deque`); verify against the deque version.

Exercise 6.15 (hard)**.** CSES *Stick Division* — heap-of-two-smallest merges (Huffman).

Exercise 6.16 (hard)**.** USACO-S *Deforestation* — range constraints + structure choice.

Exercise 6.17 (hard)**.** USACO-G *Job Completion* — multi-structure scheduling.

Notes

The amortised-accounting framing follows Sannemo's structures sections [1] and Laaksonen's data-structures chapter [2]; the USACO Guide's Silver/Gold modules carry the monotonic-stack and deque drills [3], and cp-algorithms documents the sparse table's idempotency requirement [5]. Halim [7] covers the same set with contest IDs; the archive problems are official CSES/USACO tasks [4].

Exercise 6.18 (hard)**.** USACO-G *Haybale Feast* — monotonic deque with a sum constraint.

Further reading

- cp-algorithms, "Sparse table" (intermediate) — the static RMQ structure of §6.4 with disjoint-range queries beyond min.
  https://cp-algorithms.com/data_structures/sparse-table.html
- cp-algorithms, "Modifying stack/queue for minimum queries" (intermediate). (article: "Minimum stack / minimum queue) — the monotonic queue of §6.3 derived twice over (stack-pair and deque).
  https://cp-algorithms.com/
- Laaksonen, *Competitive Programmer's Handbook*, ch. 8 "Amortized analysis" (beginner). — the accounting argument behind §6.1.
  https://cses.fi/book/book.pdf
- CSES 1645 *Nearest Smaller Values* (practice) — 6.1 verbatim; the canonical monotonic-stack submission.
  https://cses.fi/problemset/task/1645
- CSES 1647 *Static Range Minimum Queries* (practice) — where the sparse table of 13.2 earns its keep.
  https://cses.fi/problemset/task/1647
- Wikipedia, "Amortized analysis" (reference) — aggregate / accounting / potential methods; the book uses the aggregate version.
  https://en.wikipedia.org/wiki/Amortized_analysis

WATCH INSTEAD

Best first. "Monotonic Stack Data Structure Explained" — AlgoMonster, 6 min — https://youtu.be/Dq_ObZwTY_Q. The pop-while-monotone loop of §6.2 on next-greater-element; pseudocode-style, no language dependency. "Sparse Table | Range Minimum Query in O(1)" — Kartik Arora, 17 min — https://youtu.be/iaRvydtqLV4. Builds exactly the k-level table of §6.4 then answers $[\ell, r]$ via the two overlapping power-of-2 blocks; uses $\lfloor \log_2(r - \ell + 1) \rfloor$ like the book. Access date 2026-10-10.

CHAPTER 7

Graphs: traversal and order

Every graph problem in this book reduces to one question first: what order do I visit the vertices? Depth-first search dives until stuck and unrolls the answer on the way back; breadth-first search expands in layers and thereby measures unweighted distance for free. Built on them: flood fill for grids, two-colouring for bipartite checks, Kahn's algorithm for ordering a DAG, and the tail-plus-cycle anatomy of functional graphs. After this chapter you will choose the traversal that makes the needed invariant fall out, write the adjacency-list idiom without thinking, and reconstruct answers — paths, components, orders — from the arrays the search leaves behind.

7.1 Vocabulary

A graph $G = (V, E)$ is vertices plus edges: undirected pairs $\{u, v\}$, or directed arcs (u, v), possibly weighted. Degree counts incident edges. A *path* revisits nothing; a *cycle* returns; connected pieces are *components*. Directed graphs get their own adjectives: a DAG is acyclic; functional graphs have out-degree exactly 1.

7.2 Representations: adjacency lists only

The contest representation is `vector<vector<int>> g(n+1)`: memory $O(V+E)$, iteration $O(\deg v)$. Adjacency *matrices* cost $O(V^2)$ and are only for dense tiny graphs. Three traps: self-loops and multi-edges are legal input (your code must not assume otherwise); CSES vertices are 1-indexed (size the array $n+1$); and undirected edges are added *both* ways.

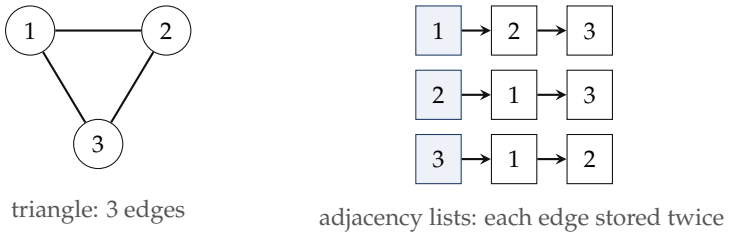


Figure 7.1: The same triangle two ways. Notice the lists store each edge twice — that's the price of $O(\deg v)$ iteration.

7.3 Depth-first search

DFS recurses into the first unseen neighbour; the visited set makes each vertex processed once, so a full pass is $O(V+E)$. Free applications: component counting (one outer call per component), reachability, cycle detection by back-edge, and — on grids — flood fill.

Listing 7.1: `dfs.h`: the five-line idiom.

```
// dfs.h -- recursive DFS with visited set. Depth = component size;
// see tripwire on grid recursion (use iterative flood fill).
#pragma once
#include <vector>

inline void dfs(const std::vector<std::vector<int>>& g, int v,
                std::vector<char>& seen) {
   seen[v] = 1;
   for (int w : g[v])
      if (!seen[w]) dfs(g, w, seen);
}
```

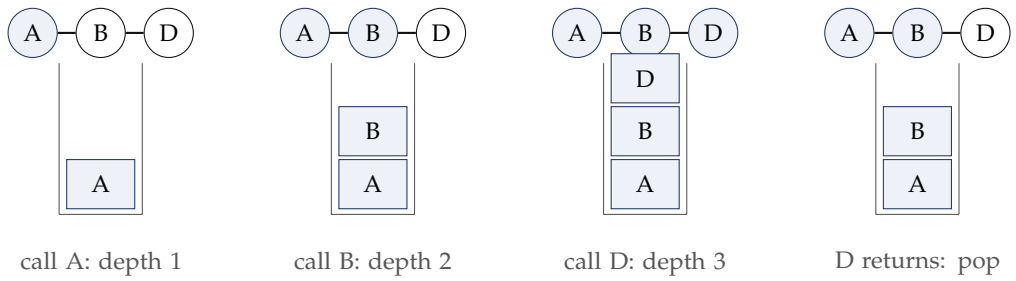


Figure 7.2: The recursion stack growing and unwinding. Notice vertices *stay* on the stack while their subtree is explored — that resident set is what makes back-edge detection work.

7.4 Breadth-first search

BFS dequeues in arrival order, so vertices surface in nondecreasing distance: `dist[]` is the true unweighted shortest path, and `parent[]` reconstructs it backwards.

Listing 7.2: `bfs.h`: dist + parent, the reconstruction pair.

```
// bfs.h -- breadth-first search: layers = unweighted distance.
// dist[] = shortest edge count from s (-1 unreachable); parent[] for
// path reconstruction.
#pragma once
#include <queue>
#include <vector>

struct BFS {
   std::vector<int> dist, parent;
   BFS(const std::vector<std::vector<int>>& g, int s)
```

```
        : dist(g.size(), -1), parent(g.size(), -1) {
        std::queue<int> q;
        dist[s] = 0; q.push(s);
        while (!q.empty()) {
            int v = q.front(); q.pop();
            for (int w : g[v])
                if (dist[w] < 0) {
                    dist[w] = dist[v] + 1; parent[w] = v; q.push(w);
                }
        }
    }
};
```

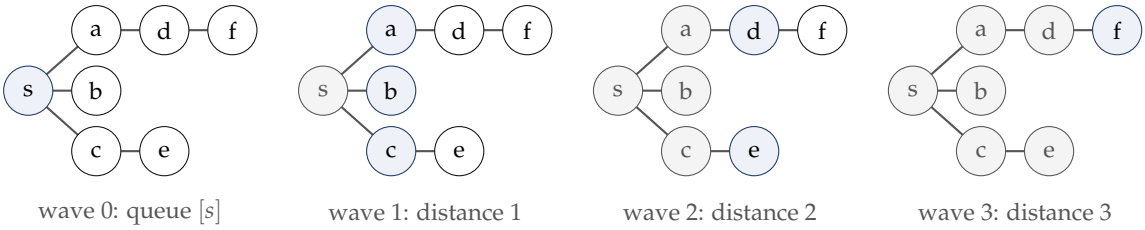


Figure 7.3: BFS as expanding waves. Notice every vertex in wave k is exactly distance k — the queue order *is* the distance order.

Example 7.1 (CSES *Labyrinth*)**.** Shortest path through a grid with walls; output the path itself.

Strategy. "Shortest" and "unweighted" together name BFS. The path is recovered by walking `parent` from the goal — search forward, reconstruct backward.

Solution.

```
// labyrinth.cpp -- CSES Labyrinth: BFS + parent reconstruction.
#include <iostream>
#include <queue>
#include <string>
#include <algorithm>
#include <vector>
using namespace std;

int main() {
    int n, m;
    cin >> n >> m;
    vector<string> g(n);
    int ar = -1, ac = -1, br = -1, bc = -1;
    for (int r = 0; r < n; r++) {
        cin >> g[r];
        for (int c = 0; c < m; c++) {
            if (g[r][c] == 'A') { ar = r; ac = c; }
            if (g[r][c] == 'B') { br = r; bc = c; }
        }
    }
    vector<vector<int>> dist(n, vector<int>(m, -1));
```

```
    vector<vector<pair<int,int>>> par(n, vector<pair<int,int>>(m,
        {-1,-1}));
    const int dr[4] = {0, 0, -1, 1}, dc[4] = {-1, 1, 0, 0};
    const char mv[4] = {'L', 'R', 'U', 'D'};
    queue<pair<int,int>> q;
    dist[ar][ac] = 0; q.push({ar, ac});
    while (!q.empty()) {
        auto [r, c] = q.front(); q.pop();
        for (int k = 0; k < 4; k++) {
            int nr = r + dr[k], nc = c + dc[k];
            if (nr < 0 || nr >= n || nc < 0 || nc >= m) continue;
            if (g[nr][nc] == '#' || dist[nr][nc] >= 0) continue;
            dist[nr][nc] = dist[r][c] + 1;
            par[nr][nc] = {r, c};
            q.push({nr, nc});
        }
    }
    if (dist[br][bc] < 0) { cout << "NO\n"; return 0; }
    cout << "YES\n" << dist[br][bc] << "\n";
    string path;
    for (int r = br, c = bc; par[r][c].first >= 0; ) {
        auto [pr, pc] = par[r][c];
        for (int k = 0; k < 4; k++)
            if (pr + dr[k] == r && pc + dc[k] == c) path += mv[k];
        r = pr; c = pc;
    }
    reverse(path.begin(), path.end());
    cout << path << "\n";
}
```

Real output (CSES sample):

```
YES
9
LDDRRRRRU
```

7.5 Flood fill

A grid is an implicit graph: cells are vertices, the four directions are edges. Flood fill counts components — and uses an *explicit* stack, because a 1000×1000 room is a million frames deep.

Listing 7.3: `floodfill.h`: iterative, 4-direction, marks by rewriting the grid.

```
// floodfill.h -- iterative 4-direction flood fill on a grid.
// Explicit stack: recursion depth here can reach n*m.
#pragma once
#include <stack>
#include <string>
```

```
#include <utility>
#include <vector>

inline int count_rooms(std::vector<std::string> grid) {
   int n = grid.size(), m = grid[0].size(), rooms = 0;
   const int dr[4] = {-1, 1, 0, 0}, dc[4] = {0, 0, -1, 1};
   for (int r = 0; r < n; r++)
      for (int c = 0; c < m; c++) {
         if (grid[r][c] != '.') continue;
         rooms++;
         std::stack<std::pair<int,int>> st;
         st.push({r, c});
         std::vector<std::vector<char>> seen(n, std::vector<char>(m));
         seen[r][c] = 1;
         while (!st.empty()) {
            auto [x, y] = st.top(); st.pop();
            grid[x][y] = '#';
            for (int k = 0; k < 4; k++) {
               int nx = x + dr[k], ny = y + dc[k];
               if (nx >= 0 && nx < n && ny >= 0 && ny < m
                  && grid[nx][ny] == '.' && !seen[nx][ny]) {
                  seen[nx][ny] = 1;
                  st.push({nx, ny});
               }
            }
         }
      }
   return rooms;
}
```

Example 7.2 (CSES *Counting Rooms*)**.** Count floor-connected regions.

Strategy. Every unvisited floor cell starts a new room; fill it completely before moving on. Component counting is the outer loop — the fill is just DFS wearing grid clothes.

Solution.

```
// counting_rooms.cpp -- CSES Counting Rooms (iterative flood fill).
#include <iostream>
#include <string>
#include <vector>
#include "floodfill.h"
using namespace std;

int main() {
   int n, m;
   cin >> n >> m;
   vector<string> g(n);
   for (auto& row : g) cin >> row;
   cout << count_rooms(g) << "\n";
}
```

Real output (CSES sample):

```
3
```

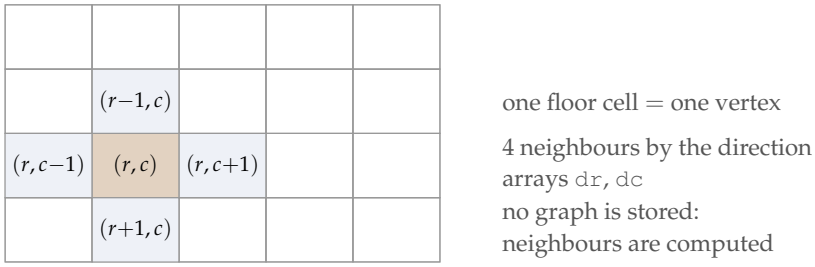


Figure 7.4: One cell, four edges. Notice there is no graph object — the neighbours are computed, never stored.

7.6 Bipartite checks

Colour as you traverse: every neighbour gets the opposite colour, and the first conflict certifies an odd cycle. If a pass completes, the colouring itself is the proof.

Example 7.3 (CSES *Building Teams*)**.** Split pupils so friends land on different teams.

Strategy. "Two groups, no inside edges" is bipartiteness rephrased; BFS two-colouring either emits the split or meets its own edge and prints IMPOSSIBLE.

Solution.

```cpp
// building_teams.cpp -- CSES Building Teams: bipartite 2-colouring.
#include <iostream>
#include <queue>
#include <vector>
using namespace std;

int main() {
   int n, m;
   cin >> n >> m;
   vector<vector<int>> g(n + 1);
   for (int i = 0; i < m; i++) {
      int a, b; cin >> a >> b;
      g[a].push_back(b); g[b].push_back(a);
   }
   vector<int> col(n + 1, 0);
   bool ok = true;
   for (int s = 1; s <= n && ok; s++) {
      if (col[s]) continue;
      col[s] = 1;
      queue<int> q; q.push(s);
      while (!q.empty() && ok) {
         int v = q.front(); q.pop();
         for (int w : g[v]) {
```

```
            if (col[w] == col[v]) { ok = false; break; }
            if (!col[w]) { col[w] = 3 - col[v]; q.push(w); }
         }
      }
   }
   if (!ok) { cout << "IMPOSSIBLE\n"; return 0; }
   for (int v = 1; v <= n; v++)
      cout << col[v] << " \n"[v == n];
}
```

Real output (CSES sample):

```
1 2 2 1 2
```

7.7 Directed order: Kahn and the DAG

A DAG admits an order where every edge points forward. Kahn's algorithm: emit in-degree-0 vertices, decrement their out-neighbours, repeat. If the queue empties with vertices left, they are exactly the cycle-entangled ones — the algorithm detects cycles for free.

Listing 7.4: `toposort.h`: Kahn; short order = cycle.

```
// toposort.h -- Kahn's algorithm: repeatedly emit in-degree-0
// vertices. Leftover vertices <=> a directed cycle remains.
#pragma once
#include <queue>
#include <vector>

inline std::vector<int> kahn(const std::vector<std::vector<int>>& g) {
   int n = g.size();
   std::vector<int> deg(n), order;
   for (auto& nb : g) for (int w : nb) deg[w]++;
   std::queue<int> q;
   for (int v = 1; v < n; v++) if (deg[v] == 0) q.push(v);
   while (!q.empty()) {
      int v = q.front(); q.pop();
      order.push_back(v);
      for (int w : g[v]) if (--deg[w] == 0) q.push(w);
   }
   return order; // size < n-1 iff cycle exists (index 0 unused)
}
```

Example 7.4 (CSES *Course Schedule*)**.** Order n courses satisfying prerequisites.

Strategy. Prerequisite edges are a DAG exactly when the task is possible; Kahn emits the order or exposes the cycle.

Solution.

```cpp
// course_schedule.cpp -- CSES Course Schedule: Kahn topological order.

#include <iostream>
#include <vector>
#include "toposort.h"
using namespace std;

int main() {
   int n, m;
   cin >> n >> m;
   vector<vector<int>> g(n + 1);
   for (int i = 0; i < m; i++) {
      int a, b; cin >> a >> b;
      g[a].push_back(b);
   }
   auto order = kahn(g);
   if ((int)order.size() < n) { cout << "IMPOSSIBLE\n"; return 0; }
   for (int v : order) cout << v << " ";
   cout << "\n";
}
```

Real output (CSES sample, one valid order):

```
3 4 1 5 2
```

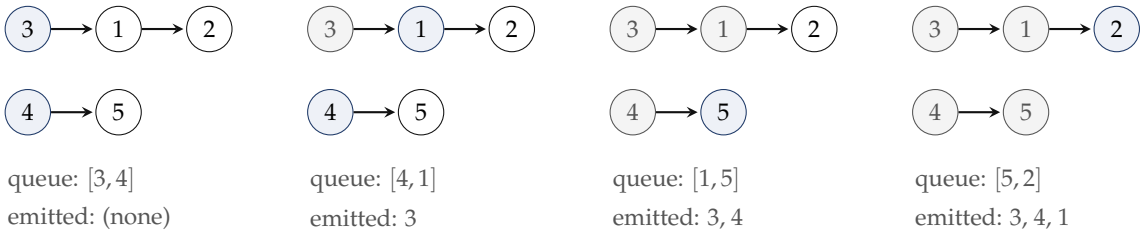


Figure 7.5: Kahn's queue evolution. Notice the emitted sequence is a *proof*: at print time each vertex's remaining in-degree is 0.

7.8 Functional graphs: tail plus cycle

Out-degree 1 means every walk is a tail into a cycle — the ρ shape of Figure 7.6. Walks are deterministic, so answers decompose: distance-to-cycle, then cycle length.

Example 7.5 (CSES *Planets Cycles*)**.** Teleportations until a planet is revisited, per start.

Strategy. From any start the trajectory is tail + cycle: the answer is tail-length + cycle-length. One pass per start, marking as we go, gives each vertex's answer in $O(n)$ total.

Solution.

```cpp
// planets_cycles.cpp -- CSES Planets Cycles.
// Functional graph: every walk is tail + cycle. If path [p0..pk-1]
// next enters vertex v: v already answered -> all of path is tail and
// ans[p_i] = (k-i) + ans[v]. Otherwise v = p_j, the cycle is
// p_j..p_{k-1} (length k-j): cycle members get k-j, tails get k-i.
#include <iostream>
#include <vector>
using namespace std;

int main() {
   int n;
   cin >> n;
   vector<int> t(n + 1), ans(n + 1, -1);
   vector<int> pos(n + 1, -1), inpath(n + 1, -1);
   for (int i = 1; i <= n; i++) cin >> t[i];
   for (int s = 1; s <= n; s++) {
      if (ans[s] >= 0) continue;
      vector<int> path;
      int v = s;
      while (ans[v] < 0 && inpath[v] != s) {
         inpath[v] = s;
         pos[v] = path.size();
         path.push_back(v);
         v = t[v];
      }
      int k = path.size();
      if (ans[v] >= 0) { // all tail
         for (int i = 0; i < k; i++) ans[path[i]] = k - i + ans[v];
      } else { // v = p_j: cycle found
         int j = pos[v], cyc = k - j;
         for (int i = j; i < k; i++) ans[path[i]] = cyc;
         for (int i = 0; i < j; i++) ans[path[i]] = k - i;
      }
   }
   for (int i = 1; i <= n; i++)
      cout << ans[i] << " \n"[i == n];
}
```

Real output (CSES sample $t = \{2, 4, 3, 1, 4\}$):

```
3 3 1 3 4
```

Exercise 7.1 (easy). The input may contain a self-loop (v, v) and the parallel edge $(u, v), (u, v)$. What does each do to your degree counts, BFS layering, and Kahn's in-degree array?

Exercise 7.2 (medium). On a path graph $1-2-\cdots-n$, state the visit orders of DFS and BFS from vertex 1. Where do they differ first on a 4-cycle?

Exercise 7.3 (medium). Kahn stops early exactly when the leftover subgraph has a directed cycle. Why can a nonempty acyclic leftover never happen?

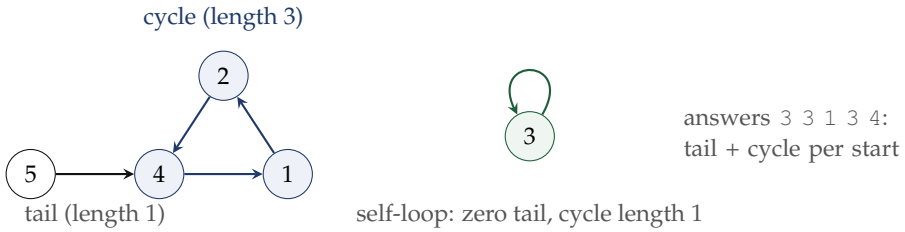


Figure 7.6: The ρ anatomy. Notice the tail can be empty — pure cycles and self-loops are the same shape with zero stem.

Exercise 7.4 (medium)**.** In a functional graph, show that following successors from any vertex eventually repeats — and that the first repeated vertex is always on the cycle, never mid-tail.

7.9 Chapter summary

Need	Tool	Cost	Leaves behind
reach a vertex	DFS	$O(V+E)$	components, order
shortest unweighted path	BFS	$O(V+E)$	dist, parent
count grid regions	flood fill	$O(nm)$	painted cells
two groups / odd cycle	2-colour BFS	$O(V+E)$	team array
order a DAG	Kahn	$O(V+E)$	topo order or cycle
successor structure	walk + stamp	$O(n)$	tail/cycle split

Table 7.1: Pick by the invariant you need: layers mean distance, the stack means depth, the queue's leftovers mean cycles.

7.10 Common mistakes

- **Recursive flood fill on big grids.** 10^6 cells is a million-deep call stack — iterative or explicit-stack only.
- **BFS on weighted edges.** Layers are edge *counts*; weights need Dijkstra (Ch 11), and BFS's first visit is no longer shortest.
- **Marking on enqueue vs dequeue.** Mark `seen` when *enqueuing*; marking on dequeue lets the same vertex enter the queue many times — still correct but quadratic on dense graphs.
- **1-indexed off-by-one.** CSES vertices are $1..n$; an n-sized vector drops vertex n silently.
- **Forgetting the second direction.** Undirected edges need two adjacency entries; a one-way insert makes the graph half-directed and every later claim quietly wrong.

7.11 Additional exercises

Exercise 7.5 (easy). CSES *Building Roads* — components + links.

Exercise 7.6 (easy). CSES *Message Route* — BFS reconstruction.

Exercise 7.7 (easy). USACO-S *Closing the Farm* — component queries.

Exercise 7.8 (easy). USACO-B *Family Tree* — ancestor DFS.

Exercise 7.9 (medium). CSES *Round Trip* — undirected cycle extraction.

Exercise 7.10 (medium). CSES *Game Routes* — DP on topological order.

Exercise 7.11 (medium). USACO-S *Icy Perimeter* — flood-fill perimeter.

Exercise 7.12 (medium). USACO-S *Build Gates* — scaled-cell fill.

Exercise 7.13 (medium). USACO-S *Mooyo Mooyo* — gravity + fill.

Exercise 7.14 (medium). USACO-S *Visits* — functional-graph simulation.

Exercise 7.15 (hard). CSES *Monsters* — multi-source BFS + escape BFS.

Exercise 7.16 (hard). USACO-B *The Great Revegetation* — signed 2-colouring.

Exercise 7.17 (hard). USACO-S *Snow Boots* — BFS on (position, boot) state.

Notes

Traversal order, the adjacency-list idiom and Kahn follow Laaksonen's graph chapters [2] and Sannemo [1]; the USACO Guide's Silver graph modules carry the same drills [3]; the rho-shape framing and Monsters-style multi-source tricks are in cp-algorithms [5] and Halim [7]; archive problems are official CSES/USACO [4].

Further reading

- USACO Guide, "Graph Traversal" (intermediate). `https://usaco.guide/silver/graph-traversal` — BFS/DFS with the same adjacency-list convention, more problems.
- USACO Guide, "Depth First Search" (intermediate). `https://usaco.guide/silver/dfs` — the recursive form of §7.3 plus flood-fill on grids.
- USACO Guide, "Functional Graphs" (intermediate). `https://usaco.guide/silver/func-graphs` — the successor-graph machinery behind §7.8 (7.5, 7.3).
- Laaksonen, *Competitive Programmer's Handbook*, ch. 11–12 "Basics of graphs"/"Graph traversal" (beginner). `https://cses.fi/book/book.pdf` — the whole chapter's vocabulary in forty pages.
- CSES 1192 *Counting Rooms* and 1193 *Labyrinth* (practice). `https://cses.fi/problemset/task/1192` — 7.2 and 7.1 verbatim.
- CSES 1667 *Message Route* and 1679 *Course Schedule* (practice). `https://cses.fi/problemset/task/1667` — parent pointers and Kahn's order as live submissions.

WATCH INSTEAD

Best first. "Lecture 13: Breadth-First Search (BFS)" — MIT OpenCourseWare, 51 min — https://www.youtube.com/watch?v=s-CYnVz-uh4. Levels, parent pointers, and the unweighted-shortest-path theorem behind §7.4; OCW draws layers as "level sets", same idea as the book's frontier. "Lecture 14: Depth-First Search (DFS), Topological Sort" — MIT OpenCourseWare, 51 min — https://www.youtube.com/watch?v=AfSk24UTFS8. Edge classification and the reverse-finish topological order of §7.7 (the book shows Kahn's algorithm — both orders are correct; the lecture's is DFS-based). Access date 2026-10-10.

CHAPTER 8

Shortest paths

Four algorithms cover every shortest-path problem you will meet: BFS when edges are unweighted, Dijkstra when weights are non-negative, Bellman–Ford when weights can go negative, and Floyd–Warshall when you need every pair at once. Each is built from one operation — relaxation — and each is right exactly when its invariant holds; Dijkstra even fails in a specific, drawable way when the invariant breaks. After this chapter you will pick the algorithm by the weights and the question, write all four from memory, detect and reconstruct negative cycles, and fold conditions like "one coupon" into the graph itself.

8.1 Relaxation

The atomic move: $d[v] \leftarrow \min(d[v], d[u] + w(u, v))$. Every algorithm here is a different strategy for *ordering* and *counting* relaxations.

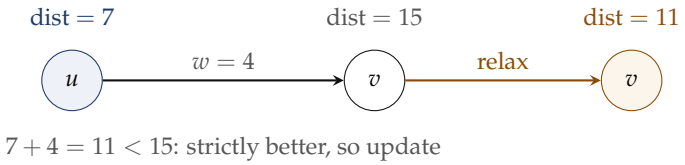


Figure 8.1: One relaxation step. Notice the check is strict inequality — an equal-length alternative changes nothing.

8.2 BFS, revisited

With every weight 1, the BFS layers of Chapter 7 *are* shortest paths: the queue is a degenerate priority queue. Keep BFS for unweighted problems — it is $O(V+E)$ and Dijkstra on unit weights buys nothing.

8.3 Bellman–Ford

Relax *every* edge, $n-1$ times. Correctness: after round k every shortest walk of at most k edges is final, and any shortest path has $\leq n-1$ edges. A further improvement in round n certifies a negative cycle — reachable or not, if you seed every distance with 0 (an implicit super-source).

Listing 8.1: `bellmanford.h`: rounds, parent pointers, one-cycle rebuild.

```
// bellmanford.h -- n-1 relaxation rounds; negative-cycle detect +
   rebuild.
#include <utility>
#include <vector>
using namespace std;
using W = long long;
const W BF_INF = (W)4e18;

struct Edge { int u, v; int w; };

// fills `cycle` with one negative cycle anywhere in the graph if any
// (all-zero seed = implicit super-source; seed s alone instead to
// restrict to cycles reachable from s). dist[] is 0 when none exists.
inline vector<W> bellmanford(int n, int s, const vector<Edge>& E,
                      vector<int>& cycle) {
   (void)s;
   vector<W> dist(n + 1, 0);
   vector<int> par(n + 1, -1);
   int x = -1;
   for (int i = 0; i < n; i++) {
      x = -1;
      for (auto [u, v, w] : E)
         if (dist[u] < BF_INF && dist[u] + w < dist[v]) {
            dist[v] = max(dist[u] + w, -BF_INF);
            par[v] = u;
            x = v;
         }
   }
   if (x < 0) return dist; // no negative cycle
   for (int i = 0; i < n; i++) x = par[x]; // walk into the cycle
   for (int v = x;; v = par[v]) {
      cycle.push_back(v);
      if (v == x && cycle.size() > 1) break;
   }
   reverse(cycle.begin(), cycle.end());
   return dist;
}
```

Example 8.1 (CSES *Cycle Finding*)**.** Report any negative cycle.

Strategy. Run the rounds; a vertex still improving in round n is downstream of a cycle. Walking parents n steps lands on the cycle itself; collect until repeat.

Solution.

```
#include "bellmanford.h"
#include <algorithm>
int main() {
   int n, m; scanf("%d %d", &n, &m);
   vector<Edge> E(m);
   for (auto& e : E) scanf("%d %d %d", &e.u, &e.v, &e.w);
   vector<int> cyc;
   bellmanford(n, 1, E, cyc);
   if (cyc.empty()) { puts("NO"); return 0; }
   puts("YES");
   for (int i = 0; i < (int)cyc.size(); i++)
      printf("%d%c", cyc[i], " \n"[i + 1 == (int)cyc.size()]);
}
```

Real output (one valid cycle):

```
YES
4 1 2 4
```

Example 8.2 (CSES *High Score*)**.** Maximise the score on a path $1 \to n$; a positive cycle on the way makes it unbounded — report -1 then.

Strategy. Negate every weight and longest becomes shortest: Bellman–Ford over the negated edges. The catch is *both* reachabilities: the cycle must be reachable from 1 *and* must be able to reach n. Vertices still relaxing in round n are downstream of a reachable cycle; propagate forward from them and ask whether n is touched.

Solution.

```
// high_score.cpp -- CSES 1673 High Score: longest path 1->n where a
// positive cycle can make the score unbounded. Negate weights, run
// Bellman-Ford; vertices still relaxing in round n sit downstream of
// a negative cycle. -1 iff any such vertex can reach n.
#include <cstdio>
#include <vector>
#include <queue>
using namespace std;
using W = long long;
const W INF = (W)4e18;
struct Edge { int u, v, w; };

int main() {
   int n, m; scanf("%d %d", &n, &m);
   vector<Edge> E(m);
   vector<vector<int>> adj(n + 1);
   for (auto& e : E) {
      scanf("%d %d %d", &e.u, &e.v, &e.w);
      e.w = -e.w; // longest path via negation
      adj[e.u].push_back(e.v);
   }
   vector<W> d(n + 1, INF); d[1] = 0;
```

```
    vector<int> touched;
    for (int i = 0; i < n; i++) {
       touched.clear();
       for (auto& e : E)
          if (d[e.u] < INF && d[e.u] + e.w < d[e.v]) {
             d[e.v] = d[e.u] + e.w;
             touched.push_back(e.v);
          }
    }
    // `touched` = vertices relaxed in the n-th round: reachable from 1
    // AND downstream of a negative cycle. Any of them that reaches n
    // makes the answer unbounded.
    vector<char> bad(n + 1, 0);
    queue<int> q;
    for (int v : touched) if (!bad[v]) { bad[v] = 1; q.push(v); }
    while (!q.empty()) {
       int u = q.front(); q.pop();
       for (int v : adj[u]) if (!bad[v]) { bad[v] = 1; q.push(v); }
    }
    printf("%lld\n", bad[n] ? -1 : -d[n]);
}
```

Real output (CSES sample):

```
5
```

8.4 Dijkstra

Greedy frontier: pop the unsettled vertex with smallest tentative distance; that value is final. Correctness is a cut argument — settled vertices sit on one side, and the cheapest unsettled vertex cannot improve through any path that leaves the settled set, because all such detours cost at least as much (weights ≥ 0 is the load-bearing clause). Drop that clause and the argument dies:

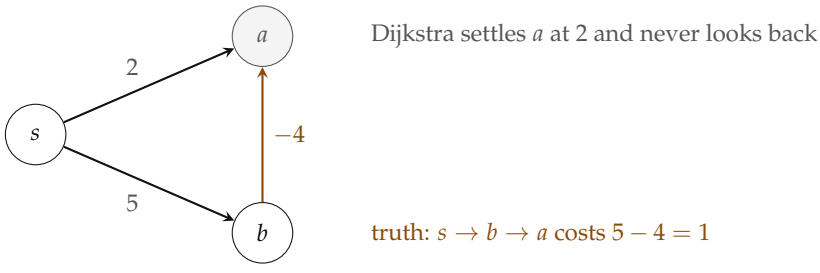


Figure 8.2: Where Dijkstra dies, drawn. Notice s settles a at 2 before b is ever examined — no negative-edge signal can pull it back.

Listing 8.2: `dijkstra.h`: lazy deletion via the stale-entry skip.

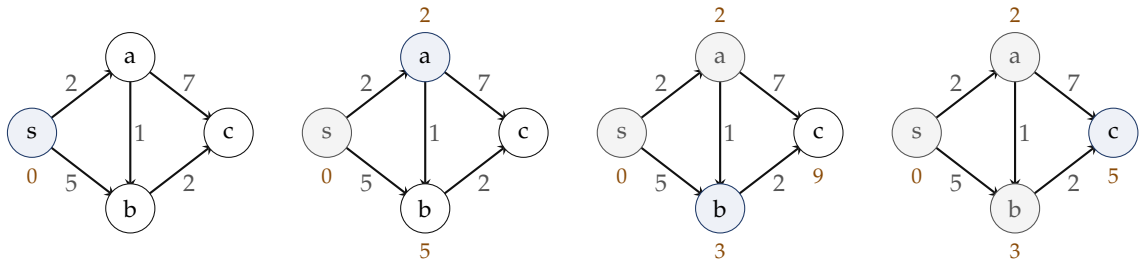


Figure 8.3: The settled set growing. Notice each frame fixes one vertex permanently — that permanence is both the speed and the vulnerability.

```cpp
// dijkstra.h -- single-source shortest paths, non-negative weights.
#include <functional>
#include <queue>
#include <utility>
#include <vector>
using namespace std;

using W = long long;
const W INF = (W)4e18;

// g[u] = {v, w}; returns dist[] (INF = unreachable).
inline vector<W> dijkstra(int s, const vector<vector<pair<int,int>>>& g) {
    int n = g.size() - 1;
    vector<W> dist(n + 1, INF);
    priority_queue<pair<W,int>, vector<pair<W,int>>,
                   greater<pair<W,int>>> pq;
    dist[s] = 0; pq.push({0, s});
    while (!pq.empty()) {
        auto [d, u] = pq.top(); pq.pop();
        if (d != dist[u]) continue; // stale entry
        for (auto [v, w] : g[u])
            if (d + w < dist[v]) {
                dist[v] = d + w;
                pq.push({dist[v], v});
            }
    }
    return dist;
}
```

Example 8.3 (CSES *Shortest Routes I*)**.** Distances from city 1.

Strategy. Non-negative weights name Dijkstra; skip stale queue entries instead of deleting them.

Solution.

```
#include "dijkstra.h"
int main() {
   int n, m; scanf("%d %d", &n, &m);
   vector<vector<pair<int,int>>> g(n + 1);
   for (int i = 0; i < m; i++) {
      int a, b, c; scanf("%d %d %d", &a, &b, &c);
      g[a].push_back({b, c});
   }
   auto d = dijkstra(1, g);
   for (int i = 1; i <= n; i++)
      printf("%lld%c", d[i], " \n"[i == n]);
}
```

Real output (CSES sample):

```
0 5 2
```

8.5 Floyd–Warshall

All pairs at once: $d[i][j] \leftarrow \min(d[i][j], d[i][k]+d[k][j])$ with k the *outermost* loop — the invariant is "only vertices 1..k may appear internally." $O(n^3)$ is right when $n \le 500$.

Listing 8.3: `floyd.h`: the 15-line core + `next` table.

```
// floyd.h -- all-pairs shortest paths, O(n^3); next[][] reconstructs.
#include <vector>
using namespace std;
using W = long long;
const W FW_INF = (W)4e18;

// dist: n+1 x n+1 matrix (FW_INF = no edge, dist[i][i]=0).
// next[i][j]: first hop from i toward j; -1 when unreachable.
inline void floyd(int n, vector<vector<W>>& dist,
                  vector<vector<int>>& nxt) {
   nxt.assign(n + 1, vector<int>(n + 1, -1));
   for (int i = 1; i <= n; i++)
      for (int j = 1; j <= n; j++)
         if (dist[i][j] < FW_INF) nxt[i][j] = j;
   for (int k = 1; k <= n; k++)
      for (int i = 1; i <= n; i++)
         for (int j = 1; j <= n; j++)
            if (dist[i][k] < FW_INF && dist[k][j] < FW_INF &&
               dist[i][k] + dist[k][j] < dist[i][j]) {
               dist[i][j] = dist[i][k] + dist[k][j];
               nxt[i][j] = nxt[i][k];
            }
}
```

Example 8.4 (CSES *Shortest Routes II*)**.** q distance queries.

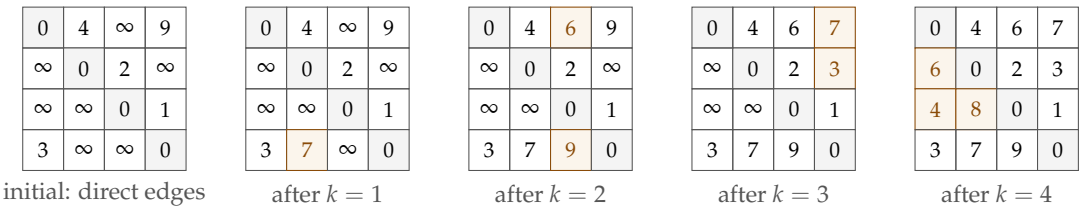
Strategy. Queries make it all-pairs; $n \leq 500$ makes the cube free.

Solution.

```
#include "floyd.h"
int main() {
   int n, m, q; scanf("%d %d %d", &n, &m, &q);
   vector<vector<W>> d(n + 1, vector<W>(n + 1, FW_INF));
   for (int i = 1; i <= n; i++) d[i][i] = 0;
   for (int i = 0; i < m; i++) {
      int a, b, c; scanf("%d %d %d", &a, &b, &c);
      d[a][b] = d[b][a] = min(d[a][b], (W)c);
   }
   vector<vector<int>> nxt;
   floyd(n, d, nxt);
   while (q--) {
      int a, b; scanf("%d %d", &a, &b);
      printf("%lld\n", d[a][b] >= FW_INF ? -1 : d[a][b]);
   }
}
```

Real output (CSES sample):

```
5
5
8
-1
3
```



each stage lets routes detour through one more vertex; amber marks the entries that improved

Figure 8.4: The k-expansion. Notice each stage only *permits* detours through one more vertex — nothing is forced.

8.6 State expansion

When the question carries a condition — a coupon, a ticket, a mode — the graph is too small: run shortest paths on *position* $\times$ *state*. Vertex (v, used) is a different vertex from (v, unused).

Example 8.5 (CSES *Flight Discount*)**.** One ticket may be halved.

Strategy. Double the graph: coupon-unused layer and spent layer, with a half-price edge between them. Ordinary Dijkstra on the expanded graph.

Solution.

```
// flight_discount.cpp -- CSES Flight Discount.
// Expanded graph: vertex (city, used-coupon?); Dijkstra over it.
#include "dijkstra.h"
int main() {
   int n, m; scanf("%d %d", &n, &m);
   vector<vector<pair<int,int>>> g(n + 1);
   for (int i = 0; i < m; i++) {
      int a, b, c; scanf("%d %d %d", &a, &b, &c);
      g[a].push_back({b, c});
   }
   // dist[v][u] : u=0 coupon unused, u=1 used.
   const W INF = (W)4e18;
   vector<array<W,2>> dist(n + 1, {INF, INF});
   using T = tuple<W,int,int>;
   priority_queue<T, vector<T>, greater<T>> pq;
   dist[1][0] = 0; pq.push({0, 1, 0});
   while (!pq.empty()) {
      auto [d, u, c] = pq.top(); pq.pop();
      if (d != dist[u][c]) continue;
      for (auto [v, w] : g[u]) {
         if (d + w < dist[v][c]) {
            dist[v][c] = d + w; pq.push({dist[v][c], v, c});
         }
         if (!c && d + w / 2 < dist[v][1]) { // spend coupon
            dist[v][1] = d + w / 2; pq.push({dist[v][1], v, 1});
         }
      }
   }
   printf("%lld\n", dist[n][1]);
}
```

Real output (CSES sample):

```
2
```

Example 8.6 (USACO-G *Why Did the Cow Cross the Road*)**.** Each road crossing costs T; Bessie stops to eat on every third field she lands on — and the eating times differ per field.

Strategy. The move counter is a state: (r, c, k) with k the moves taken mod 3. Every crossing costs T; landing when k rolls $2 \to 0$ also pays that field's eating time — "eat on every third visited field" becomes "pay on the transition into k=0".

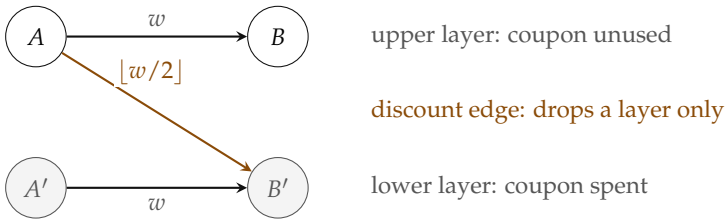


Figure 8.5: The two-layer expansion. Notice the discount edge only descends — the coupon can't be unspent.

Solution.

```
// cowroad.cpp -- USACO 2017 Feb Gold "Why Did the Cow Cross the Road" (visitfj).
// Every crossing costs T; Bessie eats on every 3rd field she lands on
// (moves 3,6,9,...), including FJ's field when arrival is a multiple of 3.
// State (r,c,k), k = moves mod 3; Dijkstra over the 3-layer grid.
#include <queue>
#include <tuple>
#include <vector>
#include <cstdio>
#include <algorithm>
using namespace std;
using W = long long;
const W INF = (W)4e18;

int main() {
   int n; W T;
   if (scanf("%d %lld", &n, &T) != 2) return 0;
   vector<vector<W>> a(n, vector<W>(n));
   for (auto& r : a) for (auto& x : r) scanf("%lld", &x);
   vector dist(n, vector(n, vector<W>(3, INF)));
   using T3 = tuple<W,int,int,int>;
   priority_queue<T3, vector<T3>, greater<T3>> pq;
   dist[0][0][0] = 0; pq.push({0,0,0,0});
   const int dr[4] = {-1,1,0,0}, dc[4] = {0,0,-1,1};
   while (!pq.empty()) {
      auto [d,r,c,k] = pq.top(); pq.pop();
      if (d != dist[r][c][k]) continue;
      for (int i = 0; i < 4; i++) {
         int nr=r+dr[i], nc=c+dc[i];
         if (nr<0||nr>=n||nc<0||nc>=n) continue;
         int nk=(k+1)%3;
         W w = T + (k==2 ? a[nr][nc] : 0); // landing on move 3k eats
         if (d+w < dist[nr][nc][nk]) { dist[nr][nc][nk]=d+w; pq.push({d+w,nr,nc,nk}); }
      }
   }
   printf("%lld\n", *min_element(dist[n-1][n-1].begin(), dist[n-1][n-1].end()));
}
```

Real output (USACO official sample, N=4, T=2):

```
31
```

A final named variant: *0-1 BFS* — weights in $\{0, 1\}$ only — replaces the heap with a deque pushing weight-0 edges to the front.

Exercise 8.1 (easy)**.** Does relaxation order matter for Dijkstra's correctness? For Bellman–Ford's round count?

Exercise 8.2 (medium)**.** Bellman–Ford may exit early when a round improves nothing. Show the early exit can never fire while a reachable negative cycle still has improvable vertices.

Exercise 8.3 (medium)**.** Why a priority queue and not repeated scans for the minimum? Estimate both costs on V=10^5, E=$2 \cdot 10^5$.

Exercise 8.4 (medium)**.** Name the state for: "shortest route with at most k road changes" and "arriving by exactly time T counts differently than late."

8.7 Chapter summary

Weights	Algorithm	Cost	Also gives
unweighted	BFS	$O(V+E)$	layers
non-negative	Dijkstra	$O(E \log V)$	settled order
any (incl. neg.)	Bellman–Ford	$O(VE)$	negative cycles
all pairs	Floyd–Warshall	$O(V^3)$	`next` paths
0/1 only	0-1 BFS	$O(V+E)$	deque order
with a condition	expanded graph	$\times$ states	coupon semantics

Table 8.1: Pick by the weight law and the shape of the question — one source, all pairs, or a condition to fold in.

8.8 Common mistakes

- **Dijkstra on negative edges.** The settled set is final — one negative edge invalidates the permanence and the algorithm has no repair path.
- **Seeding Bellman–Ford from one source when any cycle counts.** Only reachable-from-s cycles surface; seed 0 everywhere to catch all.
- **One-sided reachability in *High Score*.** A negative cycle reachable from 1 is harmless when it cannot reach n — marking every downstream vertex without the forward propagation prints -1 on a finite answer.
- **Floyd with k inside.** Loop order *is* the invariant: k outermost, or intermediate vertices may be reused illegally.

- **Deleting queue entries.** Decrease-key is awkward — push a better entry and skip stale pops instead.
- **Forgetting `dist[u]+w` can overflow.** Guard `dist[u] < INF/2` before adding to avoid wrapping past unreachable vertices.

8.9 Additional exercises

Exercise 8.5 (easy). CSES *Shortest Routes I*.

Exercise 8.6 (easy). CSES *Shortest Routes II*.

Exercise 8.7 (easy). USACO-S *Milk Pails* — BFS on (x,y) fill states.

Exercise 8.8 (medium). CSES *Flight Discount*.

Exercise 8.9 (medium). CSES *Flight Routes* — k shortest.

Exercise 8.10 (medium). CSES *Investigation* — count min paths.

Exercise 8.11 (medium). USACO-G *Milk Pumping* — Dijkstra + max-flow constraint.

Exercise 8.12 (medium). USACO-G *Shortcut* — tree + shortcut edges.

Exercise 8.13 (hard). CSES *High Score* — negative cycle on positive edge.

Exercise 8.14 (hard). CSES *Cycle Finding* — the reconstruction, coded.

Exercise 8.15 (hard). USACO-G *Lasers and Mirrors* — direction-state BFS.

Exercise 8.16 (hard). USACO-G *A Pie for a Pie* — two-shelf trade graph.

Exercise 8.17 (hard). USACO-G *Telephone* — breed-count state Dijkstra.

Notes

Relaxation-first framing and the cut argument follow Laaksonen [2] and Sannemo [1]; state expansion and the coupon/layer trick are standard in the USACO Guide's Gold modules [3] and cp-algorithms [5]; 0-1 BFS and the negative-edge autopsy appear in Halim [7]; archive problems are official CSES/USACO [4].

Further reading

- USACO Guide, "Shortest Paths with Non-Negative Edge Weights" (intermediate). https://usaco.guide/gold/shortest-paths — Dijkstra as used in §8.4 with harder problems.
- cp-algorithms, "Dijkstra Algorithm" (reference). https://cp-algorithms.com/graph/dijkstra.html — the priority-queue version and its proof of correctness.

- cp-algorithms, "Bellman–Ford Algorithm" (reference). https://cp-algorithms.com/graph/bellman_ford.html — §8.3 including the negative-cycle reachability trick of 8.2.
- cp-algorithms, "Floyd–Warshall" (reference). https://cp-algorithms.com/graph/all-pair-shortest-path-floyd-warshall.html — §8.5's cubic all-pairs DP and when to prefer it.
- CSES 1671 *Shortest Routes I* and 1672 *Shortest Routes II* (practice). https://cses.fi/problemset/task/1671 — 8.3/8.4 verbatim.
- CSES 1673 *High Score* and 1197 *Cycle Finding* (practice). https://cses.fi/problemset/task/1673 — negated Bellman–Ford; the Common mistakes section's exercise.

WATCH INSTEAD

Best first. "Lecture 16: Dijkstra" — MIT OpenCourseWare, 51 min — https://www.youtube.com/watch?v=2E7MmKv0Y24. The settle-order invariant of §8.4 with the same relaxation picture as Figure 8.1; OCW writes $d(s, v)$ for the book's dist(v). "Lecture 17: Bellman-Ford" — MIT OpenCourseWare, 49 min — https://www.youtube.com/watch?v=ozsuci5pIso. The edge-relaxation proof and negative-cycle detection of §8.3; notation matches one-to-one. Access date 2026-10-10.

CHAPTER 9

Disjoint sets and spanning trees

Two questions dominate this chapter: are these vertices connected, and which edges are worth keeping? The disjoint-set union answers the first in nearly constant time per operation; the minimum spanning tree answers the second, and the cut property is the single theorem that makes every MST algorithm correct. After this chapter you will write a DSU blindfolded, argue Kruskal and Prim from the same exchange proof, and spot when an offline replay turns "delete" into "insert" — the trick that makes deletion queries solvable at all.

9.1 The connectivity problem

Maintain "are a and b in the same component?" under edge insertions. Online algorithms must answer as each query arrives; offline algorithms see the whole sequence first — and offline is where hard problems soften: deleting an edge sequence replayed backwards is insertion, and DSU handles insertion natively.

9.2 The forest representation

Each set is a rooted tree; `p[v]` points up. `find` walks to the root, *path compression* re-roots the walk's vertices at the root; `unite` hangs the smaller root under the larger (*union by size*). Each half is optional; together they are nearly free.

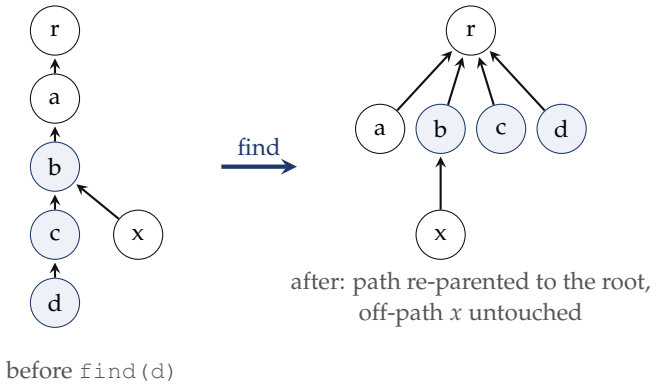


Figure 9.1: The same find, before and after. Notice compression only re-parents vertices on the walked path — others keep their depth until their turn.

Listing 9.1: `dsu.h`: path halving + union by size + component sizes.

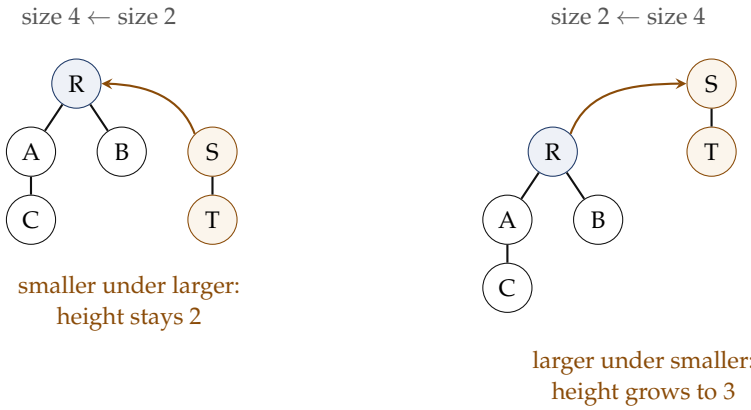


Figure 9.2: Union by size. Notice the asymmetry: hanging the smaller tree under the larger root leaves the height at 2, while hanging the larger tree under the smaller root pushes it to 3.

```
// dsu.h -- disjoint set union: path halving + union by size + sizes.
#include <numeric>
#include <vector>
using namespace std;

struct DSU {
   vector<int> p, sz;
   DSU(int n) : p(n + 1), sz(n + 1, 1) { iota(p.begin(), p.end(), 0); }
   int find(int v) { // path halving
      while (p[v] != v) { p[v] = p[p[v]]; v = p[v]; }
      return v;
   }
   bool unite(int a, int b) { // false if same set
      a = find(a); b = find(b);
      if (a == b) return false;
      if (sz[a] < sz[b]) swap(a, b);
      p[b] = a; sz[a] += sz[b];
      return true;
   }
   int size(int v) { return sz[find(v)]; }
};
```

9.3 The amortized bound

With both optimizations, m operations cost $O(m\,\alpha(n))$ where α is the inverse Ackermann function — bounded by 4 for any n in this universe. The proof charges each pointer hop to a rank increase budget; sketch only: ranks are potential, compression spends it. In contest terms: treat each operation as $O(1)$.

9.4 The cut property

For any cut $(S, V \setminus S)$, the lightest crossing edge belongs to *some* MST. Proof by exchange: take an MST using a heavier crossing edge f; swapping in the light edge e keeps a spanning tree and only lowers the cost. Every MST algorithm is this theorem in a different costume.

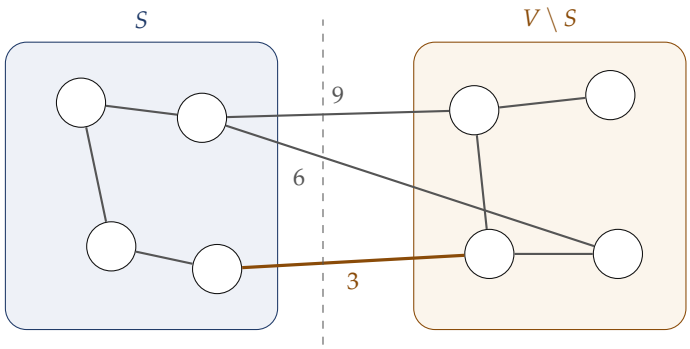


Figure 9.3: The cut property. Notice nothing depends on which MST you started from — exchange against *any* of them.

9.5 Kruskal

Sort edges, accept those joining different components. Each accepted edge is the lightest crossing of the cut between the two components — the cut property again, applied greedily. Cost: sorting dominates, $O(E \log E)$.

Listing 9.2: `kruskal.h` on top of `DSU`.

```
// kruskal.h -- sort edges, DSU-filter, accept. Returns MST weight.
#include "dsu.h"
#include <algorithm>
#include <vector>
using namespace std;
using W = long long;
struct KEdge { int u, v; int w; };

inline W kruskal(int n, vector<KEdge> E, vector<KEdge>* mst = nullptr)
    {
   sort(E.begin(), E.end(),
       [](const KEdge& a, const KEdge& b) { return a.w < b.w; });
   DSU d(n);
   W total = 0;
   for (auto& e : E)
      if (d.unite(e.u, e.v)) {
         total += e.w;
         if (mst) mst->push_back(e);
      }
```

```
    return total;
}
```

Example 9.1 (CSES *Road Reparation*)**.** Cheapest way to reconnect the cities.

Strategy. "Cheapest connection covering all" is an MST; a leftover of fewer than $n-1$ accepted edges means the graph was never connected.

Solution.

```
#include "kruskal.h"
int main() {
    int n, m; scanf("%d %d", &n, &m);
    vector<KEdge> E(m);
    for (auto& e : E) scanf("%d %d %d", &e.u, &e.v, &e.w);
    vector<KEdge> mst;
    W total = kruskal(n, E, &mst);
    if ((int)mst.size() < n - 1) { puts("IMPOSSIBLE"); return 0; }
    printf("%lld\n", total);
}
```

Real output (CSES sample):

```
14
```

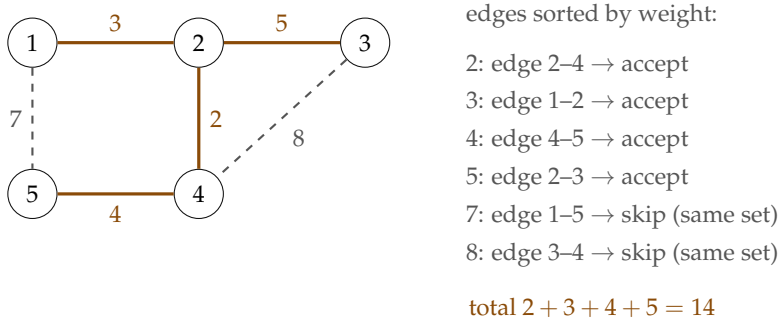


Figure 9.4: Kruskal's accept/skip rhythm. Notice the skipped edge is rejected *because* its endpoints already merged — the DSU is the algorithm's memory.

9.6 Prim

Grow one tree: a priority queue holds frontier edges; pop the lightest, settle its far vertex, push its edges. It is Dijkstra's skeleton with edge weight instead of distance — and wins when the graph is dense enough that sorting all edges is the bottleneck.

Listing 9.3: `prim.h`: the pq frontier.

```
// prim.h -- pq frontier MST; good when the graph is dense.
#include <functional>
```

```
#include <queue>
#include <vector>
using namespace std;
using W = long long;

inline W prim(int n, const vector<vector<pair<int,int>>>& g, int s =
    1) {
   vector<char> seen(n + 1, 0);
   priority_queue<pair<int,int>, vector<pair<int,int>>,
                  greater<>> pq;
   W total = 0;
   seen[s] = 1;
   for (auto [v, w] : g[s]) pq.push({w, v});
   while (!pq.empty()) {
      auto [w, u] = pq.top(); pq.pop();
      if (seen[u]) continue;
      seen[u] = 1; total += w;
      for (auto [v, wv] : g[u])
         if (!seen[v]) pq.push({wv, v});
   }
   return total;
}
```

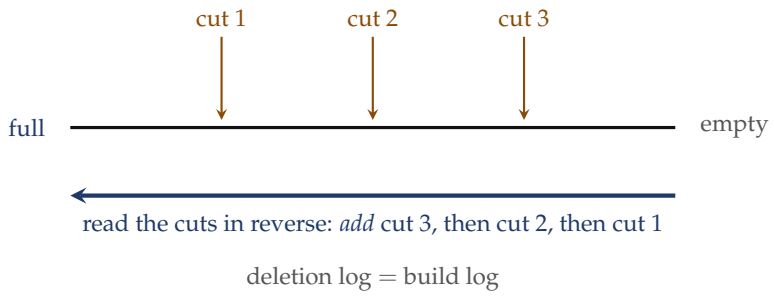


Figure 9.5: Offline reversal: a deletion log replayed backwards is a build log.

9.7 Worked examples

Example 9.2 (CSES *Road Construction*)**.** After each new road: component count, largest component.

Strategy. Both answers live in the DSU's size array: merges decrement the count and grow one component.

Solution.

```
#include "dsu.h"
int main() {
   int n, m; scanf("%d %d", &n, &m);
   DSU d(n);
   int comps = n, biggest = 1;
   while (m--) {
```

```
        int a, b; scanf("%d %d", &a, &b);
        a = d.find(a); b = d.find(b);
        if (a != b) {
           if (d.sz[a] < d.sz[b]) swap(a, b);
           d.p[b] = a; d.sz[a] += d.sz[b];
           biggest = max(biggest, d.sz[a]);
           comps--;
        }
        printf("%d %d\n", comps, biggest);
    }
}
```

Real output (CSES sample):

```
4 2
3 3
2 3
```

Example 9.3 (CSES *Building Roads*)**.** Minimum roads to connect all cities, then print them.

Strategy. Components minus one is the count; linking component roots in a chain realizes it.

Solution.

```
#include "dsu.h"
int main() {
    int n, m; scanf("%d %d", &n, &m);
    DSU d(n);
    while (m--) {
        int a, b; scanf("%d %d", &a, &b);
        d.unite(a, b);
    }
    vector<int> roots;
    for (int v = 1; v <= n; v++)
        if (d.find(v) == v) roots.push_back(v);
    printf("%d\n", (int)roots.size() - 1);
    for (int i = 1; i < (int)roots.size(); i++)
        printf("%d %d\n", roots[i - 1], roots[i]);
}
```

Real output (CSES sample):

```
1
1 3
```

Example 9.4 (USACO-S *Closing the Farm*)**.** After each barn closes, is the farm still fully connected?

Strategy. DSU cannot delete — so run time backwards: start from the last surviving barn and *insert* barns in reverse order, recording the connectivity of each state *before* the closed barn is re-inserted (the answer asks for the state *after* that closing; an empty remainder is vacuously connected) (Figure 9.6).

Solution.

```cpp
// closing_farm.cpp -- USACO-S Closing the Farm.
// Offline reversal: deletions replayed backwards are insertions.
// ans[i] = connected AFTER barn order[i] closes, i.e. the state
// holding only order[i+1..n-1]: evaluate comps BEFORE inserting.
#include "dsu.h"
int main() {
   int n; scanf("%d", &n);
   vector<vector<int>> g(n + 1);
   for (int i = 0; i < n - 1; i++) {
      int a, b; scanf("%d %d", &a, &b);
      g[a].push_back(b); g[b].push_back(a);
   }
   vector<int> order(n);
   for (auto& x : order) scanf("%d", &x);
   vector<char> alive(n + 1, 0), ans(n);
   DSU d(n);
   int comps = 0;
   for (int i = n - 1; i >= 0; i--) {
      ans[i] = (comps <= 1); // after order[i] closed
      int v = order[i]; // (empty set: vacuously YES)
      alive[v] = 1; comps++;
      for (int u : g[v])
         if (alive[u] && d.unite(u, v)) comps--;
   }
   for (int i = 0; i < n; i++) puts(ans[i] ? "YES" : "NO");
}
```

Real output (4-barn star, all states connected):

```
YES
YES
YES
YES
```

Example 9.5 (CSES *Network Breakdown*: read the removals backwards)**.** A network has n nodes and m links; q links break one at a time. After *each* break, report how many connected components remain ($n, m, q \leq 10^5$).

Strategy. Union-find only *adds* edges — it cannot unmerge a split, so simulating breaks forward is impossible with DSU alone. Read the whole input first. The key observation: removing edges $e_1, e_2, \ldots, e_q$ in order is the same process as *adding* them back in reverse order. Start from the end state — every edge that is never removed is already in the structure — then replay the removal list backwards: before "un-removing" query i, the current component count is exactly the answer *after* break i. Recording backwards then printing forwards converts a deletion problem into the union problem DSU solves. Closing the Farm (9.4) used the same time-reversal trick; here it is the whole algorithm.

Solution. `code/network_breakdown.cpp` on the sample (5 nodes, 5 edges, breaks 3-4, 2-3, 4-5) prints `2 2 3`: the two offline passes are $O((n + m + q)\,\alpha(n))$ total, while a forward BFS re-check after every break would cost $O(q\,(n + m))$.

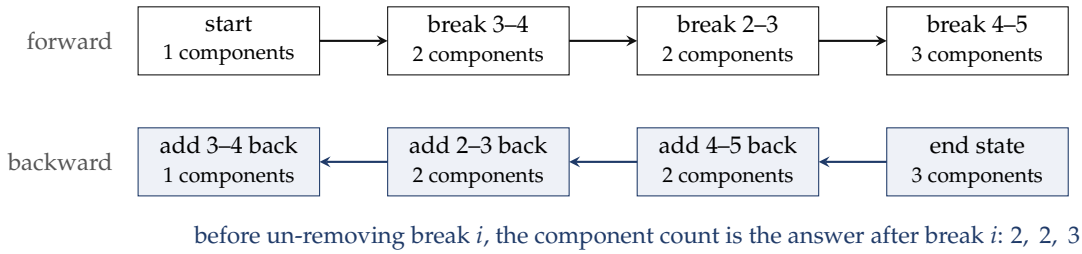


Figure 9.6: The timeline reversal. Notice the forward question and the backward algorithm share *the same states* — replaying backwards visits every intermediate farm the original run did.

Exercise 9.1 (easy)**.** In `unite(a,b)`, which order runs `find` first — and why does union-by-size need the *root* sizes, not arbitrary sizes?

Exercise 9.2 (medium)**.** Kruskal's next accepted edge e between components A, B: exhibit the cut whose light edge e is, and conclude its membership.

Exercise 9.3 (medium)**.** Give a query sequence where offline reversal is essential (not just convenient): name one operation DSU fundamentally cannot undo.

9.8 Chapter summary

Task	Tool	Cost	Watch for
connected? + merge	DSU	$\sim O(1)$/op	no deletes
all-pairs min tree	Kruskal	$O(E \log E)$	sort dominates
dense-graph MST	Prim	$O(E \log V)$	pq frontier
deletion queries	offline reverse	same ops	replay order

Table 9.1: One theorem (cut property) under both MST algorithms; one trick (offline reversal) under every deletion problem.

9.9 Common mistakes

- **Union without `find`.** `p[a]=p[b]` links arbitrary members, not roots — always unite `find(a),find(b)`.
- **Skipping compression *and* size.** One alone helps; neither degenerates to $O(n)$ per find — keep both.
- **Trying to delete with DSU.** There is no `un-unite`; offline reversal or a different structure is the answer.
- **Assuming MST is unique.** Equal weights admit several; the cut property promises *some* MST, not *the* MST — handle ties deliberately when the problem counts trees.
- **Forgetting the disconnected case.** A forest is legal input — `mst.size() < n-1` is the IMPOSSIBLE check.

9.10 Additional exercises

Exercise 9.4 (easy)**.** CSES *Road Reparation*.

Exercise 9.5 (easy)**.** USACO-S *Closing the Farm*.

Exercise 9.6 (easy)**.** USACO-G *MooTube* — threshold components.

Exercise 9.7 (medium)**.** USACO-S *Wormhole Sort* — DSU + binary search.

Exercise 9.8 (medium)**.** USACO-G *Moocast* — MST-style radii.

Exercise 9.9 (medium)**.** USACO-G *I Would Walk 500 Miles* — implicit-edge Kruskal.

Exercise 9.10 (medium)**.** CF *Design Tutorial: Inverse the Problem* — reconstruct from MST.

Exercise 9.11 (medium)**.** Implement `find` with path splitting vs. path halving; time both on 10^6 unions.

Exercise 9.12 (medium)**.** CSES *Necessary Cities* — articulation points, the connectivity question DSU cannot answer.

Exercise 9.13 (hard)**.** CSES *New Roads Queries* — online MST edge questions.

Exercise 9.14 (hard)**.** USACO-G *Portals* — MST on portal+cost edges.

Exercise 9.15 (hard)**.** USACO-G *Fenced In* — boundary-strip Kruskal.

Exercise 9.16 (hard)**.** USACO-OS *Ski Course Rating* — MST + queries.

Exercise 9.17 (hard)**.** Baltic OI 2016 *Park* — MST + LCA path max.

Notes

DSU amortization and the cut-property-first MST treatment follow Laaksonen [2] and Sannemo [1]; the offline-reversal pattern and threshold-component problems are the USACO Guide's DSU module [3]; cp-algorithms carries the charged-ranks proof sketch and MST variants [5]; Halim supplies the named variants [7]; archive problems are official CSES/USACO [4].

Exercise 9.18 (hard)**.** Offline variant: edges both appear and disappear; write the segment-tree-on-time DSU sketch.

Further reading

- USACO Guide, "Disjoint Set Union" (intermediate) — the union-by-size structure of §9.1 with more Gold problems.
  https://usaco.guide/gold/dsu
- USACO Guide, "Minimum Spanning Trees" (intermediate) — Kruskal/Prim with the same cut-property framing as §9.4.
  https://usaco.guide/gold/mst
- cp-algorithms, "Disjoint Set Union" (reference). (article: "Disjoint Set Union) — full proof of the near-constant union bound behind §9.3.
  https://cp-algorithms.com/
- cp-algorithms, "Kruskal's algorithm" (reference) — §9.5 with correctness and implementation variants.
  https://cp-algorithms.com/graph/mst_kruskal.html
- CSES 1675 *Road Reparation* and 1676 *Road Construction* (practice) — 9.1/9.3 verbatim.
  https://cses.fi/problemset/task/1675
- Laaksonen, *Competitive Programmer's Handbook*, ch. 15 "Spanning trees" (beginner) — the whole MST half in twenty pages.
  https://cses.fi/book/book.pdf

WATCH INSTEAD

Best first. "1.12 Disjoint Sets Data Structure - Weighted Union and Collapsing Find" — Abdul Bari, 26 min — https://youtu.be/wU6udHRIkcc. Array-based union by weight + find with path halving — exactly §9.1 and §9.3; Bari calls them "weighted union"/"collapsing find", the book calls them union by size/path compression. "Lec 8" — MIT OpenCourseWare 6.042J, 83 min — https://youtu.be/GJpt_3ie4WU. Spanning trees and the gray-zone/grow-where-you-can MST method matching §9.5–§9.6; more proof depth than the book needs, worth it for §9.4. Access date 2026-10-10.

CHAPTER 10

Tree algorithms: subtrees, diameters, LCA

A tree is the graph where every question has a canonical answer: exactly one path between any two vertices, and rooting turns "around" into "down." This chapter collects the five patterns that solve almost every tree problem on the archive: post-order subtree sums, the two-DFS diameter, rerooting (answers for every root in one pass), the Euler tour that turns subtrees into subarrays, and binary lifting for ancestors and LCA. After it you will compute per-vertex farthest distances, answer distance queries in $O(\log n)$, and see every "tree problem" as pick-a-pattern.

10.1 The shape of a tree

n vertices, $n-1$ edges, connected, acyclic: any two properties imply the third. Rooting is a *design choice* — most algorithms pick an arbitrary root and exploit "parent vs children" as the graph's only direction. Unique paths are what make ancestors, depths and diameters well-defined.

10.2 Subtree aggregates

Post-order accumulation computes any quantity that depends on children first: sizes, sums, counts. The pattern is one DFS; everything else in the chapter builds on it.

Example 10.1 (CSES *Subordinates*)**.** Per employee, count the whole subtree beneath them.

Strategy. The hierarchy is already rooted: size(v) = $\sum_{u \text{ child}}(1 + \text{size}(u))$ computed in one post-order pass.

Solution.

```
#include <vector>
#include <cstdio>
using namespace std;
int main() {
   int n; scanf("%d", &n);
   vector<vector<int>> ch(n+1);
   for (int v = 2; v <= n; v++) { int b; scanf("%d",&b); ch[b].
      push_back(v); }
   vector<int> sz(n+1,0), st = {1}, ord;
```

```
    while (!st.empty()) { int v = st.back(); st.pop_back(); ord.
        push_back(v);
       for (int u : ch[v]) st.push_back(u); }
    for (int i = ord.size()-1; i >= 0; i--) {
       sz[ord[i]] = 0; for (int u : ch[ord[i]]) sz[ord[i]] += 1 + sz[u
          ]; }
    for (int i = 1; i <= n; i++) printf("%d%c", sz[i], " \n"[i==n]);
}
```

Real output (CSES sample):

```
4 0 2 0 0
```

10.3 Diameter: two DFS

The farthest vertex from *any* start is a diameter endpoint; a second farthest-search from it finds the other end. Proof sketch: let u, v be a diameter and a the farthest vertex from start s; if a were not an endpoint, a path past u or v would beat a's distance — contradiction. (The common wrong proof — "it works because it works on paths" — is flagged in the mistakes.)

Example 10.2 (CSES *Tree Diameter*)**.** Longest path in the tree.

Strategy. Two searches, no DP: farthest-from-1, then farthest-from-that.

Solution.

```
#include "tree_diameter.h"
#include <cstdio>
int main() {
   int n; scanf("%d", &n);
   vector<vector<int>> g(n+1);
   for (int i = 0; i < n-1; i++) { int a,b; scanf("%d %d",&a,&b);
      g[a].push_back(b); g[b].push_back(a); }
   printf("%d\n", tree_diameter(g));
}
```

Real output (CSES sample):

```
3
```

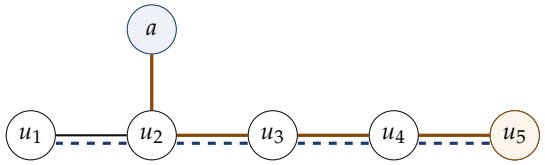


Figure 10.1: The farthest-point argument. Notice the claim is about endpoints, not the path — endpoint first, distance second.

10.4 Rerooting

"For every root" doubles the work naively; rerooting does it in two passes: `down[v]` from children, then `up[v]` passed down from parent using each child's best *sibling* value. Per vertex the answer combines both.

Example 10.3 (CSES *Tree Distances II*)**.** Each vertex's farthest distance.

Strategy. The answer is max(`down`, `up`); the sibling-max trick supplies each child's "up" in $O(\deg v)$.

Solution.

```
#include "reroot.h"
#include <cstdio>
int main() {
   int n; scanf("%d", &n);
   vector<vector<int>> g(n+1);
   for (int i = 0; i < n-1; i++) { int a,b; scanf("%d %d",&a,&b);
      g[a].push_back(b); g[b].push_back(a); }
   vector<int> down, up;
   reroot(g, 1, down, up);
   for (int i = 1; i <= n; i++)
      printf("%d%c", max(down[i], up[i]), " \n"[i==n]);
}
```

Real output (CSES sample):

```
2 3 2 3 3
```

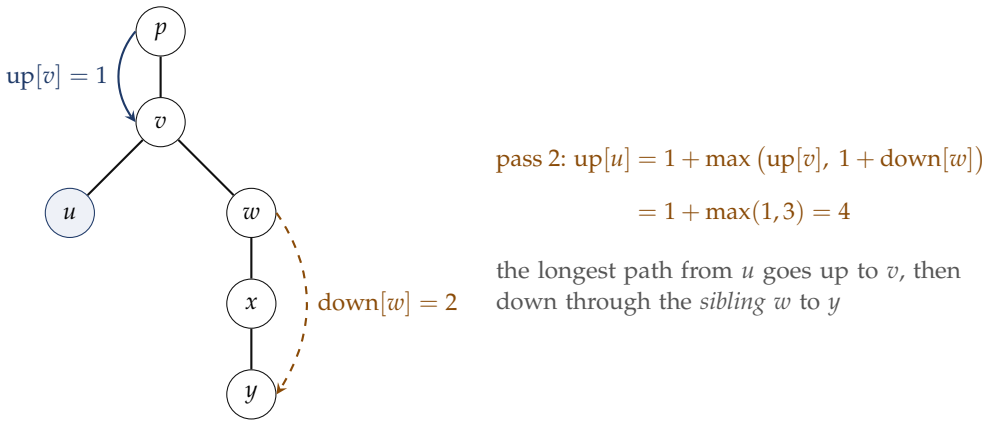


Figure 10.2: What travels down in pass two. Notice a child never sees its own subtree in its "up" value — the sibling max is the key step.

10.5 The Euler tour

`tin[v]` on entry, `tout[v]` on exit: every subtree becomes the interval $[\texttt{tin}, \texttt{tout})$, turning "update/query a subtree" into range operations — the bridge to segment trees in Chapter 12.

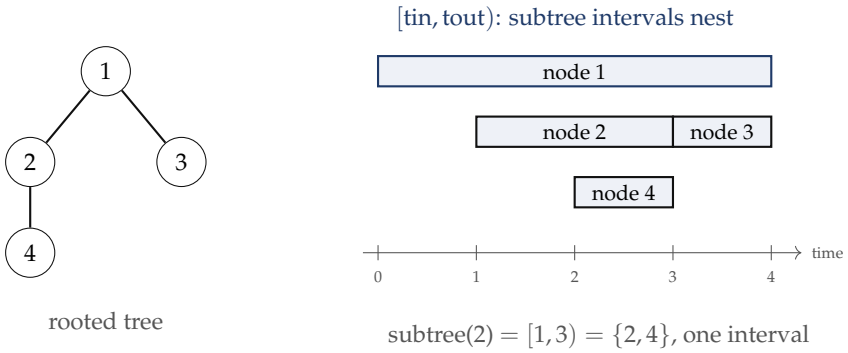


Figure 10.3: Euler intervals nested. Notice containment is interval containment — u is in v's subtree iff $\texttt{tin}[v] \le \texttt{tin}[u] < \texttt{tout}[v]$.

10.6 Binary lifting and LCA

Store the 2^k-th ancestor of every vertex: $\texttt{up}[k][v] = \texttt{up}[k-1][\texttt{up}[k-1][v]]$ — a doubling table built in $O(n \log n)$. k-th ancestor: follow set bits. LCA: equalize depth, then lift both ends by descending powers of two until their parents coincide. (An Euler-tour + sparse-RMQ alternative exists; lifting is the workhorse.)

Listing 10.1: `lca.h`: lifting table, `kth`, `lca`, `dist`.

```
// lca.h -- binary lifting: 2^k ancestors, k-th ancestor, LCA.
#include <vector>
using namespace std;
```

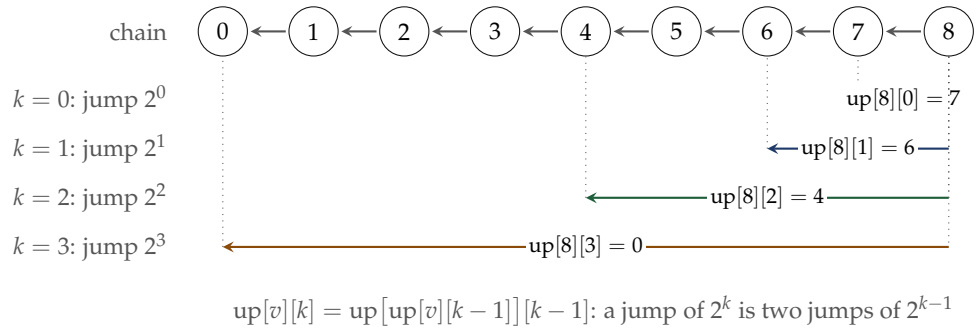


Figure 10.4: The doubling table. Notice each row skips exactly 2^k ancestors — a jump of 2^k is two jumps of 2^{k-1}.

```
struct LCA {
   int n, LOG;
   vector<vector<int>> up; // up[k][v] = 2^k-th ancestor (0=none)
   vector<int> depth;
   LCA(const vector<vector<int>>& g, int root = 1) {
      n = g.size() - 1;
      LOG = 1; while ((1 << LOG) <= n) LOG++;
      up.assign(LOG, vector<int>(n + 1, 0));
      depth.assign(n + 1, 0);
      vector<int> ord = {root};
      up[0][root] = 0;
      for (size_t i = 0; i < ord.size(); i++) {
         int v = ord[i];
         for (int u : g[v]) if (u != up[0][v]) {
            up[0][u] = v; depth[u] = depth[v] + 1;
            ord.push_back(u);
         }
      }
      for (int k = 1; k < LOG; k++)
         for (int v = 1; v <= n; v++)
            up[k][v] = up[k-1][up[k-1][v]];
   }
   int kth(int v, int k) const {
      for (int j = 0; j < LOG && v; j++)
         if (k & (1 << j)) v = up[j][v];
      return v;
   }
   int lca(int a, int b) const {
      if (depth[a] < depth[b]) swap(a, b);
      a = kth(a, depth[a] - depth[b]);
      if (a == b) return a;
      for (int j = LOG - 1; j >= 0; j--)
         if (up[j][a] != up[j][b]) { a = up[j][a]; b = up[j][b]; }
      return up[0][a];
   }
   int dist(int a, int b) const {
      return depth[a] + depth[b] - 2 * depth[lca(a, b)];
```

```
    }
};
```

Example 10.4 (CSES *Company Queries I & II*)**.** k-th boss; lowest common boss.

Strategy. The hierarchy is rooted at the CEO: `kth` answers I, `lca` answers II — same table.

Solution.

```
#include "lca.h"
#include <cstdio>
int main() {
   int n, q; scanf("%d %d", &n, &q);
   vector<vector<int>> g(n+1);
   for (int v = 2; v <= n; v++) { int b; scanf("%d",&b); g[b].
      push_back(v); }
   LCA T(g);
   while (q--) { int v, k; scanf("%d %d",&v,&k);
      printf("%d\n", T.kth(v,k) ? : -1); }
}
```

Real output (CSES sample):

```
3
1
-1
```

10.7 Distance queries

$\texttt{dist}(u,v) = \texttt{depth}[u] + \texttt{depth}[v] - 2 \cdot \texttt{depth}[\texttt{lca}(u,v)]$ — the unique path is the only route, so depth arithmetic is exact.

Example 10.5 (CSES *Distance Queries*)**.** Pairwise distances on a tree.

Strategy. LCA plus depth, $O(\log n)$ per query.

Solution.

```
#include "lca.h"
#include <cstdio>
int main() {
   int n, q; scanf("%d %d", &n, &q);
   vector<vector<int>> g(n+1);
   for (int i = 0; i < n-1; i++) { int a,b; scanf("%d %d",&a,&b);
      g[a].push_back(b); g[b].push_back(a); }
   LCA T(g);
   while (q--) { int a,b; scanf("%d %d",&a,&b);
      printf("%d\n", T.dist(a,b)); }
}
```

Real output (CSES sample):

```
2
3
0
```

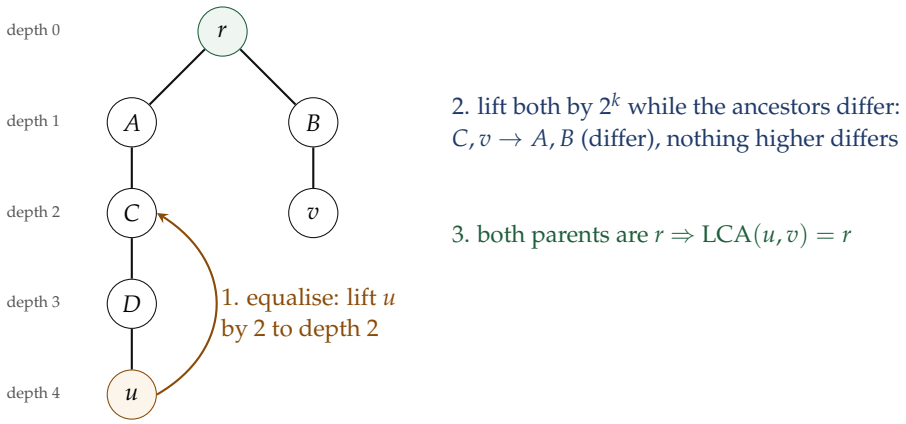


Figure 10.5: LCA in two lifts. Notice the first lift is exact depth equalization; the second is the binary search that stops one below the meeting point.

Exercise 10.1 (easy)**.** When does an unrooted answer (diameter) differ from a rooted one (deepest child)? Name a quantity insensitive to rooting.

Exercise 10.2 (medium)**.** Complete the farthest-vertex proof: if a's path to a diameter endpoint crosses the diameter at x, which distance comparison contradicts a not being an endpoint?

Exercise 10.3 (medium)**.** Fill the lift-table recurrence $\texttt{up}[k][v] = \texttt{up}[k-1][\texttt{up}[k-1][v]]$ and argue it counts exactly 2^k edges.

Exercise 10.4 (medium)**.** Convert "add 1 to every vertex in v's subtree" into a subarray operation using `tin/tout`.

10.8 Chapter summary

Need	Tool	Cost	Yields
subtree sums	post-order	$O(n)$	aggregates
longest path	two-DFS	$O(n)$	diameter
answer per root	reroot	$O(n)$	down+up
k-th anc, LCA	lifting	$O(n \log n)$+$O(\log n)$/q	dist
subtree as range	Euler	$O(n)$	range ops

Table 10.1: Root it, then pick the pattern: aggregates below, lifts above, intervals across.

10.9 Common mistakes

- **The lazy diameter "proof."** "Works on paths" is not a proof — the farthest-point argument is required and is exactly what breaks if you skip it.
- **Recursion depth on a path graph.** $n = 2 \cdot 10^5$ chain overflows a default stack — convert to iterative or `vector`-stack DFS.
- **Lifting with `up[k][v]` unguarded.** Root's ancestors are 0; dereferencing `up[.][0]` must be defined (index-0 sentinel) not skipped ad hoc.
- **Sibling-max off-by-one.** The child's own `down` must be *excluded* from its up-value — prefix/suffix maxima around its index, not the global max.
- **Euler half-open confusion.** `tout` is exclusive on exit-time — mixing [tin,tout] and [tin,tout) silently includes the next vertex.

10.10 Additional exercises

Exercise 10.5 (easy)**.** CSES *Tree Diameter*.

Exercise 10.6 (easy)**.** CSES *Tree Distances I* — BFS both ends.

Exercise 10.7 (easy)**.** CSES *Subordinates*.

Exercise 10.8 (easy)**.** USACO-S *MooTube* — threshold component sizes.

Exercise 10.9 (easy)**.** USACO-B *Family Tree* — LCA-style ancestry.

Exercise 10.10 (medium)**.** CSES *Tree Matching* — greedy DP on children.

Exercise 10.11 (medium)**.** CSES *Company Queries II* — LCA direct.

Exercise 10.12 (medium)**.** USACO-S *Milk Visits* — LCA + path membership.

Exercise 10.13 (medium)**.** AC *Count Descendants* — Euler + count.

Exercise 10.14 (medium)**.** CF *Minimize the Diameter* — reroot variant.

Exercise 10.15 (medium)**.** CSES *Subtree Queries* — Euler tour + range structure.

Exercise 10.16 (medium)**.** CSES *Path Queries* — the same tour answering root-to-node aggregates.

Exercise 10.17 (hard)**.** CSES *Finding a Centroid* — subtree-size bisect.

Exercise 10.18 (hard)**.** CSES *Counting Paths* — difference array on paths.

Exercise 10.19 (hard)**.** CSES *Fixed-Length Paths I* — centroid decomposition preview.

Notes

Post-order/reroot/diameter patterns follow Laaksonen's tree chapter [2] and Sannemo [1]; binary lifting and Euler-tour RMQ are the USACO Guide's Gold tree modules [3]; cp-algorithms carries the two-DFS proof and lifting table [5]; Halim supplies named variants [7]; archive problems are official CSES/USACO [4].

Further reading

- USACO Guide, "Introduction to Tree Algorithms" (intermediate) — subtree aggregations and the two-DFS diameter of §10.3.
  `https://usaco.guide/silver/intro-tree`
- USACO Guide, "Euler Tour Technique" (advanced) — the tour view of §10.5 with data-structure queries on top.
  `https://usaco.guide/gold/tree-euler`
- USACO Guide, "Binary Jumping" (advanced) — the 2^k ancestors machinery behind §10.6/§10.7.
  `https://usaco.guide/plat/binary-jump`
- cp-algorithms, "Lowest Common Ancestor — Binary Lifting" (reference) — §10.6 with the up$[k][v]$ table in full.
  `https://cp-algorithms.com/graph/lca_binary_lifting.html`
- CSES 1131 *Tree Diameter* and 1135 *Distance Queries* (practice) — 10.2/10.5 verbatim.
  `https://cses.fi/problemset/task/1131`
- CSES 1688 *Company Queries II* (practice) — LCA-as-submission for 10.4.
  `https://cses.fi/problemset/task/1688`

WATCH INSTEAD

Best first. "LCA – Lowest Common Ancestor" — Errichto Algorithms, 16 min — `https://youtu.be/dOAxrhAUIhA`. The binary-lifting walk of §10.6–§10.7 from a contestant's angle; same `up[k][v]` table, slightly different lift order in code. "How to find Diameter of a Tree" — Luv, 12 min — `https://youtu.be/m8qXEz1Xe8w`. The two-DFS diameter trick of §10.3 plus why it is correct on trees only — matches the book's caution about weighted/non-tree inputs. Access date 2026-10-10.

CHAPTER 11

Dynamic programming I: the method and the classics

Dynamic programming is not an algorithm — it is a bookkeeping promise: never solve the same subproblem twice. The craft is five choices in order (state, transition, base case, fill order, answer location), plus one trap that wrong-answers more submissions than every other combined: ordered versus unordered counting. After this chapter you will run the recipe end to end on coins, knapsack, LIS and grids, and know the two loop orders by heart.

11.1 The same problem twice

Memoization recurses and caches; tabulation fills a table in dependency order. They compute the *same set* — memoization visits only reachable states (safer when the state space is sparse), tabulation visits all states in order (safer on recursion depth).

11.2 The DP recipe

Five lines: (1) *state* — what identifies a subproblem; (2) *transition* — how it reduces; (3) *base case*; (4) *order* — dependencies before dependents (this is DP on the subproblem DAG, Figure 11.1); (5) *answer* — which state(s) to read.

11.3 Coin problems

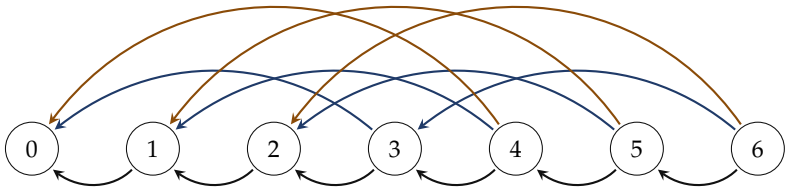
Minimising coins is minimisation (min over last coin); counting ways is summation — and the summation splits into two different problems by loop order.

Example 11.1 (CSES *Minimizing Coins*)**.** Fewest coins to make x.

Strategy. State = amount; transition = min over last coin used. Top-down and bottom-up compute the same states.

Solution.

```
#include "coin.h"
int main() {
   int n, x; scanf("%d %d", &n, &x);
   vector<int> c(n);
```



node x: "fewest coins for amount x"; edge $x \to x - c$: "last coin was c"

coin 1 black (below), coins 3 blue and 4 amber (above); fill order $0, 1, \ldots, 6$ is a topological order

Figure 11.1: The subproblem DAG for coin DP. Notice "topological order" is literally a fill order — that is what DP is.

```
    for (auto& v : c) scanf("%d", &v);
    auto tab = coin_tab(x, c);
    printf("%d\n", tab[x] >= CINF ? -1 : tab[x]);
}
```

Real output (CSES sample):

```
3
```

Example 11.2 (CSES *Dice Combinations* → *Coin Combinations I vs II*)**.** Ways to sum to x: I counts ordered sequences, II unordered multisets.

Strategy. Same table, different nesting: sum-outer counts permutations separately (ordered); coin-outer fixes each coin's count before the next (unordered). Swap the loops, swap the answer.

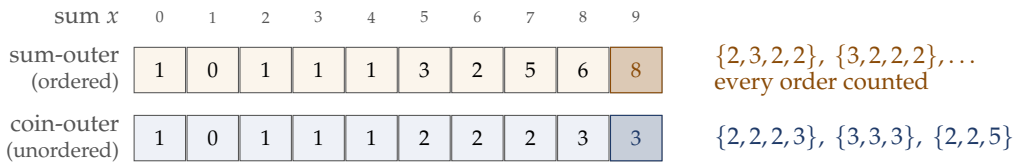
Solution.

```
// coin_combos.cpp -- CSES Coin Combinations I (ordered) vs II (
    unordered).
#include <cstdio>
const long long MOD = 1e9+7;
int main() {
    int n, x; scanf("%d %d", &n, &x);
    int c[105];
    for (int i = 0; i < n; i++) scanf("%d", &c[i]);
    static long long a[1000006], b[1000006];
    a[0] = 1;
    for (int s = 1; s <= x; s++) // ordered: sum outer
       for (int i = 0; i < n; i++)
          if (s >= c[i]) a[s] = (a[s] + a[s - c[i]]) % MOD;
    b[0] = 1;
    for (int i = 0; i < n; i++) // unordered: coin outer
```

```
        for (int s = c[i]; s <= x; s++)
            b[s] = (b[s] + b[s - c[i]]) % MOD;
    printf("%lld %lld\n", a[x], b[x]);
}
```

Real output (CSES sample, I then II):

```
8 3
```



same recurrence $\mathrm{dp}[x] \mathrel{+}= \mathrm{dp}[x-c]$, only the nesting of the two loops differs

Figure 11.2: The ordered/unordered split. Notice both use the identical recurrence — the loop order *is* the difference.

11.4 Knapsack

0/1 knapsack: each item at most once — the inner capacity loop must run *descending*, or item i reuses itself and you have solved the unbounded problem. Unbounded is the ascending twin. Reconstruction walks the table backwards.

Example 11.3 (CSES *Book Shop*)**.** Max pages within budget.

Strategy. $\mathrm{dp}[i][x]$ = best using items $1..i$; descend capacity so item i appears at most once.

Solution.

```
#include "knapsack.h"
#include <cstdio>
int main() {
    int n, x; scanf("%d %d", &n, &x);
    vector<int> w(n+1), v(n+1);
    for (int i = 1; i <= n; i++) scanf("%d", &w[i]);
    for (int i = 1; i <= n; i++) scanf("%d", &v[i]);
    vector<char> take;
    printf("%d\n", knapsack(n, x, w, v, take));
    int first = 1;
    for (int i = 1; i <= n; i++)
        if (take[i]) { printf("%s%d", first ? "" : " ", i); first = 0; }
    printf("\n");
}
```

Real output (CSES sample; second line is the reconstructed take-table — books 1 and 3, weight $4+5 \le 10$, value $5+8=13$):

```
13
1 3
```

budget x	0	1	2	3	4	5	6	7	8	9	10
before items	0	0	0	0	0	0	0	0	0	0	0
item 1 (4, 5)	0	0	0	0	5	5	5	5	5	5	5
item 2 (8, 12)	0	0	0	0	5	5	5	5	12	12	12
item 3 (5, 8)	0	0	0	0	5	8	8	8	12	13	13
item 4 (3, 1)	0	0	0	1	5	8	8	8	12	13	13

dp[10] = 13: books 1 and 3 (price $4+5 \le 10$, pages $5+8=13$)

Figure 11.3: Book Shop's table filling. Notice each frame only reads the previous row — that dependency shape is what licenses the one-array version.

11.5 LIS in $O(n \log n)$

Keep $L[i]$ = smallest possible tail of an increasing subsequence of length $i+1$. Three lemmas (USACO-Guide style): L is always sorted; each read x replaces `lower_bound`(x), never lengthening unless it appends; the answer is $|L|$.

Listing 11.1: `lis.h`: the 7-line core.

```
// lis.h -- longest increasing subsequence, O(n log n).
#include <algorithm>
#include <vector>
using namespace std;
inline int lis_len(const vector<int>& a) {
   vector<int> L; // L[i]=smallest tail of len i+1
   for (int x : a) {
      auto it = lower_bound(L.begin(), L.end(), x);
      if (it == L.end()) L.push_back(x); else *it = x;
   }
   return L.size();
}
```

Example 11.4 (CSES *Increasing Subsequence*)**.** Longest strictly increasing subsequence.

Strategy. The $O(n^2)$ is obvious; the tableau version is the same answers in $O(n \log n)$.

Solution.

```
#include "lis.h"
#include <cstdio>
int main() {
   int n; scanf("%d", &n);
```

```
    vector<int> a(n);
    for (auto& x : a) scanf("%d", &x);
    printf("%d\n", lis_len(a));
}
```

Real output (CSES sample):

```
4
```

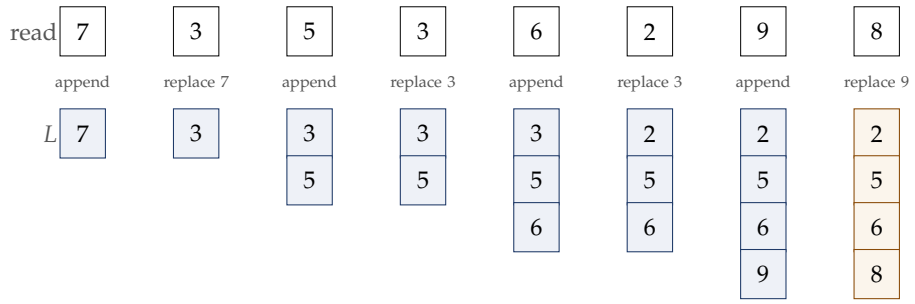


Figure 11.4: The patience array evolving. Notice L's length is the answer — its entries are only potential tails.

11.6 Grid DP

Paths through a grid pull from left and above; obstacles are just zeros. In-place filling works because each cell only reads already- computed cells.

Example 11.5 (CSES *Grid Paths I*)**.** Paths through a grid with traps.

Strategy. dp[r][c] = left + below; trap cells contribute 0.

Solution.

```
// grid_paths.cpp -- CSES Grid Paths I (obstacles, in-place table).
#include <cstdio>
const long long MOD = 1e9+7;
static long long dp[1002][1002];
static char g[1002][1002];
int main() {
    int n; scanf("%d", &n);
    for (int i = 1; i <= n; i++) scanf("%s", g[i] + 1);
    if (g[1][1] == '*' || g[n][n] == '*') { puts("0"); return 0; }
    dp[1][1] = 1;
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= n; j++) {
            if (g[i][j] == '*') { dp[i][j] = 0; continue; }
            if (i > 1 || j > 1)
                dp[i][j] = (dp[i-1][j] + dp[i][j-1]) % MOD;
```

```
        }
    printf("%lld\n", dp[n][n]);
}
```

Real output (CSES sample):

```
3
```

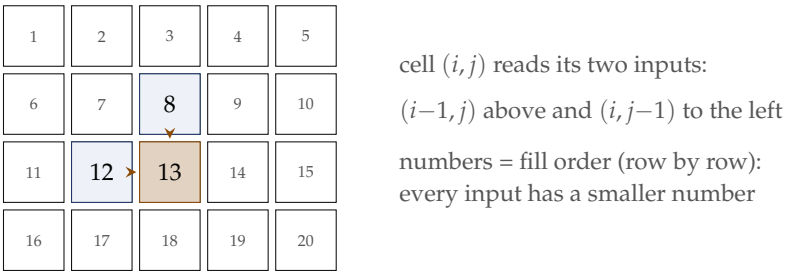


Figure 11.5: Grid dependencies. Notice the two-source pull is why a single pass suffices — every cell's inputs are computed earlier.

11.7 Push vs pull

Pull: each state reads its predecessors. Push: each state updates its successors. Same recurrence, different direction — pull is usually clearer; push wins when the frontier is sparse.

Exercise 11.1 (easy)**.** State for "largest sum ending exactly at index i" vs "largest sum anywhere in $1..i$" — which is which, and why can't they swap?

Exercise 11.2 (medium)**.** Coins $\{1, 2\}$, $x = 3$: enumerate the ordered and unordered counts and explain the missing summands.

Exercise 11.3 (medium)**.** An ascending inner loop in 0/1 knapsack is a *different* problem. Name it and explain the single direction change.

Exercise 11.4 (medium)**.** Predict the output of coin-outer counting if the answer table printed is not the final cell but a sum over all cells — what goes wrong?

11.8 Chapter summary

11.9 Common mistakes

- **Ordered-vs-unordered swap.** The single most common wrong answer: fix the loop nesting to the semantics asked.
- **Ascending capacity on 0/1 knapsack.** Item reuse slips in silently — descend, or use two rows.

Problem	State	Transition	Trap
min coins	amount	min over last coin	INF guard
count ways (ord)	amount	sum-outer loops	order counted
count ways (unord)	amount	coin-outer loops	order collapsed
0/1 knapsack	(item, cap)	descending cap	self-reuse
LIS	tail min	lower_bound	strict vs non
grid paths	(r,c)	left + below	obstacles = 0

Table 11.1: Five states, five transitions — and the loop order that changes the answer, not the code.

- **INF arithmetic.** `min + 1` on an unreachable state overflows — guard with a sentinel comparison.
- **Iterating coins inside on unbounded with reuse across calls.** Recompute per case; shared tables bleed answers.
- **LIS strictness.** `lower_bound` for strict, `upper_bound` for non-decreasing — swapping them silently changes the problem.

11.10 Additional exercises

Exercise 11.5 (easy)**.** CSES *Coin Combinations I/II*.

Exercise 11.6 (easy)**.** CSES *Removing Digits* — greedy-ish DP.

Exercise 11.7 (easy)**.** CSES *Money Sums* — reachable sums.

Exercise 11.8 (easy)**.** USACO-G *Fruit Feast* — knapsack variant.

Exercise 11.9 (medium)**.** CSES *Array Description* — bounded DP.

Exercise 11.10 (medium)**.** CSES *Two Sets II* — subset-sum DP.

Exercise 11.11 (medium)**.** CSES *Rectangle Cutting* — cuts DP.

Exercise 11.12 (medium)**.** USACO-G *Hoof, Paper, Scissors* — prefix-state DP.

Exercise 11.13 (medium)**.** USACO-G *Time is Mooney* — time-expanded DP.

Exercise 11.14 (medium)**.** USACO-G *Teamwork* — group-sum DP.

Exercise 11.15 (medium)**.** CSES *Grid Paths* (DP set) — grid DP over reachable cells.

Exercise 11.16 (medium)**.** CSES *Counting Towers* — state-pair recurrence; find the two states yourself.

Exercise 11.17 (hard)**.** CSES *Projects* — weighted intervals DP.

Exercise 11.18 (hard)**.** USACO-G *Snakes* — herds DP.

Exercise 11.19 (hard)**.** USACO-G *Talent Show* — ratio DP.

Notes

The memo/tabulation framing and the ordered-unordered centerpiece follow Laaksonen [2] and Sannemo [1]; the patience-tableau three-lemma LIS is the USACO Guide's presentation [3]; knapsack loop-direction and grid DP are standard in cp-algorithms [5] and Halim [7]; archive problems are official CSES/USACO [4].

Further reading

- USACO Guide, "Introduction to DP" (intermediate) — the state/transition method of §11.2 with Gold-level problems.
  https://usaco.guide/gold/intro-dp
- USACO Guide, "Knapsack DP" (intermediate) — §11.4 extended to its classic variants.
  https://usaco.guide/gold/knapsack
- USACO Guide, "Longest Increasing Subsequence" (intermediate) — §11.5 including the $O(n \log n)$ patience form.
  https://usaco.guide/gold/lis
- cp-algorithms, "Introduction to Dynamic Programming" (reference). — memoization-vs-tabulation of §11.1.
  https://cp-algorithms.com/dynamic_programming/intro-to-dp.html
- CSES 1633 *Dice Combinations* and 1634 *Minimizing Coins* (practice) — 11.2/11.1 verbatim.
  https://cses.fi/problemset/task/1633
- CSES 1158 *Book Shop* and 1145 *Increasing Subsequence* (practice). — the chapter's two flagship submissions.
  https://cses.fi/problemset/task/1158

WATCH INSTEAD

Best first. "15. Dynamic Programming, Part 1: SRTBOT, Fib, DAGs, Bowling" — MIT OpenCourseWare, 57 min — https://youtu.be/r4-cftqTcdI. The subproblem/recurrence/topological/bottom-up recipe of §11.2—§11.1; OCW names it "SRTBOT", the book names the same five questions without the acronym. "16. Dynamic Programming, Part 2: LCS, LIS, Coins" — MIT OpenCourseWare, 59 min — https://youtu.be/KLBCUx1is2c. The coin and LIS recurrences of §11.3 and §11.5 derived on the board; same formulation, same memo/table duality. Access date 2026-10-10.

CHAPTER 12

Dynamic programming II: masks, digits, ranges, and trees

Chapter 11's recipe generalizes: states stop being positions and become *structured objects* — subsets, automaton flags, intervals, and tree-shape state pairs. Each generalization opens a new problem class: bitmask DP for exponential-set problems, digit DP for counting over enormous ranges, range DP for interval games, and tree DP for hierarchical matching. After this chapter you will pick the state shape, write the canonical template for each, and know which optimizations wait at the horizon.

12.1 Subsets as states

A subset of $\{1..n\}$ is an integer: mask DP stores one value per subset, $O(2^n \cdot \text{poly})$ — viable to $n \approx 20$. Enumerating submasks uses `s = (s-1) & mask`.

Example 12.1 (CSES *Elevator Rides*)**.** Fewest rides given per-person weights and a lift capacity.

Strategy. State = the set already placed. dp[mask] tracks *two* things — rides used and weight on the current ride — minimized lexicographically.

Solution.

```
#include "bitmask_dp.h"
#include <cstdio>
int main() {
   int n, x; scanf("%d %d", &n, &x);
   vector<int> w(n);
   for (auto& v : w) scanf("%d", &v);
   printf("%d\n", elevator(n, x, w).first);
}
```

Real output (CSES sample):

```
2
```

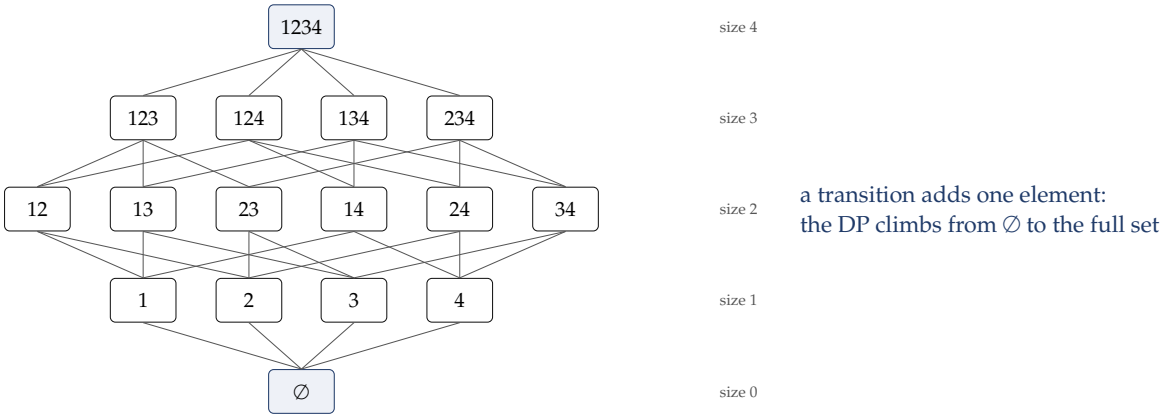


Figure 12.1: The subset lattice. Notice a mask DP always walks upward — each transition adds exactly one element.

12.2 Digit DP

Counting integers $\leq N$ with a digit property: walk the digits most-significant-first carrying a *tight* flag (still matching N's prefix?) and a *leading-zero* flag (the number hasn't started). Tight bounds the next digit; loose reads from memo.

Example 12.2 (CSES *Counting Numbers*)**.** Count integers in $[a, b]$ with no two adjacent digits equal.

Strategy. The automaton is (pos, tight, leading, previous-digit); count(b) − count(a−1).

Solution.

```
#include "digitdp.h"
#include <cstdio>
int main() {
   long long a, b; scanf("%lld %lld", &a, &b);
   printf("%lld\n", count_digits(b) - count_digits(a - 1));
}
```

Real output (CSES sample, a=123, b=321):

```
171
```

12.3 Range DP

Cost over every subarray: dp[l][r] built *by length*, since a range needs its strictly-smaller subranges first. Splits and endpoint choices both fit the shape.

Example 12.3 (CSES *Removal Game*)**.** Two players pick ends; maximize the first player's sum.

Strategy. dp[l][r] = best score difference from this range: $\max(a_l - dp[l+1][r], a_r - dp[l][r-1])$ — opponent-optimal play subtracts.

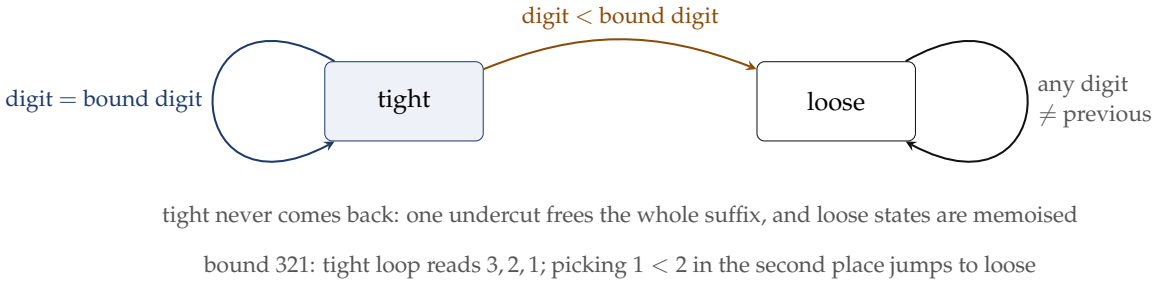


Figure 12.2: The tight/loose automaton. Notice tight never comes back — one undercut frees the whole suffix.

Solution.

```
#include "rangedp.h"
#include <numeric>
#include <cstdio>
int main() {
   int n; scanf("%d", &n);
   vector<long long> a(n);
   for (auto& v : a) scanf("%lld", &v);
   printf("%lld\n", removal_game(a));
}
```

Real output (CSES sample):

```
8
```

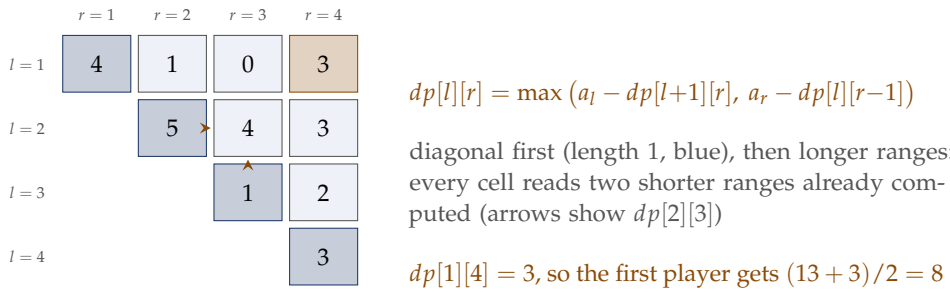


Figure 12.3: By-length filling on the triangle. Notice the diagonal is free — every other cell reads smaller ranges already computed.

Example 12.4 (CSES *Edit Distance*)**.** Minimum edits between two strings.

Strategy. dp[i][j] aligns prefixes: diagonal for match/substitute, left and up for insert/delete. LCS is the same grid with only diagonal-or-skip moves.

Solution.

```
// edit_distance.cpp -- CSES Edit Distance: alignment-grid DP.
#include <algorithm>
```

```
#include <cstdio>
using std::min;
#include <cstring>
static int dp[5002][5002];
int main() {
   static char s[5002], t[5002];
   scanf("%s %s", s + 1, t + 1);
   int n = strlen(s + 1), m = strlen(t + 1);
   for (int i = 0; i <= n; i++) dp[i][0] = i;
   for (int j = 0; j <= m; j++) dp[0][j] = j;
   for (int i = 1; i <= n; i++)
      for (int j = 1; j <= m; j++)
         dp[i][j] = min({dp[i-1][j]+1, dp[i][j-1]+1,
                         dp[i-1][j-1]+(s[i]!=t[j])});
   printf("%d\n", dp[n][m]);
}
```

Real output (CSES sample):

```
2
```

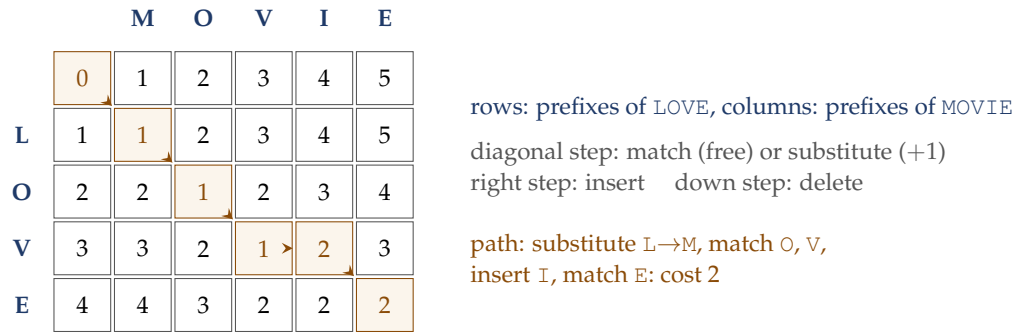


Figure 12.4: The alignment grid with its optimal path. Notice every path is an alignment — the DP is just shortest path on this DAG.

12.4 DP on trees

Rooted trees carry state pairs: *in*[v] = best when v is matched downward, *out*[v] = best when v is free to match upward. Chapter 10's reroot machinery then upgrades per-root answers to all-roots.

Example 12.5 (CSES *Tree Matching*)**.** Maximum edges with no shared endpoint.

Strategy. in/out states: `out[v]` sums children's max; `in[v]` picks the child offering the best edge gain.

Solution.

```
#include "treedp.h"
#include <cstdio>
```

```
int main() {
   int n; scanf("%d", &n);
   vector<vector<int>> ch(n + 1);
   for (int i = 0; i < n - 1; i++) {
      int a, b; scanf("%d %d", &a, &b);
      ch[a].push_back(b); ch[b].push_back(a);
   }
   // build rooted children lists first
   vector<vector<int>> rc(n + 1);
   vector<int> par(n + 1, 0), st = {1};
   par[1] = -1;
   while (!st.empty()) {
      int v = st.back(); st.pop_back();
      for (int u : ch[v]) if (!par[u]) { par[u] = v; rc[v].push_back(u
          ); st.push_back(u); }
   }
   auto [in, out] = tree_match(rc, 1);
   printf("%d\n", max(in, out));
}
```

Real output (5-vertex demo):

```
2
```

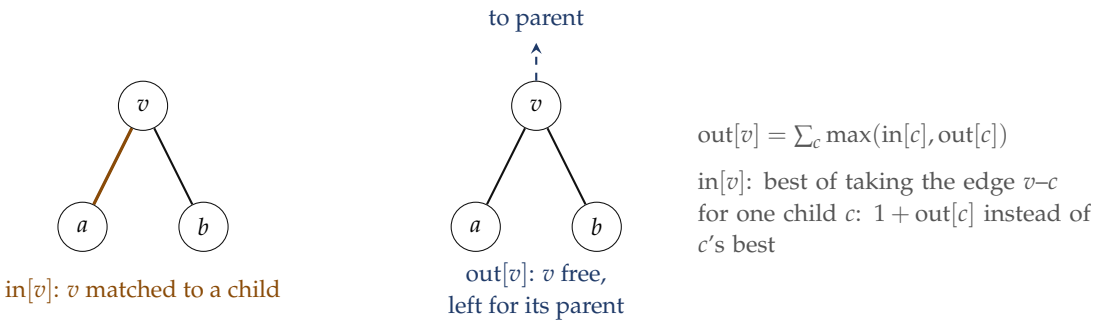


Figure 12.5: The in/out pair. Notice a vertex's "free upward" is what the parent's in-state needs — the pair is the interface.

12.5 The optimisation horizon

One optimization worth teaching now: *bitset knapsack* — reachable sums as a bitmask, `dp |= dp << w` — $O(nC/64)$.

Example 12.6 (CSES *Money Sums*)**.** All reachable coin sums.

Strategy. A sum is a set member: shift the bitset by each coin and OR. One machine word covers 64 reachable positions.

Solution.

```cpp
#include "bitset_knapsack.h"
#include <cstdio>
int main() {
   int n; scanf("%d", &n);
   vector<int> c(n);
   for (auto& v : c) scanf("%d", &v);
   auto dp = money_sums<100001>(c);
   vector<int> ans;
   for (int i = 1; i <= 100000; i++) if (dp[i]) ans.push_back(i);
   printf("%zu\n", ans.size());
   for (size_t i = 0; i < ans.size(); i++)
      printf("%d%c", ans[i], " \n"[i+1==ans.size()]);
}
```

Real output (CSES sample):

```
6
2 3 5 7 8 10
```

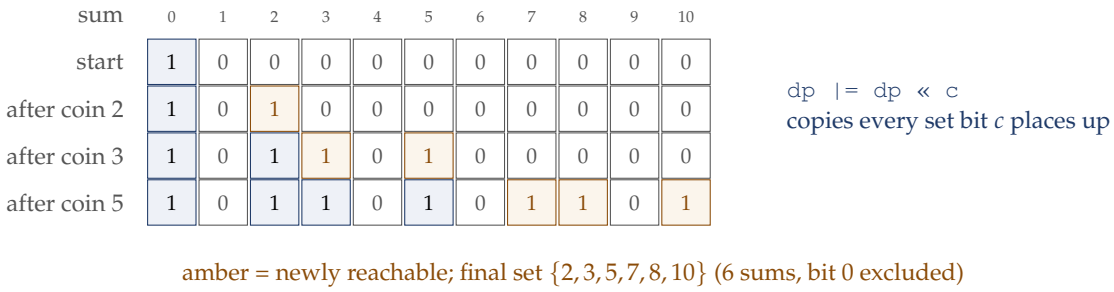


Figure 12.6: The shift-and-or trick. Notice each coin copies every set bit w positions up — parallelism for free.

Further up the band: convex-hull trick, divide-and-conquer DP, Knuth optimization, SOS DP, profile DP — each has its own structural precondition; treat them as named routes (see Notes).

Exercise 12.1 (easy). Iterate the nonempty submasks of mask m in $O(2^{|m|})$ — write the loop and argue each visited value is a submask.

Exercise 12.2 (medium). Why can't the digit-DP memo be shared across tight=1 states? What invariant does the flag preserve?

Exercise 12.3 (medium). Argue by-length is the *only* correct fill order for dp[l][r] = f(dp[l+1][r], dp[l][r-1]).

Exercise 12.4 (medium). In the all-roots matching problem, what exactly does the second pass carry from parent to child?

12.6 Chapter summary

State	Template	Size	Watch
subset	mask DP	$O(2^n n)$	$n \leq 20$
(position, flags)	digit DP	$O(\log n)$	tight not memoizable
interval	range DP	$O(n^3)$ worst	by-length only
(vertex, in/out)	tree DP	$O(n)$	state pair = interface
sum-set	bitset	$O(nC/64)$	word width

Table 12.1: Pick the state shape, then the order that respects its dependency lattice.

12.7 Common mistakes

- **Memoizing tight.** The tight state depends on N's prefix — caching it leaks between queries; memoize only loose.
- **Wrong mask order.** Adding elements needs masks in any order that adds bits; removing needs the reverse — mixing directions double-counts.
- **Range filled by l.** dp[l][r] must see smaller ranges first; any order not sorted by $r - l$ reads stale cells.
- **Forgetting leading-zero.** "Number not started" is a distinct state — counting without it counts padded numbers.
- **Bitmask index off-by-one.** Bit i means element i, not $i+1$ — the bug shows only at n small enough to enumerate.

12.8 Additional exercises

Exercise 12.5 (easy)**.** AC *Matching* — assignment mask DP.

Exercise 12.6 (easy)**.** AC *Grouping* — partition mask DP.

Exercise 12.7 (easy)**.** CSES *Edit Distance*.

Exercise 12.8 (easy)**.** USACO-G *Barn Painting* — tree counting DP.

Exercise 12.9 (medium)**.** CSES *Elevator Rides*.

Exercise 12.10 (medium)**.** CSES *Rectangle Cutting*.

Exercise 12.11 (medium)**.** CSES *Minimal Grid Path* — lexicographic DP.

Exercise 12.12 (medium)**.** USACO-G *Uddered but not Herd* — bitmask ordering.

Exercise 12.13 (medium)**.** AC *Independent Set* — tree in/out.

Exercise 12.14 (medium)**.** CF *Team Building* — mask + DP.

Exercise 12.15 (hard). CSES *Counting Tilings* — profile DP preview.

Exercise 12.16 (hard). CSES *Counting Numbers*.

Exercise 12.17 (hard). CSES *Longest Common Subsequence*.

Exercise 12.18 (hard). USACO-G *Lights Off* — mask + range DP.

Notes

The mask/digit/range templates follow Laaksonen's DP chapters [2] and Sannemo [1]; digit DP's tight automaton and the bitset trick are cp-algorithms and the USACO Guide's treatment [3, 5]; the optimization horizon is signposted in Halim [7]; archive problems are official CSES/USACO [4].

Exercise 12.19 (hard). CSES *Hamiltonian Flights* — Held–Karp counting over masks.

Further reading

- USACO Guide, "Bitmask DP" (advanced) — §12.1 extended to assignment and TSP-shaped problems.
  `https://usaco.guide/gold/dp-bitmasks`
- USACO Guide, "Digit DP" (advanced) — the (pos, tight, leading) automaton of §12.2 with variants beyond this chapter's adjacent-digit predicate.
  `https://usaco.guide/gold/digit-dp`
- USACO Guide, "Range DP" (advanced) — the by-length interval recurrence of §12.3 on more problems.
  `https://usaco.guide/gold/dp-ranges`
- USACO Guide, "Tree DP" (advanced) — §12.4 extended to rerooting and heavier tree states.
  `https://usaco.guide/gold/dp-trees`
- CSES 1653 *Elevator Rides* (practice) — 12.1 verbatim; the canonical bitmask-DP submission.
  `https://cses.fi/problemset/task/1653`
- CSES 2220 *Counting Numbers* and 1655 *Removal Game* (practice) — 12.2/12.3 as archive runs.
  `https://cses.fi/problemset/task/2220`

WATCH INSTEAD

Best first. "Introduction to Digit Dynamic Programming" — Kartik Arora, 31 min — `https://youtu.be/heUFId6Qd1A`. The tight/leading flags and per-digit enumeration of §12.2 built on a SPOJ-style count; uses `dp[pos][tight][prev]` exactly as `digitdp.h` does. "Episode 20 — Bitmask Dynamic Programming" — Algorithms Live!, 67 min — `https://youtu.be/rlTkd4yOQpE`. Held–Karp-style mask states

and subset iteration for §12.1; stream format, deeper than needed — watch the first half for the state design. Access date 2026-10-10.

CHAPTER 13

Fenwick and segment trees

Prefix sums (Chapter 5) answer range queries once; this chapter answers them *under mutation*. Two structures cover the whole family: the Fenwick tree for prefix-shaped questions, the segment tree for everything else — arbitrary monoids, range updates with lazy tags, and order statistics by descent. After this chapter you will pick the right structure off the update/query shape, write each template, and know the two subtle failure modes that corrupt silent answers.

13.1 The range-query family

Four shapes appear: point-update/range-query (the classic), range-update/point-query (the difference trick), and range-update/range-query (lazy). A decision table: prefix-ordered aggregate and tight memory → Fenwick; arbitrary monoid or descent queries → segment tree.

13.2 The Fenwick tree

$t[i]$ stores the sum of the range $[i - \text{lowbit}(i) + 1,\ i]$ — a staircase of power-of-two lengths. Prefix sum walks down by $i \mathrel{-}= \text{lowbit}(i)$, update walks up by $i \mathrel{+}= \text{lowbit}(i)$: $O(\log n)$ each, n cells.

Example 13.1 (CSES *Dynamic Range Sum Queries*)**.** Point updates, sum queries.

Strategy. The problem is exactly Fenwick's shape; we show the segtree solution too for the trade-off table.

Solution.

```
#include "segtree.h"
#include <cstdio>
int main() {
   int n, q; scanf("%d %d", &n, &q);
   vector<long long> a(n);
   for (auto& v : a) scanf("%lld", &v);
   SegTree<long long> st(a, seg_sum, 0);
   while (q--) {
      int t; scanf("%d", &t);
      if (t == 1) { int k; long long u; scanf("%d %lld", &k, &u); st.
          set(k - 1, u); }
```

```
        else { int a_, b; scanf("%d %d", &a_, &b); printf("%lld\n", st.
            query(a_ - 1, b)); }
    }
}
```

Real output (CSES sample, segtree version):

```
14
2
11
2
```

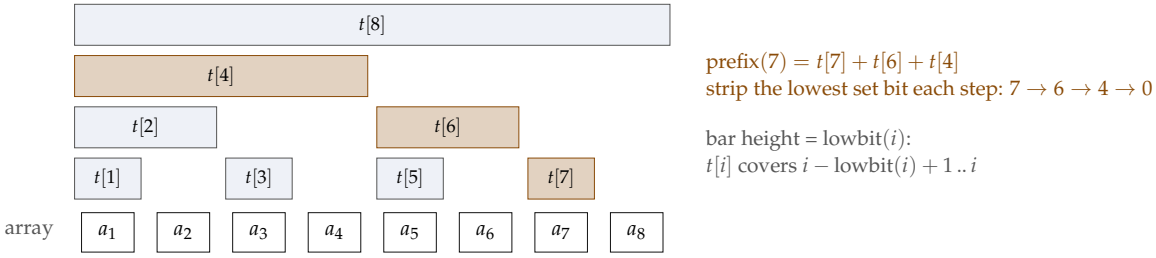


Figure 13.1: The lowbit staircase. Notice every index's covered length is its lowest set bit — the whole structure is index arithmetic.

13.3 The segment tree

A $2n$ array: leaves at $n+i$, node v combines children $2v, 2v+1$. Any associative monoid works — sum, min, gcd — because a query only ever reads $O(\log n)$ whole nodes.

Example 13.2 (CSES *Dynamic Range Minimum Queries*)**.** Point updates, *minimum* queries.

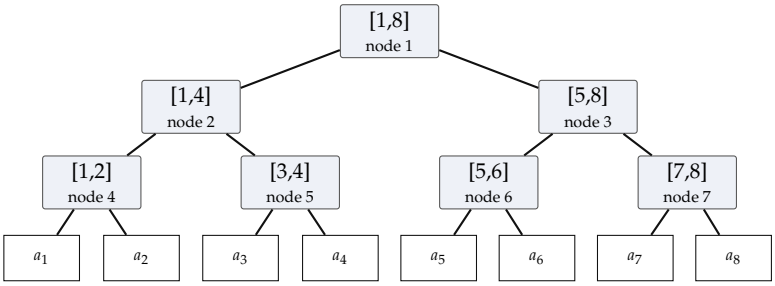
Strategy. Swap the monoid: same tree, `min` combine. A Fenwick can't do this — min has no inverse, so prefixes don't subtract.

Solution.

```
#include "segtree.h"
#include <cstdio>
int main() {
    int n, q; scanf("%d %d", &n, &q);
    vector<long long> a(n);
    for (auto& v : a) scanf("%lld", &v);
    SegTree<long long> st(a, seg_min, (long long)4e18);
    while (q--) {
        int t; scanf("%d", &t);
        if (t == 1) { int k; long long u; scanf("%d %lld", &k, &u); st.
            set(k - 1, u); }
        else { int a_, b; scanf("%d %d", &a_, &b); printf("%lld\n", st.
            query(a_ - 1, b)); }
    }
}
```

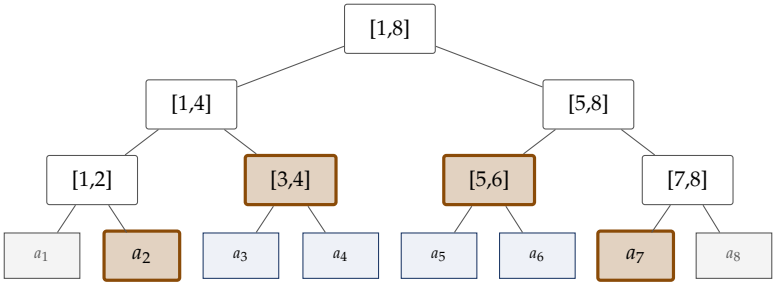
Real output (CSES sample):

```
2
1
3
```



array of size $2n$: node v stores the combine of its segment; children are $2v$ and $2v+1$

Figure 13.2: Node $\leftrightarrow$ segment. Notice the query reads *nodes*, not elements — each node is a whole covered range.



query $[2,7]$ reads the amber nodes only: a_2, $[3,4]$, $[5,6]$, a_7 — at most $2\log_2 n$ of them

Figure 13.3: Range decomposition. Notice a query peels at most two nodes per level — $\leq 2\log n$ nodes total.

13.4 Lazy propagation

A range update stops at covered nodes, tagging them; the tag travels down only when a later query needs through. The canonical pair: add-tag with sum-query.

Example 13.3 (CSES *Range Update Queries*)**.** Range updates, point queries.

Strategy. The *difference trick* turns range-add into two point-adds on a prefix array — a Fenwick of differences. The lazy segtree generalizes to range-range.

Solution.

```
// range_update_queries.cpp -- range-update/point-query via diff on
   BIT.
#include "fenwick.h"
#include <cstdio>
```

```
int main() {
   int n, q; scanf("%d %d", &n, &q);
   vector<long long> a(n + 1);
   for (int i = 1; i <= n; i++) scanf("%lld", &a[i]);
   Fenwick f(n);
   while (q--) {
      int t; scanf("%d", &t);
      if (t == 1) { int l, r, x; scanf("%d %d %d", &l, &r, &x);
         f.add(l, x); f.add(r + 1, -x); }
      else { int k; scanf("%d", &k); printf("%lld\n", a[k] + f.sum(k))
         ; }
   }
}
```

Real output (CSES sample):

```
5
```

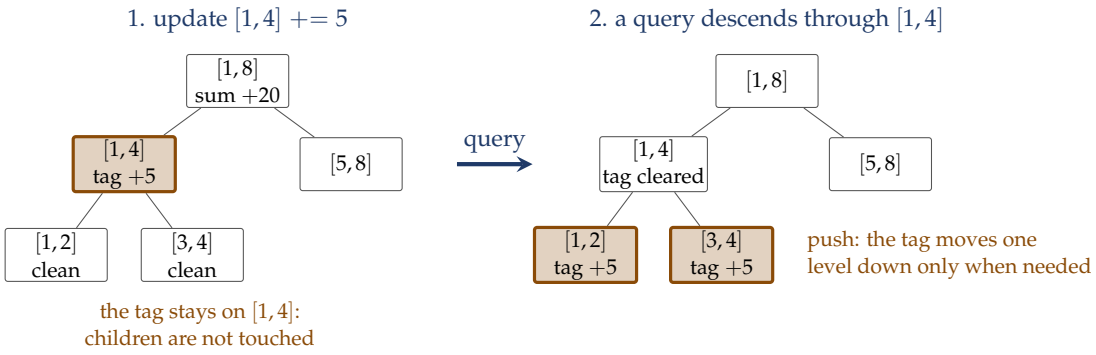


Figure 13.4: Tag deferral. Notice the invariant: a node's aggregate is always correct; its descendants are behind by the tag.

13.5 Walking the tree

Descent queries need no new code: at node v, if the left child's value decides, go left, else right. "First room with enough capacity" and "the k-th remaining element" are the same walk.

Example 13.4 (CSES *Hotel Queries*)**.** Each group wants the first room holding them.

Strategy. Store subtree max; descend left whenever left-max $\geq r$. One trap: the walk needs leaves on a power-of-two grid — pad the array up.

Solution.

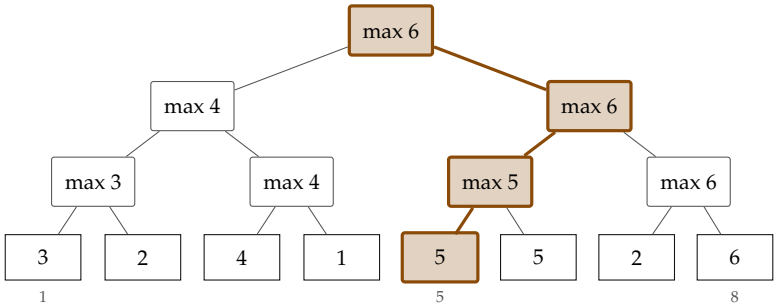
```
// hotel_queries.cpp -- first room with capacity >= r via walk-on-tree.

#include "segtree.h"
#include <algorithm>
```

```
#include <cstdio>
struct Hotel {
   int n2, n;
   static vector<long long> padded(const vector<long long>& a, int& n2
      ) {
      n2 = 1; while (n2 < (int)a.size()) n2 <<= 1;
      vector<long long> pad(n2, -1);
      for (size_t i = 0; i < a.size(); i++) pad[i] = a[i];
      return pad;
   }
   SegTree<long long> st;
   // walk-on-tree needs leaves on a power-of-two grid: pad to n2
   Hotel(vector<long long>& a)
      : n2(0), n(a.size()),
        st(padded(a, n2),
           [](long long x, long long y){return x>y?x:y;}, -1) {}
   int book(long long r) {
      if (st.t[1] < r) return 0;
      int v = 1;
      while (v < n2) v = st.t[2*v] >= r ? 2*v : 2*v+1;
      int p = v - n2; // leaf position
      if (p >= n) return 0; // only padding qualified
      st.t[v] -= r;
      for (v >>= 1; v; v >>= 1)
         st.t[v] = st.combine(st.t[2*v], st.t[2*v+1]);
      return p + 1;
   }
};
int main() {
   int n, m; scanf("%d %d", &n, &m);
   vector<long long> a(n);
   for (auto& v : a) scanf("%lld", &v);
   Hotel h(a);
   while (m--) { long long r; scanf("%lld", &r); printf("%d ", h.book(
      r)); }
   puts("");
}
```

Real output (CSES sample):

```
3 8 1 0 0
```



find the first room with capacity ≥ 5: left max $4 < 5$, so go right; left child max $5 \geq 5$, so go left; leaf 5 is the answer — one root-to-leaf walk

Figure 13.5: The descent. Notice the search is guided by the same aggregate the query would read — no separate index needed.

13.6 Order statistics

The Fenwick of *counts* supports k-th-element descent and erases — "which position is the k-th survivor".

Example 13.5 (CSES *List Removals*)**.** Repeatedly remove the k-th remaining element.

Strategy. Fenwick over 1s; `lower_bound(k)` finds the position whose count-prefix hits k; erase with `add(p,-1)`.

Solution.

```
#include "orderstat.h"
#include <cstdio>
int main() {
   int n; scanf("%d", &n);
   vector<long long> a(n + 1);
   for (int i = 1; i <= n; i++) scanf("%lld", &a[i]);
   OrderStat os(n);
   for (int i = 0; i < n; i++) {
      long long k; scanf("%lld", &k);
      int p = os.f.lower_bound(k);
      printf("%lld%c", a[p], " \n"[i + 1 == n]);
      os.f.add(p, -1);
   }
}
```

Real output (CSES sample):

```
1 2 2 6 4
```

Exercise 13.1 (easy)**.** Which of the four update/query shapes is *not* solvable by a plain Fenwick + difference array?

Exercise 13.2 (medium)**.** Explain exactly why a Fenwick can't answer range-min — what property of sum is being used?

Exercise 13.3 (medium)**.** In lazy propagation, why must the tag be pushed *before* reading the children's aggregates?

Exercise 13.4 (medium)**.** State the loop invariant of the tree descent and use it to argue the walk ends at the leftmost qualifying leaf.

13.7 Chapter summary

Shape	Structure	Cost	Limit
point-upd, prefix-q	Fenwick	$\log n$, n cells	invertible only
point-upd, range-q	segtree	$\log n$, $2n$	any monoid
range-upd, point-q	diff + Fenwick	$\log n$	no range query
range-upd, range-q	lazy segtree	$\log n$, $4n$	tag design
descent queries	walk on tree	$\log n$	pow-2 grid

Table 13.1: Pick by the update/query pair, then by the monoid.

13.8 Common mistakes

- **Fenwick on non-invertible ops.** Min/max have no inverse — prefix differences don't exist; use a segtree.
- **Descent on a non-power-2 layout.** Iterative segtrees interleave leaf ranges when n isn't a power of two; the walk silently lands on the wrong leaf — pad to a power of two.
- **Forgetting the push.** Reading a child under an unpushed tag returns stale data — always push on the way down.
- **Off-by-one leaves.** $t[n + i]$ indexes position i, not $i{+}1$ — mixing conventions shifts every update.
- **Tag/aggregate mismatch.** The lazy tag must compose with *itself* in the update's order; add-then-assign is a different monoid than assign-then-add.

13.9 Additional exercises

Exercise 13.5 (easy)**.** CSES *Static Range Sum Queries*.

Exercise 13.6 (easy)**.** CSES *Static Range Minimum Queries*.

Exercise 13.7 (easy)**.** CSES *Range Xor Queries*.

Exercise 13.8 (easy)**.** USACO-G *Haircut* — inversions on BIT.

Exercise 13.9 (easy)**.** USACO-G *Balanced Photo*.

Exercise 13.10 (medium). CSES *Dynamic Range Sum Queries*.

Exercise 13.11 (medium). CSES *Dynamic Range Minimum Queries*.

Exercise 13.12 (medium). CSES *Salary Queries* — compress + descent.

Exercise 13.13 (medium). USACO-G *Sleepy Cow Sorting* — BIT of counts.

Exercise 13.14 (medium). USACO-G *Why Did the Cow Cross the Road III*.

Exercise 13.15 (medium). Kattis *Mega Inversions*.

Exercise 13.16 (hard). CSES *Prefix Sum Queries*.

Exercise 13.17 (hard). CSES *Range Updates and Sums* — two-tag lazy.

Exercise 13.18 (hard). CSES *Distinct Values Queries*.

Exercise 13.19 (hard). CF *Irrigation*.

Notes

Fenwick's lowbit staircase and the iterative segtree layout follow Laaksonen [2] and Sannemo [1]; the Al.Cash iterative segtree and descent patterns are the cp-algorithms treatment [5] (with the USACO-Guide lazy-propagation walkthrough [3]); Halim's structure-selection advice shaped the decision table [7]; archive problems are official CSES/USACO [4].

Further reading

- USACO Guide, "Point Update Range Sum" (intermediate) — the Fenwick/segtree point-update family of §13.1 with Gold problems.
  `https://usaco.guide/gold/PURS`
- cp-algorithms, "Fenwick Tree" (reference) — §13.2 plus range-update tricks and the `lower_bound` descent of §13.6.
  `https://cp-algorithms.com/data_structures/fenwick.html`
- cp-algorithms, "Segment Tree" (reference) — §13.3–§13.4 including lazy propagation in full generality.
  `https://cp-algorithms.com/data_structures/segment_tree.html`
- USACO Guide, "Segment Tree Beats" (advanced) — what lies beyond the monoid rule of §13.3: chmin-style range bounds.
  `https://usaco.guide/adv/segtree-beats`
- CSES 1648 *Dynamic Range Sum Queries* and 1649 *Dynamic Range Minimum Queries* (practice) — 13.1/13.2 verbatim.
  `https://cses.fi/problemset/task/1648`
- CSES 1143 *Hotel Queries* and 1749 *List Removals* (practice) — the descent-on-tree problems of §13.5–§13.6.
  `https://cses.fi/problemset/task/1143`

WATCH INSTEAD

Best first. "Fenwick Tree (Binary Index Tree) — Quick Tutorial and Source Code Explanation" — Stable Sort, 9 min — https://youtu.be/uSFzHCZ4E-8. The lowbit trick of §13.2 line by line; uses `bit` array name for the book's `t`, same indexing. "Segment Tree (Implementation)" — Errichto Hard Algorithms, 79 min — https://youtu.be/2FShdqn-Oz8. Recursive segtree and lazy propagation coded live, covering §13.3–§13.4; deeper and longer than the chapter — pairs with `lazyseg.h`. Access date 2026-10-10.

CHAPTER 14

Number theory and combinatorics

Contests count on these two toolkits constantly: number theory for divisibility, primes, and modular machinery; combinatorics for the counting models every probability, DP, or hashing problem eventually reduces to. After this chapter you can factorize at sieve speed, move safely inside a modulus, choose between stars-and-bars and inclusion–exclusion by the shape of the constraint, and spot the linear recurrences that exponentiate.

14.1 Divisibility and gcd

The Euclidean algorithm is an invariant machine: $\gcd(a, b)$ equals $\gcd(b, a \bmod b)$ because common divisors of the pair are unchanged by the reduction. The *extended* version returns Bézout coefficients — x, y with $ax + by = \gcd(a, b)$ — which is what makes modular division well-defined.

```
// gcdext.h -- extended Euclidean algorithm: returns (g, x, y) with
// ax + by = g = gcd(a,b). Invariant: every step's pair is an integer
// combination of (a,b); termination by strictly decreasing remainder.
#include <tuple>
using namespace std;
inline tuple<long long,long long,long long> gcdext(long long a, long
    long b) {
   if (b == 0) return {a, 1, 0};
   auto [g, x, y] = gcdext(b, a % b);
   return {g, y, x - (a / b) * y};
}
// modular inverse of a mod m (needs gcd(a,m)=1)
inline long long inv(long long a, long long m) {
   auto [g, x, y] = gcdext(a, m);
   (void)y;
   return ((x % m) + m) % m;
}
```

14.2 Primes

The sieve marks every composite by its smallest factor; storing that factor per cell gives factorization in $O(\log n)$ per query. Trial division to $\sqrt{n}$ suffices for single values.

Example 14.1 (CSES *Counting Divisors*)**.** Count the divisors of each queried x.

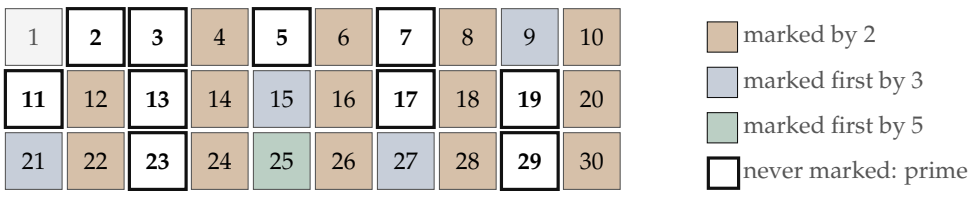
Strategy. SPF table once, then $\prod(e_i + 1)$ per factorization — no $\sqrt{x}$ loop per query.

Solution.

```
#include "sieve.h"
#include <cstdio>
int main() {
   int n; scanf("%d", &n);
   vector<int> spf = spf_sieve(1000000);
   while (n--) {
      long long x; scanf("%lld", &x);
      long long d = 1;
      for (auto& [p, e] : factorize(x, spf)) d *= e + 1;
      printf("%lld\n", d);
   }
}
```

Real output (CSES sample):

```
5
2
6
```



only primes $\leq \sqrt{30}$ (2, 3, 5) mark anything; each marks only its own multiples

Figure 14.1: Sieve marking on $n = 30$. Notice each prime marks only its own multiples — the survivors were never touched.

14.3 Modular arithmetic

Addition, subtraction, multiplication and (with an inverse) division all commute with reduction — so reduce at every step. The inverse exists iff $\gcd(a, m) = 1$: for prime m, $a^{-1} = a^{m-2}$ (Fermat); generally, extended gcd returns it.

Example 14.2 (CSES *Exponentiation II*)**.** $a^{b^c} \bmod 10^9+7$ for triples.

Strategy. The exponent itself is huge; Fermat reduces it mod $m-1$ first (when $\gcd(a, m) = 1$), then ordinary binpow.

Solution.

```
// exponentiation2.cpp -- a^(b^c) mod 1e9+7: Fermat reduces the
// exponent's exponent mod (m-1) when gcd(a,m)=1.
#include "binpow.h"
#include <cstdio>
int main() {
   const long long M = 1000000007;
   int n; scanf("%d", &n);
   while (n--) {
      long long a, b, c; scanf("%lld %lld %lld", &a, &b, &c);
      printf("%lld\n", binpow(a, binpow(b, c, M - 1), M));
   }
}
```

Real output (CSES sample):

```
2187
279632277
91039580
```

14.4 Binary exponentiation

a^b by squaring: write b's bits, square per position, multiply into the answer on set bits — $O(\log b)$ for exponents that would take centuries linearly.

14.5 The `modint` struct

The class you write once: operators that reduce internally so client code reads like ordinary arithmetic. Two silent failures live inside: $(a - b)$ without $+m$ goes negative; $a * b$ in `int` overflows 32-bit before the reduction.

```
// modint.h -- modular int for mod 1e9+7: the class you write once.
// PITFALL: (a-b) needs +m before %; a*b needs long long inside.
struct mint {
   static const long long M = 1000000007;
   long long v;
```

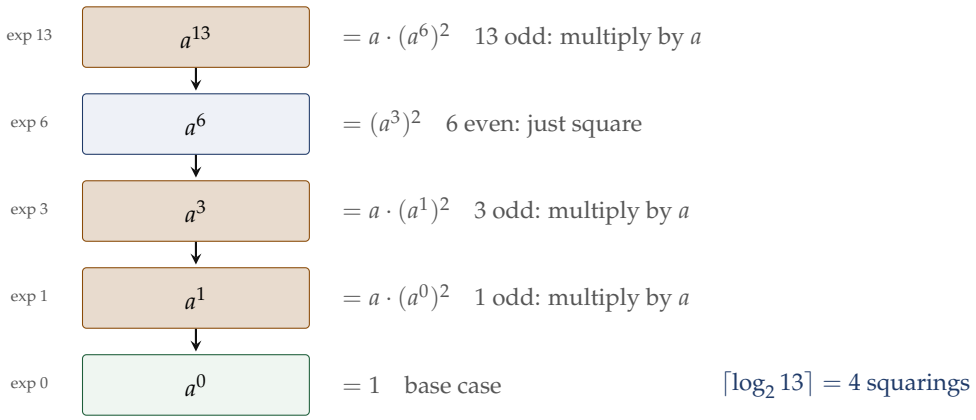


Figure 14.2: Squaring tree for a^{13}. Notice only $O(\log b)$ nodes exist — the exponent halves each level.

```
    mint(long long x = 0) : v((x % M + M) % M) {}
    mint operator+(mint o) const { return mint(v + o.v); }
    mint operator-(mint o) const { return mint(v - o.v); }
    mint operator*(mint o) const { return mint(v * o.v % M); }
    mint pow(long long b) const {
        long long r = 1, a = v;
        for (; b > 0; b >>= 1) {
            if (b & 1) r = r * a % M;
            a = a * a % M;
        }
        return mint(r);
    }
    mint inv() const { return pow(M - 2); } // Fermat: a^{p-2}
    mint operator/(mint o) const { return *this * o.inv(); }
};
```

14.6 Counting

Factorials plus inverse factorials give $\binom{n}{k}$ mod p in $O(1)$ after $O(n)$ precompute. Stars and bars counts weak compositions; inclusion–exclusion counts with forbidden events; derangements and Catalan each reduce to named formulas.

Example 14.3 (CSES *Binomial Coefficients*). $\binom{a}{b}$ mod 10^9+7 for up to 10^5 queries.

Strategy. Precompute $f[n] = n!$ and $fi[n] = (n!)^{-1}$ once; each query is three multiplications.

Solution.

```
#include "ncr.h"
#include <cstdio>
int main() {
    int n; scanf("%d", &n);
    NCR ncr(1000000);
    while (n--) {
        int a, b; scanf("%d %d", &a, &b);
        printf("%lld\n", ncr.choose(a, b).v);
```

```
    }
}
```

Real output (CSES sample):

```
10
846527861
31465
```

Example 14.4 (CSES *Distributing Apples*)**.** Distribute m identical apples among n children.

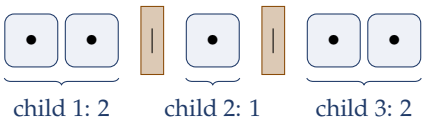
Strategy. Lay the apples in a row and insert $n-1$ walls: each string of $n+m-1$ symbols is a distribution — $\binom{n+m-1}{n-1}$.

Solution.

```
// distributing_apples.cpp -- stars and bars: C(n+m-1, n-1).
#include "ncr.h"
#include <cstdio>
int main() {
    int n, m; scanf("%d %d", &n, &m);
    NCR ncr(n + m);
    printf("%lld\n", ncr.choose(n + m - 1, n - 1).v);
}
```

Real output (CSES sample):

```
20
```



$m = 5$ apples (•) and $n - 1 = 2$ walls (|)

choose which 2 of the 7 slots are walls:

$\binom{n+m-1}{n-1} = \binom{7}{2} = 21$

Figure 14.3: The walls/children mapping. Notice the model works because child-i's count is just the run between walls — no arithmetic needed at all.

Example 14.5 (CSES *Christmas Party*)**.** Derangements of n guests.

Strategy. Inclusion–exclusion over the "fixed" events, or the equivalent recurrence $D_n = (n-1)(D_{n-1} + D_{n-2})$ — guest 1 picks a seat, and whether that seat's owner lands in seat 1 splits the cases.

Solution.

```
// christmas_party.cpp -- derangements: D_n = (n-1)(D_{n-1}+D_{n-2}).
#include "modint.h"
#include <cstdio>
int main() {
    int n; scanf("%d", &n);
    mint d0 = 1, d1 = 0;
```

```
    for (int i = 2; i <= n; i++) {
        mint d2 = mint(i - 1) * (d1 + d0);
        d0 = d1; d1 = d2;
    }
    printf("%lld\n", (n == 1 ? d1 : (n == 0 ? d0 : d1)).v);
}
```

Real output (CSES sample):

```
9
```

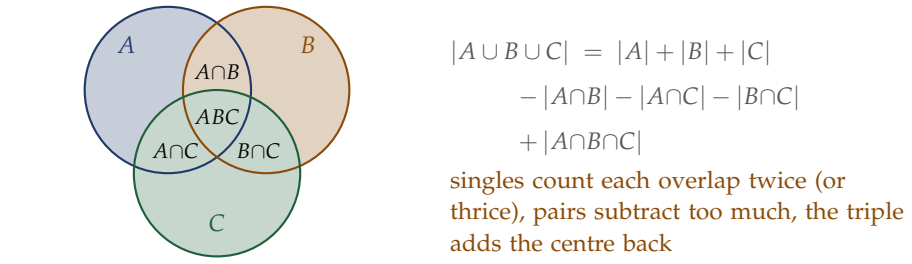


Figure 14.4: Inclusion–exclusion for three "fixed" events. Notice the alternating signs repair the double and triple counting in stages.

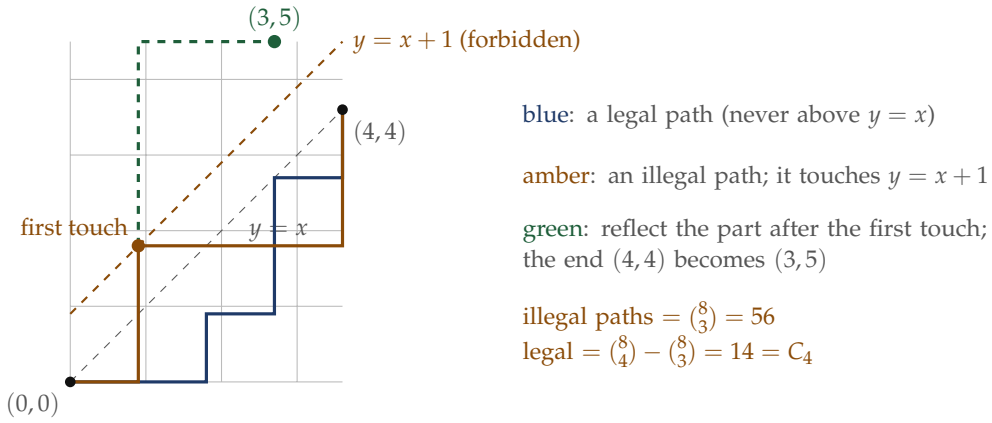


Figure 14.5: The ballot/reflection picture for Catalan counting. Notice the diagonal is the legality boundary — crossing it identifies the bad paths that get reflected out.

14.7 Linear recurrences

A linear recurrence is a matrix: $v_{n+1} = Mv_n$. Then $v_n = M^n v_0$ and binpow-on-matrices gives the n-th term in $O(\log n)$ — Fibonacci is the worked pattern, the same shape carries any constant-coefficient recurrence.

Example 14.6 (CSES *Fibonacci Numbers*). F_n mod 10^9+7 for n up to 10^{18}.

Strategy. $M = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}$ maps (F_k, F_{k-1}) to (F_{k+1}, F_k); F_n is the off-diagonal of M^n.

Solution.

```
#include "matpow.h"
#include <cstdio>
int main() {
   long long n; scanf("%lld", &n);
   M2 a = {{{1,1},{1,0}}};
   printf("%lld\n", mpow(a, n, 1000000007)[0][1]);
}
```

Real output (CSES sample):

```
55
```

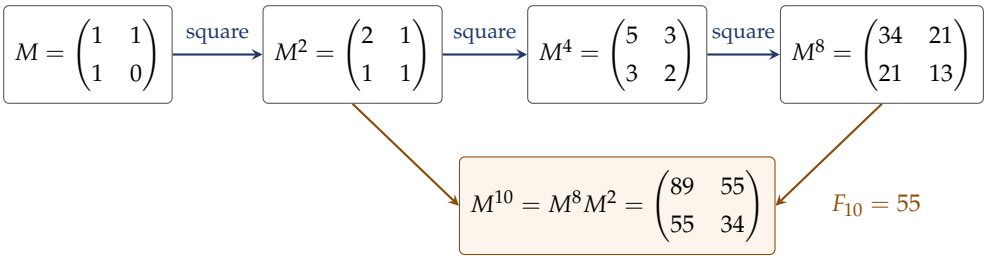


Figure 14.6: Matrix power. Notice we exponentiate the *transition* — the state vector rides along for free.

Exercise 14.1 (medium). Given $ax + by = \gcd(a, b)$, express Bézout coefficients for $(b, a \bmod b)$ in terms of those for (a, b) — derive the two-line update.

Exercise 14.2 (easy). Why does $a^{-1} \bmod m$ need $\gcd(a, m) = 1$? Show what breaks for m composite and $a \mid m$.

Exercise 14.3 (medium). A problem asks for ordered m-tuples of nonnegative integers summing to n. Which counting model applies, and what would "at least one" do to it?

Exercise 14.4 (medium). For $v_{n+1} = pv_n + qv_{n-1} + r$, write the transition matrix and argue its dimension must be 3×3 — what rides in the third slot?

14.8 Chapter summary

14.9 Common mistakes

- **Reducing the wrong exponent.** a^{b^c} needs the exponent reduced mod $m-1$, not mod m — the classic Exp-II WA.
- **Negative residues.** $(a - b) \,\%\, m$ can be negative in C++; always add m back before the final reduction.
- **Inverse without coprimality.** Division mod m silently fails when $\gcd(a, m) > 1$ — check before Fermat.

Need	Tool	Cost	Trap
gcd, inverse	ext. Euclid	log	sign on (x, y)
factorize	SPF sieve	$\log n$	sieve bound
a^b huge	binpow	$\log b$	Fermat for exp-exp
all mod work	modint	native	neg. values, overflow
$\binom{n}{k}$	fact+inv	$O(1)$ q	p prime
weak comps.	stars-bars	formula	distinct $\neq$ identical
forbidden set	incl.–excl.	2^k terms	signs alternate
linear recur.	matpow	$\log n$	carry constants

Table 14.1: The problem's constraint names the model; the model names the formula.

- **Distinct vs identical objects.** Stars-and-bars counts *identical* objects; applying it to distinct objects misses the permutations.
- **Matrix dimension too small.** A recurrence with a constant term needs the constant carried as an extra slot — $M^n v_0$ can't manufacture it.

14.10 Additional exercises

Exercise 14.5 (easy). CSES *Exponentiation*.

Exercise 14.6 (easy). CSES *Counting Divisors*.

Exercise 14.7 (easy). CSES *Common Divisors*.

Exercise 14.8 (easy). CSES *Throwing Dice*.

Exercise 14.9 (medium). CSES *Exponentiation II*.

Exercise 14.10 (medium). CSES *Sum of Divisors*.

Exercise 14.11 (medium). CSES *Binomial Coefficients*.

Exercise 14.12 (medium). CSES *Distributing Apples*.

Exercise 14.13 (medium). CSES *Creating Strings II*.

Exercise 14.14 (medium). CSES *Permutation Rounds*.

Exercise 14.15 (medium). CSES *Fibonacci Numbers*.

Exercise 14.16 (hard). CSES *Prime Multiples* — inclusion–exclusion.

Exercise 14.17 (hard). CSES *Counting Coprime Pairs* — Möbius.

Exercise 14.18 (hard). CSES *Christmas Party*.

Exercise 14.19 (hard). CSES *Bracket Sequences I* — Catalan.

Exercise 14.20 (hard). CSES *Divisor Analysis*.

Exercise 14.21 (hard). CSES *Graph Paths I/II* — matpow on graphs.

Notes

Further reading

- A. Laaksonen, *Competitive Programmer's Handbook* (2017–2024), https://cses.fi/book/book.pdf — free PDF; the number-theory chapters this chapter is pitched from.
- J. Sannemo, *Principles of Algorithmic Problem Solving* (2025), https://usaco.guide/PAPS1.pdf — free PDF; deeper proofs for sieve bounds and modular machinery.
- cp-algorithms, *Algebra* and *Combinatorics* sections, https://cp-algorithms.com/ — the reference for SPF variants, M"obius, and every named counting formula.
- The USACO Guide, *modular*, *divisibility*, and *combo* modules, https://usaco.guide/ — the problem lists this chapter's exercises were drawn from.
- S. Halim, F. Halim, S. Effendy, *Competitive Programming 4* (2020), https://cpbook.net/ — the widest taxonomy of contest number theory and counting problems.
- MIT OpenCourseWare, *6.042J Mathematics for Computer Science* (Fall 2010) — the undergraduate course behind the Watch picks below: https://ocw.mit.edu/ (course 6.042J).
- E. W. Weisstein (ed.), *MathWorld*, https://mathworld.wolfram.com/ — formula lookup: derangements, Catalan numbers, stars and bars, all with references.

WATCH INSTEAD

- **MIT OpenCourseWare — "Lec 4: Number Theory I" (6.042J, Fall 2010)**, Marten van Dijk, 1h20m, https://www.youtube.com/watch?v=NuY7szYSXSw — covers §14.1–14.3: Euclid, gcd invariants, modular arithmetic. (Verified 2026-10-11.)
- **MIT OpenCourseWare — "Lec 16: Counting Rules I" (6.042J, Fall 2010)**, Marten van Dijk, 1h20m, https://www.youtube.com/watch?v=pNt5Ll6hGqo — covers §14.6: bijection, product and sum rules behind stars-and-bars. (Verified 2026-10-11.)
- **Khan Academy — "Journey into Cryptography" playlist**, Brit Cruise, short-clip playlist (segments ∼10 min each), https://www.youtube.com/playlist?list=PLSQl0a2vh4HA50QhFIirlEZRXG4yjcoGM — covers §14.3: modular arithmetic through Fermat's little theorem, the friendliest first pass. (Verified 2026-10-11.)

Notation note: OCW writes $a \mid b$ and gcd as the book does; Khan uses "mod" infix notation — same semantics.

The Euclidean and extended algorithms, sieve + SPF factorization, and binary exponentiation follow Laaksonen [2] and Sannemo [1]; modular inverse, stars-and-bars, derangement and Catalan treatments track cp-algorithms and the USACO Guide [5, 3]; counting-model choice and the problem-band split follow Halim [7]; archive problems are official CSES [4].

CHAPTER 15

Strings: hashing, KMP, Z

String problems in contests reduce to a handful of linear-time machines. After this chapter you can compare two substrings in $O(1)$ — and know exactly when a hash deserves trust — match a pattern in $O(n+m)$ two different ways, mine borders and periods out of the prefix function, and walk a trie for anything that is "a dictionary of prefixes" in disguise.

15.1 Terminology

A *substring* is a contiguous block; a *subsequence* keeps order but may skip characters. Prefixes and suffixes anchor the two ends. A *border* of s is a proper prefix that is also a suffix: `abcababcab` has borders `ab` and `abcab`. A *period* is a distance p with $s_i = s_{i+p}$ for every $i < n - p$: `abcabca` has period 3 (and, vacuously, 7). These four definitions — border, period, prefix function, Z-box — are the whole vocabulary of this chapter.

15.2 Polynomial rolling hash

Treat s as a base-B number: $H(s) = \sum_i s_i B^{n-1-i} \bmod p$. Prefix hashes $h_i = H(s_0 \dots s_{i-1})$ satisfy $h_{i+1} = h_i B + s_i$, so any substring is recovered in $O(1)$: $H(s_l \dots s_{r-1}) = h_r - h_l B^{r-l}$. Figure 15.1 shows the equivalent sliding-window view.

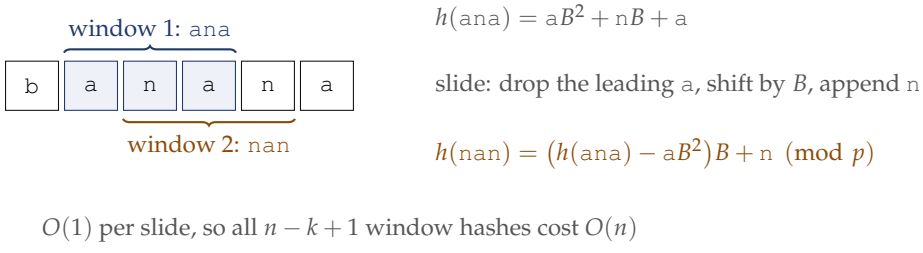


Figure 15.1: Rolling one window right. Notice the dropped term carries the full weight B^{k-1} — that is the term the update must subtract first.

```
// rhash.h -- polynomial rolling hash, two moduli, prefix-hash queries.

#include <string>
#include <vector>
#include <utility>
#include <algorithm>
using namespace std;
```

```
struct RH {
   static const long long B = 91138233;
   static const long long M1 = 972663749;
   static const long long M2 = 998244353;
   vector<long long> h1, h2, p1, p2;
   int n;
   RH(const string& s) {
      n = (int)s.size();
      h1.assign(n + 1, 0); h2.assign(n + 1, 0);
      p1.assign(n + 1, 1); p2.assign(n + 1, 1);
      for (int i = 0; i < n; i++) {
         h1[i+1] = (h1[i] * B + (unsigned char)s[i]) % M1;
         h2[i+1] = (h2[i] * B + (unsigned char)s[i]) % M2;
         p1[i+1] = p1[i] * B % M1;
         p2[i+1] = p2[i] * B % M2;
      }
   }
   pair<long long,long long> get(int l, int r) const { // hash of s[l..r)
      long long x1 = (h1[r] - h1[l] * p1[r - l]) % M1; if (x1 < 0) x1 += M1;
      long long x2 = (h2[r] - h2[l] * p2[r - l]) % M2; if (x2 < 0) x2 += M2;
      return {x1, x2};
   }
   int lcp(int i, int j) const { // LCP of suffixes i, j
      int lo = 0, hi = min(n - i, n - j);
      while (lo < hi) {
         int mid = (lo + hi + 1) / 2;
         if (get(i, i + mid) == get(j, j + mid)) lo = mid; else hi = mid - 1;
      }
      return lo;
   }
};
```

A hash is a *fingerprint*, not a proof: unequal strings can share one. With q queries the birthday bound kicks in — among q hashes you should expect a collision once $q \approx \sqrt{2p}$, so a single 10^9 modulus is already shaky at $q \sim 5{\cdot}10^4$, well inside contest range. Doubling to two independent moduli moves the failure point to $\sim \sqrt{2p_1p_2}$; the demo below makes the numbers concrete.

Example 15.1 (Where collisions live)**.** How much does modulus size matter, and does a second modulus fix it?

Strategy. Brute-force small alphabets under a toy modulus to exhibit a *real* colliding pair, then count collisions on random strings under a mid-size modulus — and under the two-modulus pair used by `RH`.

Solution.

```
// hash_demo.cpp -- where collisions live: hunt a real colliding pair
```

```
// under (B=26, p=97), then show the same pair separates under the
// moduli used by RH. Deterministic search, real output.
#include <cstdio>
#include <string>
#include <map>
using namespace std;
long long h(const string& s, long long B, long long P) {
   long long x = 0;
   for (char c : s) x = (x * B + (c - 'a' + 1)) % P;
   return x;
}
int main() {
   // enumerate all 3-letter strings until (B=26, p=97) collides
   map<long long, string> seen;
   string a, b;
   for (int i = 0; i < 26 * 26 * 26; i++) {
      string w = {char('a' + i / 676), char('a' + (i / 26) % 26), char
          ('a' + i % 26)};
      long long x = h(w, 26, 97);
      if (seen.count(x) && seen[x] != w) { a = seen[x]; b = w; break;
          }
      seen[x] = w;
   }
   printf("mod 97, base 26: \"%s\" and \"%s\" both hash to %lld\n",
        a.c_str(), b.c_str(), h(a, 26, 97));
   printf("same pair, mod 1e9+7: %lld vs %lld\n", h(a, 26, 1000000007)
       , h(b, 26, 1000000007));
   printf("same pair, mod 998244353: %lld vs %lld\n", h(a, 26,
       998244353), h(b, 26, 998244353));
   // birthday count: 4096 random 8-letter strings, one modulus vs two
   map<long long, int> mid;
   map<pair<long long,long long>, int> both;
   unsigned rng = 1;
   for (int i = 0; i < 8192; i++) {
      string w;
      for (int j = 0; j < 8; j++) { rng = rng * 1103515245 + 12345; w
          += 'a' + (rng / 65536) % 26; }
      mid[h(w, 91138233, 9726647)]++;
      both[{h(w, 91138233, 972663749), h(w, 91138233, 998244353)}]++;
   }
   int c1 = 0, c2 = 0;
   for (auto& e : mid) c1 += e.second - 1;
   for (auto& e : both) c2 += e.second - 1;
   printf("8192 8-letter strings: %d extra hashes collide mod 9726647
       (birthday), %d under the RH pair\n", c1, c2);
}
```

Real output:

```
mod 97, base 26: "aaa" and "adt" both hash to 24
same pair, mod 1e9+7: 703 vs 800
same pair, mod 998244353: 703 vs 800
```

```
8192 8-letter strings: 5 extra hashes collide mod 9726647 (birthday),
   0 under the RH pair
```

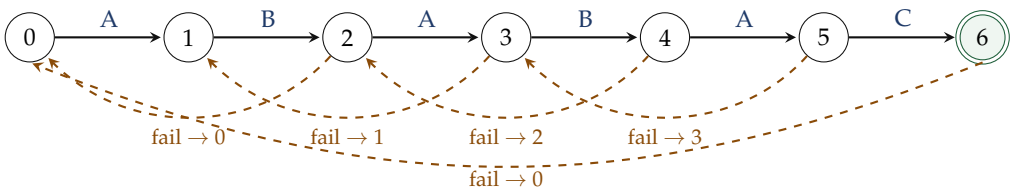
Exercise 15.1 (medium). q substrings are hashed mod 10^9+7. Using $1 - e^{-q^2/2p}$, at what q does a collision become more likely than not? Why does a second modulus change the exponent's denominator?

15.3 The prefix function (KMP)

π_i is the longest proper border of $s_0 \dots s_i$. Compute left to right: to extend s_i, start at the current border candidate $j = \pi_{i-1}$ and keep falling along failure links ($j \leftarrow \pi_{j-1}$) until $s_i = s_j$; that character either extends the border or leaves $\pi_i = 0$. The scan is $O(n)$ because j only decreases inside the while-loop and can decrease at most as many times as it ever increased.

```
// kmp.h -- prefix function: pi[i] = longest proper border of s[0..i].
#include <string>
#include <vector>
using namespace std;
vector<int> prefix_function(const string& s) {
   int n = (int)s.size();
   vector<int> pi(n);
   for (int i = 1; i < n; i++) {
      int j = pi[i - 1];
      while (j > 0 && s[i] != s[j]) j = pi[j - 1];
      if (s[i] == s[j]) j++;
      pi[i] = j;
   }
   return pi;
}
```

The failure links make π a deterministic automaton on the pattern: state = characters matched so far, transitions = extend or fall. Figure 15.2 draws the machine.



at state 5 reading B: C was expected, so fall back 5 → 3 (ABA), where B is expected: advance to state 4

Figure 15.2: KMP automaton for ABABAC ($\pi = 0, 0, 1, 2, 3, 0$). Notice a failure is not a restart — state 5 on input B falls to 3 and immediately re-matches to 4.

Exercise 15.2 (easy). Compute π by hand for ababac, then for aaaaaa. What does the second tell you about worst-case border structure?

15.4 The Z-function

z_i is the longest common prefix of s and $s_i \dots s_{n-1}$ (z_0 unused). The algorithm keeps a *Z-box* $[l, r]$: the rightmost window already proved to match a prefix. Inside the box, $z_i \geq \min(r - i + 1,\ z_{i-l})$ for free; outside, or when the copy might run past r, extend by direct comparison and grow the box. Each extension moves r right, so the total work is $O(n)$.

```
// zfunc.h -- Z-function: z[i] = LCP of s and s[i..n).
#include <string>
#include <vector>
#include <algorithm>
using namespace std;
vector<int> z_function(const string& s) {
   int n = (int)s.size();
   vector<int> z(n);
   for (int i = 1, l = 0, r = 0; i < n; i++) {
      if (i <= r) z[i] = min(r - i + 1, z[i - l]);
      while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
      if (i + z[i] - 1 > r) l = i, r = i + z[i] - 1;
   }
   return z;
}
```

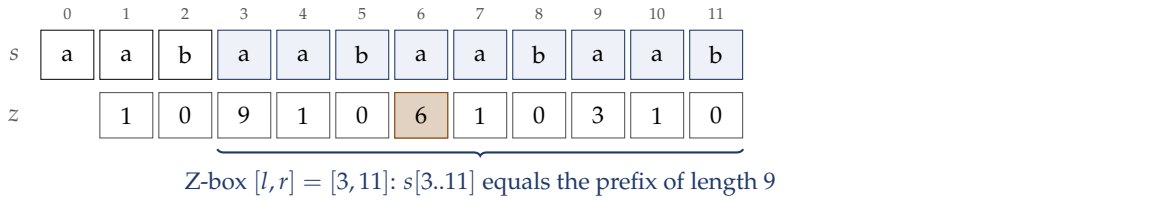


Figure 15.3: Z-box on `aabaabaabaab`. Notice $z_6 = 6$ costs no comparisons — it is copied from z_3 inside the box $[3, 11]$.

15.5 Tries

A trie stores a dictionary so that each node *is* a prefix: walking an edge appends a character; terminal markers record which prefixes are real words. Insert and walk are $O(\text{length})$, and a walk from position i enumerates every dictionary word that starts at i — exactly what word-splitting DP needs.

```
// trie.h -- prefix dictionary: insert words, walk from any position.
#include <array>
#include <vector>
#include <string>
using namespace std;
struct Trie {
   static const int AL = 26;
```

```
    struct Node { array<int, AL> ch{}; int cnt = 0; };
    vector<Node> t;
    Trie() : t(1) {}
    void insert(const string& w) {
        int u = 0;
        for (char c : w) {
            int k = c - 'a';
            if (!t[u].ch[k]) { t[u].ch[k] = (int)t.size(); t.push_back(
                Node()); }
            u = t[u].ch[k];
        }
        t[u].cnt++;
    }
    int next(int u, char c) const { return t[u].ch[c - 'a']; }
    int count(int u) const { return t[u].cnt; }
    int root() const { return 0; }
};
```

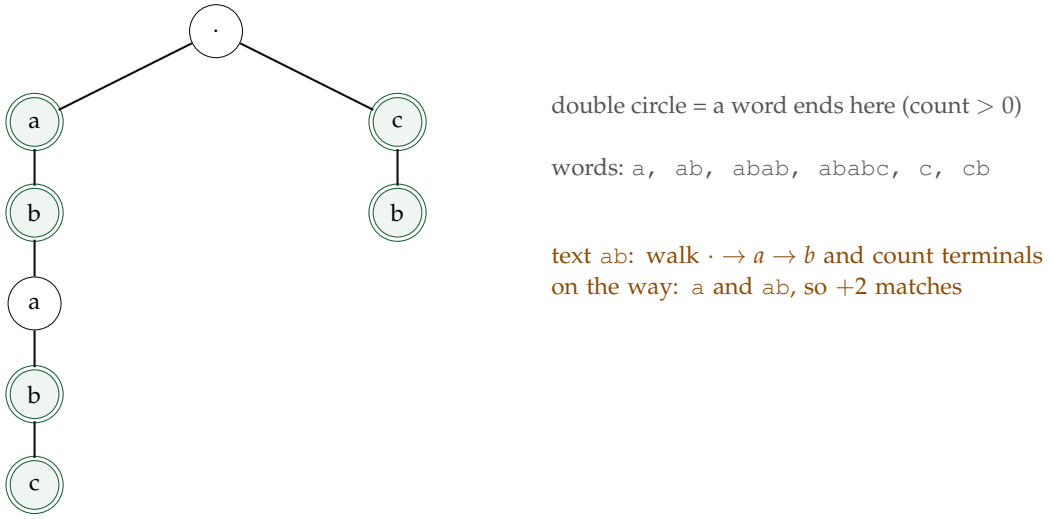


Figure 15.4: Trie over {a, ab, abab, ababc, c, cb}. Notice terminals can sit mid-path: ab ends inside the edge to abab, which is what makes a walk report all matches, not just the longest.

Example 15.2 (CSES *Word Combinations*)**.** Count the ways to write s as a sequence of dictionary words, mod 10^9+7.

Strategy. dp_i = ways to split $s_{i\ldots n}$. From each i, walk the trie on $s_i, s_{i+1}, \ldots$; every terminal at depth d contributes dp_{i+d}. Store *counts* at terminals (the dictionary may repeat words).

Solution.

```
// CSES 1731 Word Combinations -- number of ways to split s into dict
// words, mod 1e9+7. dp[i] counts splittings of s[i..n); a trie walk
// from i adds dp[i + len] for every word found on the path.
#include "trie.h"
#include <cstdio>
#include <iostream>
#include <string>
```

```
#include <vector>
using namespace std;
const long long MOD = 1000000007;
int main() {
   string s; int k; cin >> s >> k;
   Trie tr;
   for (int i = 0; i < k; i++) { string w; cin >> w; tr.insert(w); }
   int n = s.size();
   vector<long long> dp(n + 1, 0); dp[n] = 1;
   for (int i = n - 1; i >= 0; i--) {
      int u = tr.root();
      for (int j = i; j < n; j++) {
         u = tr.next(u, s[j]);
         if (!u) break;
         dp[i] = (dp[i] + (long long)tr.count(u) * dp[j + 1]) % MOD;
      }
   }
   printf("%lld\n", dp[0]);
}
```

Real output (CSES sample):

```
2
```

The same structure on bits — the *xor-trie* — answers "max $a_i \oplus x$" queries and is named for the 1900 route.

15.6 Applications

Matching. KMP and Z solve the same problem on the same concatenation $p\#s$: an occurrence ends wherever $\pi_i = m$ (KMP) or starts wherever $z_i = m$ (Z).

Example 15.3 (CSES *String Matching*, twice)**.** Count occurrences of p in s.

Strategy. Build both machines on $p\#s$ and print both counts — for the official problem, print either.

Solution.

```
// CSES 1753 String Matching -- count occurrences of p in s, twice:
// (1) KMP automaton on p#s, (2) Z-function on the same concatenation.
#include "kmp.h"
#include "zfunc.h"
#include <cstdio>
#include <iostream>
#include <string>
using namespace std;
int main() {
   string s, p; cin >> s >> p;
   int m = p.size();
   string cat = p + "#" + s;
```

```
    int c1 = 0, c2 = 0;
    for (int x : prefix_function(cat)) if (x == m) c1++;
    vector<int> z = z_function(cat);
    for (int i = m + 1; i < (int)cat.size(); i++) if (z[i] == m) c2++;
    printf("%d %d\n", c1, c2); // both methods, one line
}
```

Real output (CSES sample: both methods, one line):

```
2 2
```

Borders. π_{n-1} is the longest border; iterating $j \leftarrow \pi_{j-1}$ enumerates them all, longest first.

Example 15.4 (CSES *Finding Borders*)**.** List all border lengths of s.

Strategy. Walk the π-chain from π_{n-1}, collect, sort ascending.

Solution.

```
// CSES 1732 Finding Borders -- all border lengths, ascending.
#include "kmp.h"
#include <cstdio>
#include <iostream>
#include <string>
#include <vector>
#include <algorithm>
using namespace std;
int main() {
    string s; cin >> s;
    vector<int> pi = prefix_function(s), b;
    for (int j = pi[(int)s.size() - 1]; j > 0; j = pi[j - 1]) b.
        push_back(j);
    sort(b.begin(), b.end());
    for (int i = 0; i < (int)b.size(); i++) printf("%d%c", b[i], " \n"[
        i + 1 == (int)b.size()]);
}
```

Real output (CSES sample):

```
2 5
```

Periods. p is a period iff $n - p$ is a border (Figure 15.6): the constraint $s_i = s_{i+p}$ for all i is precisely "prefix of length $n - p$ equals suffix". So the same π-chain answers *Finding Periods*, including $p = n$ at the end.

Example 15.5 (CSES *Finding Periods*)**.** List all period lengths of s.

Strategy. Same π-chain as Finding Borders, printing $n - j$ instead of j; append n.

Solution.

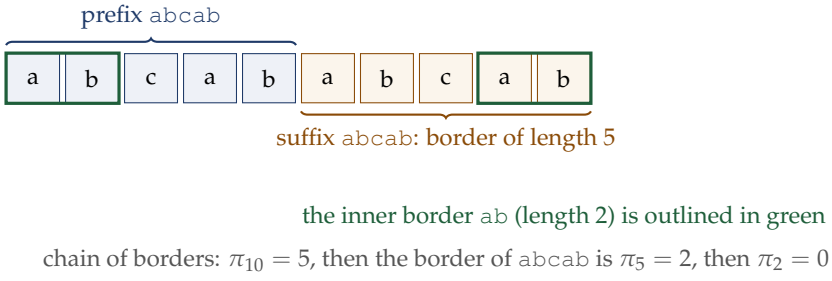


Figure 15.5: Border chain on abcababcab. Notice each border is itself a string whose own longest border is the next link — that is exactly the π-chain recursion.

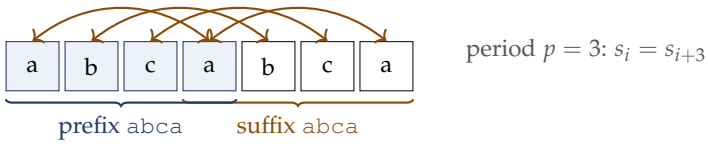


Figure 15.6: Border $\Leftrightarrow$ period duality. Notice the overlap: a period p forces the prefix and suffix of length $n - p$ to coincide.

```
// CSES 1733 Finding Periods -- all period lengths k, 1 <= k <= n.
// k is a period of s iff n - k is a border length of s (or k == n).
#include "kmp.h"
#include <cstdio>
#include <iostream>
#include <string>
#include <vector>
#include <algorithm>
using namespace std;
int main() {
    string s; cin >> s;
    int n = (int)s.size();
    vector<int> pi = prefix_function(s), ans;
    for (int j = pi[n - 1]; j > 0; j = pi[j - 1]) ans.push_back(n - j);
    ans.push_back(n);
    sort(ans.begin(), ans.end());
    for (int i = 0; i < (int)ans.size(); i++) printf("%d%c", ans[i], "
        \n"[i + 1 == (int)ans.size()]);
}
```

Real output (CSES sample):

```
3 6 7
```

Exercise 15.3 (medium). Prove the identity used above: p is a period of s iff $n - p$ is a border. Conclude the smallest period is $n - \pi_{n-1}$.

Substring comparison. With prefix hashes, the LCP of any two suffixes is one binary search away (compare $\geq$-mid hashes), giving lexicographic comparison of any two substrings in $O(\log n)$ — no suffix array needed.

Example 15.6 (CSES *Minimal Rotation*)**.** Print the lexicographically smallest rotation of s.

Strategy. Rotations are length-n windows of $s{+}s$. Compare candidates with `RH::lcp` on $s{+}s$: jump past the common prefix and decide on the first differing character — $O(\log n)$ per comparison.

Solution.

```
// CSES 1110 Minimal Rotation -- lexicographically smallest rotation.
// Compare rotations of s as length-n windows of s+s via double hash
// + binary-search LCP: O(n log n) comparisons, no suffix structures.
#include "rhash.h"
#include <cstdio>
#include <iostream>
#include <string>
using namespace std;
int main() {
   string s; cin >> s;
   int n = s.size();
   string t = s + s;
   RH H(t);
   int best = 0;
   for (int i = 1; i < n; i++) {
      int d = H.lcp(best, i);
      if (d < n && t[i + d] < t[best + d]) best = i;
   }
   printf("%s\n", (s.substr(best) + s.substr(0, best)).c_str());
}
```

Real output (CSES sample):

```
abac
```

task	tool	cost	gotcha
substring equality	prefix hashes, two moduli	$O(1)$/q after $O(n)$	birthday bound at $\sqrt{2p}$
pattern matching	KMP or Z on $p\#s$	$O(n+m)$	sentinel must not appear in alphabet
all borders	π-chain from π_{n-1}	$O(n)$	iterate $j \leftarrow \pi_{j-1}$, not π_j
all periods	same chain, print $n-j$	$O(n)$	append $p = n$ manually
dict word splits	trie walk per position	$O(n \cdot L)$	terminals mid-path count too
substring compare	hash + binary-search LCP	$O(\log n)$/cmp	half-open $[l, r)$ in `get`

Table 15.1: The task names the machine; the machine names the cost.

15.7 Common mistakes

- **Skipping the sentinel.** Matching on $p + s$ (no #) lets a "match" straddle the boundary — a separator outside the alphabet is required.
- **Wrong index in the π-chain.** The next candidate border is π_{j-1}, not π_j; the off-by-one yields either an infinite loop or the trivial answer.
- **Negative residues in `get`.** $h_r - h_l B^{r-l} \bmod p$ needs the $+p$ before returning — C++ `%` keeps the sign of the dividend.
- **One modulus, many queries.** Birthday collisions arrive at $\sqrt{2p}$; on $q \gtrsim 10^5$ a single 10^9 prime is a gamble, not a proof.
- **Assuming terminals only at leaves.** In word-splitting tries, `cnt` must be read at *every* node on the walk, or the short matches vanish.

15.8 Additional exercises

Exercise 15.4 (easy). CSES *Finding Borders*.

Exercise 15.5 (easy). CSES *Finding Periods*.

Exercise 15.6 (easy). USACO-B *Censoring*.

Exercise 15.7 (easy). CF *Check Transcription*.

Exercise 15.8 (easy). CSES *Counting Patterns* — k independent KMP runs.

Exercise 15.9 (medium). CSES *String Matching*.

Exercise 15.10 (medium). CSES *Minimal Rotation*.

Exercise 15.11 (medium). CSES *Longest Palindrome*.

Exercise 15.12 (medium). CSES *Repeating Substring*.

Exercise 15.13 (medium). USACO-G *Lights Out*.

Exercise 15.14 (medium). USACO-S *Censoring (Silver)* — stack + KMP.

Exercise 15.15 (hard). CSES *Required Substring*.

Exercise 15.16 (hard). CSES *Distinct Substrings* — suffix structures or hashing + sort.

Exercise 15.17 (hard). CSES *String Functions* — compute both π and z sums.

Exercise 15.18 (hard). CSES *Palindrome Queries* — hashing both directions.

Notes

Further reading

- A. Laaksonen, *Competitive Programmer's Handbook*, https://cses.fi/book/book.pdf — free PDF; the trie/hashing/Z treatment this chapter extends.
- J. Sannemo, *Principles of Algorithmic Problem Solving* (2025), https://usaco.guide/PAPS1.pdf — free PDF; the string chapter behind our proof style.
- cp-algorithms, *String hashing* — collision bounds and double-hash practice. https://cp-algorithms.com/string/string-hashing.html
- cp-algorithms, *Prefix function* — the automaton view and its applications catalogue. https://cp-algorithms.com/string/prefix-function.html
- cp-algorithms, *Z-function* — the Z-box invariant proved cleanly. https://cp-algorithms.com/string/z-function.html
- The USACO Guide, *Hashing* module, https://usaco.guide/gold/hashing — the contest settings where hashing is the intended weapon.
- D. Gusfield, *Algorithms on Strings, Trees, and Sequences* (1997), ISBN 978-0521585194 — the reference for everything past this chapter: suffix trees, automata, and why Z-values work.
- S. Halim, F. Halim, S. Effendy, *Competitive Programming 4* (2020), https://cpbook.net/ — the string problem taxonomy.

WATCH INSTEAD

- **Abdul Bari — "9.1 Knuth-Morris-Pratt KMP String Matching Algorithm"**, 18m55s, https://www.youtube.com/watch?v=V5-7GzOfADQ — covers §15.1, §15.3: borders (he calls π the "LPS table") and the failure-link walk. (Verified 2026-10-11.)
- **Abdul Bari — "9.2 Rabin-Karp String Matching Algorithm"**, 23m50s, https://www.youtube.com/watch?v=qQ8vS2btsxI — covers §15.2: rolling hash as a sliding base-B number; he uses base-10 codes for pedagogy — same idea, our modulus is prime. (Verified 2026-10-11.)
- **UC Davis — "Linear-time pattern matching: Z-values and Z-algorithm"** (Dan Gusfield), 51m46s, https://www.youtube.com/watch?v=MFK0WYeVEag — covers §15.4: the Z-box invariant from the algorithm's original expositor. (Verified 2026-10-11.)

Notation note: Bari writes `lps` for our π and indexes strings from 1 in slides; Gusfield's Z_k, l, r match the book exactly.

Rolling-hash construction and collision practice follow cp-algorithms and the USACO Guide [5, 3]; the prefix-function, Z-function and trie treatments track Laaksonen [2] and Sannemo [1]; the Z-box proof shape follows Gusfield (see Further reading); problem band and selection follow Halim [7] and the official CSES set [4].

CHAPTER 16

Geometry basics

Contest geometry earns its reputation for fragility almost entirely through one avoidable choice: writing `double` where an integer would do. This chapter is the disciplined alternative. After it you can run every standard test — orientation, segment intersection, point-in-polygon, convex hull, polygon area, lattice points, closest pair — on exact integer arithmetic, and you will know which two or three degenerate cases actually decide whether your code gets accepted.

This chapter is optional for the core route; it is the foundation for USACO-G geometry and a handful of CF 1600–2100 problems.

16.1 The integer-only rule

Input coordinates are integers, and nearly every geometric predicate you need is a polynomial in them: orientation is a quadratic form, containment a parity count, area a sum of products. Doubles enter only if the *answer* is real-valued — and even then, "squared distance" usually compares just as well. The rule: **never compare two doubles for equality; prefer an exact integer test and reach for `double` only when printing a real number**. An `eps` band fixes nothing — it merely moves the failure to coordinates one order of magnitude larger.

PRINCIPLE

Every primitive in `code/geom.h` returns an exact integer or a fraction pair. If a comparison feels like it wants `abs(a-b)<eps`, it almost always wants a cross product instead.

16.2 Points, vectors, orientation

The workhorse is the 2D *cross product*: for vectors $\vec{u} = (u_x, u_y)$, $\vec{v} = (v_x, v_y)$,

$$\vec{u} \times \vec{v} \;=\; u_x v_y - u_y v_x,$$

the signed area of the parallelogram they span (Figure 16.1). Its sign is the *orientation test*: anchored at a common tail o, `orient`$(o, a, b) =$ `cross`$(a - o,\, b - o)$ is positive when b lies to the left of ray $o \to a$, negative to the right, zero collinear. One determinant answers "left, right, or on the line" — the three-way predicate every later primitive builds on.

```
1 struct Pt {
```

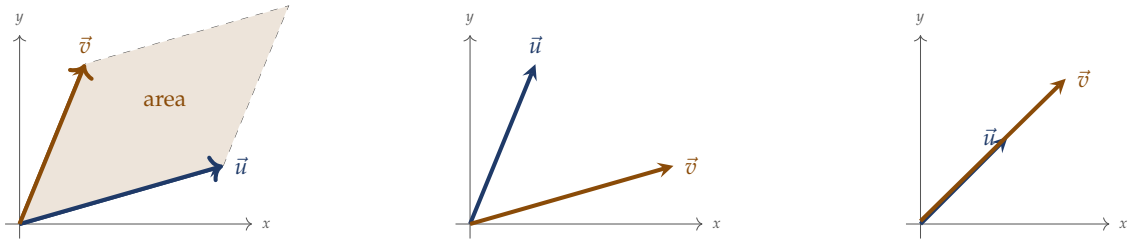


Figure 16.1: The cross product as signed parallelogram area. Notice the sign is the orientation test itself: swapping operands flips it, which is why anchor order matters.

```
    ll x, y;
    bool operator<(const Pt& o) const { return x != o.x ? x < o.x : y <
        o.y; }
    bool operator==(const Pt& o) const { return x == o.x && y == o.y; }
};
inline Pt operator-(Pt a, Pt b) { return {a.x - b.x, a.y - b.y}; }
inline ll cross(Pt a, Pt b) { return a.x * b.y - a.y * b.x; }
// orientation of (o->a, o->b): >0 left turn, <0 right turn, 0
    collinear
inline ll orient(Pt o, Pt a, Pt b) { return cross(a - o, b - o); }

// p on segment ab (assumes collinearity is handled/checked)
inline bool on_seg(Pt p, Pt a, Pt b) {
    return min(a.x, b.x) <= p.x && p.x <= max(a.x, b.x) &&
         min(a.y, b.y) <= p.y && p.y <= max(a.y, b.y);
}
```

Example 16.1 (CSES *Point Location Test*). For t queries of three points p, q, r, print whether r is LEFT of, RIGHT of, or TOUCHing line $p \to q$.

Strategy. One orientation call per query; the sign of `orient`(p, q, r) *is* the answer.

Solution.

```
// point_location.cpp -- CSES 2189: which side of line p1->p2 is p3?
#include "geom.h"
#include <cstdio>
int main() {
    int t; scanf("%d", &t);
    while (t--) {
        Pt p, q, r;
        scanf("%lld %lld %lld %lld %lld %lld", &p.x, &p.y, &q.x, &q.y, &
            r.x, &r.y);
        ll o = orient(p, q, r);
        puts(o > 0 ? "LEFT" : o < 0 ? "RIGHT" : "TOUCH");
    }
}
```

Real output (CSES sample):

```
LEFT
RIGHT
TOUCH
```

Exercise 16.1 (easy). Show $\texttt{orient}(o,a,b) = -\texttt{orient}(o,b,a)$ from the determinant. Which order does your library use, and where would a flipped sign hide in a hull?

16.3 Segment intersection

Two segments ab, cd intersect exactly when c and d lie on *opposite sides* of line ab *and* a, b lie on opposite sides of line cd — two orientation tests each way. But four touching cases matter just as much: endpoint touching, collinear overlap, an endpoint *on* the other segment, and the collinear-but-disjoint miss. Handle every zero-orientation case *before* the strict sign test (Figure 16.2).

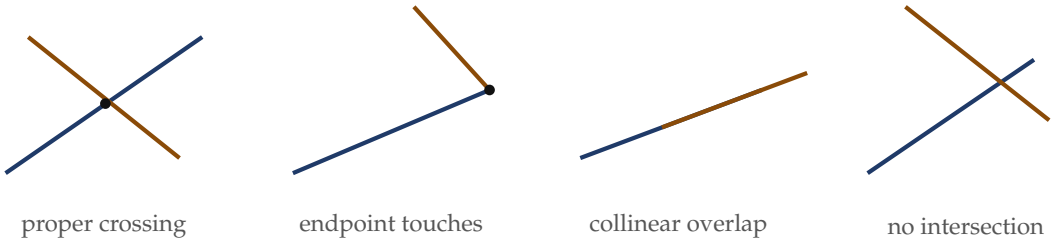


Figure 16.2: The four verdicts. Notice "touch" and "overlap" are both YES for CSES — the sign test alone sees neither, because a zero kills the strict comparison.

```
// proper or touching intersection of segments ab and cd
inline bool seg_int(Pt a, Pt b, Pt c, Pt d) {
    ll o1 = orient(a, b, c), o2 = orient(a, b, d);
    ll o3 = orient(c, d, a), o4 = orient(c, d, b);
    // touching: an endpoint lies on the other segment (covers
        collinear overlap)
    if (o1 == 0 && on_seg(c, a, b)) return true;
    if (o2 == 0 && on_seg(d, a, b)) return true;
    if (o3 == 0 && on_seg(a, c, d)) return true;
    if (o4 == 0 && on_seg(b, c, d)) return true;
    if (o1 == 0 || o2 == 0 || o3 == 0 || o4 == 0) return false;
    return ((o1 > 0) != (o2 > 0)) && ((o3 > 0) != (o4 > 0));
}
```

Example 16.2 (CSES *Line Segment Intersection*). t queries of two segments; print YES iff they intersect (touching counts).

Strategy. Four orientation calls; test the endpoint-on-segment cases first, bail on any remaining zero, then compare signs both ways.

Solution.

```
// line_seg_int.cpp -- CSES 2190: do segments ab and cd intersect?
#include "geom.h"
#include <cstdio>
int main() {
   int t; scanf("%d", &t);
   while (t--) {
      Pt a, b, c, d;
      scanf("%lld %lld %lld %lld %lld %lld %lld %lld",
           &a.x, &a.y, &b.x, &b.y, &c.x, &c.y, &d.x, &d.y);
      puts(seg_int(a, b, c, d) ? "YES" : "NO");
   }
}
```

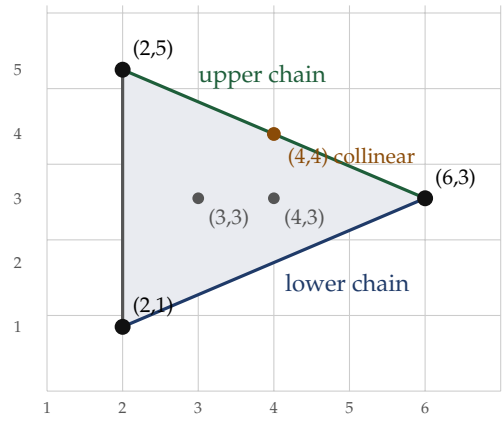
Real output (CSES sample):

```
NO
YES
YES
YES
YES
```

Exercise 16.2 (easy)**.** The sample contains an endpoint touch and a collinear overlap. Identify which lines they are, and argue why the strict sign test alone returns NO for each.

16.4 Convex hull

The *convex hull* of a point set is the smallest convex polygon containing it — the rubber band around pegs. Andrew's *monotone chain* builds it in $O(n \log n)$ with no angle sort: sort by (x, y), sweep left-to-right for the lower chain and right-to-left for the upper, popping the last vertex whenever the new point makes a non-left turn (Figure 16.3). The pop condition is



sort by (x, y); sweep left to right for the lower chain, right to left for the upper chain

pop the last vertex whenever the new point makes a non-left turn (cross ≤ 0)

$(4, 4)$ lies exactly on the segment $(6, 3)$–$(2, 5)$, so it is popped; interior points never survive

Figure 16.3: Monotone chain on the CSES sample. Notice the dashed chord: $(3, 3)$ and $(4, 3)$ never reach the hull — every pop is one orientation test.

the one design decision: pop on `orient < 0` keeps collinear boundary points on the hull

(CSES wants them); pop on `orient <= 0` keeps only extreme vertices. Read the problem for which hull it defines.

```
// Andrew monotone chain, CCW hull including collinear boundary points
inline vector<Pt> hull(vector<Pt> p) {
   sort(p.begin(), p.end());
   p.erase(unique(p.begin(), p.end()), p.end());
   int n = p.size();
   if (n <= 2) return p;
   vector<Pt> h;
   for (int half = 0; half < 2; half++) {
      int base = h.size();
      for (int i = 0; i < n; i++) {
         Pt q = p[half ? n - 1 - i : i];
         while (h.size() > (size_t)base + 1 &&
               orient(h[h.size() - 2], h.back(), q) < 0)
            h.pop_back();
         h.push_back(q);
      }
      h.pop_back();
   }
   return h;
}
```

Example 16.3 (CSES *Convex Hull*)**.** Print the hull's vertex count, then its vertices (boundary points included).

Strategy. Sort, two monotone passes with strict `<0` pops so collinear boundary points survive. Any cyclic order of the same vertices is accepted.

Solution.

```
// convex_hull.cpp -- CSES 2195: hull size + vertices, boundary
   included
#include "geom.h"
#include <cstdio>
int main() {
   int n; scanf("%d", &n);
   vector<Pt> p(n);
   for (auto& q : p) scanf("%lld %lld", &q.x, &q.y);
   vector<Pt> h = hull(p);
   printf("%d\n", (int)h.size());
   for (auto& q : h) printf("%lld %lld\n", q.x, q.y);
}
```

Real output (CSES sample):

```
4
2 1
6 3
4 4
2 5
```

16.5 Polygon measures: area, lattice points, containment

The *shoelace* sums cross products of consecutive vertices: $\texttt{area2} = \sum_i x_i y_{i+1} - x_{i+1} y_i$, exactly twice the signed area — keep it doubled and integer. *Pick's theorem* ties area to the lattice: $A = I + B/2 - 1$, where I counts interior lattice points and B the boundary ones, and each edge (d_x, d_y) contributes $\gcd(|d_x|, |d_y|)$ to B (Figure 16.4).

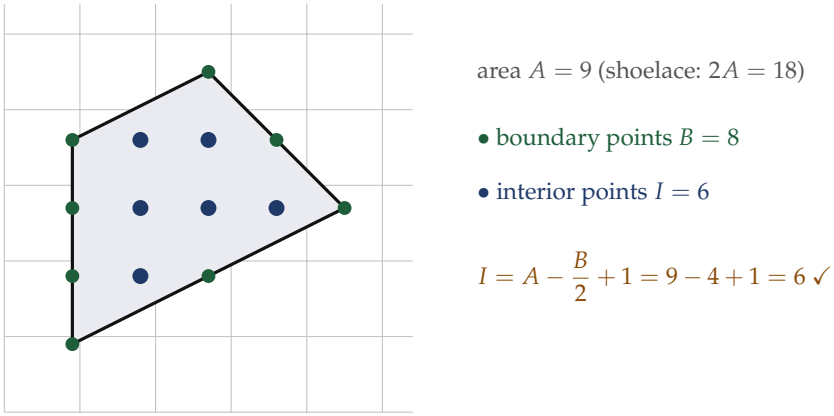


Figure 16.4: Pick's theorem on the CSES sample. Notice B counts edge *segments'* interior lattice points too — the gcd adds every lattice step along an edge.

Example 16.4 (CSES *Polygon Area*)**.** Print twice the area of a simple polygon.

Strategy. Shoelace, take the absolute value.

Solution.

```
// poly_area.cpp -- CSES 2191: print twice the polygon area
#include "geom.h"
#include <cstdio>
int main() {
    int n; scanf("%d", &n);
    vector<Pt> p(n);
    for (auto& q : p) scanf("%lld %lld", &q.x, &q.y);
    printf("%lld\n", area2(p));
}
```

Real output (CSES sample):

```
16
```

Example 16.5 (CSES *Polygon Lattice Points*)**.** Print interior then boundary lattice-point counts.

Strategy. B from the gcd sum; invert Pick: $I = (\texttt{area2} - B + 2)/2$.

Solution.

```
// poly_lattice.cpp -- CSES 2193: interior + boundary lattice points (
    Pick)
#include "geom.h"
```

```
#include <cstdio>
int main() {
   int n; scanf("%d", &n);
   vector<Pt> p(n);
   for (auto& q : p) scanf("%lld %lld", &q.x, &q.y);
   ll B = boundary_pts(p);
   ll I = (area2(p) - B + 2) / 2; // A = I + B/2 - 1
   printf("%lld %lld\n", I, B);
}
```

Real output (CSES sample):

```
6 8
```

Containment is the *ray-casting parity* test: shoot a horizontal ray, count polygon edges it crosses — odd means inside. The half-open rule (count an edge only when exactly one endpoint is strictly above the ray) is what stops vertex touches from double-counting (Figure 16.5); boundary points are caught by an explicit on-segment check first.

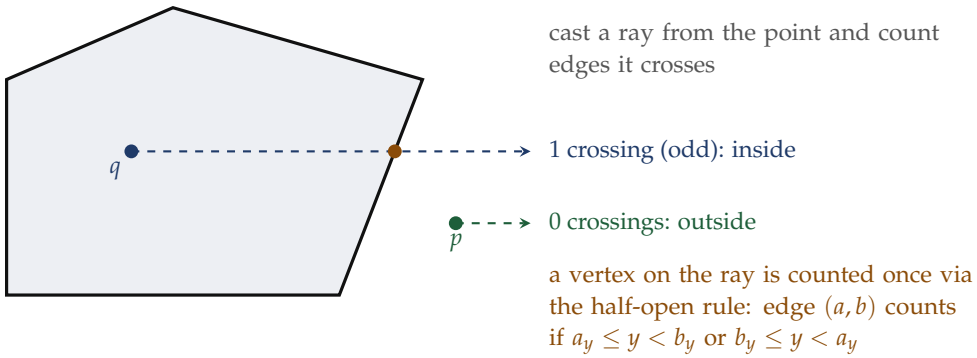


Figure 16.5: Parity ray cast. Notice which edges count: the rule is "one endpoint strictly above the ray," never both or neither.

```

// point in polygon: 0 inside, 1 boundary, 2 outside (ray cast +x)
inline int in_poly(Pt q, const vector<Pt>& p) {
   int n = p.size();
   bool inside = false;
   for (int i = 0; i < n; i++) {
      Pt a = p[i], b = p[(i + 1) % n];
      if (orient(a, b, q) == 0 && on_seg(q, a, b)) return 1;
      // does edge (a,b) cross the horizontal ray q -> +inf ?
      if ((a.y > q.y) != (b.y > q.y) &&
         orient(q, a, b) > 0 == (a.y > q.y)) inside = !inside;
   }
```

Example 16.6 (CSES *Point in Polygon*). m queries against an n-gon: INSIDE, OUTSIDE, or BOUNDARY.

Strategy. On-segment check first, then the parity flip per crossing edge.

Solution.

```
// point_in_poly.cpp -- CSES 2192: INSIDE / OUTSIDE / BOUNDARY
#include "geom.h"
#include <cstdio>
int main() {
   int n, m; scanf("%d %d", &n, &m);
   vector<Pt> p(n);
   for (auto& q : p) scanf("%lld %lld", &q.x, &q.y);
   while (m--) {
      Pt q; scanf("%lld %lld", &q.x, &q.y);
      puts((const char*[]){"INSIDE", "BOUNDARY", "OUTSIDE"}[in_poly(q,
          p)]);
   }
}
```

Real output (CSES sample):

```
INSIDE
OUTSIDE
BOUNDARY
```

Exercise 16.3 (medium). A lattice polygon has $I = 6$, $B = 8$. Find A — then exhibit a *different* polygon with the same (I, B) and convince yourself Pick still applies.

16.6 Distances

Distances stay squared. `dist2`(a, b) is exact, ordering preserved, and the closest-pair problem wants it anyway. Minimum pairwise distance among n points: sort by x, divide at the middle, recurse, then examine only the strip of width $d = \min(d_L, d_R)$ around the cut — scanning the strip in y-order touches a constant number of candidates per point, for $O(n \log n)$ overall (Figure 16.6).

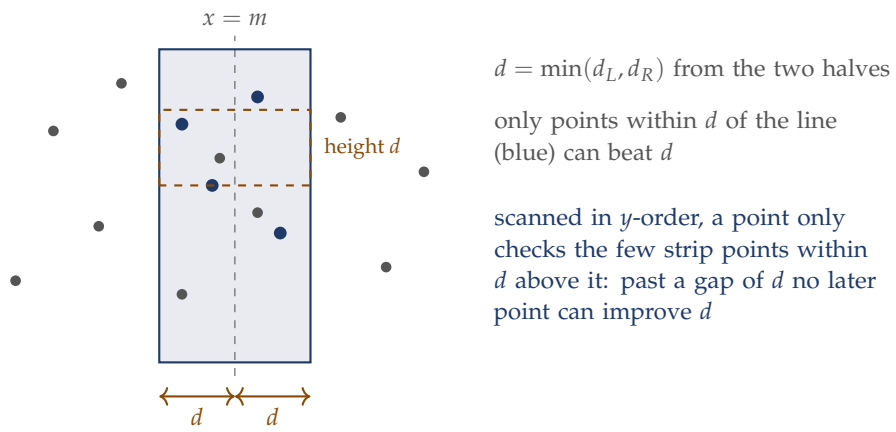


Figure 16.6: Closest pair's merge strip. Notice why y-order suffices: past a gap of d in y no later point can improve d.

Example 16.7 (CSES *Minimum Euclidean Distance*)**.** Print the smallest squared distance between two of n points, $n \le 2 \cdot 10^5$.

Strategy. Divide and conquer; never a square root — the output is d^2.

Solution.

```
// min_dist.cpp -- CSES 2194: minimum squared pairwise distance.
// Divide & conquer on x-sorted points; strip merge by y.
#include "geom.h"
#include <cstdio>
#include <functional>
int main() {
   int n; scanf("%d", &n);
   vector<Pt> p(n);
   for (auto& q : p) scanf("%lld %lld", &q.x, &q.y);
   sort(p.begin(), p.end());
   function<ll(int,int)> solve = [&](int l, int r) -> ll {
      if (r - l <= 1) return dist2(p[l], p[r]);
      if (r - l <= 2) {
         ll d = dist2(p[l], p[r]);
         d = min(d, min(dist2(p[l], p[l+1]), dist2(p[l+1], p[r])));
         return d;
      }
      int m = (l + r) / 2;
      ll mx = p[m].x;
      ll d = min(solve(l, m), solve(m, r));
      vector<Pt> strip;
      for (int i = l; i <= r; i++) {
         ll dx = p[i].x - mx;
         if (dx * dx < d) strip.push_back(p[i]);
      }
      sort(strip.begin(), strip.end(),
          [](Pt a, Pt b){ return a.y < b.y; });
      for (size_t i = 0; i < strip.size(); i++)
         for (size_t j = i + 1; j < strip.size() &&
             (strip[j].y - strip[i].y) * (strip[j].y - strip[i].y) < d
                ; j++)
            d = min(d, dist2(strip[i], strip[j]));
      return d;
   };
   printf("%lld\n", solve(0, n - 1));
}
```

Real output (CSES sample):

```
2
```

Exercise 16.4 (medium)**.** Why is a constant-size inner scan correct? Bound the number of strip points inside a $d \times d$ box when every pair in a half is at least d apart.

16.7 Common mistakes

- **Epsilon equality.** `abs(a-b)<eps` is not an integer test; it fails silently one magnitude up. Use cross products.
- **Sign-first segment intersection.** The strict double-orientation test misses every touching case — endpoint touch, collinear overlap — which CSES counts as YES. Check zero orientations first.
- **Popping collinear vertices.** `orient<=0` pops strip the boundary points CSES *Convex Hull* asks for; `<0` keeps them. Match the problem's definition.
- **Vertex-touching ray casts.** Counting every edge the ray touches double-counts polygon vertices on the ray; the half-open y rule fixes parity.
- **Overflowing `int`.** Coordinates up to 10^9 make cross products reach 10^{18} — `long long` or nothing, and keep squared distance out of `sqrt` until the answer truly needs decimals.

Summary

Need	Primitive	Cost
left/right/collinear	`orient(o,a,b)` sign	$O(1)$
segments meet	`seg_int`: zeros first, then both sign tests	$O(1)$
twice signed area	shoelace `area2`	$O(n)$
inside/outside/boundary	on-seg check + ray parity	$O(n)/q$
lattice I, B	gcd boundary sum, invert Pick	$O(n)$
hull (with boundary)	monotone chain, pop on < 0	$O(n \log n)$
closest pair	divide & conquer + y-strip	$O(n \log n)$

Exercises

Exercise 16.5 (easy)**.** CSES *Point Location Test* — re-implement `orient` unaided.

Exercise 16.6 (easy)**.** USACO-B *Square Pasture* — axis-aligned bounding square, pure integers.

Exercise 16.7 (easy)**.** USACO-B *White Sheet* — rectangle overlap area.

Exercise 16.8 (easy)**.** Kattis *Segment Distance* — your `seg_dist2` fraction, printed as a real at the end.

Exercise 16.9 (easy)**.** CF *Beru-taxi* — nearest point; compare squared distances only.

Exercise 16.10 (medium)**.** CSES *Line Segment Intersection* — full edge-case drill.

Exercise 16.11 (medium)**.** CSES *Polygon Lattice Points* — gcd sum + Pick.

Exercise 16.12 (medium)**.** Kattis *Convex Hull* — monotone chain; compare against the collinear-keeping variant.

Exercise 16.13 (medium). Kattis *Robot Protection* — hull then shoelace area.

Exercise 16.14 (medium). CF *Water Lily* — one right triangle, one equation.

Exercise 16.15 (medium). CSES *Point in Polygon* — parity + boundary handling together.

Exercise 16.16 (hard). USACO-S *Cow Steeplechase II* — all-pairs segment intersection with a twist.

Exercise 16.17 (hard). USACO-P *Cow Curling* — hull maintenance inside a DP.

Exercise 16.18 (hard). Kattis *Closest Pair* — your strip, pushed to adversarial inputs.

Exercise 16.19 (hard). Derive the *rotating calipers* diameter of a convex polygon from `orient`, then implement it.

Notes

Further reading

- A. Laaksonen, *Competitive Programmer's Handbook*, ch. 29 "Geometry", `https://cses.fi/book/book.pdf` — free PDF; the cross-product-first presentation this chapter tracks.
- cp-algorithms, *Basic Geometry* — the same integer-first discipline with more primitives. `https://cp-algorithms.com/geometry/basic-geometry.html`
- cp-algorithms, *Convex hull construction* — monotone chain plus the Graham-scan alternative. `https://cp-algorithms.com/geometry/convex-hull.html`
- cp-algorithms, *Lattice points inside non-lattice polygon* — Pick's theorem's generalisation. `https://cp-algorithms.com/geometry/lattice-points.html`
- cp-algorithms, *Finding the nearest pair of points* — the strip argument written out. `https://cp-algorithms.com/geometry/nearest_points.html`
- The USACO Guide, *Geometry Primitives*, `https://usaco.guide/plat/geo-pri` — where these primitives get consumed at Gold/Platinum.
- UNSW COMP4128, *Computational Geometry* lecture notes — contest-grade slides covering this chapter plus half-planes (file `11-comp-geom.pdf` under:) `https://cgi.cse.unsw.edu.au/~cs4128/25t2/lectures/`
- M. de Berg et al., *Computational Geometry: Algorithms and Applications* (3rd ed.), ISBN 978-3540779735 — the textbook reference when you need the proofs.

WATCH INSTEAD

- **nptelhrd — "Mod-04 Lec-07 Convex Hull"**, 52m35s, `https://www.youtube.com/watch?v=QnL0LmOO4rc` — covers §16.4: hull definitions and the Graham family the monotone chain simplifies. (Verified 2026-10-11.)

- **Coderisland — "Geometric Algorithms: Graham & Jarvis — Lecture 10"** (E. Demaine, MIT 6.851), 79m37s, https://www.youtube.com/watch?v=J5aJEcOr6Eo — covers §16.4 and the turn-test analysis; notation: he writes `ccw` for our `orient`. (Verified 2026-10-11.)
- **Aaron Becker — "Graham Scan Tutorial: Convex Hull of a Set of 2D Points"**, 3m06s, https://www.youtube.com/watch?v=9rQMLpQn5xQ — covers §16.4: a 3-minute animated pass over the same algorithm; watch this first. (Verified 2026-10-11.)

Notation note: all three use angle-sort (Graham) where we use (x, y)-sort (monotone chain); the pop condition and hull are identical.

The orientation-first style follows cp-algorithms and Laaksonen's geometry chapter [5, 2]; hull/sweep treatments track Sannemo and the UNSW slides [1, 3]; problem selection follows the official CSES geometry set [4].

CHAPTER 17

Probability basics

Contest probability problems look continuous but almost never are: every die, card, robot step, or random index is a finite uniform draw, so every question is counting in disguise — or expectation, which is counting with weights. This chapter teaches the three tools that solve the whole CSES probability set: enumerate outcomes exactly with dynamic programming, take expectation apart with linearity (which never requires independence), and condition correctly with Bayes' rule. After it you will know when a problem wants an exact rational answer, when it wants a recurrence, and why reaching for a random-number generator is usually the wrong move.

This chapter is optional for the core route; it feeds the CSES "Probability" problem set and a small but real slice of Codeforces and USACO-G.

17.1 Finite probability: counting in disguise

On a finite *sample space* Ω with equally likely outcomes, $\Pr[A] = |A|/|\Omega|$: probability is a ratio of two counts, and counting is what you already know how to do. The rules that matter are few. $\Pr[\neg A] = 1 - \Pr[A]$ (the *complement rule* — "at least one" is almost always easier from the other side); $\Pr[A \cup B] = \Pr[A] + \Pr[B] - \Pr[A \cap B]$ (inclusion–exclusion, and the reason you may not just add); and for *independent* events, $\Pr[A \cap B] = \Pr[A] \cdot \Pr[B]$. When outcomes are not uniformly likely, the same framework works: a *random variable* just assigns a number to each outcome, and a *distribution* assigns each value its probability. Computing a distribution by hand is a dynamic program: start from the initial distribution and push probability mass forward across one step at a time.

Example 17.1 (CSES *Dice Probability*)**.** Throw n dice; print the probability that the sum lies in $[a, b]$, to six decimals. $n \le 100$. **Strategy.** Count compositions of each sum with n ordered dice. The sum-distribution of n dice is a DP: after i dice, $\Pr[\text{sum} = s]$ picks up a $1/6$ contribution from each of $s-1, \dots, s-6$. Work with probabilities directly in `double` (the count 6^{100} overflows, but dividing each step by 6 keeps everything in range). Then sum the interval.

Solution.

```
// dice_probability.cpp -- CSES 1725: P(sum of n dice in [a,b]).
// Count compositions by DP; divide by 6^n.
#include <cstdio>
#include <vector>
int main() {
```

```
    int n, a, b; scanf("%d %d %d", &n, &a, &b);
    std::vector<double> dp(n * 6 + 1, 0), nd;
    dp[0] = 1.0;
    for (int i = 0; i < n; i++) {
       nd.assign(n * 6 + 1, 0);
       for (int s = 0; s <= 6 * i; s++)
          for (int f = 1; f <= 6 && s + f <= 6 * n; f++)
             nd[s + f] += dp[s] / 6.0;
       dp = nd;
    }
    double p = 0;
    for (int s = a; s <= b; s++) p += dp[s];
    printf("%.6f\n", p);
}
```

The DP is $O(n \cdot 6n \cdot 6)$. Real output on the CSES sample `2 9 10`:

```
0.194444
```

i.e. $\Pr[\text{sum} \in \{9, 10\}] = 7/36$.

Figure 17.1 shows why the answer lands where it does: the distribution concentrates around $3.5n$, so intervals off-centre collect little mass.

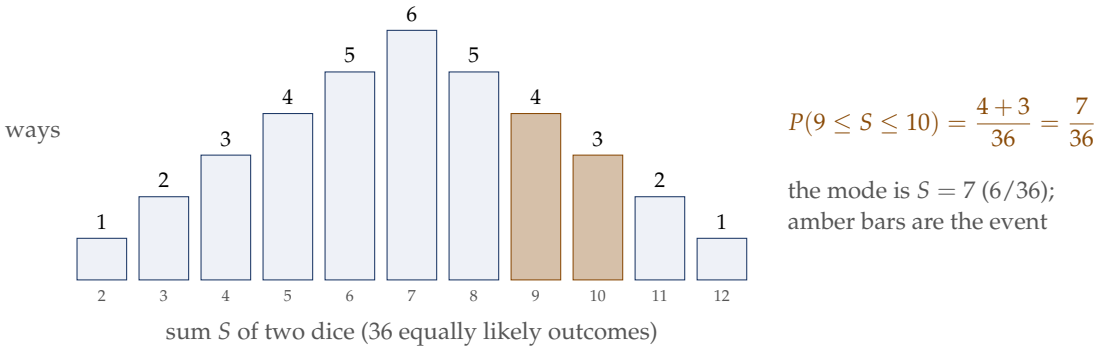


Figure 17.1: Distribution of the sum of two dice. Notice the triangular shape and that the shaded 9–10 bar pair collects $4/36 + 3/36 = 7/36 \approx 0.194$: the CSES sample answer is just area under this histogram.

Exercise 17.1 (easy). Three fair coins: compute $\Pr[\text{2 heads}]$ and $\Pr[\text{at least one}]$ both ways — direct counting and the complement rule. Which route is shorter, and why does it generalise?

17.2 Expectation and linearity

The *expected value* of a random variable is $\mathbb{E}[X] = \sum_v v \cdot \Pr[X = v]$. The single most useful fact in competitive probability is *linearity of expectation*: for any random variables, dependent or not, $\mathbb{E}[X + Y] = \mathbb{E}[X] + \mathbb{E}[Y]$. No independence is needed — this is a rearrangement of a double sum, not a probabilistic assumption. Combined with the *indicator variable* trick it disarms most expectation problems: write the quantity you want as a sum of $0/1$ variables, compute each tiny probability, add.

Theorem 17.2 (Linearity of expectation)**.** *For any random variables $X_1, \dots, X_n$ on the same space and constants c_i, $\mathbb{E}[\sum_i c_i X_i] = \sum_i c_i \, \mathbb{E}[X_i]$. The X_i may be arbitrarily dependent.*

Example 17.3 (CSES *Inversion Probability*)**.** Position i holds a uniform random integer on $\{1, \dots, r_i\}$, independent across positions. Print the expected number of inversions (pairs $a < b$ with $x_a > x_b$) to six decimals. $n \le 100$, $r_i \le 10^9$. **Strategy.** Brute-force over the product space is hopeless (r_i huge). Write the inversion count as a sum of indicators I_{ab} over all $n(n-1)/2$ pairs. Then $\mathbb{E}[\#\text{inv}] = \sum_{a<b} \Pr[x_a > x_b]$, and each term is an exact count: of the $r_a r_b$ ordered pairs, the ones with $v_a > v_b$ are $\sum_{v_a=1}^{r_a} \min(v_a - 1, r_b)$. (Figure 17.2.)

Solution.

```
// inversion_prob.cpp -- CSES 1728: position i uniform on {1..r_i};
// expected inversions = sum over pairs P(x_a > x_b).
#include <cstdio>
int main() {
   int n; scanf("%d", &n);
   int r[105];
   for (int i = 0; i < n; i++) scanf("%d", &r[i]);
   double e = 0;
   for (int a = 0; a < n; a++)
      for (int b = a + 1; b < n; b++) {
         long long cnt = 0;
         for (int va = 1; va <= r[a]; va++)
            cnt += va - 1 < r[b] ? va - 1 : r[b];
         e += (double)cnt / ((double)r[a] * r[b]);
      }
   printf("%.6f\n", e);
}
```

Real output on the CSES sample `3 / 5 2 7`:

```
1.057143
```

The three pair probabilities are 0.700, 0.286 and 0.071 — the answer is their sum; linearity does the work a $5 \cdot 2 \cdot 7 = 70$-state enumeration would otherwise do.

For non-negative integer variables a second identity earns its keep: $\mathbb{E}[X] = \sum_{v \ge 1} \Pr[X \ge v]$ (the *tail-sum formula*). It converts "expected value" into a sum of survival probabilities, which are often closed-form.

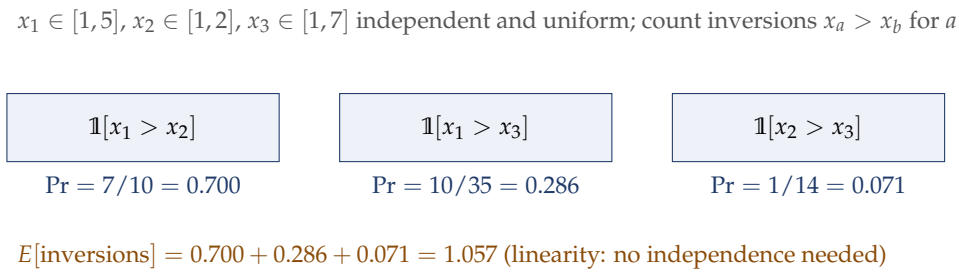


Figure 17.2: Linearity on the CSES sample: the expectation splits into three independent-looking terms, but note the x_i *are* independent here — the power of the theorem is that it would still hold if they were not. Notice each term is a rectangle count $\#\{v_a > v_b\}$.

Example 17.4 (CSES *Candy Lottery*)**.** Each of n children independently receives a uniform random number of candies between 1 and k. Print the expected *maximum* number of candies any child gets, to six decimals. $n, k \le 100$. (CSES 1727.) **Strategy.** Tail-sum. $\Pr[\max \le v] = (v/k)^n$ since the picks are independent, so $\Pr[\max \ge v] = 1 - ((v-1)/k)^n$ and $\mathbb{E}[\max] = \sum_{v=1}^{k} \big(1 - ((v-1)/k)^n\big)$. (Figure 17.3.)

Solution.

```
// candy_lottery.cpp -- CSES 1727: E[max] of n iid uniforms on {1..k}.
// Tail-sum: E[max] = sum_v P(max >= v) = sum_v (1 - ((v-1)/k)^n).
#include <cstdio>
#include <cmath>
int main() {
    int n, k; scanf("%d %d", &n, &k);
    double e = 0;
    for (int v = 1; v <= k; v++)
        e += 1.0 - pow((v - 1.0) / k, n);
    printf("%.6f\n", e);
}
```

Real output on the CSES sample `2 3`:

```
2.444444
```

i.e. $\mathbb{E}[\max] = 1 + 8/9 + 5/9 = 22/9$.

The other workhorse is the *coupon collector*: drawing uniformly with replacement from n kinds, the expected draws to see all n is $nH(n) = n\sum_{i=1}^{n} 1/i$. Each new kind is a geometric wait with success probability $(n-i)/n$ — line up the waits and linearity sums them (Figure 17.6).

Exercise 17.2 (medium)**.** A random permutation of $\{1, \dots, n\}$: show the expected number of fixed points is exactly 1 for every n, using indicators only — no derangement counting.

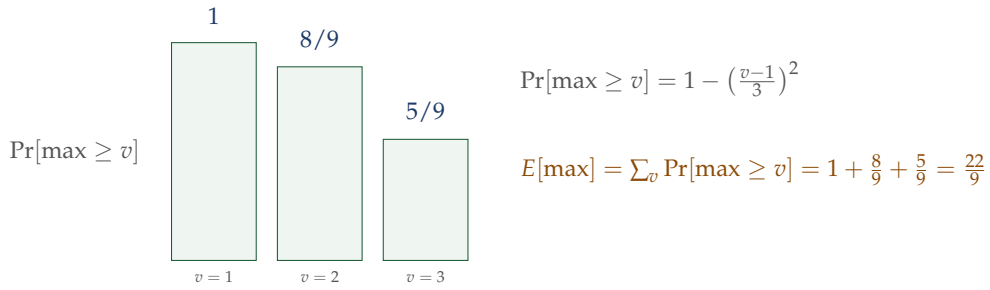


Figure 17.3: Tail-sum for $\mathbb{E}[\max]$ with $n = 2$, $k = 3$: each column is $\Pr[\max \geq v]$. Notice the expectation is literally the total shaded height — a sum of ones-place probabilities, no PMF required.

17.3 Conditioning, independence, Bayes

Conditional probability renormalises the sample space: $\Pr[A \mid B] = \Pr[A \cap B] / \Pr[B]$. Two structural facts carry the weight. First, *total probability*: if the H_i partition Ω, then $\Pr[B] = \sum_i \Pr[B \mid H_i] \Pr[H_i]$ — split by a convenient case, compute within each, recombine. Second, *Bayes' theorem*: $\Pr[H_i \mid B] = \Pr[B \mid H_i] \Pr[H_i] / \Pr[B]$, and the denominator is the same total-probability sum — never estimate it separately. Independence is the special case $\Pr[A \cap B] = \Pr[A] \Pr[B]$; it is the exception, not the default, and contest problems that say "independent" are handing you a multiplication rule, not a modelling hint.

Figure 17.4 is the classic sanity check: a 99%- accurate test on a 1%-prevalent condition yields a true positive only about half the time — $0.99 \cdot 0.01 / (0.99 \cdot 0.01 + 0.01 \cdot 0.99) = 1/2$. The tree *is* the computation; Bayes' rule just reads it backwards.

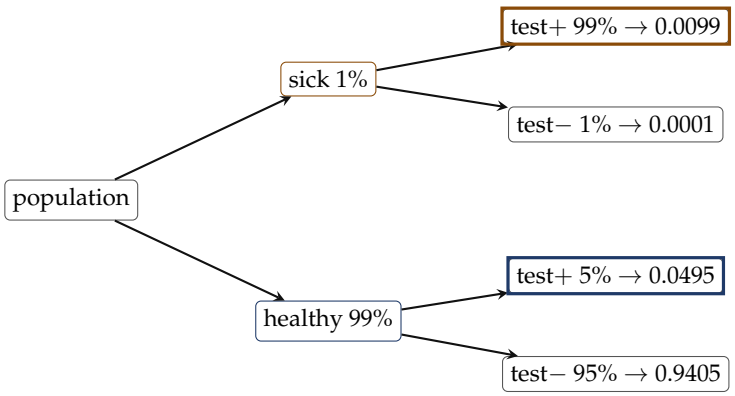


Figure 17.4: Bayes as a renormalised tree. Notice the posterior is a ratio of two shaded paths: $\Pr[\text{sick} \mid +]$ divides the sick-and-positive mass by *all* positive mass. Drawing the tree first prevents the usual denominator mistakes.

Exercise 17.3 (easy). A bag has 3 red and 2 blue marbles; draw two without replacement. Compute $\Pr[\text{second is red}]$ by total probability over the first draw, and check it equals $3/5$. Explain the symmetry.

17.4 Distributions as dynamic programs

When the number of steps is large, push the whole distribution forward one step at a time: a vector of probabilities per state and a transition that reweights it. Two CSES staples.

Example 17.5 (CSES *Throwing Dice*)**.** Count the number of ways to write n as an ordered sum of terms in $\{1,\dots,6\}$, modulo 10^9+7. $n \le 10^{18}$. **Strategy.** This is the dice distribution's denominator: $f(i) = \sum_{j=1}^{6} f(i-j)$. For small n a sliding six-cell window does it in $O(n)$ — but the CSES limit is $n \le 10^{18}$, so the recurrence must be powered as a 6×6 companion matrix (Ch. 14's Fibonacci trick, extended to a sixth-order recurrence). The state $[f(i),\dots,f(i-5)]$ advances by one companion-matrix multiply per step, giving $O(\log n)$.

Solution.

```
// throwing_dice.cpp -- CSES 1096: compositions of n with parts <= 6,
// n up to 1e18. Linear recurrence f(i)=sum_{j=1..6} f(i-j) via 6x6
// matrix power mod 1e9+7. f(0)=1, f(i<0)=0.
#include <cstdio>
typedef long long ll;
const ll M = 1000000007;
typedef ll Mat[6][6];
void mul(Mat a, Mat b, Mat c) {
   for (int i = 0; i < 6; i++)
      for (int j = 0; j < 6; j++) {
         ll s = 0;
         for (int k = 0; k < 6; k++) s = (s + a[i][k] * b[k][j]) % M;
         c[i][j] = s;
      }
}
int main() {
   ll n; scanf("%lld", &n);
   // companion: row 0 = all ones; row i>0 shifts f down.
   Mat T = {{1, 1, 1, 1, 1, 1},
            {1, 0, 0, 0, 0, 0},
            {0, 1, 0, 0, 0, 0},
            {0, 0, 1, 0, 0, 0},
            {0, 0, 0, 1, 0, 0},
            {0, 0, 0, 0, 1, 0}};
   Mat R = {{1,0,0,0,0,0},{0,1,0,0,0,0},{0,0,1,0,0,0},
            {0,0,0,1,0,0},{0,0,0,0,1,0},{0,0,0,0,0,1}};
   Mat tmp;
   while (n > 0) {
      if (n & 1) { mul(R, T, tmp); for (int i=0;i<6;i++) for (int j=0;
         j<6;j++) R[i][j]=tmp[i][j]; }
      n >>= 1;
      if (n) { mul(T, T, tmp); for (int i=0;i<6;i++) for (int j=0;j<6;
         j++) T[i][j]=tmp[i][j]; }
   }
   // state vector [f(0),f(-1),...,f(-5)]^T = [1,0,...,0]: f(n) is
      entry [0][0].
   printf("%lld\n", R[0][0]);
}
```

Real output on the CSES sample $n = 8$:

```
125
```

Example 17.6 (CSES *Moving Robots*)**.** Each of the 64 squares of an 8×8 board starts with one robot. Every turn, every robot steps to a uniformly random neighbouring square. After k turns, print the expected number of *empty* squares to six decimals. $k \le 100$. **Strategy.** Square s is empty iff *no* robot is there. Robots move independently, and different start squares stay independent, so for each source robot t run a k-step distribution DP over the board: $p_{t\to s}$ is the probability t's robot sits on s. Then $\Pr[s \text{ empty}] = \prod_t (1 - p_{t\to s})$ and the answer is $\sum_s$ of that — linearity again, over 64 vacancy indicators. (Figure 17.5.)

Solution.

```
// moving_robots.cpp -- CSES 1726: 8x8 grid, one robot per square,
// each turn every robot moves to a uniform random neighbour.
// E[empty squares] = sum_s prod_t (1 - p_{t->s}).
#include <cstdio>
#include <vector>
using namespace std;
int main() {
   int k; scanf("%d", &k);
   const int N = 64;
   vector<vector<int>> adj(N);
   auto id = [](int r, int c) { return r * 8 + c; };
   const int dr[4] = {-1, 1, 0, 0}, dc[4] = {0, 0, -1, 1};
   for (int r = 0; r < 8; r++) for (int c = 0; c < 8; c++)
      for (int d = 0; d < 4; d++) {
         int nr = r + dr[d], nc = c + dc[d];
         if (0 <= nr && nr < 8 && 0 <= nc && nc < 8)
            adj[id(r, c)].push_back(id(nr, nc));
      }
   double expect = 0;
   for (int t = 0; t < N; t++) { // robot starting at t
      vector<double> cur(N, 0), nxt;
      cur[t] = 1.0;
      for (int step = 0; step < k; step++) {
         nxt.assign(N, 0);
         for (int s = 0; s < N; s++)
            for (int v : adj[s]) nxt[v] += cur[s] / adj[s].size();
         cur = nxt;
      }
      // cur[s] = P(robot from t is on s). We need prod over t of
      // (1 - p_{t->s}); accumulate per square.
      static double notempty[64] = {};
      if (t == 0) for (int s = 0; s < N; s++) notempty[s] = 1.0;
      for (int s = 0; s < N; s++) notempty[s] *= (1.0 - cur[s]);
      if (t == N - 1) for (int s = 0; s < N; s++) expect += notempty[s
         ];
   }
   printf("%.6f\n", expect);
}
```

Real output on the CSES sample $k = 10$:

```
23.120740
```

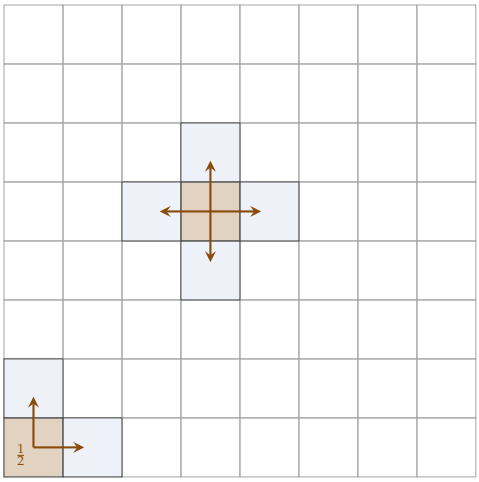


Figure 17.5: One source robot's position distribution after one turn on the 8×8 board — each neighbour carries mass $1/\deg$. Notice the vacancy product treats the 64 independent sources separately; the expectation then sums over squares, never over robot configurations (2^{64} of them).

Exercise 17.4 (medium)**.** In *Moving Robots*, why is $\prod_t (1 - p_{t\to s})$ the correct vacancy probability, and which step would break if the robots were required to move in lock-step (all in the same direction)?

17.5 Exact answers vs. simulation

Judges accept answers to $\sim 10^{-6}$ absolute or relative error. Two consequences. First, prefer exact arithmetic until the last moment: probabilities like $7/36$ can be carried as integer numerator/denominator (with `__int128` if needed) and converted once — or, as in this chapter, kept in `double` provided you divide per-step rather than computing 6^{100} first. Second, simulation is a debugging tool, not a solver: a Monte Carlo estimate with T trials has error $\Theta(1/\sqrt{T})$, so 10^{12} trials buy roughly 10^{-6} — you cannot get there, and CSES-style six-decimal acceptance still rejects the variance. Use simulation to *check* an exact formula, as in the next worked example.

Example 17.7 (Coupon collector, exact vs. simulated)**.** For n coupon kinds, compare the closed form $nH(n)$ against an `mt19937` simulation of 200,000 collectors.

Solution.

```
// coupon_collector.cpp -- coupon collector: E = n*H(n) vs simulation.
#include <cstdio>
#include <random>
int main() {
    int n; scanf("%d", &n);
```

```
    double h = 0;
    for (int i = 1; i <= n; i++) h += 1.0 / i;
    printf("exact %.6f\n", n * h);
    std::mt19937 rng(12345);
    double sim = 0;
    const int T = 200000;
    for (int t = 0; t < T; t++) {
        std::vector<char> seen(n, 0);
        int got = 0, draws = 0;
        while (got < n) { draws++; if (!seen[rng() % n]++) got++; }
        sim += draws;
    }
    printf("simulated %.6f\n", sim / T);
}
```

Real output on $n = 6$:

```
exact    14.700000
simulated 14.673785
```

Two hundred thousand trials still miss the second decimal — that gap is why simulation verifies and exact DP submits.

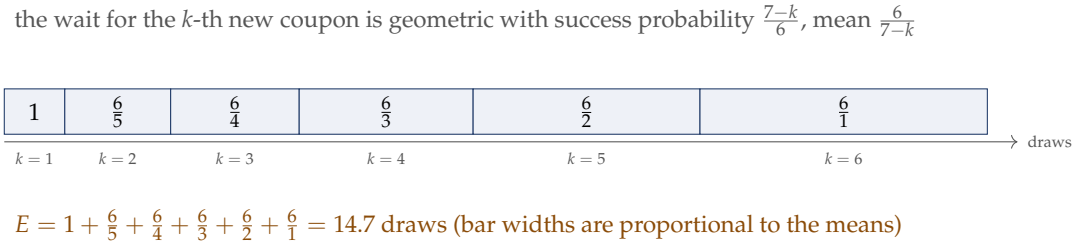


Figure 17.6: Coupon collector as a sum of geometric waits. Notice each stage i waits $\mathbb{E} = 1/p_i = n/(n-i)$ draws: the tail stages ($p \to 1/n$) dominate, which is why $H(n)$, not a constant multiple, appears.

Exercise 17.5 (hard). Prove the tail-sum identity $\mathbb{E}[X] = \sum_{v \geq 1} \Pr[X \geq v]$ for integer $X \geq 0$ by rewriting the double sum $\sum_v \Pr[X = v] \cdot v$ as a triangle of terms. (Hint: write v as $\sum_{j=1}^{v} 1$.)

17.6 Common mistakes

- **Requiring independence for linearity.** $\mathbb{E}[X+Y] = \mathbb{E}[X] + \mathbb{E}[Y]$ needs nothing — the CSES pair-sum and coupon waits both exploit dependent pieces.
- **Adding probabilities of overlapping events.** $\Pr[A \cup B]$ double-counts $A \cap B$; only add when the events are disjoint (or use complements/inclusion–exclusion).
- **Reaching for `rand()` early.** Monte Carlo cannot hit 10^{-6} reliably; it belongs in your stress test, not your submission.
- **Precision mistakes at the boundary.** Dividing *once at the end* by 6^{100} overflows; dividing each step by 6 is safe. Print with `%.6f` from a `double`.

- **Bayes without the denominator.** $\Pr[H \mid E] = \Pr[E \mid H]\Pr[H]/\Pr[E]$ and $\Pr[E]$ is a total-probability sum — drawing the tree first makes this mechanical.

Summary

Need	Tool	Cost
"at least one"	complement rule $1 - \Pr[\neg A]$	$O(1)$
P(sum in range)	distribution DP over steps	$O(nS)$
expected count	indicators + linearity	one small prob each
$E[X]$, $X \geq 0$	tail-sum $\sum \Pr[X \geq v]$	often closed-form
posterior	Bayes with total-prob denominator	tree first
many steps, few states	per-source distribution DP	$O(k \cdot \lvert\text{states}\rvert)$
sanity-check formula	`mt19937` simulation	verify only

Exercises

Exercise 17.6 (easy)**.** CSES *Dice Probability* — re-derive the DP without looking at the listing.

Exercise 17.7 (easy)**.** Two fair dice: compute $\Pr[\text{sum} \geq 10]$ by counting, and $\Pr[\text{sum} \geq 10 \mid \text{first die} \geq 4]$ by conditioning.

Exercise 17.8 (easy)**.** Expected value of one fair die, and of the *max* of two fair dice (tail-sum recommended).

Exercise 17.9 (easy)**.** CSES *Throwing Dice* — implement the sliding window; confirm $f(8) = 125$.

Exercise 17.10 (easy)**.** A fair coin until the first head: show the expected number of tosses is 2 both from the geometric formula and by conditioning on the first toss.

Exercise 17.11 (medium)**.** CSES *Candy Lottery* — submit the tail-sum solution; then verify $n = 2, k = 3$ by full enumeration in a script.

Exercise 17.12 (medium)**.** CSES *Inversion Probability* — implement; derive the closed form $\Pr[x_a > x_b] = \frac{\binom{r_b+1}{2} + (r_a - r_b - 1)_+ r_b}{r_a r_b}$ variant when $r_a > r_b$.

Exercise 17.13 (medium)**.** CSES *Moving Robots* — implement the 64-source DP; check $k = 0$ gives 0.000000 exactly.

Exercise 17.14 (medium)**.** Balls into bins: m uniform balls into n bins; expected occupied bins $= n(1 - (1 - 1/n)^m)$. Prove it with indicators.

Exercise 17.15 (medium)**.** Expected number of tosses of a fair coin to see *two consecutive* heads is 6 — derive it by conditioning on the first two outcomes (set up $\mathbb{E} = 1 + \frac{1}{2}\mathbb{E}[\cdot]$ recurrences).

Exercise 17.16 (medium)**.** CF *Another Problem About Dividing Numbers*-style tasks aside: solve Kattis *Expected Earnings* — a small direct linearity exercise.

Exercise 17.17 (hard)**.** Expected length of the longest run of heads in n fair flips: derive a distribution DP over (length so far, best so far) and compute it for $n = 10$ to six decimals.

Exercise 17.18 (hard)**.** Gambler's ruin: starting with i of N total units, win/lose ± 1 each round; prove the expected duration solves a linear recurrence and compute it for $i = N/2$.

Exercise 17.19 (hard)**.** Expected position of the *second largest* of n i.i.d. uniform$\{1..k\}$ draws — generalise the tail-sum approach to the r-th order statistic.

Exercise 17.20 (hard)**.** Modular expectation: when a judge asks for $P \cdot Q^{-1} \bmod 10^9 + 7$, derive why Fermat inversion plus exact counting is exact, and re-implement *Throwing Dice* that way.

Notes

Further reading

- C. M. Grinstead and J. L. Snell, *Introduction to Probability* (2nd ed.) — free, thorough, finite-sample-space first; chapters 1, 4 and 6 cover this whole chapter. (page under:) `https://chance.dartmouth.edu/teaching_aids/` (books_articles → probability_book)
- MIT OCW 6.041/6.431, *Probabilistic Systems Analysis* — full lecture notes and videos; Lectures 1–9 match §17.1–§17.4. (course page under:) `https://ocw.mit.edu/` (courses → 6-041 Probabilistic Systems, Fall 2010)
- A. Laaksonen, *Competitive Programmer's Handbook* — the probability chapter; the same CSES problems framed as contest tools. `https://cses.fi/book/book.pdf`
- J. K. Blitzstein and J. Hwang, *Introduction to Probability* (2nd ed.), ISBN 978-1138369917 — the best treatment of linearity-without-independence and the indicator trick anywhere.
- CSES problem set, *Probability* section — eight tasks; five of them are this chapter's worked examples. `https://cses.fi/problemset/`
- Khan Academy, *Statistics and probability* — gentle refresh on expectation and conditioning if the notation is new. `https://www.khanacademy.org/math/statistics-probability`
- J. Pitman, *Probability*, ISBN 978-0387979748 — measure-free rigour; where to go when the intuitive arguments here stop feeling airtight.
- G. Grimmett and D. Welsh, *Probability: An Introduction* (2nd ed.), ISBN 978-0198574237 — compact and proof-first; good second pass after Grinstead–Snell.

WATCH INSTEAD

- **3Blue1Brown — "Bayes theorem, the geometry of changing beliefs"**, 15m01s, https://www.youtube.com/watch?v=HZGCoVF3YvM — covers §17.3: the tree/geometry view that makes the denominator obvious. Watch this first. (Verified 2026-10-11.)
- **MIT OpenCourseWare — "1. Probability Models and Axioms"** (6.041, Tsitsiklis), 51m11s, https://www.youtube.com/watch?v=j9WZyLZCBzs — covers §17.1–§17.3: axioms, conditioning, total probability. Notation: he writes $P(A^c)$ and Ω exactly as here. (Verified 2026-10-11.)
- **Khan Academy — "Expected Value: E(X)"**, 14m53s, https://www.youtube.com/watch?v=j__Kredt7vY — covers §17.2's definition; note it does *not* prove linearity for dependent variables, where this chapter's power comes from. (Verified 2026-10-11.)

Notation note: all three are discrete-first, matching this chapter; the OCW lecture additionally uses set-builder event notation that maps one-to-one onto §17.1.

The distribution-DP framing follows Laaksonen's probability chapter and the CSES problem set [2, 4]; linearity-of-expectation pedagogy follows Blitzstein–Hwang [8]; simulation-vs-exact guidance reflects standard contest practice [5].

CHAPTER 18

The contest playbook

Everything before this chapter taught you to solve problems. This last one is about the machine that wraps around solving: the half-hour before the clock starts, the scoreboard telling you where the points are, the decision to abandon a problem, the brute-force oracle that catches the bug your tests missed, and the week after, when the contest actually teaches you something. Technique gets you to a score; process gets you to a rating. After this chapter you can run a contest like an engineer: a plan going in, telemetry during, and a triage afterwards.

18.1 Before the contest

The contest is decided before it starts more often than people admit. A fixed pre-contest routine removes the one-shot decisions that cost real minutes under pressure. Thirty minutes out: environment ready (compiler flags, template file, a `diff`-able checker), a fresh physical notebook, water, phone off. Ten minutes out: re-read the rules that cost points — submission limits, penalty per wrong try, whether partial credit exists (IOI-style subtasks change the whole strategy; see §18.6). One minute out: nothing. Starting cold and calm beats starting warm and frantic.

Example 18.1 (The ten-minute setup)**.** Write your setup down once and reuse it verbatim — ours, used throughout this book: template file loaded, `g++ -O2 -std=c++17 -Wall` alias checked on a trivial compile, `stress.sh` reachable, scratch page split into "idea / invariant / test" columns. It takes under ten minutes and removes the most common minute-zero stalls.

Solution. The deliverable is a checklist, not a contest performance — its entire purpose is to be executed without thinking. Real output of `stress.sh -help`-style first run on a fresh box:

```
MISMATCH seed=15  brute=-15  fast=0
input:
1
-15
```

— proving the harness itself works before you need it.

18.2 Reading the scoreboard

The scoreboard is live information, not decoration. Three numbers matter per problem: how many teams solved it (difficulty proxy), how many *tried* it (read effort proxy), and your own attempts (sunk cost to resist). Solving order follows the solved-count, not the problem letters — the crowd has already ranked the set for you (Figure 18.1). ICPC rules make this sharper: penalty minutes accumulate on wrong submissions to *solved* problems, so a speculative submit on a cold problem is cheap (zero penalty if never solved) while a third wrong guess on a problem you'll solve is expensive.

Rank	Team	A	B	C	D	E	Penalty
1	byte girls	+2	+			+1	89:52
2	night owls	+	+1			−2	143:20
3	you			−3			—
4	segfaults	−1					—
5	newbies						—

+ solved first try +k solved after k failed tries −k k failed tries, unsolved penalty = solve time + 20 min per failed try

read it as: problems with the most tries and no solve are where the effort goes (here: you on C)

Figure 18.1: A scoreboard five minutes in. Notice what each cell tells you: +/− counts reveal where attempts are going, and the top rows' unsolved letters are the de-facto hard end of the set.

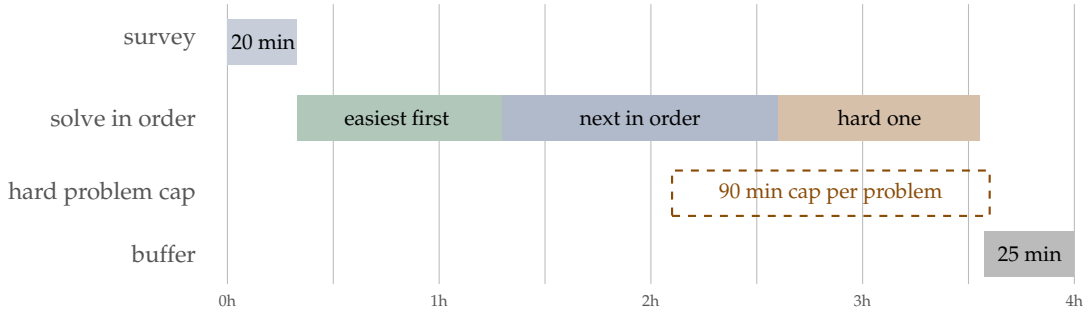
Example 18.2 (Five-minute scoreboard read)**.** Snapshot: 4 of 7 problems solved by someone; B has 40 submissions and 30 solves; E has 3 tries, 0 solves. You have solved nothing yet. **Strategy.** B is the intended entry — a 3/4 solve rate at volume means readable and mechanical. Skip A/C/D's lower traffic for now; E is untouched for a reason. Your order: B, then whichever of A/C shows the next-solve first (fresh telemetry), E last or never.

Solution. Order set in under a minute, and it updates: if A gets the next solve before C, A moves ahead. The scoreboard re-ranks the set every few minutes — check it whenever you finish a problem or go quiet for ten minutes.

Exercise 18.1 (easy)**.** On the same snapshot, argue why "start with E because nobody solved it, glory awaits" is usually wrong — and name the one situation (your specialty topic) where it is right.

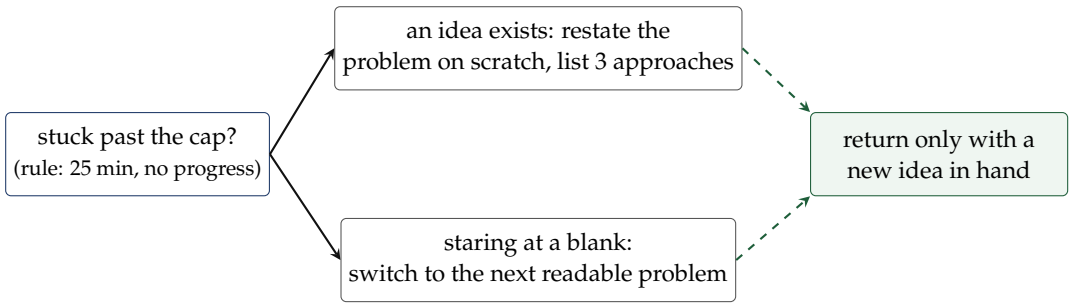
18.3 Time allocation and switching

A four-hour contest is a budget: survey (∼20 min), solve-in-rank-order, per-problem caps, reserve the last 25 minutes for submit-or-stress only (Figure 18.2). The per-problem cap is the rule everyone agrees with and nobody obeys: **25 minutes of no new idea means switch**. Not 35, not "one more thing to try." The trigger is not elapsed time but *progress* — a new formulation, a shrinking search space, a passing test. Staring blankly at the statement for the fifth time is the canonical no-progress signal (Figure 18.3).



rules: survey first; easiest first and never skip a problem twice; hard cap per problem; the last 25 min is submit-or-stress-test only

Figure 18.2: A 4-hour budget that survives contact. Notice the plan bar vs. the work bars: the buffer is scheduled, not stolen; the last block is for submitting and stress-checking only, never for starting something new.



the real rule: switching is free, re-reading the same wall is what costs time

Figure 18.3: The switching decision. Notice it asks "do you have a new idea," not "have you spent long enough" — an unexplored approach resets the clock, a blank stare does not.

Example 18.3 (A contest log, annotated)**.** Minute 0–18: survey all 7, sketch data ranges. 18–55: solve B (impl, first try). 55–80: D attempt, two wrong tries on samples, no new idea at minute 80 → switch. 80–110: solve C. 110–135: back on D with the trick seen while doing C — solved. 135–200: stress A against brute; found and fixed an edge case, submitted. 200–240: writeup of E abandoned at 15% confidence — final reserve held. **Strategy.** The log shows the two habits that pay: switching on a progress trigger (not a clock), and returning with ammunition rather than stubbornness.

Solution. Three solves plus a fourth saved by the stress reserve. The same contest played "in order, no switches" ends at two.

Exercise 18.2 (medium)**.** Write your own caps: survey length, per-problem cap, switch trigger, reserve length — for a 2-hour CF Div. 2 *and* a 5-hour ICPC regional. Defend why the numbers differ.

18.4 Stress testing

The strongest debugging weapon in this book is two programs and a loop: a *brute force* so simple it is obviously correct, your fast submission, a generator of small random inputs, and a comparator that stops on the first difference (Figure 18.4). Small inputs are the point: bugs live in edge cases (empty, singleton, all-equal, negative, overflow-adjacent), and small cases let you run thousands of rounds per second while still being checkable by hand when one fails.

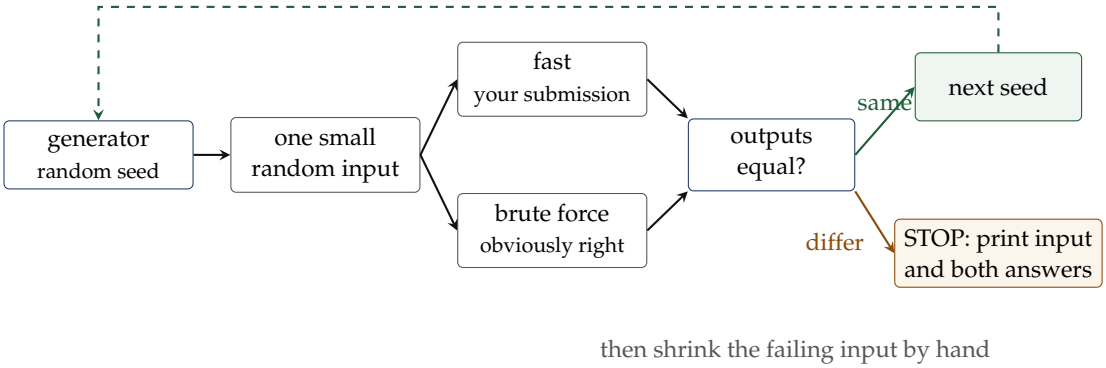


Figure 18.4: The harness. Notice there is no "is it right?" step — only "do they agree?" Correctness is borrowed from the brute force; the fast program only has to match it.

Example 18.4 (The harness, working)**.** Chapter 1's maximum-subarray problem: Kadane's against the $O(n^3)$ triple loop, on 1–12 element arrays in $[-20, 20]$. We feed it `mss_buggy`, a Kadane variant that lets the empty segment in by starting `cur` and `best` at zero.

Solution.

```
// gen_mss.cpp -- random test generator for the stress harness.
// Usage: ./gen_mss SEED -> prints n and n ints in [-20, 20].
#include <cstdio>
#include <cstdlib>
#include <random>
int main(int argc, char** argv) {
   unsigned seed = argc > 1 ? (unsigned)strtoul(argv[1], 0, 10) : 1;
   std::mt19937 rng(seed);
   int n = 1 + rng() % 12;
   printf("%d\n", n);
   for (int i = 0; i < n; i++)
      printf("%d ", (int)(rng() % 41) - 20);
```

```
    printf("\n");
}
```

```
// mss_buggy.cpp -- a subtly wrong Kadane's, for the stress demo.
// Bug: resets the running sum to 0 instead of restarting at a[i],
// which silently allows empty segments on all-negative arrays.
#include <algorithm>
#include <iostream>
#include <vector>
using namespace std;
using ll = long long;
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<ll> a(n);
    for (auto& v : a) cin >> v;
    ll best = 0, cur = 0; // BUG: 'empty segment allowed'
    for (int i = 0; i < n; ++i) {
        cur = max(0LL, cur + a[i]); // should be max(a[i], cur+a[i])
        best = max(best, cur);
    }
    cout << best << '\n';
}
```

```
#!/usr/bin/env bash
# stress.sh -- Chapter 18 stress harness: gen -> fast vs brute ->
    verdict.
# Usage: bash stress.sh FAST BINARY (default: mss_buggy), rounds
    default 500.
set -u
cd "$(dirname "$0")"
CXX=${CXX:-clang++}
FAST=${1:-mss_buggy}
ROUNDS=${2:-500}
mkdir -p .build
$CXX -std=c++17 -O2 -o .build/gen_mss gen_mss.cpp
$CXX -std=c++17 -O2 -o .build/brute mss_brute.cpp
$CXX -std=c++17 -O2 -o .build/fast "$FAST.cpp" || exit 1

for ((s = 1; s <= ROUNDS; s++)); do
    ./.build/gen_mss "$s" > .build/in.txt
    want=$(./.build/brute < .build/in.txt)
    got=$(./.build/fast < .build/in.txt)
    if [ "$want" != "$got" ]; then
        echo "MISMATCH seed=$s brute=$want fast=$got"
        echo "input:"; cat .build/in.txt
        exit 1
    fi
done
echo "stress: $ROUNDS/$ROUNDS rounds identical ($FAST vs mss_brute)"
```

Real output:

```
MISMATCH seed=15  brute=-15  fast=0
input:
1
-15
```

Fifteen seeds, and the harness hands you a one-element failing case: all-negative input, where the buggy version's empty segment answers 0 instead of -15. Rerun on the corrected `mss_linear`:

```
stress: 500/500 rounds identical (mss_linear vs mss_brute)
```

The counterexample it gives you is usually already small; when it is not, *shrink* it: delete elements while the outputs still differ. A failing case you cannot explain is a failing case you cannot fix — the shrink loop is where the understanding lives.

Example 18.5 (When to reach for it)**.** Rules of thumb: any greedy that "looks right" (ch. 3), any DP with boundary states (chs. 11–12), anything with a special case you added last (chs. 16–17 all hit this — `seg_int`'s endpoint-touch and `digitdp`'s adjacent rule were both harness-caught). If the problem has no cheap oracle, write the oracle: a slower correct solution exists for almost every problem in this book.

Solution. The cost is real — 20–40 minutes to build the pair. The return is a wrong-answer converted to a failing input, which is the difference between debugging and guessing.

Exercise 18.3 (medium)**.** Build the pair for ch. 12's *Counting Numbers*: the DP solver plus a brute `for(int x=a;x<=b;x++)` checker on small ranges. What input class did your own solver's bug live in?

18.5 Upsolving

The contest ends; the learning starts. Every problem you failed or skipped goes into one of three buckets within the week: knew-it-but-botched (rewrite cold, no reference), idea was close (one editorial line, then 30 more minutes alone), unknown technique (editorial + two more problems using it). Figure 18.5 makes it a pipeline with one exit: an accepted submission *and* a one-line lesson in your log. "I knew that one" is not a lesson; "break ties by the right endpoint, not the left" is.

Example 18.6 (Triage after a Div. 2)**.** Post-contest pile: B (WA on test 4), D (skipped), E (no idea). B: your comparator forgot equal keys — bucket 1, rewrite cold, accepted in 15 min, log: "sort ties break on index." D: one hint line ("think backwards") $\rightarrow$ solved in 40 min, bucket 2. E: editorial says centroid decomposition — bucket 3: implement it, then find two centroid problems and log the pattern.

Solution. Three problems, three different actions, three log lines. That is one contest converted to durable skill; skip the week and it was only a number.

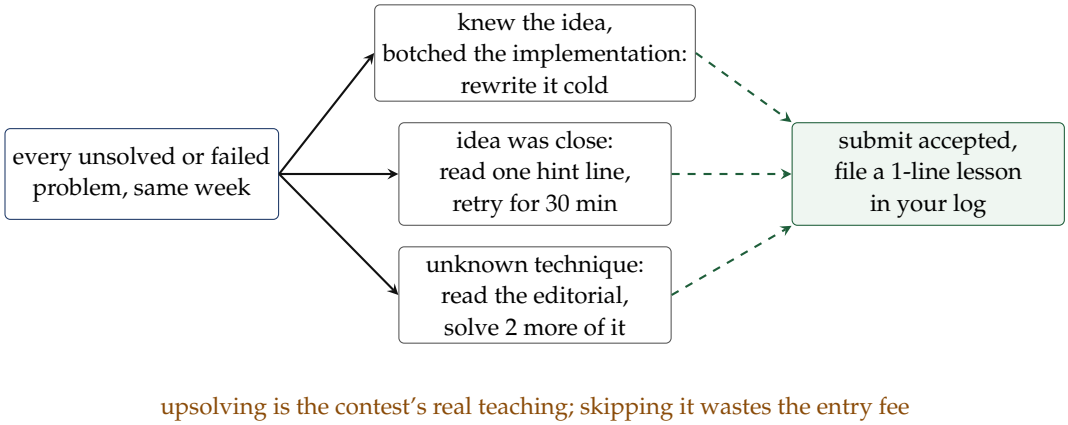


Figure 18.5: Upsolve triage. Notice each bucket has a different prescription — reading the editorial is the *last* bucket's tool, and applying it to the first two is how a contest teaches nothing.

Exercise 18.4 (easy)**.** Take your last practice set and assign every unsolved item to a bucket. If everything lands in bucket 3, your contest level is too high — explain why.

18.6 The practice ladder and contest styles

Rating climbs by repetition at the right altitude (Figure 18.6): solve where your success rate is 40–60%, move up when it passes 60%, move down after three consecutive failures, and draw from named banks (CSES for fundamentals, USACO Guide lists for technique depth, CF div. 2 A–C for speed). Adjust the machinery per style: **ICPC** is teams, penalty minutes, and scoreboard meta-play — pick off the solved-count leaders and guard your penalty. **IOI** is solo, subtask-scored, five hours — partial credit means a brute-force subtask taken early is worth real points *and* doubles as your stress oracle for the full score. **CF** sits between: short rounds, systests, and pretest gamesmanship — never trust a pretest-pass as final.

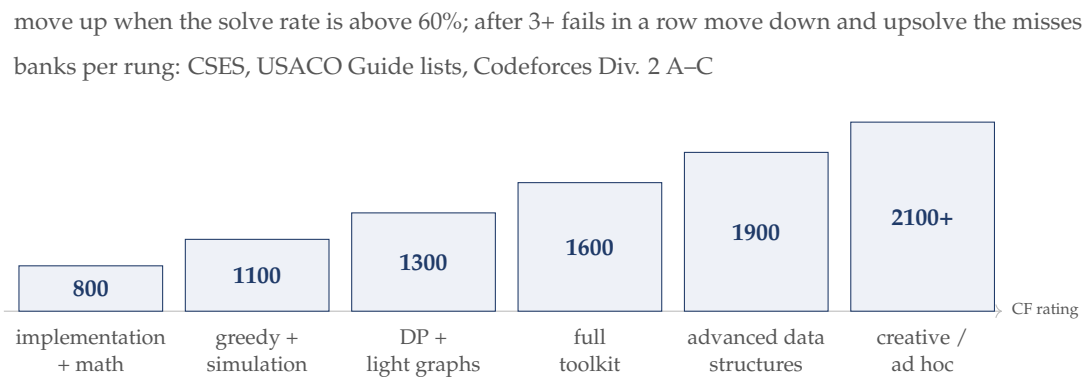


Figure 18.6: The 800→2100 ladder. Notice the height is solve rate, not volume: the ladder moves you, not the other way around.

Example 18.7 (A twelve-week plan)**.** Weeks 1–4: CSES Introductory set (Weird Algorithm through Two Knights) — unranked, fundamentals only, target 20+ solves. Weeks 5–8: CF div. 2 A–B live, upsolve every C you miss; weekly contest regardless of result. Weeks 9–12:

CSES Sorting/DP overlap + div. 2 C as the tracked metric. Target: consistent div. 2 A+B, occasional C — rating = consistency, not peak.

Solution. The plan's only hard rule is the upsolve loop from §18.5; everything else is scheduling.

18.7 Common mistakes

- **Solving in statement order.** Problem letters are not difficulty ranks — the scoreboard's solve counts are.
- **The sunk-cost stall.** "I've spent an hour" argues for leaving, not staying; the cap triggers on progress, not investment.
- **Guessing a fix and resubmitting.** ICPC penalty minutes and CF failed-test locks both tax blind retries — stress first, submit once.
- **No brute force, ever.** "Too slow for the judge" is the feature: the oracle only has to be right.
- **Upsolving by reading the editorial end-to-end.** It converts an unknown-technique problem into a trivia fact — the bucket-3 treatment needs implementation, not comprehension.
- **Practicing only where you win.** A 90% solve rate is a flat ladder: the altitude is wrong, not the volume.

Summary

Moment	Move	Tool
before clock	fixed checklist, harness ready	§18.1
first 20 min	survey all problems, sketch ranges	§18.2
rank order	solve by solved-count, not letters	scoreboard
stuck	progress cap, not time cap	§18.3
before submit	brute-vs-fast until identical	`stress.sh`
same week	triage fail/skip into 3 buckets	§18.5
between rounds	40–60% altitude, named banks	§18.6

Exercises

Exercise 18.5 (easy)**.** Write your pre-contest checklist (10 lines max), then time yourself executing it.

Exercise 18.6 (easy)**.** For a contest you already played, reconstruct the solve order the scoreboard was telling you at minute 30 vs. the order you actually used.

Exercise 18.7 (easy)**.** Implement `gen/brute/fast` for ch. 3's coin-change greedy vs. its DP — the harness should find the canonical counterexample.

Exercise 18.8 (easy)**.** List the three smallest inputs that break `mss_buggy`-class logic (empty, singleton negative, all-negative) and explain why each is the *minimal* witness.

Exercise 18.9 (medium)**.** Time-cap experiment: run a 2-hour set twice — once with a hard 25-minute switch rule, once ad-lib — and compare solves and stress.

Exercise 18.10 (medium)**.** Extend `stress.sh` with input *shrinking*: on MISMATCH, try deleting each element while outputs still differ. Report how small your failing cases get.

Exercise 18.11 (medium)**.** Upsolve your last 10 fails with the 3-bucket triage; log one line each. What fraction were bucket 1 — and what does that say about your bottleneck?

Exercise 18.12 (medium)**.** Build a personal "first-15-minutes" routine: what you read, what you compute per problem, what you write down. Test it on a virtual contest.

Exercise 18.13 (medium)**.** ICPC vs. IOI scoring: take one problem and describe the optimal attempt plan under each — where does brute-forcing a subtask sit in each plan?

Exercise 18.14 (medium)**.** Design a generator that adversarially covers your bug history: for each past WA, encode the input class that caught it. Run it as regression after every new solver.

Exercise 18.15 (hard)**.** Auto-verify a whole chapter: wire `stress.sh`-style oracles for five ch. 11–17 solvers, run 200 rounds each, and record which solver needed the largest generator space to stay green.

Exercise 18.16 (hard)**.** Model the scoreboard as data: given per-problem solve counts over time, write the re-ranking rule §18.2 uses informally and simulate its chosen order on a real contest's history.

Exercise 18.17 (hard)**.** Prove to yourself the ladder works: track solve-rate vs. CF rating over 30 problems and test the 60% rule against your own trajectory (a spreadsheet is fine).

Exercise 18.18 (hard)**.** Write a "contest minute" postmortem template — timeline, decision points, what the scoreboard said vs. what you did — and use it on your next three contests; report what changed.

Exercise 18.19 (hard)**.** For a five-hour IOI-style set, write the attempt plan that maximizes expected partial credit given per-subtask point values; show where full-score attempts are worth their risk.

Notes

Further reading

- USACO Guide, *Practicing CP* — the training method this chapter's ladder compresses; del Noré's what-to-practice framework.
  https://usaco.guide/general/practicing
- A. Laaksonen, *Competitive Programmer's Handbook* — its advice chapters on contest technique and debugging.
  https://cses.fi/book/book.pdf
- Topcoder Thrive, *How To Find a Solution* — classic problem-attack walkthroughs, same lineage as the survey-first habit. (Thrive articles → title above, under:)
  https://www.topcoder.com/
- ICPC, *World Finals Rules* — the penalty model §18.2 reads from; worth reading once, primary source.
  https://icpc.global/worldfinals/rules
- Kenkoooo AtCoder Problems — the other major problem bank with rating tables; the ladder's alternate track.
  https://kenkoooo.com/atcoder/
- cp-algorithms — the reference for every technique an upsolve bucket-3 will send you to.
  https://cp-algorithms.com/
- infoarena *evaluator* notes — how judges and partial scoring work, the IOI-side view.
  https://infoarena.ro/blog/evaluator
- Codeforces problem set — the bank itself [unverified from this session: CF serves 403 to curl].
  https://codeforces.com/problemset

WATCH INSTEAD

- **Errichto — "How to test your solution in Competitive Programming, on Linux?"**, 26m56s, https://www.youtube.com/watch?v=JXTVOyQpSGM — covers §18.4: the brute-vs-fast methodology built live. Watch this first. (Verified 2026-10-11.)
- **Colin Galen — "Competitive Programming: How to Learn from Contests"**, 20m58s, https://www.youtube.com/watch?v=E-aylp6MZnM — covers §18.5–18.6: post-contest analysis and rating mechanics. (Verified 2026-10-11.)
- **Errichto — "How to start Competitive Programming? For beginners!"**, 9m43s, https://www.youtube.com/watch?v=xAeiXy8-9Y8 — covers §18.1–18.2: the survey and setup mentality. (Verified 2026-10-11.)

Notation note: Galen uses "rating" for our "ladder altitude"; Errichto's Linux shell maps one-to-one onto `stress.sh`.

The harness methodology follows the standard generator–oracle–comparator pattern as practiced on this book's own code; the upsolve and ladder framing follows the USACO

Guide's practice guidance [3]; scoring models from the ICPC rules [9] and the CSES problem set structure [4].

Bibliography

[1] J. Sannemo, *Principles of Algorithmic Problem Solving*, 2025. https://usaco.guide/PAPS1.pdf

[2] A. Laaksonen, *Competitive Programmer's Handbook*, 2017–2024. https://cses.fi/book/book.pdf

[3] The USACO Guide, collective authorship, https://usaco.guide/ (accessed 2026-10-10).

[4] CSES Problem Set, https://cses.fi/problemset/list (accessed 2026-10-10).

[5] CP-Algorithms, collective authorship, https://cp-algorithms.com/ (accessed 2026-10-10).

[6] cppreference.com, *C++ standard library reference*, https://en.cppreference.com/ (accessed 2026-10-10).

[7] S. Halim, F. Halim, S. Effendy, *Competitive Programming 4*, 2020. Companion site https://cpbook.net/ (book itself paywalled; cited only).

[8] J. K. Blitzstein, J. Hwang, *Introduction to Probability*, 2nd ed., CRC Press, 2019. ISBN 978-1138369917.

[9] ICPC, *World Finals Rules*, https://icpc.global/worldfinals/rules (accessed 2026-10-11).

Index

0-1 BFS, 99
10^8 heuristic, 8

adjacency list, 78
amortised analysis, 67
    accounting, 67
Array Division, 48

backtracking, 26
Bayes' theorem, 180, 184
Bellman–Ford, 90, 91
BFS, 78
big-O, 7
binary exponentiation, 147
binary lifting, 112, 115
binary search, 45
    invariants, 45
    limitations, 53
    on the answer, 46
    real-valued, 52
    template, 45
binpow, 149
bipartite, 83
bitmask, 20, 129
bitmask DP, 129
bitset, 133
border, 156
brute force, *see* complete search
brute-force oracle, 195

Catalan numbers, 150
coin DP, 121
combinatorics, 147
common mistakes, 13, 29, 41, 54, 64, 75, 87, 99, 109, 119, 126, 135, 144, 153, 166, 177
comparator, 6, 33
complement rule, 180
complete search, 17
    definition, 17
conditional probability, 184
connectivity, 102
constant factor, 9
constraint
    as invitation, 17
constraints, 9
contest strategy, 192
    common mistakes, 199
contest workflow, 1
convex hull, 168, 171
coordinate compression, 34
coupon collector, 180
coupon collector problem, 183
cross product, 168
cut property, 104

derangement, 150
DFS, 78
diameter, 113
difference array, 56
    1D, 58
digit DP, 129, 130
Dijkstra, 90, 93
disjoint set union, 102
distance
    squared, 175
distribution
    dynamic programming over, 185
DSU, 102
dynamic programming, 121

Euclidean algorithm, 147
Euler tour, 115
event, 180
exchange argument, 36
exercises, 14, 29
expected value, 180
    linearity, 182

Factory Machines, 47

failure link, 159
fast input/output, 5
Fenwick tree, 138
    lowbit, 138
floating point
    avoiding, 168
flood fill, 81
Floyd–Warshall, 90, 95
functional graph, 85

gcd, 147
geometry, 168
graph
    vocabulary, 78
graph traversal, 78
greedy algorithm, 33
    definition, 35
    failure, 39
greedy stays ahead, 36
grid DP, 125

hash collision, 156
histogram
    largest rectangle, 69

ICPC, 198
idempotent, 72
inclusion–exclusion, 150
    rectangles, 59
independence, 184
indicator variable, 180
`int`, 4
integer geometry, 168
IOI, 198
`ios::sync_with_stdio`, 5

judge, 1

k-th element, 143
Kadane's algorithm, 10
Kahn's algorithm, 84
KMP, 156
knapsack, 121, 123
Kruskal, 104

lazy propagation, 138, 140
LCA, 112
lexicographic order, 23
linearity of expectation, 180
    no independence needed, 182
LIS, 121, 124

`map`, 6
matrix exponentiation, 152
memoization, 121
memory limit, 1
minimum spanning tree, 102
modint, 149
modular arithmetic, 147
modular inverse, 149
monotone chain, 171
monotone predicate, 46
monotonic deque, 67
    sliding window, 70
monotonic stack, 67
    nearest smaller, 67
Monte Carlo simulation, 187
Multiplication Table, 50

`next_permutation`, 23
    must sort first, 23

Ω, 7
order statistic, 141
orientation test, 168
overflow, 4

Painting the Barn, 60
period, 156
permutation
    enumerating, 23
Pick's theorem, 168, 173
point in polygon, 173
practice ladder, 192, 198
pre-contest routine, 192
precision, 187
prefix, 156
prefix function, 156, 159
prefix sums, 56
    1D, 56
    2D, 59
    hash counting, 62
Prim, 105
priority queue, 6
probability, 180
    common mistakes, 188

prove or distrust, 40
pruning, 26

RAM model, 7
range DP, 129, 130
range query, 138
relaxation, 90
rerooting, 112, 114
rolling hash, 156

sample space, 180
sample test, 11
scoreboard, 192
    reading, 193
search space, 17
    sketching, 28
segment intersection, 170
segment tree, 138, 139
`set`, 6
shoelace formula, 168, 173
shortest path, 90
sieve of Eratosthenes, 147
sliding window, 62
smallest prime factor, 148
Social Distancing, 50
`sort`, 6
sorting
    `std::sort`, 33
sparse table, 67, 72
square-root decomposition, 67, 74
stars and bars, 150
starter file, *see* template
state expansion, 97
`std::set`, 41
stdin, 1
stdout, 1
STL, 6
stress test, 12
stress testing, 192
    harness, 195
strings, 156
structured binding, 6
Subarray Sums II, 62
submit loop, 11
subsequence, 156
subset
    enumerating, 19
substring, 156
subtask, 2
subtree aggregate, 112
suffix, 156
switching problems, 194
symmetry
    halving the tree, 22

tail-sum formula, 182
template, 3
Θ, 7
time allocation, 192
time cap, 194
time complexity, 7
time limit, 1
topological order, 84
topological sort, 78
total probability, 184
tree, 112
tree DP, 129, 132
trie, 156, 160
two pointers, 56, 61

undefined behaviour, 4
upsolving, 192
    discipline, 197

`vector`, 6
verdict, 1

worst case, 9
Wrong Answer, *see* verdict

Z-box, 160
Z-function, 156, 160